GOOGLE | CODE ONLY | HELPERS + MAIN CODE

Study path, code only

288 questions in 43 groups, in the same order and with the same numbers as the study path book. Each question shows its statement and examples, then only the code you write for it: the helper functions first, then the main answer, with the same syntax colours as the main book. The brute force, the follow-up code and the tests are not here; they are in the study path book, whose groups also start with the code template. The italic line under each title says what you change in the group's template for that question.

38 EASY	197 MEDIUM	53 HARD

1. Frontier search (BFS or DFS), Tier 1: spread, count and measure distance (9)

1. Flood Fill
2. Number of Islands
3. Rotten Oranges / Multi-Source BFS (taxis)
4. Solve Grid and Counting Problems
5. Find shortest relationship path using BFS
6. Farthest City for a New Data Center
7. Compute distance to nearest taxi in grid
8. Infection Spread on a Grid (Cellular Automaton)
9. Shortest Distance From All Buildings

2. Sweep over events in time order (9)

10. Meeting Rooms / Interval Scheduling Variants
11. Train / Flight Schedule Reachability
12. Solve Rooms and Top-K Streams
13. Compute servers needed for daily recurring jobs
14. Gate Entry / Exit Time Resolution with Tie-Break Rules
15. Meeting Rooms III Variant
16. Count Overlapping Rectangle Updates on a Grid
17. Compute city skyline outline
18. Compute Turnstile Crossing Times

3. Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward (9)

19. Given two trees, determine if the pre-order traversals of two trees are same or not.
20. Calculate the height of a binary tree.
21. Find right-side view of binary tree
22. Expression Tree Single-Leaf Mutation
23. Longest Consecutive Path in Binary Tree
24. Given the root of a binary tree, return the length of the diameter of the tree.
25. Implement zigzag level-order traversal
26. Repeatedly Delete Leaves Matching a Target Value From a Binary Tree
27. Delete an Entire Binary Tree

4. A heap of what is available: pop the best, put back the changed one (9)

28. Streaming Median (LC 295) with Follow-ups
29. Compute minimum number of rooms needed
30. Implement a Reusable Locker Allocation System
31. Simulate room assignments for scheduled meetings
32. Trip Expense Settlement
33. Playlist Shuffle No Adjacent Artists
34. Minimum Makespan on Identical CPUs
35. Design server allocation for multi-type nodes
36. Huffman Tree Construction

5. Recurrence on a position or a range (memo or table) (9)

37. Target Expressions from Ordered Digits
38. Count Word Decompositions into Chemical Element Symbols

39. Find the longest palindromic subsequence using dynamic programming.
40. Jump Game with Profit
41. Fence Painting Minimum Operations (Gemini Team)
42. Maximize 1D deliveries within distance and minimize total distance
43. Edit distance
44. Count Perfect Wake Numbers up to N
45. Word Break: All Sentences

6. Best-first search with a heap: Dijkstra and its minimax twin (9)

46. Path Minimizing the Maximum Cell (Swim in Rising Water)
47. Nearest Favorite City by Travel Time
48. Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time.
49. Find the shortest path in a graph.
50. Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat.
51. Minimize maximum height along a grid path
52. Find minimum threshold enabling grid path
53. Minimize travel time with optional meeting point
54. Compute minimax grid path and network delay

7. Backtracking: choose, recurse, undo (9)

55. Mahjong Winning Hand Detection (14 Tiles)
56. Count Prefix-Matching Simple Paths in a Labeled Graph
57. Generate Parentheses
58. American Football Score Combinations
59. Permutations
60. Determine if a 14-tile hand is winning
61. Count Passcode Patterns on a Keypad
62. Match people to questions using tags and priorities
63. Matrix Flower Placement with House Adjacency

8. Runs and groups: extend while it fits, restart or close when it does not (9)

64. Group Timestamps Into Close Intervals
65. First Out-of-Order Character
66. Longest Non-Decreasing Subarray with One Replacement
67. Piano Hand Movement Grouping
68. Find longest increasing contiguous subarray
69. Sum of Good Subarrays (Adjacent Diff ±1)
70. Sustained-Error Time Window
71. Longest Non-Decreasing Subarray From Two Arrays
72. Count Super Streaks

9. Union-find: merge the pairs, then ask who is together (8)

73. Count Islands in a Grid and Track the Count as Land Cells Are Added
74. 2D Points Connected Components by Radius
75. Count Clusters of 2D Points Within a Radius
76. Find A Threshold-Limited Path With Minimum Required Safety
77. Movie Recommendations via Transitive Similarity
78. Determine Reporting Relationships
79. Determine whether two nodes are related
80. Friend Graph with Timeline (Friend / Unfriend Events)

10. A dict of per-key state that you update, expire, or rank (8)

81. Robot Status Message Deduplication Window
82. Active Communication Top Users
83. Filter Repeated Robot Status Messages
84. Next-Word Predictor
85. Batch and Streaming Log Deduplication
86. Design a logger that returns top-K messages
87. Hit Counter
88. Website Activity Analytics: Unique Users + Session Time

11. Binary search on a yes/no condition (8)

89. First Index Reaching a Threshold (Binary Search)

90. Cutting Ribbons (LC 1891)
91. First Bad Version with Parallel Search Follow-up
92. Solve Banana Speed and Interval Merge
93. Given a list with response time for each server, calculate how long time it will take to process n requests.
94. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant
95. Find the median of two sorted arrays.
96. Find longest duplicated substring

12. Greedy: scan once, carry the best choice so far, act only when forced (8)

97. Maximum Profit
98. Alarm for the Next Upcoming Birthday
99. Coins on the Table
100. Visible People in a Line (Last Person, Then Everyone)
101. Bounded Knapsack Variant
102. Find the Largest Monotone Increasing Number
103. Min boards to cover roof holes
104. Jump Game II (Minimum Jumps)

13. Sets and counters: remember the first collection, check the second against it (8)

105. Exclude First-K-Unique From Another List
106. Find the Duplicates
107. Positional Matches Between Two Lists
108. Contains Duplicate
109. Filter Common Elements Per Window (Two Lists)
110. Reconstruct original array from doubled shuffle
111. Min deletions to avoid overlap in first k
112. Smallest Missing Positive Integer in Linear Time and Constant Extra Space

14. bisect on a sorted list: find a position, count a range, keep it sorted (8)

113. Count People in a Postal Code Range
114. Sliding-Window Mean After Removing the Largest K Values
115. Russian Doll Envelopes
116. Count Sorted Strings With a Given Prefix
117. Streaming Insert + Find K-th Largest
118. Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.
119. Find Maximum Envelope Nesting
120. Count of Smaller Numbers After Self

15. Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) (8)

121. Parity Alternating Subarray Query
122. Cake Balance Line
123. Find the Pivot Index (Balanced Split) in O(n)
124. Range Duplicate Check with Constant-Time Queries
125. Find a Horizontal Cut That Bisects Rectangle Area
126. Check pivot after removing one element
127. Gold Chain Split into Two Equal Halves
128. Count the number of ways to split a string.

16. Sorted intervals: merge while they touch, then read the gaps or search (8)

129. Check an Interval Against Existing Intervals for Conflicts
130. Find Containing Range
131. Union of Sorted Interval Lists
132. Range Module (Insert and Point Query)
133. Meetings Minus a Do-Not-Schedule Block
134. Merge Intervals
135. Find safe travel intervals between planet influences
136. Implement a function that processes a stream of interval data.

17. Trie: nested dicts, one character per level (8)

137. URL Request Handler Router
138. Trie Prefix Search / Autocomplete
139. Build Prefix Lookup with a Trie
140. Greedy Longest-Match Dictionary Tokenizer
141. Check if all substrings are dictionary words

142. Return dictionary words matching a prefix
143. Implement Trie Insert and Exact-Word Search
144. Implement Longest-Match Text Replacement

18. A dict for lookup plus an ordered container for who is next (lazy deletion) (8)

145. Implement Batched Undo/Redo Layer
146. Design a restaurant waitlist system
147. Implement Python's itertools.tee
148. Store With Cheapest-Item Removal
149. Round-Robin Job Scheduler
150. Aggregate FizzBuzz Offer Matches
151. Restaurant Waitlist: join / delete / find_first_match
152. Implement LRU Cache.

19. Rolling state: the next state comes only from the previous one (7)

153. Climbing Stairs
154. Fibonacci Numbers
155. Maximize sum without choosing adjacent elements
156. Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.
157. Paint House II
158. Write SQL code to publish the Fibonacci series.
159. Constrained Dice Roll Sequences

20. Two pointers: move i and j toward each other (or along two lists) (7)

160. Reverse a linked list.
161. Sort a binary array in linear time.
162. Is this a valid palindrome?
163. Reverse a Sentence
164. Find median of merged RLE arrays
165. Run-Length Vector Storage + Dot Product
166. Count Unique Triplets That Sum to Zero

21. Walk two strings at once with two indices (7)

167. Find the First Occurrence of a Substring
168. Determine if two words are sorted in lexicographic order.
169. Return all unique substrings of a given string.
170. Can board states be transformed?
171. Decide string transform with directional tokens and walls
172. Determine if two sentences are similar.
173. Find Dictionary Matches for a Jammed String

22. Simulate a machine: a class with state and one branch per kind of event (7)

174. Mario-Style Score Tracker (OOP)
175. Tic-Tac-Toe System with K Players and 3-in-a-Row Win
176. Design Tic-Tac-Toe With K Players
177. Infinite Deck Card Duel
178. Implement iterator merging lists with filtering
179. Design an elevator control system
180. Simulate In-Place Cellular State Updates

23. Top-K by frequency: Counter plus nsmallest (6)

181. Top-K Frequent in Large Logs
182. Rearrange Tasks With Cooldown
183. Implement frequency + distance top■K queries
184. Solve chat, grid paths, and car rentals
185. Top-K Chatters by Word Count
186. Find top/bottom-k words in list or stream

24. Digits and characters: peel one at a time and carry (6)

187. Caesar Cipher: Recover Key, Encrypt, and Decrypt
188. Add two big integers from digit lists
189. Solve several coding interview problems
190. Check Whether Digits Are Monotone Increasing
191. Check Digits and Combine Ranges

192. Permutation Sequence

25. Look at the input once and keep only a small summary (and count your calls) (6)

193. Two-Day Log Intersection under Memory Limit
194. Find shared objects across two log files
195. Streaming Points: Emit K Within Distance
196. Minimize calls to find all bad test pairs
197. Find All Bad Pairs Using runTest Oracle
198. Find maximum value in huge text file

26. Grid DP: each cell from its neighbours above and to the left (5)

199. Grid Path Count with Diagonal Moves and Checkpoints
200. Longest Same-Color Grid Sequence
201. Count Same-Color Squares in Matrix
202. Find the Longest Straight Run of One Color
203. Count same-color squares in a character grid

27. Nested structures: a stack of contexts, or recursion with a cache (5)

204. Decode String
205. Template Variable Expansion with Follow-Ups
206. Expand Recursive Key-Value Placeholders in a Template
207. Nested List Weight Sum (LC 339) with Follow-Ups
208. Nested Function Expression Evaluator

28. Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly (5)

209. Shortest Path Nodes and Escaping a Pursuer
210. Radio Broadcast Reachability
211. Minimum Direction Violations in a Directed Graph
212. Compute shortest paths with blocked nodes
213. Shortest Directed Cycle Through a Node

29. Topological sort (Kahn) and cycle detection (5)

214. Course Scheduling and Dependency Variants
215. Validate Race Results (Cycle Detection)
216. Solve tree, graph, sliding-window problems
217. Validate course catalog dependencies
218. Course Schedule

30. Counting pairs by looking up each partner in a Counter or a bucket (5)

219. Progressive Array Partitioning
220. K-diff Pairs in an Array
221. Count Axis-Aligned Squares Among Grid Points
222. Sum of Two Squares in Two Ways
223. Maximum-Area Rectangle from Planar Points

31. Tree DP: build a parent order once, then add up from the leaves (5)

224. Compute File System Total Size
225. Minimum-Cost Tree Disconnection
226. Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.
227. Compute distance-sum from every tree node
228. Tree Distance Sum (Re-root DP)

32. Counting or reaching an amount: the knapsack loop (5)

229. Maximize Shelf Space (Unbounded Knapsack)
230. Coin Change
231. Partition an array into two sub-arrays with equal sum.
232. Explain how to find a target sum in an array.
233. Reverse Coin Change

33. Prefix sums, Tier 2: running total plus a dict of earlier totals (5)

234. Maximum-Sum Subarray With Equal Endpoints
235. Maximum Subarray Sum With a Minimum Length Constraint
236. Solve array-sum and city-community problems
237. Find the maximum subarray sum.

238. Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k.

34. Group by a canonical key (5)

239. Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.
240. Check if all substrings are anagrams of words
241. Group anagrams
242. Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.
243. Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second?

35. K-way merge: a heap of the next item from each source (5)

244. Find k pairs with smallest sums
245. K Largest Elements Across Sorted Lists
246. Sort a nearly sorted (or K sorted) array.
247. Merge k sorted linked lists.
248. Return top-k subset sums in descending order

36. Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up (5)

249. Next Birthday (BST In-Order Successor with Wraparound)
250. Lowest Common Ancestor
251. Determine if a given binary tree is a binary search tree (BST).
252. Find mode in a trinary search tree
253. Next Upcoming Birthday in a Family Tree

37. Random pick without repeats: swap the chosen item with the last one and pop (5)

254. Random Generator Without Repeats
255. Random Song Shuffler With a No-Repeat Window
256. Design set with O(1) random access
257. Unique Random Number Generator
258. Shuffle array using random integer API

38. Build and serialize: write a structure out, read it back with the same walk (5)

259. Construct a binary tree given its preorder and inorder traversals.
260. Write functions to serialize and deserialize a list of strings.
261. Print a Directory Tree
262. Implement a decode and encode function of a general N-ary tree.
263. Serialize and Deserialize a Graph

39. Games: win or lose with a cache, and guessing with feedback (5)

264. Choose optimal guesses for green-only Wordle
265. Wordle-Style 5-Letter Minimum-Guess Strategy
266. Determine winner in stack-merging game
267. Find secret word with match-count feedback
268. Babylon Tile-Stack Merge Game

40. Sliding window: grow on the right, shrink on the left while it is too bad (5)

269. Temperature Monitoring Data Structure
270. Shortest Subarray with K Distinct Integers
271. Find the minimum window substring.
272. Footpath Beauty After Repainting
273. Necklace Cut into Two Halves with Equal D/R Counts

41. Parent pointers: store who is above each node and climb (5)

274. Tree Node.
275. Employee Hierarchy Add / Delete
276. Validate parent array forms a tree
277. Lowest Common Ancestor of a Binary Tree III
278. Root a Tree So It Becomes a Binary Tree

42. Build or transform a grid with index arithmetic (5)

279. Filled ASCII Diamond
280. Connected Crop Arrangement on a Grid
281. Rotate Any m×n 2D Matrix

282. Solve matrix groups and recipe inventory
283. Range Sum Query 2D (Immutable Matrix)

43. Split into pieces, fix each piece, join (and pack words into lines) (5)

284. Implement string formatting and request tracking tasks
285. Snake Case to Camel Case with Repository-Wide Refactor
286. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)
287. Implement a function to justify a list of words within a given width.
288. Convert Valid Snake Case to Lower Camel Case

GROUP 1 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE | 1 E 6 M 2 H

Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

1 Flood Fill — EASY

Template fit: starts = [(sr, sc)]; usable = same colour as the start; repaint every cell you reach; return early if old colour == new colour

QUESTION

You are given an m x n grid image where image[r][c] is the integer color of the pixel at row r , column c . You are also given a starting pixel (sr, sc) and a new_color . Perform a flood fill: starting from (sr, sc) , repaint that pixel and every pixel connected to it 4-directionally (up, down, left, right) that shares the SAME original color as the starting pixel, changing them all to new_color . Pixels connected only diagonally are not filled. Return the modified grid.

Function signature

```
from typing import List
def flood_fill(image: List[List[int]], sr: int, sc: int, new_color: int) ->
List[List[int]]:
    ...
```

Follow-up (as reported): Implement with DFS (recursive or explicit stack) or BFS; discuss the base case that prevents re-visiting when new_color equals the original color.

EXAMPLES

Example 1	
Input	`image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, new_color = 2`
Output	`[[2,2,2],[2,2,0],[2,0,1]]`
Why	`Starting color is 1. All 1-pixels connected 4-directionally to (1,1) become 2. The bottom-right 1 is only diagonally reachable, so it stays.`

Example 2	
Input	`image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, new_color = 5`
Output	`[[5,5,5],[5,5,5]]`
Why	`The whole grid is one connected region of color 0.`

Example 3	
Input	`image = [[0,0,0],[0,1,0]], sr = 1, sc = 1, new_color = 1`
Output	`[[0,0,0],[0,1,0]]`
Why	`The start pixel already equals new_color, so nothing changes.`

CONSTRAINTS

- 1 <= m, n <= 50
- 0 <= image[r][c], new_color < 2^16
- 0 <= sr < m , 0 <= sc < n
- If the starting pixel already has new_color , the grid is returned unchanged (guard against infinite recursion).

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Iterative DFS with a guard
# Idea: Spread over same-coloured neighbours and repaint;
# skip everything when the colour does not change.
def flood_fill(image, sr, sc, new_color):
    old = image[sr][sc]
    if old == new_color:
```

```
        return image
    stack = [(sr, sc)]
    image[sr][sc] = new_color
    while stack:
        r, c = stack.pop()
        for nr, nc in (
            (r + 1, c),
            (r - 1, c),
            (r, c + 1),
            (r, c - 1),
        ):
            if (
                0 <= nr < len(image)
                and 0 <= nc < len(image[0])
                and image[nr][nc] == old
            ):
                image[nr][nc] = new_color
                stack.append((nr, nc))
    return image
```

2 Number of Islands — MEDIUM

Template fit: loop over every cell; each unseen land cell starts a new frontier_search (share one seen set); islands += 1 per start

QUESTION

EXAMPLES

```
[[1,1,0,0],[1,1,0,0],[0,0,1,1],[0,0,1,1]] -> 2.
Streaming: [[0,0],[0,1],[1,1]] adds -> counts [1, 1, 1] (last addition merges the two land cells).
```

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))

def neighbors(r, c, rows, cols):
    # the cells beside (r, c) that are inside the grid
    for dr, dc in DIRS:
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            yield nr, nc

def flood(grid, start, seen):
    # visit all land joined to start; return the count
    rows, cols = len(grid), len(grid[0])
    seen.add(start)
    stack, size = [start], 0
    # a stack instead of recursion: no recursion limit
    while stack:
        r, c = stack.pop()
        size += 1
        for nb in neighbors(r, c, rows, cols):
            if grid[nb[0]][nb[1]] == 1 and nb not in seen:
                seen.add(nb)
                stack.append(nb)
    return size

def island_sizes(grid):
    if not grid:
        return []
    seen, sizes = set(), []
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            # an island starts at land not visited yet
```

```
            if grid[r][c] == 1 and (r, c) not in seen:
                sizes.append(flood(grid, (r, c), seen))
    return sizes

# BASE QUESTION (optimal): Iterative DFS flood fill
# Idea: Scan every cell; each unvisited land cell starts a
# new island, and an explicit stack marks all connected
# land as visited. Iterative DFS avoids Python's recursion
# limit on large grids.
def num_islands(grid):
    # one entry per island
    return len(island_sizes(grid))
```

3 Rotten Oranges / Multi-Source BFS (taxis) MEDIUM

Template fit: starts = every rotten orange; usable = fresh orange; answer = rounds taken, or -1 if a fresh orange stays unreached

QUESTION

EXAMPLES

```
Grid [[2,1,1],[1,1,0],[0,1,1]] (rotting-oranges variant): return number of minutes until all reachable
    1s become 2 (4).
```

Taxi variant: for each empty cell, distance to nearest taxi.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# HELPERS for the base question
def multi_source_bfs(rows, cols, sources, open_cell):
    # distance from the nearest source to every cell
    dist = {s: 0 for s in sources}
    queue = deque(sources)
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c),
                       (r, c + 1), (r, c - 1)):
            inside = 0 <= nr < rows and 0 <= nc < cols
            if inside and (nr, nc) not in dist:
                if open_cell(nr, nc):
                    dist[(nr, nc)] = dist[(r, c)] + 1
                    queue.append((nr, nc))
    return dist

# BASE QUESTION (optimal): Multi-source BFS by minute
# Idea: Push every rotten orange into the queue at minute
# 0 and expand level by level; each level is one minute.
# Count the fresh oranges and decrement as they rot, so
# the answer is -1 if any remain at the end.
def rotting_minutes(grid):
    rows, cols = len(grid), len(grid[0])
    cells = [(r, c) for r in range(rows)
             for c in range(cols)]
    rotten = [x for x in cells if grid[x[0]][x[1]] == 2]
    fresh = [x for x in cells if grid[x[0]][x[1]] == 1]
    # rot spreads through any non-empty cell
    dist = multi_source_bfs(
        rows, cols, rotten, lambda r, c: grid[r][c] != 0)
    # a fresh orange that is never reached means -1
    if any(x not in dist for x in fresh):
        return -1
    return max((dist[x] for x in fresh), default=0)
```

4 Solve Grid and Counting Problems

MEDIUM

Template fit: the island count again (queue version); the second part of the question is a short counting formula, no search

QUESTION

A software engineering intern phone screen included two coding tasks: 1.
Count islands in a grid Given an `m x n` grid of characters where `'1'` represents land and `'0'` represents water, return the number of islands. An island is formed by connecting adjacent land cells horizontally or vertically. You may assume all four edges of the grid are surrounded by water. 2.
Count ways to form a target word Given an array of uppercase letters, count how many different ways you can choose letters from the array so that the chosen letters can be rearranged to spell **`GOOGLE`**. Each array element can be used at most once, and letters at different indices are considered different choices even if they contain the same character.
Design efficient algorithms for both tasks, and discuss the time and space complexity of your approach.

Part 1: Count Islands in a GridGiven a 2D grid of characters where each cell is either '1' (land) or '0' (water), return the number of islands. An island is a group of adjacent land cells connected only horizontally or vertically. Diagonal cells do not count as connected. You may assume the grid edges are surrounded by water.
Constraints
0 <= number of rows <= 3000 <= number of columns <= 300grid[r][c] is either '0' or '1'All rows have the same length

EXAMPLES

```
Input: []
```

Expected Output:
Explanation: Empty grid contains no land, so there are no islands.

```
Input: [['1']]
```

Expected Output:
Explanation: A single land cell forms one island.

Hints
Whenever you find an unvisited land cell, start a DFS or BFS to mark the entire island.Only 4-directional neighbors matter: up, down, left, and right.
Part 2: Count Ways to Form the Word GOOGLEGiven an array of uppercase letters, count how many different ways you can choose letters so that the chosen letters can be rearranged to spell GOOGLE. Each array element can be used at most once. Letters at different indices count as different choices even if they contain the same character. The word GOOGLE requires exactly 2 G's, 2 O's, 1 L, and 1 E.
Constraints
0 <= len(letters) <= 200000Each element is an uppercase English letter from 'A' to 'Z'The answer may be large, but fits in Python's integer type
Examples
Input: []
Expected Output:
Explanation: No letters means it is impossible to form GOOGLE.
Input: ['G','O','O','G','L','E']
Expected Output:
Explanation: There is exactly one way to choose all six letters.
Hints
First count how many G, O, L, and E letters appear in the array.You need to choose 2 of the G positions and 2 of the O positions, then 1 L and 1 E.

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))

# BASE QUESTION (optimal): Islands: iterative BFS
# Idea: Same scan, but spread with a queue instead of
# recursion, so the depth of an island cannot overflow the
# stack. Marking visited cells in place (or with a set)
# keeps each cell processed once.
def num_islands(grid):
    # '1' is land: a BFS marks each island once
    if not grid:
        return 0
    m, n = len(grid), len(grid[0])
    seen, count = set(), 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] != "1" or (r, c) in seen:
                continue
            count += 1
            seen.add((r, c))
            queue = deque([(r, c)])
            while queue:
                x, y = queue.popleft()
                for dx, dy in STEPS:
                    a, b = x + dx, y + dy
                    if not (0 <= a < m and 0 <= b < n):
                        continue
                    land = grid[a][b] == "1"
                    if land and (a, b) not in seen:
                        seen.add((a, b))
                        queue.append((a, b))
    return count
```

5 Find shortest relationship path using BFS — MEDIUM

Template fit: one start; also store parent[v] = u when you first reach v; stop at the target and walk parent back to rebuild the path

QUESTION

Overview: This question evaluates proficiency in graph representations and traversal algorithms—specifically BFS and adjacency-structure construction—for computing unweighted shortest paths and reasoning about reachability.

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# BASE QUESTION (optimal): Breadth-first search with a
# parent map
# Idea: Queue the source and mark it visited. Pop a
# person, queue each unvisited neighbour with parent =
# this person, and stop when the target is queued. Follow
# parents back to the source and reverse to get the path.
def shortest_path(adj, s, t):
    # BFS: the first time we reach a node is by a shortest
    # path
    if s == t:
        return [s]
    parent, queue = {s: None}, deque([s])
    while queue:
        u = queue.popleft()
        for v in adj.get(u, ()):
            if v not in parent:
```

```
                parent[v] = u
                if v == t:
                    # walk the parent links back to s
                    path = [v]
                    while parent[path[-1]] is not None:
                        path.append(parent[path[-1]])
                    return path[::-1]
                queue.append(v)
    return None
```

6 Farthest City for a New Data Center

MEDIUM

Template fit: same on a graph: starts = the data centres, neighbors = adjacency list; answer = the city with the largest distance

QUESTION

Cities are nodes 0..num_cities-1 of an undirected, unweighted graph; edges[i] = [u, v] is a direct connection. You are given a non-empty list dcs of cities that already host a data center. You want to build a new data center in the city that is as far as possible from the nearest existing data center — that is, the city whose shortest-path distance (in number of edges) to its closest data center is maximized. Return that city's index. Break ties by choosing the smallest index. It is guaranteed every city can reach at least one data center.

Function signature

def farthest_city(num_cities: int, edges: list[list[int]], dcs: list[int]) -> int:

...

Follow-up (as reported): Run a single BFS seeded from all data centers at once (distance 0) instead of a BFS per data center.

EXAMPLES

Example 1	
Input	num_cities = 5, edges = [[0,1],[1,2],[2,3],[3,4]], dcs = [0]
Output	4
Why	Distances from city 0 are 0,1,2,3,4; city 4 is farthest.

Example 2	
Input	num_cities = 5, edges = [[0,1],[1,2],[2,3],[3,4]], dcs = [0, 4]
Output	2
Why	Nearest-DC distances are 0->0, 1->1, 2->2, 3->1, 4->0; city 2 is farthest at distance 2.

CONSTRAINTS

- 1 <= num_cities <= 10**5 , 0 <= len(edges) <= 2*10**5 .
- 1 <= len(dcs) <= num_cities , dcs are distinct valid city indices.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# BASE QUESTION (optimal): One multi-source BFS
# Idea: Seed the queue with every data center at distance
# 0; the first visit is the nearest one.
def farthest_city(num_cities, edges, dcs):
    adj = [[] for _ in range(num_cities)]
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)
```

```
    dist = [-1] * num_cities
    queue = deque(dcs)
    for d in dcs:
        dist[d] = 0
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if dist[v] < 0:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist.index(max(dist))
```

7 Compute distance to nearest taxi in grid — MEDIUM

Template fit: starts = every taxi cell; usable = not a wall; the dist table is the answer, cells never reached stay -1

QUESTION

Overview: This question evaluates understanding of graph traversal and shortest-path reasoning on grids, algorithmic problem-solving for computing distances from many targets, and the ability to manage time and space complexity on large inputs.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))

def spread(grid, dist, queue, better=False):
    # grow outward; with better=True a cell may be updated
    # when a
    # new, closer source reaches it (distances only go
    # down)
    R, C = len(grid), len(grid[0])
    while queue:
        r, c = queue.popleft()
        for dr, dc in STEPS:
            a, b = r + dr, c + dc
            if not (0 <= a < R and 0 <= b < C):
                continue
            if grid[a][b] == "#":
                continue
            new = dist[r][c] + 1
            unseen = dist[a][b] == -1
            if unseen or (better and new < dist[a][b]):
                dist[a][b] = new
                queue.append((a, b))

# BASE QUESTION (optimal): One multi-source BFS
# Idea: Initialise the queue with every taxi at distance 0
# and expand in rounds. A cell receives its distance the
# first time it is reached; blocked cells are never
# entered; cells that no taxi can reach stay at -1.
def taxi_distances(grid):
    # 'T' taxi, '.' road, '#' wall. One BFS that starts
    # from
    # every taxi at once gives each cell its nearest taxi.
    R, C = len(grid), len(grid[0])
    dist = [[-1] * C for _ in range(R)]
    queue = deque()
    for r in range(R):
        for c in range(C):
```

```
            if grid[r][c] == "T":
                dist[r][c] = 0
                queue.append((r, c))
    spread(grid, dist, queue)
    return dist
```

8 Infection Spread on a Grid (Cellular Automaton) HARD

Template fit: starts = every infected cell; usable = healthy cell; the answer is the largest distance reached (number of rounds)

QUESTION

Overview: This question tests a candidate's ability to implement and optimize a synchronous cellular automaton simulation on a 2D grid, assessing proficiency in BFS-based propagation, state machine design, and incremental computation. It falls under algorithms and grid simulation, evaluating practical coding skill and the ability to handle progressively complex constraints including recovery delays, barriers, and early-termination fixed-point detection.

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))

# TIME each cell gets infected
def infection_times(grid):
    # step at which each cell gets infected (a BFS level);
    # 'I' = infected at time 0, '.' = healthy, '#' =
    # blocked
    rows, cols = len(grid), len(grid[0])
    when = {}
    frontier = [(r, c) for r in range(rows)
                for c in range(cols) if grid[r][c] == "I"]
    for cell in frontier:
        when[cell] = 0
    step = 0
    while frontier:
        step += 1
        nxt = []
        for r, c in frontier:
            for dr, dc in STEPS:
                a, b = r + dr, c + dc
                inside = 0 <= a < rows and 0 <= b < cols
                # every healthy neighbour is infected next
                # step
                if inside and grid[a][b] == "." \
                        and (a, b) not in when:
                    when[(a, b)] = step
                    nxt.append((a, b))
        frontier = nxt
    return when

# BASE QUESTION (optimal): Frontier BFS (only recently
# infected cells spread)
# Idea: Put every initially infected cell in the frontier.
# Each step, look only at the neighbors of frontier cells,
# infect the healthy ones, and make them the next
# frontier.
def spread_steps(grid):
    # steps until nothing changes any more
    when = infection_times(grid)
    return max(when.values(), default=0)
```

9 Shortest Distance From All Buildings HARD

Template fit: run it once per building (one start each); add each run into a total table and count how many buildings reached each cell

QUESTION

You are given a 2D grid where each cell is:
0 = empty land (a school can be built here), 1 = a house, 2 = a tree (an obstacle that cannot be entered or crossed). You want to build one school on an empty cell so that the total travel distance from every house to the school is minimized. Distance is measured by 4-directional steps (up/down/left/right); trees block movement. A valid school location must be reachable from every house. Return the coordinates (row, col) of the empty cell with the minimum total distance to all houses. If several cells tie on total distance, return the one that comes first in row-major order (smallest row, then smallest column). If no empty cell can reach all houses, return (-1, -1) .

Function signature

```
def shortest_distance_from_all_buildings(grid: list[list[int]]) -> tuple[int, int]:
    ...
```

Follow-up (as reported): (Reported) Minimize the average distance instead of the total. Since the number of houses is fixed, the minimizing cell is the same, but track the count to report the average.

EXAMPLES

Example 1	
Input	`grid = [[1, 0, 2, 0, 1], [0, 0, 0, 0, 0], [0, 0, 1, 0, 0]]`
Output	`(1, 2)`
Why	`Cell (1,2) reaches all three houses with total distance 3 + 3 + 1 = 7, the minimum over all reachable empty cells.`

Example 2	
Input	`grid = [[1, 0, 1]]`
Output	`(0, 1)`
Why	`The only empty cell is distance 1 from each house; total 2.`

Example 3	
Input	`grid = [[1, 2, 1]]`
Output	`(-1, -1)`
Why	`There is no empty cell, so no valid location exists.`

CONSTRAINTS

- 1 <= rows, cols <= 50
- Each cell is 0 , 1 , or 2 .
- There is at least one house.

MAIN CODE | helper functions first, then the answer

```python
from collections import deque
from collections import deque


# BASE QUESTION (optimal): One BFS per house
# Idea: Every house adds its distance to the empty cells
# it reaches; valid cells were reached by all houses.
def school_totals(grid):
    rows, cols = len(grid), len(grid[0])
    total = [[0] * cols for _ in range(rows)]
    reached = [[0] * cols for _ in range(rows)]
    houses = 0
```

```
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] != 1:
                continue
            houses += 1
            seen, queue = {(r, c)}, deque([(r, c, 0)])
            while queue:
                x, y, d = queue.popleft()
                for nx, ny in (
                    (x + 1, y),
                    (x - 1, y),
                    (x, y + 1),
                    (x, y - 1),
                ):
                    if (
                        0 <= nx < rows
                        and 0 <= ny < cols
                        and (nx, ny) not in seen
                        and grid[nx][ny] == 0
                    ):
                        seen.add((nx, ny))
                        total[nx][ny] += d + 1
                        reached[nx][ny] += 1
                        queue.append((nx, ny, d + 1))
    return total, reached, houses

# BASE QUESTION (optimal): One BFS per house
# Idea: Every house adds its distance to the empty cells
# it reaches; valid cells were reached by all houses.
def shortest_distance_from_all_buildings(grid):
    total, reached, houses = school_totals(grid)
    best, spot = float("inf"), (-1, -1)
    for r in range(len(grid)):
```

```
        for c in range(len(grid[0])):
            if (
                grid[r][c] == 0
                and reached[r][c] == houses
                and total[r][c] < best
            ):
                best, spot = total[r][c], (r, c)
    return spot
```

Sweep over events in time order

10 Meeting Rooms / Interval Scheduling Variants

MEDIUM

Template fit: the template as written: the largest running count is the number of rooms

QUESTION

LC 253 (Meeting Rooms II) appears in many forms across phone screen and onsite coding rounds — minimum number of rooms, room assignment, cyclic time, daily server jobs, gate entry/exit. Common follow-ups push toward sweep line, min-heap, or distributed processing.

The base task in every variant: given a list of intervals (start, end) — or pickup/return pairs, meeting time pairs, server job runs — answer one or more of:

Min rooms / cars / servers needed to honor all intervals concurrently (the canonical min-meeting-rooms problem).
Assignment: which interval goes to which room/car, so the caller can use the rental record.
Single-interval overlap test between two intervals.

Variant: parking-lot capacity

Arrival and departure intervals represent cars occupying a parking lot. Compute the peak simultaneous occupancy using the same interval-overlap model.

Variant: cyclic time (24h wrap)

Intervals wrap around midnight; if an interval ends after the day boundary, it carries into the next day.
Follow-up: "if N is huge, how do you optimize?" — expect a sweep-line discussion.

Variant: daily-recurring server jobs

Same set of jobs runs every day. If a job can't finish in one day, truncate at midnight and resume next day.
A server can run multiple identical jobs that overlap on the same machine; different jobs need separate servers.
Follow-up: a single job that runs many days needs its own dedicated server.

Variant: on-call rotation consolidation

Input: on-call rotations {name, start, end} using half-open intervals.
Output maximal contiguous segments during which the active on-call set does not change; omit gaps with nobody on call.
Segments are sorted by start, and names in each segment are the active set for that time range.
Clean approach: sort starts, keep a min-heap by end time, and emit a segment whenever the next start or next end changes the active set.

Variant: gate entry / exit with priority rules

Array of (timestamp, action) where action in {enter, exit}; each index is a different person.
Return for each person what time they actually enter/exit, applying tie-break rules:

If the prior tick had an enter, enter wins on a tie.
If the prior tick had an exit, exit wins on a tie.
If the prior tick had no activity, exit wins.
Within the winning class, lower index wins.

Optimization: instead of stepping time tick-by-tick, jump through the sorted input timestamps to reach O(n).

Variant: Google Calendar free-time intersection

Input: requested attendees plus each person's busy calendar intervals.
Return every time range where all requested attendees are free.
Normalize each person's busy blocks into free gaps, then intersect the candidate ranges across attendees.

Variant: car-rental assignment

QUESTION

Input: rental orders containing an order ID, pickup time, and return time.
Assign every order to a car so that no assigned intervals overlap, minimize the number of cars, and return each car's complete order list. Model each car explicitly with its own rental record.
A current output variant instead returns a dictionary from each reservation ID to its assigned car ID; the follow-up asks for the final fleet size.
A return at time t and another pickup at time t may reuse the same car.
Follow-up: if the fleet is capped at K cars and all requests cannot be served, define how the allocator detects and handles the failure.

Variant: exact-concurrency intervals

Compute the maximum number of simultaneously active meetings.
Given a target concurrency X, also return every maximal time range during which exactly X meetings are active.
Define the event ordering when one meeting starts at the same timestamp another ends; this choice controls whether a zero-width overlap is counted.

Variant: maximum-profit job scheduling

Each job has startTime, endTime, and profit. Choose a set of non-overlapping jobs maximizing total profit. The candidate identified it as LC 1235 (Maximum Profit in Job Scheduling).
Asked as the first question of an L4 onsite round, followed by an unrelated range-query problem in the same slot.

On-call employee metadata variant

Each duty record includes employee ID, employee name, start time, and end time. Return the employee count and identifying information for each time segment.

EXAMPLES

```
Canonical min-meeting-rooms example: [[0,30],[5,10],[15,20]] -> 2.
On-call consolidation: A[10,50], B[20,60], C[30,40], D[30,40] -> [10,20]: A, [20,30]: A,B, [30,40]:
    A,B,C,D, [40,50]: A,B, [50,60]: B.
Single-interval overlap: intervalsOverlap([1,5],[4,7]) -> true.
```

Long-running job: a 36-hour daily job is impossible under the truncate rule; reserve a separate server.

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Sweep line over start and end
# events
# Idea: Create (time, +1) for each start and (time, -1)
# for each end, sort so that an end at time t comes before
# a start at t (rooms free up the moment a meeting ends)
# and track the running maximum.
def min_rooms(intervals):
    # +1 at every start, -1 at every end
    events = []
    for s, e in intervals:
        events.append((s, 1))
        events.append((e, -1))
    # at equal times ends (-1) sort before starts (+1)
    events.sort()
    cur = best = 0
    for _, delta in events:
        cur += delta
        best = max(best, cur)
    return best
```

11 Train / Flight Schedule Reachability

MEDIUM

Template fit: sort legs by departure; a leg is usable if you have already reached its origin before it leaves; keep the earliest arrival per place

QUESTION

Given trains (or flights) as (from, to, departure, arrival), decide whether you can travel from a source to a destination, boarding a later leg only if it departs after you arrive. Graph reachability with time state, solved with BFS / DFS.

Input: a schedule of legs, each with a departure station, an arrival station, a departure time, and an arrival time; plus a source and a destination.
Output: whether the destination is reachable from the source, where a leg can only be taken if it departs after you arrive at its departure station.
The same prompt appears as a flight schedule between airports in new-grad R1.

EXAMPLES

```
trains = [
  ("A", "Y", dep=1, arr=20),
  ("A", "Y", dep=2, arr=5),
  ("Y", "B", dep=6, arr=9)
]
source = "A", destination = "B"

Output: true — the first A->Y train arrives too late for the Y->B connection, but the second one makes it
.
```

MAIN CODE | helper functions first, then the answer

```python
# HELPERS used by the code below
INF = float("inf")

# BASE QUESTION (optimal): Sort by departure and keep
# earliest arrivals
# Idea: Sort the legs by departure time. Keep
# earliest[station] = earliest known arrival, with the
# source at minus infinity.
def earliest_arrivals(legs, source, strict=True):
    """Time is part of the state: keep the earliest time
    we can be at each station. Legs go in departure
    order, so every leg that could feed this one is
    already done."""
    earliest = {source: -INF}
    by_departure = sorted(legs, key=lambda x: x[2])
    for frm, to, dep, arr in by_departure:
        have = earliest.get(frm, INF)
        # we must already be there before it leaves
        if have < dep or (not strict and have == dep):
            if arr < earliest.get(to, INF):
                earliest[to] = arr
    return earliest

# BASE QUESTION (optimal): Sort by departure and keep
# earliest arrivals
# Idea: Sort the legs by departure time. Keep
# earliest[station] = earliest known arrival, with the
# source at minus infinity.
def reachable(legs, source, dest, strict=True):
    return dest in earliest_arrivals(legs, source, strict)
```

12 Solve Rooms and Top-K Streams

MEDIUM

Template fit: sort starts and ends separately and walk two pointers: a start before the earliest end needs a new room; the stream part is a size-K heap

QUESTION

Overview: This question evaluates algorithmic problem-solving skills across interval scheduling/resource allocation and streaming top-k maintenance, testing competency in data structures for ordered queries, tie-breaking logic, stateful stream processing, and analysis of time and space trade-offs.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Rooms: sorted starts and ends
# with two pointers
# Idea: Sort starts and ends separately. For each start:
# if it is at or after the earliest unfinished end, that
# room is reused (advance the end pointer); otherwise a
# new room is needed.
def min_rooms(intervals):
    # two sorted lists: every start with no room free
    # needs
    # a new room, otherwise it reuses one that just ended
    starts = sorted(s for s, _ in intervals)
    ends = sorted(e for _, e in intervals)
    rooms = j = 0
    for s in starts:
        if s >= ends[j]:
            # the earliest-ending meeting is over: reuse
            # it
            j += 1
        else:
            rooms += 1
    return rooms
```

13 Compute servers needed for daily recurring jobs

MEDIUM

Template fit: merge each job's own intervals first so one job counts once, then sweep all of them together

QUESTION

You operate a cluster of identical servers that run recurring **daily** jobs. For a single day, you are given a list of job execution intervals. Each interval has:

- **`job_id`** (string or integer),
- **`start_time`** (timestamp within that day),
- **`end_time`** (timestamp within that day), with **`start_time < end_time`**. Interpret each interval as a half-open time range **`[start_time, end_time)`** during which that job must be running on some server **on that day**. Rules and constraints:
- Each physical server can execute at most **one job at a time**.
- However, if multiple intervals in the input correspond to the **same `job_id`**, they may overlap in time **on the same server** (they are considered parts or retries of the same job and do not consume extra capacity).
- If a job's true execution would extend past midnight into the next day, it is conceptually **truncated at the end of the current day** for scheduling purposes; for each day you only see the portion that falls within that day's 24-hour window.

Tasks

1. Given all job intervals for a single day (potentially with repeated **`job_id`**s and overlapping intervals), compute the **minimum number of servers** required to schedule all the jobs under the above rules.
2. Describe the time and space complexity of your algorithm in terms of the number of intervals (m).

QUESTION

Follow-up

Some rare jobs are extremely long-running and conceptually span **multiple days** without restart. For such a multi-day job, you decide to reserve a **dedicated server** that is not shared with any other jobs, but which may still handle different intervals of that same **`job_id`**. Explain how you would extend or adapt your solution to:

- Detect which jobs require a dedicated server based on their multi-day duration or configuration.
- Account for these dedicated servers when computing the total number of servers needed for the system.

Part 1: Minimum Servers for One Day with Overlapping Same-Job IntervalsYou are given all job execution intervals for a single day. Each interval is half-open, meaning [start_time, end_time), and represents a time range during which a job must be running. A server can run at most one job at a time. However, if multiple intervals have the same job_id, they are considered parts or retries of the same job, so overlapping intervals with the same job_id consume only one server total during the overlap. Return the minimum number of servers required to run all intervals for the day.
Constraints
0 <= len(intervals) <= 2000000 <= job_id <= 10^90 <= start_time < end_time <= 1440Intervals are half-open: an interval ending at time t does not overlap an interval starting at time t.Multiple intervals may have the same job_id.

EXAMPLES

```
Input: ([],)
Expected Output: 0
```

Explanation: There are no jobs, so no servers are needed.

```
Input: ([[5, 100, 200], [5, 150, 250], [5, 240, 300]],)
Expected Output: 1
```

Explanation: All intervals belong to the same job_id. Even though they overlap, they can share one server.

Hints

At any moment, one server is needed for each distinct active job_id, not for each active interval.Try merging all intervals belonging to the same job_id first, then run a sweep-line algorithm over the merged intervals.
Part 2: Total Servers with Dedicated Multi-Day JobsYou are given true job execution intervals that may span multiple days. Some jobs require dedicated servers. A job is dedicated if its job_id is listed in configured_dedicated, or if any one of its true intervals has duration at least min_dedicated_duration minutes. A dedicated job uses one reserved server for that job_id and does not share that server with other jobs, though multiple intervals of the same dedicated job_id may use the same dedicated server. For a given target_day, compute the total number of

MAIN CODE | helper functions first, then the answer

```python
from collections import defaultdict


# HELPERS for the base question
def merge_by_job(intervals):
    # intervals of the SAME job may overlap on one server,
    # so
    # merge them first: (job, start, end) -> disjoint
    # blocks
    by_job = defaultdict(list)
    for job, start, end in intervals:
        by_job[job].append((start, end))
    blocks = []
    for job, spans in by_job.items():
        cur = None
        for s, e in sorted(spans):
            if cur and s <= cur[1]:
                cur[1] = max(cur[1], e)
            else:
                if cur:
```

```
                    blocks.append((cur[0], cur[1], job))
                cur = [s, e]
        blocks.append((cur[0], cur[1], job))
    return blocks

# BASE QUESTION (optimal): Merge each job's intervals,
# then sweep start and end events
# Idea: Group the intervals by job_id, sort and merge each
# group's overlapping or touching intervals, then collect
# all merged intervals.
def servers_needed(intervals, dedicated=()):
    dedicated = set(dedicated)
    shared = [i for i in intervals
              if i[0] not in dedicated]
    events = []
    for start, end, _ in merge_by_job(shared):
        events.append((start, 1))
        # [start, end): at equal times an end comes first
        events.append((end, -1))
    run = best = 0
    for _, delta in sorted(events):
        run += delta
        best = max(best, run)
    used = {i[0] for i in intervals if i[0] in dedicated}
    return best + len(used)
```

14 Gate Entry / Exit Time Resolution with Tie-Break Rules — MEDIUM

Template fit: the same two-queue simulation with the tie-break rules from the statement

QUESTION

Onsite coding round 2: given an array indexed by person id, each containing **`(timestamp, action in {enter, exit})`**, return per-person the actual time they passed through the gate after applying tie-break rules between simultaneous enters and exits.SWEsimulationalgorithmmediumFrequencySingle reportLast asked2026-05-04Stageonsite-coding

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# BASE QUESTION (optimal): Event-driven simulation with
# two queues
# Idea: Sort people by (arrival time, id) and keep one
# FIFO queue for waiting enters and one for exits. At each
# second pick from the queues using the previous-second
# state.
def gate_pass_times(events, idle_first="exit"):
    # events[i] = (arrival time, "enter" or "exit")
    n = len(events)
    # people sorted by arrival, then by id
    order = sorted(range(n),
                   key=lambda i: (events[i][0], i))
    queues = {"enter": deque(), "exit": deque()}
    result = [0] * n
    t, i, prev, last_used = 0, 0, None, -10
    while i < n or queues["enter"] or queues["exit"]:
        if not queues["enter"] and not queues["exit"]:
            # nobody waits: jump the clock to the next
            # arrival
            t = max(t, events[order[i]][0])
            # a gap of idle time forgets the previous
            # action
```

```
            if t > last_used + 1:
                prev = None
        # everyone who arrived by now joins their line
        while i < n and events[order[i]][0] <= t:
            person = order[i]
            queues[events[person][1]].append(person)
            i += 1
        both = queues["enter"] and queues["exit"]
        if both:
            want = prev or idle_first
        else:
            want = "enter" if queues["enter"] else "exit"
        person = queues[want].popleft()
        result[person] = t
        prev, last_used = want, t
        t += 1
    return result
```

15 Meeting Rooms III Variant

MEDIUM

Template fit: split meetings that cross midnight into two pieces first, then the same sweep

QUESTION

Given an array of meeting time intervals where each interval is [start, end], determine the minimum number of conference rooms required to hold all meetings without overlap. Variant: times are cyclic within a 24-hour period (meetings can wrap around midnight).

EXAMPLES

Example 1	
Input	`intervals = [[0, 30], [5, 10], [15, 20]]`
Output	`2`
Why	`Meeting [0,30] overlaps with [5,10] and [15,20], so we need 2 rooms.`

Example 2	
Input	`intervals = [[7, 10], [2, 4]]`
Output	`1`
Why	`No overlap, 1 room is sufficient.`

Example 3	
Input	`intervals = [[22, 2], [23, 1], [0, 3]]`
Output	`3`
Why	`All three meetings overlap around midnight.`

CONSTRAINTS

- 1 <= intervals.length <= 10^4
- 0 <= start < end <= 24 * 60 (minutes in a day)
- Meetings may wrap around (e.g., [23:00, 01:00])

MAIN CODE | helper functions first, then the answer

```
# HELPERS used by the code below
DAY = 24

# HELPERS for the base question
def pieces(meetings, day=DAY):
    # a meeting whose end is before its start wraps past
```

```
    # midnight: split it into the part before midnight and
    # the part after it
    for s, e in meetings:
        if e < s:
            yield s, day
            yield 0, e
        else:
            yield s, e

# BASE QUESTION (optimal): Split wrapped meetings and
# sweep (maximum overlap)
# Idea: Turn every meeting into one or two ordinary
# intervals, create +1 events at starts and -1 events at
# ends, sort them (ends before starts at the same minute,
# because intervals are half-open), and take the maximum
# running count.
def min_rooms_sweep(meetings, day=DAY):
    # sweep the time line: +1 at a start, -1 at an end. At
    # equal times an end comes first (a meeting ending at
    # 5
    # and one starting at 5 can share a room).
    events = []
    for s, e in pieces(meetings, day):
        events.append((s, 1))
        events.append((e, -1))
    events.sort()
    cur = best = 0
    for _, step in events:
        cur += step
        best = max(best, cur)
    return best
```

16 Count Overlapping Rectangle Updates on a Grid — HARD

Template fit: the same idea in two dimensions: a difference array (+1, -1, -1, +1 at the corners) and two running-sum passes

QUESTION

Count Overlapping Rectangle Updates on a Grid Implement

`range_add_counts(n, rectangles)`. Start with an **`n x n`** matrix of zeroes. Each rectangle is represented as **`[row1, col1, row2, col2]`** with inclusive, zero-based coordinates. For every rectangle, add one to each matrix cell inside that rectangle. Return the final matrix, where each cell therefore contains the number of input rectangles covering it.

Example

For **`n = 3`** and **`rectangles = [[0, 0, 1, 1], [1, 1, 2, 2]]`**, return:

```
[ [1, 1, 0], [1, 2, 1], [0, 1, 1] ]
```

Constraints

- **`1 <= n <= 500`**
- **`0 <= len(rectangles) <= 100_000`**
- **`0 <= row1 <= row2 < n`**
- **`0 <= col1 <= col2 < n`**

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): 2D difference array and prefix
# sums
# Idea: For each rectangle add +1 at (r1, c1), -1 at (r1,
# c2 + 1), -1 at (r2 + 1, c1) and +1 at (r2 + 1, c2 + 1)
```

```python
# in an (n + 1) x (n + 1) array.
def range_add_counts(n, rects):
    # rects are [r1, c1, r2, c2], inclusive. Mark only the
    # 4
    # corners of each rectangle in a difference array ...
    d = [[0] * (n + 1) for _ in range(n + 1)]
    for r1, c1, r2, c2 in rects:
        d[r1][c1] += 1
        d[r1][c2 + 1] -= 1
        d[r2 + 1][c1] -= 1
        d[r2 + 1][c2 + 1] += 1
    # ... then two prefix sums spread them over the cells
    for r in range(n + 1):
        for c in range(1, n + 1):
            d[r][c] += d[r][c - 1]
    for c in range(n + 1):
        for r in range(1, n + 1):
            d[r][c] += d[r - 1][c]
    return [row[:n] for row in d[:n]]
```

17 Compute city skyline outline — HARD

Template fit: events at each building's left and right edge; a max-heap of active heights; record a point when the top height changes

QUESTION

You are given a list of rectangular buildings in a 2D city skyline. Each building is represented by three integers **`[L, R, H]`**:

- **`L`**: the x-coordinate of the left edge of the building,
- **`R`**: the x-coordinate of the right edge of the building (**`L < R`**),
- **`H`**: the height of the building. All buildings stand on a flat horizontal line (y = 0) and extend vertically up to height **`H`**. Buildings may overlap in the x-dimension. Your task is to compute the *skyline* formed by these buildings, as seen from a distance. The skyline should be represented as a list of **key points `[x, height]`** that describe the outer contour of the union of all buildings. A **key point** is a point where the height of the skyline changes. The output list must satisfy all of the following:

1. The list is sorted by increasing **`x`**.
2. No two consecutive points have the same height (i.e., the height must change at each key point).
3. The first point has a height greater than 0.
4. The last point has height 0, marking the end of the skyline.

Input

- An integer **`n`** — the number of buildings.
- A list of **`n`** buildings, where each building is given as a triple **`[L, R, H]`** of integers. You may assume:
- **`1 <= n <= 10^4`**
- **`0 <= L < R <= 10^9`**
- **`1 <= H <= 10^9`**

Output

Return a list of key points representing the skyline. Each key point is a pair **`[x, height]`** (or a 2-element array/tuple) of integers.

Example

Input:

- **`n = 3`** - buildings = **`[[2, 9, 10], [3, 7, 15], [5, 12, 12]]`** One valid output:
- **`[[2, 10], [3, 15], [7, 12], [12, 0]]`** Explanation (informal):

QUESTION

- From x = 2 to 3, the highest building has height 10.
- From x = 3 to 7, the highest building has height 15.
- From x = 7 to 12, the remaining highest building has height 12.
- After x = 12, there are no buildings, so the height drops to 0. Design an algorithm to compute the skyline key points for the given buildings.

MAIN CODE | helper functions first, then the answer

```
import heapq


# BASE QUESTION (optimal): Sweep line with a max-heap and
# lazy deletion
# Idea: Make events (x, -height) for left edges and (x, 0,
# right) for right edges, sorted so that at the same x
# taller starts come first and ends come after starts.
# Keep a heap of (-height, right edge).
def skyline(buildings):
    # buildings are [left, right, height]
    # start events carry -height so taller starts sort
    # first;
    # end events carry 0
    events = sorted([(l, -h, r) for l, r, h in buildings]
                    + [(b[1], 0, 0) for b in buildings])
    out = []
    # the active buildings: (-height, right edge), ground
    # first
    heap = [(0, float("inf"))]
    for x, neg_h, right in events:
        # buildings that ended at or before x are gone
        while heap[0][1] <= x:
            heapq.heappop(heap)
        if neg_h:
            heapq.heappush(heap, (neg_h, right))
        top = -heap[0][0]
        # a key point only where the height changes
        if not out or out[-1][1] != top:
            out.append([x, top])
    return out
```

18 Compute Turnstile Crossing Times — HARD

Template fit: sort people by arrival time; two queues (entering, exiting) with a rule for who goes first at each second

QUESTION

Overview: This question evaluates proficiency in queue simulation, event-driven scheduling, and handling priority and tie-breaking rules while achieving linear time complexity.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# BASE QUESTION (optimal): Event-driven simulation with
# two queues
# Idea: Sort arrivals once. When both queues are empty,
# jump straight to the next arrival time; if the jump
# leaves at least one idle second, reset the previous
# state to 'none'.
def turnstile_times(events, idle_first="exit"):
```

```
    # events[i] = (arrival time, "enter" or "exit");
    # the result is the second each person goes through
    n = len(events)
    order = sorted(range(n),
                   key=lambda i: (events[i][0], i))
    lines = {"enter": deque(), "exit": deque()}
    result = [0] * n
    t, i, prev, last_used = 0, 0, None, -10
    while i < n or lines["enter"] or lines["exit"]:
        if not lines["enter"] and not lines["exit"]:
            # nobody is waiting: jump to the next arrival
            t = max(t, events[order[i]][0])
            # an idle second forgets the last direction
            if t > last_used + 1:
                prev = None
        # everyone who has arrived by now joins a line
        while i < n and events[order[i]][0] <= t:
            person = order[i]
            lines[events[person][1]].append(person)
            i += 1
        if lines["enter"] and lines["exit"]:
            want = prev or idle_first
        else:
            want = "enter" if lines["enter"] else "exit"
        person = lines[want].popleft()
        result[person] = t
        prev, last_used = want, t
        t += 1
    return result
```

GROUP 3 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE | 3 E 4 M 2 H

Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

19 Given two trees, determine if the pre-order traversals of two trees are same or not. EASY

Template fit: walk both trees in preorder with an explicit stack (push right, then left) and compare value by value; stop at the first difference

QUESTION

Given two trees, determine if the pre-order traversals of two trees are same or not.

MAIN CODE | helper functions first, then the answer

```
from itertools import zip_longest


# HELPERS for the base question
MISSING = object()

def preorder(root):
    # root, then left, then right; iterative with a stack
    stack = [root] if root else []
    while stack:
        node = stack.pop()
        yield node.val
        # right goes first so the left child comes out
        # first
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)

# BASE QUESTION (optimal): Lazy pre-order generators
# compared in lockstep
# Idea: Generate each pre-order lazily with an explicit
# stack (push right before left), and compare the two
# streams element by element with zip_longest, so the
# first mismatch stops the work and no list is built.
def same_preorder(a, b):
    # lazily compare the two sequences value by value; a
    # shorter sequence is padded with MISSING and so
    # differs
    pairs = zip_longest(preorder(a), preorder(b),
                        fillvalue=MISSING)
    return all(x == y for x, y in pairs)
```

20 Calculate the height of a binary tree. EASY

Template fit: no update needed; return 1 + max(left, right) and read the value at the root

QUESTION

Calculate the height of a binary tree.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# BASE QUESTION (optimal): Level-order traversal counting
# levels
```

```python
# Idea: Process the tree one level at a time with a queue
# and count the levels. No recursion, so depth cannot
# overflow the stack; memory is the widest level.
def height(root):
    # height in NODES: level order, count the levels. An
    # empty tree is 0 and a single node is 1. (In edges,
    # subtract one: empty is -1, single node is 0.)
    levels, queue = 0, deque([root] if root else [])
    while queue:
        levels += 1
        for _ in range(len(queue)):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return levels
```

21 Find right-side view of binary tree — EASY

Template fit: pass depth down; the first node you reach at a new depth (visiting right before left) is the one you can see

QUESTION

You are given the root of a binary tree. From the **right side** of the tree, at each depth you can see exactly one node: the rightmost node at that depth. Write a function that returns a list of the values of the nodes visible when looking at the tree from the right side, ordered from **top to bottom**. ---

Function Signature

(one possible form)

```
rightSideView(root: TreeNode) -> List[int]
```

Where **`TreeNode`** is defined as:

```
class TreeNode { int val; TreeNode left; TreeNode right; }
```

Example

Consider this binary tree:

```
1 / \ 2 3 \ \ 5 4
```

The right-side view is:

```
[1, 3, 4]
```

Explain the algorithm you would use, including its time and space complexity. You may use either breadth-first search (BFS) or depth-first search (DFS).

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): DFS visiting right before left,
# first node at each depth
# Idea: Visit the right child first. The first node you
# meet at a depth that is not yet recorded is the
# rightmost one at that depth. Only O(height) memory is
# needed.
def right_side_view(root):
    # DFS, right child first: the first node seen at each
    # depth is the rightmost one
    out = []

    def dfs(node, depth):
```

```
        if node is None:
            return
        if depth == len(out):
            out.append(node.val)
        dfs(node.right, depth + 1)
        dfs(node.left, depth + 1)

    dfs(root, 0)
    return out
```

22 Expression Tree Single-Leaf Mutation — MEDIUM

Template fit: post-order evaluation (children first, then apply the operator); to test one leaf flip, re-evaluate with that leaf changed

QUESTION

Evaluate a boolean expression tree, then answer the root value after each single-leaf flip. Current variants include XOR/OR/AND/NOT operators, no parent pointer on the node interface, and an expectation that the candidate cache parent links or paths during the initial traversal.SWEtreedfssimulationmediumstate-machineFrequencyLowLast asked2026-09-27Stageonsite-coding

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
def combine(op, a, b=None):
    # value of one gate from its input values
    if op == "AND":
        return a and b
    if op == "OR":
        return a or b
    if op == "XOR":
        return a != b
    # NOT has one input
    return not a

def right_value(value, node):
    # NOT has no right child
    return value[id(node.right)] if node.right else None

def build_cache(root):
    # one traversal caches parents, values and leaves
    parent, value, leaves = {id(root): None}, {}, []
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                parent[id(child)] = node
                stack.append(child)
    # reversed order visits children before parents
    for node in reversed(order):
        if node.op is None:
            value[id(node)] = node.val
            leaves.append(node)
        else:
            a = value[id(node.left)]
            b = right_value(value, node)
            value[id(node)] = combine(node.op, a, b)
    return parent, value, leaves

# BASE QUESTION (optimal): Cache values and parents,
# recompute only the path with early stop
# Idea: One traversal stores every node's value and its
```

```
# parent.
def flip_answers(root):
    parent, value, leaves = build_cache(root)
    answers = []
```

```
    for leaf in leaves:
        undo, node = [], leaf
        new = not value[id(leaf)]
        # walk up only while a value really changes
        while node is not None and value[id(node)] != new:
            undo.append((node, value[id(node)]))
            value[id(node)] = new
            node = parent[id(node)]
            if node is None:
                break
            a = value[id(node.left)]
            b = right_value(value, node)
            new = combine(node.op, a, b)
        answers.append(value[id(root)])
        # put every changed value back for the next leaf
        for node, old in reversed(undo):
            value[id(node)] = old
    return leaves, answers
```

23 Longest Consecutive Path in Binary Tree — MEDIUM

Template fit: pass the parent's value and the run length down; extend the run when value == parent + 1, else restart at 1

QUESTION

Given a binary tree, return the length of the longest parent-to-child path whose values increase consecutively by 1. Follow-up: allow the path to extend in any direction through the tree, not only from parent to child.SWEtreedfsrecursionmediumFrequencySingle reportLast asked2026-07-02Stagephone-screen

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Top-down DFS carrying the
# current run
# Idea: Pass the length of the run ending at the parent.
# If the child is exactly parent + 1 the run grows,
# otherwise it restarts at 1. Track the global best.
# Length is counted in nodes; state that choice out loud.
def longest_consecutive(root, want_path=False):
    # parent -> child path whose values go up by exactly 1
    best, best_path = 0, []
    stack = [(root, None, [])] if root else []
    while stack:
        node, parent, run = stack.pop()
        # extend the run or start a new one at this node
        if parent is not None and node.val == parent + 1:
            run = run + [node.val]
        else:
            run = [node.val]
        if len(run) > best:
            best, best_path = len(run), run
        for child in (node.left, node.right):
            if child is not None:
                stack.append((child, node.val, run))
    return best_path if want_path else best
```

24 Given the root of a binary tree, return the length of the diameter of the tree. MEDIUM

Template fit: best = max(best, left + right); return 1 + max(left, right) (the diameter)

QUESTION

Given the root of a binary tree, create a function diameterOfTree that determines and returns the length of the diameter of the tree, where the diameter of a tree is defined as the number of nodes along the longest path between any two nodes in the tree. For example, given the following binary tree: // tree 1 / \ 2 3 / \ 4 5 The diameter is 4, which is the length of the path [4, 2, 1, 3] or [5, 2, 1, 3].Watch Ali, SDE @ Amazon, answer the question: "Given the root of a binary tree, return the length of the diameter of the tree."No answers contributed by the community yet.Interview experiences6 shared‹›Uber · 3 months ago"interview process is very lengthy took around 5 weeks, and expectations is very high in each round, for LLD working code is required, also dont mess your HM round since i felt like it contained more weightage"Software Engineer · Mid Level / L4Meta · 4 months ago"The interview went smoothly overall. 1 round of phone screen followed by the full loop of 4 rounds packed in a day. 2 coding 1 behavior 1 system design."Software Engineer · Senior / L5Google · 9 months ago"I interviewed for an AI Engineer internship, but the process was mostly DSA-heavy. I had an online coding round first, then a technical round with binary tree and DP questions, then another harder DSA round with DP and linked list questions, and finally an HR round. The online round covered arrays and strings, and the later rounds got progressively more difficult. The only non-DSA part was the last

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): One post-order pass returning
# heights and updating the best
# Idea: depth(node) returns the height of the node's
# subtree and, on the way, updates best = max(best, left +
# right). Every node is visited once, and the answer is
# best when the traversal ends (edges, not nodes).
def diameter_nodes(root):
    # longest path between any two nodes, counted in
    # NODES.
    # Iterative post-order: height of every node, then the
    # best path through each node = left + right + 1.
    if root is None:
        return 0
    height, best, stack = {None: 0}, 0, [(root, False)]
    while stack:
        node, done = stack.pop()
        if not done:
            stack.append((node, True))
            for child in (node.left, node.right):
                if child:
                    stack.append((child, False))
            continue
        lh, rh = height[node.left], height[node.right]
        height[node] = 1 + max(lh, rh)
        best = max(best, lh + rh + 1)
    return best
```

25 Implement zigzag level-order traversal MEDIUM

Template fit: collect values by depth; reverse every odd level (or fill each level from the other end)

QUESTION

Given the root of a binary tree, return the node values in zigzag level order: visit nodes level by level, reading the first level from left to right, the next from right to left, and so on alternating each level. Output a list of lists of integers where each inner list contains the values for one level in the required direction. Aim for O(n) time and O(w) extra space, where n is the number of nodes and w is the maximum width of the tree.

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# BASE QUESTION (optimal): BFS filling each level from the
# left or the right
# Idea: Keep a left-to-right flag. For each level build a
# deque; append on the right when going left to right and
# on the left when going right to left. Flip the flag
# after each level.
def zigzag_level_order(root):
    # BFS by levels; flip the direction on every level
    out, ltr = [], True
    queue = deque([root] if root else [])
    while queue:
        level = deque()
        for _ in range(len(queue)):
            node = queue.popleft()
            if ltr:
                level.append(node.val)
            else:
                level.appendleft(node.val)
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
        out.append(list(level))
        ltr = not ltr
    return out
```

26 Repeatedly Delete Leaves Matching a Target Value From a Binary Tree

HARD

Template fit: set node.left = dfs(node.left) and node.right likewise; return None when the node became a leaf with the target value

QUESTION

Given a binary tree and an integer **`target`**, delete every leaf whose value equals **`target`**. Deleting a leaf can turn its parent into a leaf; if that parent's value also equals **`target`**, it must be deleted as well, and so on, until no leaf with value **`target`** remains. Return the resulting tree, which is empty if the root itself ends up deleted.

Function Signature

`remove_target_leaves(tree: list[int | None], target: int) -> list[int | None]`

Tree Encoding

The input and the output use the same level-order encoding:

- An empty tree is **`[]`**.
- Otherwise the first element is the root's value. Present nodes are then visited in breadth-first order, left to right, and for each one its left child and then its right child are appended, using **`None`** for an absent child. Children of absent nodes are not listed.
- Trailing **`None`** values are omitted. The input list is always well formed, and the output must be in exactly this canonical form.

Rules

- A leaf is a node with no children.
- A node whose value equals **`target`** but that still has at least one child after all deletions is kept.
- Kept nodes never move: every kept node keeps its original parent and its side (left or right).
- The returned tree contains no leaf whose value equals **`target`**.

Constraints

- The input tree has between 1 and 5000 nodes, and its height is at most 500.
- Node values and **`target`** are integers in **`[-1000000000, 1000000000]`**.

Examples

QUESTION

Input: **`tree = [5,7,7,7,3,None,7], target = 7`**
Output: **`[5,7,None,None,3]`** The leaf 7 under the root's left child is deleted, but that left child still has the child 3, so it stays. On the right side, deleting the lower 7 turns the root's right child into a leaf with value 7, so it is deleted too.
Input: **`tree = [3,3,1], target = 3`**
Output: **`[3,None,1]`** Only the left leaf is deleted. The root's value is 3, but it still has a child.
Input: **`tree = [4,4,4], target = 4`**
Output: **`[]`**

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): One post-order pass
# Idea: Process both children first, then look at the
# node: if it is now a leaf and its value is the target,
# return None to its parent.
def remove_leaves(root, target):
    # recursive post-order: children first, then the node
    deleted = 0

    def dfs(node):
        nonlocal deleted
        if node is None:
            return None
        node.left = dfs(node.left)
        node.right = dfs(node.right)
        # a leaf with the target value disappears
        if not node.left and not node.right \
                and node.val == target:
            deleted += 1
            return None
        return node

    return dfs(root), deleted
```

27 Delete an Entire Binary Tree — HARD

Template fit: post-order: free the children first, then the node itself

QUESTION

You are given the root of a binary tree (not a BST); each node has only left and right child pointers. You must "free" every node so that after the call the tree is empty. Because a node must not be freed before its children, the correct freeing order is a post-order traversal. Implement delete_tree(root) that frees all nodes and returns the list of node values in the exact order they are freed (post-order: left subtree, right subtree, then node).

Function signature

```
from typing import Optional
class TreeNode:
    def __init__(self, val: int = 0,
                 left: "Optional[TreeNode]" = None,
                 right: "Optional[TreeNode]" = None) -> None:
        self.val = val
        self.left = left
        self.right = right
def delete_tree(root: Optional[TreeNode]) -> list[int]:
    """Free every node and return the values in the order freed (post-order)."""
    ...
```

QUESTION

Follow-up (as reported): Recursive DFS uses O(h) stack space. Achieve O(1) extra space by repeatedly rotating the tree (Morris-style): while the root has a left child, rotate it up; once the root has no left child, free the root and move to its right child.

EXAMPLES

Example 1	
Input	`root = 1 / \ 2 3`
Output	`[2, 3, 1]`
Why	`Post-order frees the left child, then the right child, then the root.`

Example 2	
Input	`root = None`
Output	`[]`
Why	`An empty tree frees nothing.`

Example 3	
Input	`root = 1 / 2 / 3`
Output	`[3, 2, 1]`
Why	`A left-leaning chain frees deepest-first.`

CONSTRAINTS

- 0 <= number of nodes <= 10^5 .
- Node values are integers.

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Recursive post-order
# Idea: Free the left subtree, the right subtree and then
# the node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

# BASE QUESTION (optimal): Recursive post-order
# Idea: Free the left subtree, the right subtree and then
# the node.
def delete_tree(root):
    freed = []

    def free(node):
        if node is None:
            return
        free(node.left)
        free(node.right)
        freed.append(node.val)
        node.left = node.right = None

    free(root)
    return freed
```

GROUP 4 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE | 8 M 1 H

A heap of what is available: pop the best, put back the changed one

28 Streaming Median (LC 295) with Follow-ups — MEDIUM

Template fit: two heaps: a max-heap for the lower half and a min-heap for the upper half; keep them balanced

QUESTION

A near-canonical LC 295 streaming-median implementation using two heaps, followed by two design extensions: exploit a very small value domain, then support an arbitrary k-th-smallest query instead of only the median.SWEheapstreamingmediumFrequencySingle reportLast asked2026-08-03Stagetech-screen

MAIN CODE | helper functions first, then the answer

```python
import heapq


# BASE QUESTION (optimal): Two heaps
# Idea: Lower half in a max-heap, upper half in a min-
# heap, sizes differing by at most one and every lower
# value <= every upper value. The median is the larger
# heap's root, or the average of both roots.
class MedianFinder:
    def __init__(self):
        # low: max-heap (stored negated); high: min-heap
        self.low, self.high = [], []

    def add(self, x):
        # push through low so the halves stay ordered
        heapq.heappush(self.low, -x)
        top = -heapq.heappop(self.low)
        heapq.heappush(self.high, top)
        # keep low the same size as high, or one bigger
        if len(self.high) > len(self.low):
            top = heapq.heappop(self.high)
            heapq.heappush(self.low, -top)

    def median(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2
```

29 Compute minimum number of rooms needed — MEDIUM

Template fit: heap of end times; a meeting reuses the room that frees first if it ended, otherwise push a new room

QUESTION

Problem

You are given a list of meetings, each with a start time and end time. A single room can host only one meeting at a time. Two meetings **overlap** if one starts before the other ends (treat meetings as half-open intervals **[start, end)** so a meeting ending at time `t` does not conflict with another starting at time `t`).

Task

Return the **minimum number of rooms** required to host all meetings.

Input

- `intervals`: an array of `n` pairs `[start, end]` where `0 <= start <= end`.

Output

QUESTION

- An integer: the minimum number of rooms needed.

Constraints (typical interview bounds)

- **`1 <= n <= 200000`**
- Times are integers in a reasonable range (e.g., **`0 . 10^9`**).

Example

- Input: **`[[0,30],[5,10],[15,20]]`**
- Output: **`2`** (One room can host **`[0,30]`**, the other can host **`[5,10]`** then **`[15,20]`**.)

You are given a list of meetings, where each meeting is represented as [start, end]. A single room can host only one meeting at a time. Meetings are treated as half-open intervals [start, end), so a meeting ending at time t does not conflict with another meeting starting at time t. Return the minimum number of rooms required to host all meetings. If start == end, the meeting occupies no time and does not require a room.
Constraints
0 <= n <= 2000000 <= start <= end <= 10^9

EXAMPLES

```
Input: ([[0,30],[5,10],[15,20]],)
Expected Output: 2
Explanation: [0,30] overlaps with both [5,10] and [15,20], so at most 2 rooms are needed at the same
    time.
Input: ([[7,10],[2,4],[4,7],[10,12]],)
Expected Output: 1
```

Explanation: These meetings only touch at endpoints, so one room can host all of them in sequence.

Hints

Try turning each meeting into two events: one when it starts and one when it ends.Because intervals are half-open, if a meeting ends at the same time another starts, process the ending event first.

MAIN CODE | helper functions first, then the answer

```python
import heapq


# BASE QUESTION (optimal): Sort by start with a min-heap
# of end times
# Idea: Process meetings by start time. If the earliest-
# ending room has finished by this meeting's start, reuse
# it (pop and push the new end); otherwise open a new room
# (push).
def min_meeting_rooms(intervals):
    # half-open [start, end): a meeting ending at t does
    # not
    # clash with one starting at t. A heap holds the end
    # times
    # of the rooms in use.
    ends = []
    meetings = sorted(m for m in intervals if m[0] < m[1])
    for start, end in meetings:
        if ends and ends[0] <= start:
            # the earliest room is free: reuse it
            heapq.heapreplace(ends, end)
        else:
            heapq.heappush(ends, end)
    return len(ends)
```

30 Implement a Reusable Locker Allocation System

MEDIUM

Template fit: heap of freed locker numbers; take the smallest freed one before handing out a new number

QUESTION

Problem

A locker bank has lockers numbered from 1 through **n**. Process operations in order. **`store(customer)`** assigns the smallest available locker and returns its number, or **`-1`** if that customer already has a locker or none is available. **`pickup(customer)`** releases and returns that customer's locker, or **`-1`** if no locker is assigned. **`last()`** returns the locker number from the most recent successful **`store`**, or **`-1`** if there has not been one. Return one result per operation.

Constraints & Assumptions

- **`1 <= n <= 200,000`**.
- There are at most 200,000 operations.
- Customer IDs are non-empty strings.
- A released locker becomes immediately available for reuse.

Clarifications

- Failed store and pickup operations do not change **`last()`**.
- The allocation rule is always the smallest currently free locker number.
- A customer can store again after a successful pickup.

Examples

```
n = 3 operations = [store("a"), store("b"), pickup("a"), store("c"), last()] output = [1, 2, 1, 1, 1]
```

Hints

```
Track free lockers Use an ordered structure that can remove and reinsert the minimum locker efficiently.
```

```
Track ownership A map from customer ID to locker supports validation and pickup.
```

Process a fresh locker bank with lockers numbered 1 through n. Operations are ["store", customer], ["pickup", customer], or ["last"]. Store assigns and returns the smallest available locker, or -1 if the customer already owns one or none is free. Pickup releases and returns that customer's locker, or -1 if none is assigned. Last reports the locker from the most recent successful store, or -1 before any success. Return one integer per operation.
Constraints
1 <= n <= 200000.operations contains at most 200000 encoded store, pickup, or last lists.Customer IDs are non-empty strings.A released locker becomes immediately available for reuse.Failed store and pickup operations do not change last.A customer can store again after a successful pickup.

EXAMPLES

```
Input: (3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])
Expected Output: [1, 2, 1, 1, 1]
```

Explanation: This is the source example; the released smallest locker is reused.

```
Input: (5, [])
Expected Output: []
```

Explanation: An empty batch produces no per-operation results.

Hints

Keep available locker numbers in a min-oriented structure.Use a customer-to-locker map to validate stores and pickups.

MAIN CODE | helper functions first, then the answer

```python
import heapq


# BASE QUESTION (optimal): Min-heap of released numbers
# plus a next-unused counter
# Idea: store: if the customer already has a locker return
# -1; take the smallest released number from the heap if
# any, else the next never-used number if it is at most n,
# else -1.
class Lockers:
    def __init__(self, n):
        self.n = n
        # the next never-used locker number
        self.next_new = 1
        # released lockers, smallest on top
        self.free = []
        # customer -> locker number
        self.of = {}
        # locker of the latest successful store
        self.last_store = -1

    def store(self, customer):
        if customer in self.of:
            return -1
        if self.free:
            # a released locker is always smaller than any
            # new one
            locker = heapq.heappop(self.free)
        elif self.next_new <= self.n:
            locker = self.next_new
            self.next_new += 1
        else:
            return -1
        self.of[customer] = locker
        self.last_store = locker
        return locker

    def pickup(self, customer):
        locker = self.of.pop(customer, None)
        if locker is None:
            return -1
        # it becomes available again at once
        heapq.heappush(self.free, locker)
        return locker
```

```python
    def last(self):
        # failed operations never change this
        return self.last_store
```

31 Simulate room assignments for scheduled meetings

MEDIUM

Template fit: a heap of free room numbers and a heap of (end time, room); a delayed meeting takes the earliest-ending room and shifts later

QUESTION

Overview: This question evaluates scheduling and event-simulation skills, including time-ordered processing, resource allocation with tie-breaking, and efficient tracking of room availability and usage counts using appropriate data structures.

MAIN CODE | helper functions first, then the answer

```python
import heapq


# BASE QUESTION (optimal): Two heaps: free rooms and busy
# rooms
```

```python
# Idea: Process meetings in start order. Move rooms that
# finished by the meeting's start into the free heap
# (lowest index first). If a room is free, use it.
def most_booked_room(n, meetings, by="count"):
    # a meeting takes the lowest-numbered free room; if
    # none is
    # free it waits for the earliest room and keeps its
    # length
    free = list(range(n))
    heapq.heapify(free)
    # busy rooms: (time it becomes free, room)
    busy = []
    used = [0] * n
    for start, end in sorted(meetings):
        # rooms that finished by now are free again
        while busy and busy[0][0] <= start:
            _, room = heapq.heappop(busy)
            heapq.heappush(free, room)
        if free:
            room = heapq.heappop(free)
            heapq.heappush(busy, (end, room))
        else:
            # delayed: starts when the earliest room frees
            # up
            finish, room = heapq.heappop(busy)
            delayed_end = finish + (end - start)
            heapq.heappush(busy, (delayed_end, room))
        used[room] += 1 if by == "count" else end - start
    # ties go to the lowest room number
    return used.index(max(used))
```

32 Trip Expense Settlement

MEDIUM

Template fit: a heap of creditors and a heap of debtors; the biggest of each settles min(amount) and the remainder goes back

QUESTION

A group of friends go on a trip. Each friend may pay for others. Given payment records, compute minimal payments required to settle all debts.

EXAMPLES

Example 1	
Input	`[{"payer": "alice", "amount": 4000, "payees": ["bob", "jess", "alice", "sam"]}, {"payer": "jess", "amount": 2000, "payees": ["jess", "alice"]}]`
Output	`[{"payer": "bob", "amount": 1000, "payee": "alice"}, {"payer": "sam", "amount": 1000, "payee": "alice"}]`
Why	Calculate net balance for each person, then settle with minimum transactions.

Example 2	
Input	`expenses = [{"payer": "Alice", "amount": 40, "participants": ["Alice", "Bob"]}, {"payer": "Bob", "amount": 20, "participants": ["Alice", "Bob"]}]`
Output	`["Bob pays Alice 10.00"]`

CONSTRAINTS

- 1 <= payments.length <= 1000
- 1 <= amount <= 10^6
- 1 <= payees.length <= 100

MAIN CODE | helper functions first, then the answer

```python
import heapq


# HELPERS used by the code below
def balances(records):
    # net balance per person: what they paid minus what
    # they
    # owe. Positive: the group owes them. All sum to zero.
    net = {}
    for rec in records:
        payer = rec["payer"]
        net[payer] = net.get(payer, 0) + rec["amount"]
        for person, share in shares_of(rec).items():
            net[person] = net.get(person, 0) - share
    return {p: v for p, v in net.items() if v}

# BASE QUESTION (optimal): Heaps: match the largest debtor
# with the largest creditor
# Idea: Push creditors (positive net) in a max-heap and
# debtors (negative net) in another. Repeatedly pay
# min(debt, credit) from the biggest debtor to the biggest
# creditor, and push back whoever still has a remainder.
def settle_greedy(records):
    # the biggest debtor pays the biggest creditor, as
    # much
    # as one side can: at most n - 1 payments
    net = balances(records)
    owed = [(-v, p) for p, v in net.items() if v > 0]
    debt = [(v, p) for p, v in net.items() if v < 0]
    heapq.heapify(owed)
    heapq.heapify(debt)
    out = []
    while owed and debt:
        o_amt, creditor = heapq.heappop(owed)
        d_amt, debtor = heapq.heappop(debt)
        pay = min(-o_amt, -d_amt)
        out.append({"payer": debtor, "amount": pay,
                    "payee": creditor})
        if -o_amt > pay:
            heapq.heappush(owed, (o_amt + pay, creditor))
        if -d_amt > pay:
            heapq.heappush(debt, (d_amt + pay, debtor))
    return out

def split_equally(amount, people):
```

```python
    # follow-up: cents that do not divide evenly. Work in
    # whole cents; the leftover cents go to the first
    # people
    # in sorted order, so the rule is always the same.
    people = sorted(people)
    base, extra = divmod(amount, len(people))
    return {p: base + (1 if i < extra else 0)
            for i, p in enumerate(people)}

def shares_of(record):
    # follow-up: explicit shares per person instead of
    # equal
    payees = record["payees"]
    if isinstance(payees, dict):
        if sum(payees.values()) != record["amount"]:
            raise ValueError("shares do not add up")
        return payees
    return split_equally(record["amount"], payees)
```

33 Playlist Shuffle No Adjacent Artists — MEDIUM

Template fit: pop the artist with the most songs left, hold it for one turn, then push it back with a smaller count

QUESTION

Given a playlist of songs where each song has an artist, implement a shuffle function that minimizes (or eliminates) adjacent songs by the same artist.

EXAMPLES

Example 1	
Input	`[("A", "song1"), ("A", "song2"), ("B", "song3"), ("B", "song4"), ("C", "song5")]`
Output	`[("A", "song1"), ("B", "song3"), ("A", "song2"), ("B", "song4"), ("C", "song5")]`
Why	`No adjacent songs have same artist.`

CONSTRAINTS

- 1 <= songs.length <= 10^4

MAIN CODE | helper functions first, then the answer

```
import heapq
import random
from collections import deque


# HELPERS for the base question
def release(waiting, heap):
    # the oldest waiting artist may play again (if it has
    # any song left)
    entry = waiting.popleft()
    if entry[0]:
        heapq.heappush(heap, entry)

def take_oldest(waiting, heap):
    # break the gap on purpose: the first waiting artist
    # that still has songs plays early
    while waiting:
        entry = waiting.popleft()
        if entry[0]:
            heapq.heappush(heap, entry)
            return

# BASE QUESTION (optimal): Greedy with a max-heap, holding
# back the last artist
# Idea: Keep a heap of (remaining songs, artist). Each
# step pop the artist with the most songs left; play one
# of their songs, and push them back only after the next
# artist has played (so they cannot repeat).
def shuffle_playlist(songs, gap=2, strict=False,
                     rng=random):
    # songs are (artist, title). The artist with the most
    # songs left plays next and then waits `gap - 1` other
    # songs before it may play again (gap 2 = never twice
    # in
    # a row). Ties between artists and the song picked
    # inside
    # an artist are random, so every run looks different.
    by_artist = {}
    for artist, title in songs:
        by_artist.setdefault(artist, []).append(title)
    for titles in by_artist.values():
        rng.shuffle(titles)
    heap = [(-len(t), rng.random(), a)
            for a, t in by_artist.items()]
    heapq.heapify(heap)
```

```
    waiting, out = deque(), []
    while len(out) < len(songs):
        if not heap:
            # everyone left is still waiting
            if strict:
                return None
            take_oldest(waiting, heap)
        left, _, artist = heapq.heappop(heap)
        out.append((artist, by_artist[artist].pop()))
        # an artist stays in the queue even with no songs
        # left: it still marks one position of spacing
        waiting.append((left + 1, rng.random(), artist))
        if len(waiting) >= gap:
            release(waiting, heap)
    return out
```

34 Minimum Makespan on Identical CPUs

MEDIUM

Template fit: the template as written: heap of the times the CPUs become free

QUESTION

You have C identical CPUs and a list release_times , where release_times[i] is the earliest time job i may start. Every job runs non-preemptively for the same fixed duration S . When a job's release time arrives it may start on any free CPU; if all CPUs are busy it waits and starts on the first CPU that becomes free (never before its release time). Each CPU runs one job at a time. Scheduling optimally, compute the minimum possible makespan: the time at which the last job finishes.

Function signature

def min_makespan(release_times: list[int], num_cpus: int, duration: int) -> int:

    ...

Follow-up (as reported): (Reported) The naive approach tracked an O(n*C) table of CPU availability; reduce it to a min-heap of the C next-free times, popping the earliest-free CPU per job after sorting jobs by release time.

EXAMPLES

Example 1	
Input	`release_times = [0, 1, 2, 3], num_cpus = 2, duration = 3`
Output	`7`
Why	`CPU-A: job@0 -> [0,3), job@2 starts at 3 -> [3,6). CPU-B: job@1 -> [1,4), job@3 starts at 4 -> [4,7). Last finish = 7.`

Example 2	
Input	`release_times = [0, 0, 0], num_cpus = 1, duration = 2`
Output	`6`
Why	`One CPU runs them back to back: [0,2), [2,4), [4,6).`

Example 3	
Input	`release_times = [5], num_cpus = 3, duration = 4`
Output	`9`
Why	`The single job starts at its release time 5, finishes at 9.`

CONSTRAINTS

- 1 <= C <= 10^5
- 0 <= len(release_times) <= 10^5
- 0 <= release_times[i] <= 10^9
- 1 <= S <= 10^9
- If there are no jobs, the makespan is 0 .

QUESTION

- Target complexity: O(n log C) time, O(C) space.

MAIN CODE | helper functions first, then the answer

```python
import heapq


# BASE QUESTION (optimal): Sorted jobs and a heap of free
# times
# Idea: Each job takes the CPU that frees first, but never
# starts before its release.
def min_makespan(release_times, num_cpus, duration):
    free = [0] * num_cpus
    last = 0
    for r in sorted(release_times):
        start = max(r, heapq.heappop(free))
        heapq.heappush(free, start + duration)
        last = max(last, start + duration)
    return last
```

35 Design server allocation for multi-type nodes

MEDIUM

Template fit: a heap per type of (flexibility, server); skip servers already allocated (lazy deletion)

QUESTION

Design a service to manage a large pool (about 100, 000) of server nodes. Each node can satisfy multiple instance types; clients call getServer(reqNumOfServer, instanceTypes[]) to request servers that collectively meet the requested types, and returnServer(ids[]) to release them. Propose data structures and indexes for fast matching, availability tracking, fragmentation minimization, and scaling. Address concurrency, starvation, and failure handling.

MAIN CODE | helper functions first, then the answer

```python
import heapq
from collections import defaultdict


# BASE QUESTION (optimal): Per-type heaps ordered by
# flexibility, with lazy deletion
# Idea: For each type keep a heap of (number of supported
# types, id) for the servers that support it.
class ServerPool:
    """Servers with several instance types. Each type has
    a heap of (number of types, id): the LEAST flexible
    server goes first, which keeps versatile ones free."""

    def __init__(self, servers):
        # servers: id -> set of instance types
        self.types = {s: frozenset(t)
                      for s, t in servers.items()}
        self.allocated = set()
        self.heaps = defaultdict(list)
        self.lease = {}
        for s, ts in self.types.items():
            for t in ts:
                heap = self.heaps[t]
                heapq.heappush(heap, (len(ts), s))

    def best_for(self, t, skip):
        # the best free server of type t that is not in
        # `skip`;
        # allocated entries are dropped lazily
        heap, held = self.heaps[t], []
```

```
        while heap:
            flex, s = heap[0]
            if s in self.allocated:
                heapq.heappop(heap)
            elif s in skip:
                held.append(heapq.heappop(heap))
            else:
                break
        best = heap[0] if heap else None
        # put back the entries we only looked past
        for item in held:
            heapq.heappush(heap, item)
        return best

    def get_server(self, count, types):
```

```
        # pick servers first, commit only if the whole
        # request works
        chosen = []
        for t in types:
            # a server already chosen may cover this type
            if any(t in self.types[c] for c in chosen):
                continue
            top = self.best_for(t, set(chosen))
            if top is None:
                return None
            chosen.append(top[1])
        # fill up to `count` with the least flexible free
        # servers
        while len(chosen) < count:
            skip = set(chosen)
            found = [self.best_for(t, skip)
                     for t in types]
            options = [o for o in found if o]
            if not options:
                return None
            chosen.append(min(options)[1])
        # more servers than allowed were needed to cover
        # the types
        if len(chosen) > count:
            return None
        self.allocated |= set(chosen)
        return chosen

    def release(self, ids):
        for s in ids:
            if s in self.allocated:
                self.allocated.discard(s)
                self.lease.pop(s, None)
                flex = len(self.types[s])
                for t in self.types[s]:
                    heap = self.heaps[t]
                    heapq.heappush(heap, (flex, s))

    def rent(self, count, types, now, ttl):
        # a client that dies must not hold servers forever
        got = self.get_server(count, types)
        for s in got or []:
            self.lease[s] = now + ttl
        return got
```

```
    def expire(self, now):
        # return servers whose lease ran out to the free
        # pool
        late = [s for s in self.lease
                if self.lease[s] <= now]
```

```
        self.release(late)
        return late
```

36 Huffman Tree Construction — HARD

Template fit: pop the two smallest weights, push their sum as a new node; repeat until one node remains

QUESTION

EXAMPLES

```
{"a": 5, "b": 9, "c": 12, "d": 13, "e": 16, "f": 45} -> standard Huffman tree with f near the root.
```

MAIN CODE | helper functions first, then the answer

```
import heapq


# HELPERS for the base question
def huffman_lengths(freq):
    # code length of every symbol (depth in the tree)
    if not freq:
        return {}
    if len(freq) == 1:
        # a lone symbol still needs one bit
        return {next(iter(freq)): 1}
    # heap entries: (weight, tie-break, symbols below)
    heap = [(w, i, [s]) for i, (s, w) in
            enumerate(sorted(freq.items()))]
    heapq.heapify(heap)
    depth = {s: 0 for s in freq}
    tie = len(heap)
    while len(heap) > 1:
        # merge the two lightest; everything below goes
        # deeper
        w1, _, a = heapq.heappop(heap)
        w2, _, b = heapq.heappop(heap)
        for s in a + b:
            depth[s] += 1
        heapq.heappush(heap, (w1 + w2, tie, a + b))
        tie += 1
    return depth

def canonical_codes(lengths):
    # assign consecutive binary numbers by (length,
    # symbol)
    code, prev, out = 0, 0, {}
    for sym, length in sorted(lengths.items(),
                              key=lambda x: (x[1], x[0])):
        # moving to a longer length shifts the code left
        code <<= length - prev
        out[sym] = format(code, f"0{length}b")
        code += 1
        prev = length
    return out

# BASE QUESTION (optimal): Min-heap merge of the two
# lightest nodes
# Idea: Put every symbol in a min-heap keyed by frequency,
# repeatedly pop the two lightest nodes, merge them under
```

```
# a new internal node with the summed weight and push it
# back. The last node is the root.
def huffman_codes(freq):
    return canonical_codes(huffman_lengths(freq))
```

GROUP 5 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE | 6 M 3 H

Recurrence on a position or a range (memo or table)

37 Target Expressions from Ordered Digits — MEDIUM

*Template fit: best(l, r) = every value reachable from digits l..r; split at m and combine the two sides with +, - and **

QUESTION

EXAMPLES

```
For digits [2, 3, 4] and target 14, valid outputs include:
2 * (3 + 4)
2 + 3 * 4
```

MAIN CODE | helper functions first, then the answer

```python
from collections import defaultdict


# HELPERS for the base question
def build(digits):
    # table[(l, r)] = {value: {(text, precedence)}}
    table = {}

    def solve(l, r):
        if (l, r) in table:
            return table[(l, r)]
        res = defaultdict(set)
        if r - l == 1:
            # a single digit is an atom (precedence 3)
            res[digits[l]].add((str(digits[l]), 3))
        # try every split point
        for m in range(l + 1, r):
            for lv, ls in solve(l, m).items():
                for rv, rs in solve(m, r).items():
                    combine(res, lv, ls, rv, rs)
        table[(l, r)] = res
        return res

    return solve(0, len(digits))

def combine(res, lv, ls, rv, rs):
    ops = (("+", 1, lv + rv), ("-", 1, lv - rv),
           ("*", 2, lv * rv))
    for lt, lp in ls:
        for rt, rp in rs:
            for op, prec, val in ops:
                # brackets only where precedence needs
                left = f"({lt})" if lp < prec else lt
                need = rp < prec
                if rp == prec and op == "-":
                    need = True
                right = f"({rt})" if need else rt
                res[val].add((left + op + right, prec))

# BASE QUESTION (optimal): Interval DP: all expressions
# for digits[l:r]
# Idea: Every expression over digits[l:r] is two sub-
# expressions over a split point joined by +, - or *.
# Memoize, for each interval, a map from numeric value to
# the set of expression strings that produce it.
```

```python
def expressions_for_target(digits, target):
    found = build(digits).get(target, ())
    return sorted({text for text, _ in found})
```

38 Count Word Decompositions into Chemical Element Symbols

MEDIUM

Template fit: bottom-up ways[i]: add ways[i+1] if the one-letter symbol exists and ways[i+2] if the two-letter symbol exists

QUESTION

EXAMPLES

Physics has four stated decompositions

P - H - Y - Si - C - S P - H - Y - Si - Cs P - H - Y - S - I - C - S P - H - Y - S - I - Cs The returned count is 4.

MAIN CODE | helper functions first, then the answer

```python
# HELPERS used by the code below
ELEMENTS = set("""h he li be b c n o f ne na mg al si
p s cl ar k ca sc ti v cr mn fe co ni cu zn ga ge as
se br kr rb sr y zr nb mo tc ru rh pd ag cd in sn sb
te i xe cs ba la ce pr nd pm sm eu gd tb dy ho er tm
yb lu hf ta w re os ir pt au hg tl pb bi po at rn fr
ra ac th pa u np pu am cm bk cf es fm md no lr rf db
sg bh hs mt ds rg cn nh fl mc lv ts og""".split())

# BASE QUESTION (optimal): Suffix-count DP
# Idea: ways[i] is the number of decompositions of
# word[i:]. A symbol has one or two letters, so ways[i] =
# ways[i + 1] (if word[i] is a symbol) + ways[i + 2] (if
# word[i:i + 2] is a symbol).
def count_decompositions(word, symbols=ELEMENTS,
                         mod=None):
    w, n = word.lower(), len(word)
    # ways[i]: ways to split the suffix starting at i
    ways = [0] * (n + 2)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        # a one-letter symbol, then a two-letter symbol
        if w[i] in symbols:
            ways[i] += ways[i + 1]
        if i + 2 <= n and w[i:i + 2] in symbols:
            ways[i] += ways[i + 2]
        if mod:
            ways[i] %= mod
    return ways[0]
```

39 Find the longest palindromic subsequence using dynamic programming.

MEDIUM

Template fit: table over (i, j): equal ends add 2 to the inside; otherwise the better of dropping either end

QUESTION

Given a string s, determine the longest palindromic substring in s. A substring is considered a palindrome if it reads the same forward and backward. A substring is a contiguous sequence of characters within a string. The longest palindromic substring is the substring with the maximum length that satisfies this condition. Examples Input: "abacc" Output: "aba" Explanation: "aba" is longer than "cc"

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
def lps_table(s):
    # dp[i][j] = longest palindromic SUBSEQUENCE of
    # s[i..j].
    # Equal ends are both used; otherwise drop one end.
```

```
    n = len(s)
    dp = [[0] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):
        dp[i][i] = 1
        for j in range(i + 1, n):
            if s[i] == s[j]:
                dp[i][j] = dp[i + 1][j - 1] + 2
            else:
                dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
    return dp

# BASE QUESTION (optimal): Interval DP with a rolling row
# Idea: dp[i][j] is the length of the longest palindromic
# subsequence of s[i..j]. Fill i from the end to the start
# and j from i + 1 up: equal ends give dp[i + 1][j - 1] +
# 2, otherwise max(dp[i + 1][j], dp[i][j - 1]).
def lps_length(s):
    return lps_table(s)[0][-1] if s else 0
```

40 Jump Game with Profit — MEDIUM

Template fit: best[i] = max over every j before i that can reach i (j + arr[j] >= i) of best[j] + profit[i]

QUESTION

EXAMPLES

Variant-dependent; the principal recurrence is the one above.

MAIN CODE | helper functions first, then the answer

```
import heapq


# HELPERS used by the code below
NEG = float("-inf")

# BASE QUESTION (optimal): Reading A: arr[i] is a maximum
# jump, profit[i] is gained on landing, maximize total
# profit to the last index
# Idea: Let best[i] be the largest profit achievable when
# standing on i. Landing on i from any j < i with j +
# arr[j] >= i gives best[i] = profit[i] + max(best[j]).
# Unreachable cells stay at minus infinity.
def max_profit_fast(arr, profit):
    # a max-heap of (-best, reach) with lazy deletion
    n = len(arr)
    best = [NEG] * n
    best[0] = profit[0]
    heap = [(-best[0], arr[0])]
    for i in range(1, n):
        # forget cells that can no longer reach i
        while heap and heap[0][1] < i:
            heapq.heappop(heap)
        if heap:
            best[i] = -heap[0][0] + profit[i]
            heapq.heappush(heap, (-best[i], i + arr[i]))
    return -1 if best[-1] == NEG else best[-1]
```

41 Fence Painting Minimum Operations (Gemini Team) MEDIUM

Template fit: range (l, r): paint one stripe at the lowest height, then solve the pieces on both sides; a sparse table finds the lowest point fast

QUESTION

EXAMPLES

```
n = 5, a = [2, 2, 1, 2, 1]
Optimal:
  1) Horizontal stroke at h=1 across [1..5]   (covers row 1 across whole fence)
  2) Horizontal stroke at h=2 across [1..2]
  3) Vertical stroke at plank 4 (height 2 still missing row 2)
Optimal answer: 3.
```

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
def sparse_table(a):
    # table[j][i] = index of the smallest plank in a[i : i
    # + 2^j]
    table = [list(range(len(a)))]
    j = 1
    while (1 << j) <= len(a):
        prev, row = table[-1], []
        for i in range(len(a) - (1 << j) + 1):
            x, y = prev[i], prev[i + (1 << (j - 1))]
            row.append(x if a[x] <= a[y] else y)
        table.append(row)
        j += 1
    return table

def paint_strokes(a, width=None):
    # returns (number of strokes, the list of strokes)
    n = len(a)
    if n == 0:
        return 0, []
    table = sparse_table(a)

    def argmin(l, r):
        # smallest plank in a[l..r] in O(1)
        k = (r - l + 1).bit_length() - 1
        x, y = table[k][l], table[k][r - (1 << k) + 1]
        return x if a[x] <= a[y] else y

    def solve(l, r, base):
        # paint planks l..r whose bottom `base` rows are
        # done
        if l > r:
            return 0, []
        m = argmin(l, r)
        size = r - l + 1
        # a band of rows over the whole range needs one
        # stroke per row (or per `width` planks)
        per_row = 1
        if width is not None:
            per_row = -(-size // width)
        band = (a[m] - base) * per_row
        lc, ls = solve(l, m - 1, a[m])
        rc, rs = solve(m + 1, r, a[m])
        horizontal = band + lc + rc
        # or paint every plank by itself, top to bottom
```

```python
        if size <= horizontal:
            planks = range(l, r + 1)
            return size, [("vertical", i) for i in planks]
        rows = [("horizontal", h, l, r)
```

```
                for h in range(base + 1, a[m] + 1)]
        return horizontal, rows + ls + rs

    return solve(0, n - 1, 0)

# BASE QUESTION (optimal): Same recursion with a sparse
# table for range minimum
# Idea: The recursion is correct; only the minimum lookup
# is slow. A sparse table answers range-minimum-index
# queries in O(1), and each plank becomes the split point
# at most once, so the recursion visits n ranges.
def paint(a, width=None):
    return paint_strokes(a, width)[0]
```

42 Maximize 1D deliveries within distance and minimize total distance — MEDIUM

Template fit: table over (orders used, drivers used): skip an order, skip a driver, or match them when they are within D (count, then minus distance)

QUESTION

Overview: This question evaluates combinatorial optimization and constrained matching skills, focusing on reasoning about distance-based pairings, lexicographic objective prioritization, and edge-case handling such as duplicates and empty inputs.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): DP over sorted orders and
# drivers, maximising (matches, -distance)
# Idea: Sort both lists. dp[i][j] is the best (matches,
# -total distance) using the first i orders and the first
# j drivers: skip order i, skip driver j, or match them if
# their distance is at most D.
def deliveries(orders, drivers, D):
    # first maximise the number of deliveries, then
    # minimise
    # the total distance: a DP over the sorted positions
    o, d = sorted(orders), sorted(drivers)
    # dp[i][j] = (deliveries, -distance) using the first i
    # orders and j drivers; tuples compare
    # lexicographically
    dp = [[(0, 0)] * (len(d) + 1)
          for _ in range(len(o) + 1)]
    for i in range(1, len(o) + 1):
        for j in range(1, len(d) + 1):
            # skip this order or skip this driver ...
            best = max(dp[i - 1][j], dp[i][j - 1])
            gap = abs(o[i - 1] - d[j - 1])
            if gap <= D:
                # ... or pair them
                count, neg = dp[i - 1][j - 1]
                best = max(best, (count + 1, neg - gap))
            dp[i][j] = best
    count, neg = dp[-1][-1]
    return count, -neg
```

43 Edit distance — HARD

Template fit: table over two strings, two rows: equal letters copy the diagonal, otherwise 1 + the smallest of the three neighbours

QUESTION

Determine the Edit Distance in a Word Ladder: Given two words (beginWord and endWord), and a dictionary's word list, find the minimum number of operations needed to change beginWord into endWord. You can change only one letter at a time, and each intermediate word must exist in the word list. If there is no possible transformation, return None (Python), -1 (Java & C++), null (Javascript). Examples beginWord = 'hit' endWord = 'cog' wordList = ["hit", "hot", "dot", "dog", "cog"] output: 4

hit -> hot -> dot

-> dog -> cog beginWord = 'word' endWord = 'word' wordList = ['word', 'ward'] output: 0 beginWord = 'hit' endWord = 'cog' wordList = ["hit", "hot", "dot", "dog"] output: None

because no 'cog' in list

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): DP table with a rolling row
# Idea: dp[j] holds the distance from the current prefix
# of a to the first j characters of b.
def edit_distance(a, b):
    # dp[i][j]: edits to turn a[:i] into b[:j]
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                swap = dp[i - 1][j - 1]
                drop = dp[i - 1][j]
                add = dp[i][j - 1]
                dp[i][j] = 1 + min(swap, drop, add)
    return dp[m][n], dp
```

44 Count Perfect Wake Numbers up to N — HARD

Template fit: digit DP: cache best(remaining digits, used-digit mask, last digit, falling?) and walk along N with a tight flag

QUESTION

EXAMPLES

```
196   -> perfect (digits 1,9,6 all distinct, no 0, 9 is not a valley)
23    -> perfect
12463 -> perfect (1<2, 2<4, 4>6? No, 4<6, 6>3 — 6 is not a valley because its right neighbor 3 is
    smaller)
1546  -> not perfect (4 is adjacent to 5 and 6, both strictly larger)
320   -> not perfect (contains 0)
34321 -> not perfect (digit 3 repeats)
```

MAIN CODE | helper functions first, then the answer

```python
from math import comb


# HELPERS for the base question
def with_k_digits(k):
```

```
    # choose the k digits; the largest is the peak and
    # each
    # other digit goes left or right of it
    return comb(9, k) * 2 ** (k - 1) if k >= 1 else 0

# BASE QUESTION (optimal): test a single number
# Idea: Check the three rules directly on the digit list:
# no '0', all digits distinct, and for each interior digit
# not (left > digit < right). The first and last digits
# are never valleys.
def is_perfect(n):
    # distinct digits, no zero, and no valley: no digit is
    # smaller than both of its neighbours
    d = list(map(int, str(n)))
    if 0 in d or len(set(d)) != len(d):
        return False
    return all(not (d[i - 1] > d[i] < d[i + 1])
               for i in range(1, len(d) - 1))
```

45 Word Break: All Sentences — HARD

Template fit: best(start) = the list of sentences for the suffix; for every word that matches at start, put it in front of each sentence of best(end)

QUESTION

You are given a string s and a dictionary word_dict of distinct words. Return ALL possible ways to segment s into a sequence of space-separated words such that every word in the sequence appears in word_dict . A dictionary word may be reused any number of times. Each returned sentence is a single string of the chosen words joined by single spaces. The sentences may be returned in any order.

Function signature

from typing import List
def word_break_all(s: str, word_dict: List[str]) -> List[str]:
    ...

Follow-up (as reported): Solve with backtracking: at each position try every dictionary word that is a prefix of the remaining string, then recurse. Add memoization keyed on the suffix (or start index) to reuse partial results and avoid exponential re-work.

EXAMPLES

Example 1	
Input	`s = "catsanddog", word_dict = ["cat","cats","and","sand","dog"]`
Output	`["cats and dog", "cat sand dog"]`
Why	`Two segmentations use only dictionary words.`

Example 2	
Input	`s = "pineapplepenapple", word_dict = ["apple","pen","applepen","pine","pineapple"]`
Output	`["pine apple pen apple", "pineapple pen apple", "pine applepen apple"]`
Why	`All three segmentations consist entirely of dictionary words.`

Example 3	
Input	`s = "catsandog", word_dict = ["cats","dog","sand","and","cat"]`
Output	`[]`
Why	`No segmentation covers the whole string with dictionary words.`

CONSTRAINTS

- 1 <= len(s) <= 20

QUESTION

- 1 <= len(word_dict) <= 1000
- 1 <= len(word) <= 10 for each word; words consist of lowercase letters and are distinct.
- If no valid segmentation exists, return an empty list [] .

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Memoized DFS on the start index
# Idea: The sentences of a suffix do not depend on how you
# reached it, so compute them once.
def word_break_all(s, word_dict):
    words = set(word_dict)
    memo = {}

    def sentences(start):
        if start == len(s):
            return [""]
        if start in memo:
            return memo[start]
        out = []
        for end in range(start + 1, len(s) + 1):
            if s[start:end] in words:
                for rest in sentences(end):
                    out.append(
                        s[start:end]
                        + (" " + rest if rest else "")
                    )
        memo[start] = out
        return out

    return sentences(0)
```

Best-first search with a heap: Dijkstra and its minimax twin

46 Path Minimizing the Maximum Cell (Swim in Rising Water) MEDIUM

Template fit: the same minimax loop with a given start and end cell (Swim in Rising Water)

QUESTION

A bottleneck-path family: minimize the maximum value encountered on a grid path, then generalize the same reasoning to a weighted graph whose edges carry safety values and an admissibility threshold.

Grid variant

Input is a grid of heights and start/end cells; movement is through four-neighbors.
Cost of a path is the maximum cell value along it.
Return the minimum possible path cost.

Weighted-graph safety variant

Input is a graph whose edges carry safety values, a start node, an end node, and a safety threshold.
Find the lowest safety value required by any valid start-to-end path while every traversed edge remains under the supplied threshold.
Return -1 when no admissible path exists.
Clarify whether the threshold comparison is strict or inclusive before coding.

Mouse-and-cat safest-path variant (L4 phone screen)

Warm-up: an N x N grid with a start S, a target T, and impassable water cells; with 4-directional moves, decide whether any path from S to T exists.
Main question: the grid also contains a cat at C. The mouse moves only on land, 4-directionally, from S to the cheese at T. Find a path that maximizes the minimum Manhattan distance to the cat over all cells on the path.
This flips the objective (maximize a minimum instead of minimize a maximum), but it is the same bottleneck-path structure once each cell's distance to the cat is known.

Follow-up: Network Delay Time variant

In a directed graph with edge weights, return the maximum shortest-path distance from a source to all nodes, or -1 if any node is unreachable.

EXAMPLES
Grid

```
[ 0  1  2  3  4 ]
[24 23 22 21  5 ]
[12 13 14 15 16 ]
[11 17 18 19 20 ]
[10  9  8  7  6 ]
```

Start=(0,0), End=(4,0) in (row, col) — the bottom-left cell with value 10. Answer: 16 (the path's max cell is 16).

MAIN CODE | helper functions first, then the answer

```python
import heapq


# HELPERS for the base question
INF = float("inf")

def around(r, c, n, m):
    # four-neighbour cells inside an n x m grid
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < n and 0 <= b < m:
```

```
            yield a, b

# BASE QUESTION (optimal): Dijkstra on the running maximum
# Idea: Heap entries are (cost so far, cell). Popping a
# cell fixes its cost; each neighbour gets max(cost,
# neighbour value). Stop when the end cell is popped. Skip
# stale heap entries.
def min_max_path(g, start, end):
    n, m = len(g), len(g[0])
    # best[cell] = smallest possible "highest cell so far"
    best = {start: g[start[0]][start[1]]}
    heap = [(best[start], start)]
    while heap:
        cost, (r, c) = heapq.heappop(heap)
        # skip outdated heap entries
        if cost > best[(r, c)]:
            continue
        if (r, c) == end:
            return cost
        for a, b in around(r, c, n, m):
            # path cost = highest cell so far or this cell
            nxt = max(cost, g[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, (a, b)))
    return None
```

47 Nearest Favorite City by Travel Time

MEDIUM

Template fit: combine = cost + w; stop when a favourite is popped (keep popping equal distances to break ties on the smaller index)

QUESTION

You are given a network of num_cities cities (0..num_cities-1) connected by weighted, undirected roads roads[i] = [u, v, w] where w > 0 is the travel time. Given a start city and a non-empty list favorites of favorite cities, return the favorite city reachable from start in the shortest total travel time. Break ties by smallest city index. If none of the favorite cities is reachable, return -1 .

Function signature

```
def nearest_favorite(num_cities: int, roads: list[list[int]],
                     start: int, favorites: list[int]) -> int:
    ...
```

EXAMPLES

Example 1	
Input	`num_cities = 5, roads = [[0,1,2],[0,2,5],[1,3,1],[2,4,1]], start = 0, favorites = [3, 4]`
Output	`3`
Why	`Shortest times from 0: to 3 = 2+1 = 3, to 4 = 5+1 = 6. City 3 is nearer.`

Example 2	
Input	`num_cities = 3, roads = [[0,1,4]], start = 0, favorites = [2]`
Output	`-1`
Why	`City 2 is disconnected from start, so no favorite is reachable.`

CONSTRAINTS

- 1 <= num_cities <= 10**5 , 0 <= len(roads) <= 2*10**5 , 1 <= w <= 10**9 .
- The interviewer required the optimal solution (Dijkstra with a priority queue).

MAIN CODE | helper functions first, then the answer

```
import heapq


# BASE QUESTION (optimal): Dijkstra from the start
# Idea: Cities are finalised in order of distance, so the
# first favourite reached is the nearest.
def nearest_favorite(
    num_cities, roads, start, favorites
):
    adj = [[] for _ in range(num_cities)]
    for u, v, w in roads:
        adj[u].append((v, w))
        adj[v].append((u, w))
    dist = [float("inf")] * num_cities
    dist[start] = 0
    heap, wanted = [(0, start)], set(favorites)
    best_d, best_c = None, -1
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        if best_d is not None and d > best_d:
            break
        if u in wanted and (
            best_d is None or u < best_c
        ):
            best_d, best_c = d, u
        for v, w in adj[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (dist[v], v))
    return best_c
```

48 Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time. MEDIUM

Template fit: cost = arrival time; neighbors only offers trains that leave at or after the current time; combine = the train's arrival time

QUESTION

Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time.

MAIN CODE | helper functions first, then the answer

```
import heapq


# BASE QUESTION (optimal): Dijkstra on earliest arrival
# times, with the train sequence
# Idea: best[city] is the earliest time you can be there.
def earliest_route(trains, source, target, layover=0):
    # Dijkstra on arrival time. You may board a train if
    # it
    # leaves at or after your arrival plus the layover (no
    # layover is needed at the origin).
    by_city = {}
    for tr in trains:
        by_city.setdefault(tr.src, []).append(tr)
    best = {source: 0}
    took = {}
    heap = [(0, source)]
    while heap:
```

```
        time, city = heapq.heappop(heap)
        if time > best.get(city, float("inf")):
            continue
        if city == target:
            break
        wait = 0 if city == source else layover
        for tr in by_city.get(city, []):
            old = best.get(tr.dst, float("inf"))
            if tr.dep >= time + wait and tr.arr < old:
                best[tr.dst] = tr.arr
                took[tr.dst] = tr
                heapq.heappush(heap, (tr.arr, tr.dst))
    if target not in best:
        return None
    # walk back from the target to list the trains used
    path, city = [], target
    while city != source:
        path.append(took[city])
        city = took[city].src
    return best[target], path[::-1]
```

49 Find the shortest path in a graph. MEDIUM

Template fit: combine = cost + w; also store parent[v]; stop at the target and rebuild the path

QUESTION

Find the shortest path in a graph.

MAIN CODE | helper functions first, then the answer

```
import heapq
from collections import defaultdict


# HELPERS used by the code below
INF = float("inf")

# BASE QUESTION (optimal): Dijkstra with a heap (non-
# negative weights), returning the path
# Idea: Keep the best known distance to each node and a
# min-heap of (distance, node). Pop the closest unsettled
# node, relax its edges, and remember the parent of every
# improvement.
def dijkstra_path(edges, s, t):
    # weighted, non-negative edges (u, v, w)
    graph = defaultdict(list)
    for u, v, w in edges:
        graph[u].append((v, w))
    dist, parent, heap = {s: 0}, {s: None}, [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        if u == t:
            # the first pop of t is final: rebuild the
            # path
            path = []
            while u is not None:
                path.append(u)
                u = parent[u]
            return d, path[::-1]
        for v, w in graph[u]:
            if d + w < dist.get(v, INF):
                dist[v], parent[v] = d + w, u
```

```
                heapq.heappush(heap, (d + w, v))
    return None
```

50 Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat. MEDIUM

Template fit: maximise the minimum distance to the cat along the path: push negated values, combine = min(value, cat distance of the cell)

QUESTION

Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat.

MAIN CODE | helper functions first, then the answer

```
import heapq
from collections import deque


# HELPERS used by the code below
INF = 10 ** 9

def around(r, c, grid):
    # the open neighbours of a cell (obstacles are "#")
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < len(grid) and 0 <= nc < len(grid[0]):
            if grid[nr][nc] != "#":
                yield nr, nc

def find(grid, ch):
    for r, row in enumerate(grid):
        if ch in row:
            return r, row.index(ch)
    return None

def cat_distances(grid):
    # one BFS from the cat gives every cell's distance to
    # it
    dist = [[INF] * len(grid[0]) for _ in grid]
    cat = find(grid, "C")
    if cat is None:
        return dist
    dist[cat[0]][cat[1]] = 0
    queue = deque([cat])
    while queue:
        r, c = queue.popleft()
        for nr, nc in around(r, c, grid):
            if dist[nr][nc] == INF:
                dist[nr][nc] = dist[r][c] + 1
                queue.append((nr, nc))
    return dist

# BASE QUESTION (optimal): Multi-source BFS from the cat,
# then a max-heap bottleneck search
# Idea: BFS from every cat gives the distance of each
# cell.
def safest_value(grid):
    # a path is as safe as its closest cell to the cat.
    # Always extend the safest frontier cell (like
```

```
    # Dijkstra
    # with max and min instead of min and sum).
    dist = cat_distances(grid)
```

```
    start, goal = find(grid, "R"), find(grid, "B")
    best = {start: dist[start[0]][start[1]]}
    heap = [(-best[start], start)]
    while heap:
        neg, cell = heapq.heappop(heap)
        if cell == goal:
            return -neg
        if -neg < best[cell]:
            continue
        for nr, nc in around(cell[0], cell[1], grid):
            safety = min(-neg, dist[nr][nc])
            if safety > best.get((nr, nc), -1):
                best[(nr, nc)] = safety
                heapq.heappush(heap, (-safety, (nr, nc)))
    return None
```

51 Minimize maximum height along a grid path — HARD

Template fit: the same minimax loop on a height grid

QUESTION

Overview: This question evaluates algorithmic pathfinding and graph-modeling skills, specifically reasoning about grid-based route selection where the objective is defined by an aggregate maximum over visited cell heights.

MAIN CODE | helper functions first, then the answer

```
import heapq


# HELPERS for the base question
INF = float("inf")

def around(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b

# BASE QUESTION (optimal): Dijkstra with cost = maximum
# cell on the path
# Idea: Heap of (largest cell so far, cell). Pop the cell
# with the smallest value, extend to each neighbour with
# cost max(current, neighbour), and keep only
# improvements.
def min_max_path(H, with_path=False):
    # Dijkstra where a path costs the MAX of its cells
    m, n = len(H), len(H[0])
    best, parent = {(0, 0): H[0][0]}, {}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            if not with_path:
                return cost
            path = [(r, c)]
            while (r, c) in parent:
                r, c = parent[(r, c)]
                path.append((r, c))
            return cost, path[::-1]
        for a, b in around(r, c, m, n):
            nxt = max(cost, H[a][b])
            if nxt < best.get((a, b), INF):
```

```
                best[(a, b)] = nxt
                parent[(a, b)] = (r, c)
                heapq.heappush(heap, (nxt, a, b))
```

52 Find minimum threshold enabling grid path — HARD

Template fit: the same minimax loop: the smallest threshold that lets you walk from corner to corner

QUESTION

Overview: This question evaluates a candidate's understanding of grid pathfinding, reachability under constraints, and algorithmic optimization, focusing on skills such as graph traversal and handling threshold-based feasibility.

MAIN CODE | helper functions first, then the answer

```
import heapq


# HELPERS for the base question
INF = float("inf")

def around(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b

# ALSO: the Dijkstra version
def best_path(H, step_cost, start_cost):
    # Dijkstra where a path costs the MAX of its steps;
    # the
    # first time a cell leaves the heap its cost is final
    m, n = len(H), len(H[0])
    best = {(0, 0): start_cost}
    parent = {}
    heap = [(start_cost, 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            path = [(r, c)]
            while (r, c) in parent:
                r, c = parent[(r, c)]
                path.append((r, c))
            return cost, path[::-1]
        for a, b in around(r, c, m, n):
            nxt = max(cost, step_cost(H, r, c, a, b))
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                parent[(a, b)] = (r, c)
                heapq.heappush(heap, (nxt, a, b))
    return None, []

# BASE QUESTION (optimal): Dijkstra with cost = maximum
# cell on the path
# Idea: Heap of (largest cell so far, cell). Pop the cell
# with the smallest value, extend to each neighbour with
# cost max(current, neighbour), and keep only
# improvements.
```

```
def min_max_path(H):
    # cost of a path = its largest cell value
    # (water rising over time is the same problem)
```

```
    def step(H, r, c, a, b):
        return H[a][b]
    return best_path(H, step, H[0][0])
```

53 Minimize travel time with optional meeting point — HARD

Template fit: three runs: from a, from b, and from the destination on the reversed graph; answer = min over meeting points m of max(da, db) + dd

QUESTION

Overview: This question evaluates a candidate's understanding of graph shortest-path computation and meeting-point optimization, testing skills in handling weighted edges, aggregating path costs, and reasoning about simultaneous movements on a network.

MAIN CODE | helper functions first, then the answer

```
import heapq


# HELPERS used by the code below
INF = float("inf")

# HELPERS for the base question
def dijkstra(graph, source):
    dist = [INF] * len(graph)
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return dist

def build(n, edges, directed):
    # forward graph, and the reversed graph for "to dest"
    fwd = [[] for _ in range(n)]
    rev = [[] for _ in range(n)]
    for u, v, w in edges:
        fwd[u].append((v, w))
        rev[v].append((u, w))
        if not directed:
            fwd[v].append((u, w))
            rev[u].append((v, w))
    return fwd, rev

# BASE QUESTION (optimal): Three Dijkstra runs (from A,
# from B, and into the destination)
# Idea: distA and distB come from forward searches. The
# distance from every node to the destination is a search
# from the destination on the reversed graph (the same
# graph if undirected).
def meeting_point(n, edges, a, b, dest, directed=False):
    # both start at the same time, meet at node m, wait
    # for
    # each other, then travel together to dest. Meeting at
    # dest itself covers the "no meeting" plan.
    fwd, rev = build(n, edges, directed)
```

```
    da, db = dijkstra(fwd, a), dijkstra(fwd, b)
    # distance from every node TO dest = search the
```

```
    # reverse graph
    to_dest = dijkstra(rev, dest)
    best = (INF, -1)
    for m in range(n):
        total = max(da[m], db[m]) + to_dest[m]
        if total < best[0]:
            best = (total, m)
    return [-1, -1] if best[0] == INF else list(best)
```

54 Compute minimax grid path and network delay — HARD

Template fit: combine = max(cost, height of the next cell); the answer is the cost at the bottom-right cell

QUESTION

Overview: This question evaluates algorithmic problem-solving in shortest-path and path-optimization contexts, specifically minimax pathfinding on a grid and single-source reachability/timing in a weighted directed graph, testing graph modeling, alternative path cost metrics, edge-case handling, and working with large numeric weights.

MAIN CODE | helper functions first, then the answer

```
import heapq


# HELPERS for the base question
INF = float("inf")

def neighbors(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b

# BASE QUESTION (optimal): Dijkstra where the cost of a
# path is its maximum cell
# Idea: Heap of (best maximum so far, cell). Pop the cell
# with the smallest bottleneck, and for each neighbour the
# new bottleneck is max(current, neighbour). Improve if
# smaller.
def minimax_path(H):
    # Part 1: the path whose highest cell is as low as
    # possible
    m, n = len(H), len(H[0])
    best = {(0, 0): H[0][0]}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            return cost
        for a, b in neighbors(r, c, m, n):
            # the cost of a path is its maximum, never a
            # sum
            nxt = max(cost, H[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, a, b))
    return -1
```

Backtracking: choose, recurse, undo

55 Mahjong Winning Hand Detection (14 Tiles) — MEDIUM

Template fit: take the smallest tile; try it as a triple, then as the start of a run; undo the counts after each try

QUESTION

Mountain View onsite coding round 1: given a hand of 14 tiles, decide whether it is a winning mahjong hand. The candidate must clarify the rule set (typically: 4 melds of three identical tiles or three consecutive ranks + 1 pair).

Input: array of 14 integers representing tile values (clarify suit / range during the interview).
A winning hand consists of:

One pair (two identical tiles), and
Four melds, each being either:

A triplet (3 identical tiles), or
A run (3 consecutive ranks in the same suit).

Minority variant: A newer version instead specifies one pair plus two melds while retaining a 14-tile input. Because that accounts for only eight tiles, clarify the intended meld count and how the remaining six tiles should be handled before coding.

Output: boolean.

MAIN CODE | helper functions first, then the answer

```python
from collections import Counter


# HELPERS for the base question
def is_run_start(tile, count):
    # numbered suits are 0..2 with ranks 1..9; the tile
    # id is suit * 10 + rank. Honours (30+) never run.
    ok = tile < 30 and tile % 10 <= 7
    nxt, nxt2 = count.get(tile + 1), count.get(tile + 2)
    return ok and nxt and nxt2

def melds_left(count, need):
    # can the tiles in `count` form exactly `need` melds?
    live = [t for t in sorted(count) if count[t]]
    tile = live[0] if live else None
    if tile is None:
        return need == 0
    if need == 0:
        return False
    # the smallest tile is either in a triplet ...
    if count[tile] >= 3:
        count[tile] -= 3
        ok = melds_left(count, need - 1)
        count[tile] += 3
        if ok:
            return True
    # ... or starts a run of three
    if is_run_start(tile, count):
        for t in (tile, tile + 1, tile + 2):
            count[t] -= 1
        ok = melds_left(count, need - 1)
        for t in (tile, tile + 1, tile + 2):
            count[t] += 1
        return ok
    return False
```

```
# BASE QUESTION (optimal): Try each pair, then greedy-
# smallest with two options
# Idea: Tiles are encoded as suit * 10 + rank for the
# numbered suits (ranks 1 to 9, suits 0 to 2) and 30 and
# above for honours, which never form runs.
def is_winning_hand(hand, melds=4):
    if len(hand) != 2 + 3 * melds:
        return False
    count = Counter(hand)
```

```
    # try every tile value as the pair
    for tile in list(count):
        if count[tile] >= 2:
            count[tile] -= 2
            ok = melds_left(count, melds)
            count[tile] += 2
            if ok:
                return True
    return False
```

56 Count Prefix-Matching Simple Paths in a Labeled Graph

MEDIUM

Template fit: extend the path along edges whose label matches the next query letter, with a bitmask of visited nodes; count per depth

QUESTION

Overview: Count distinct simple graph paths whose labels match every nonempty prefix of a query word. Paths may start anywhere but cannot revisit a vertex, so the task rewards subset-state search and careful counting over a small labeled graph.

MAIN CODE | helper functions first, then the answer

```
from collections import defaultdict


# BASE QUESTION (optimal): DFS with label pruning and a
# visited bitmask
# Idea: Start from every vertex whose label equals the
# first query letter. Extend to an unvisited neighbour
# only if its label equals the next query letter, adding
# one to the count for the current length at each step.
def prefix_paths(labels, edges, query, undirected=False):
    # counts[d]: simple paths whose first d + 1 labels
    # spell
    # the first d + 1 letters of the query
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        if undirected:
            adj[v].append(u)
    counts = [0] * len(query)

    def go(u, depth, mask):
        counts[depth] += 1
        if depth + 1 == len(query):
            return
        for w in adj[u]:
            # a simple path never revisits a vertex
            if mask >> w & 1:
                continue
            if labels[w] == query[depth + 1]:
                go(w, depth + 1, mask | 1 << w)
```

```
    # a path may start at any vertex with the first letter
    for s in range(len(labels)):
        if query and labels[s] == query[0]:
            go(s, 0, 1 << s)
    return counts
```

57 Generate Parentheses

MEDIUM

Template fit: choices are '(' and ')'; allowed when opened < n, and ')' only when closed < opened; complete at length 2n

QUESTION

Generate all combinations of well-formed parentheses. A parenthesis combination is well-formed if each opening parenthesis "(" is closed by a matching closing parenthesis ")" in the correct order. For example, "()", "()()", and "((()))" are all combinations of well-formed parentheses, while ")(", "((" and "(()" are not. Examples n = 1 output: ["()"] n = 2 output: ["(())", "()()"] n = 3 output: ["((()))", "(()())", "(())()", "()(())", "()()()"] n = 4 output should contain: ["(((())))", "((()()))", "((())())", "((()))()", "(()(()))", "(()()())", "(())(())", "(())()()", "()((()))", "()(()())", "()(())()", "()()(())", "()()()()"]The problem is to generate all combinations of well-formed parentheses given n pairs of parentheses. A combination is well-formed if every opening parenthesis has a corresponding closing parenthesis and they are properly nested. Recursive Approach The most common and intuitive way to solve this problem is by using recursion with backtracking. Here's a step-by-step breakdown of the approach: Define the Problem: We need to build strings of length 2n using n opening (and n closing) parentheses. At any point in the string, the number of closing parentheses should not exceed the number of opening parentheses to ensure the sequence is valid. Recursive Function: We use a recursive helper function to build the strings character by character. The function tracks the remaining number of opening and closing parentheses that can still be added. It also

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Backtracking with open and
# close counters
# Idea: Track how many '(' and ')' have been placed. Add
# '(' if open < n; add ')' if close < open. When the
# string reaches length 2n it is valid.
def generate(n):
    # build the string one character at a time. Adding "("
    # is allowed while opens < n; adding ")" only while
    # closes < opens: that rule keeps every string valid.
    out = []

    def build(text, opens, closes):
        if len(text) == 2 * n:
            out.append(text)
            return
        if opens < n:
            build(text + "(", opens + 1, closes)
        if closes < opens:
            build(text + ")", opens, closes + 1)

    build("", 0, 0)
    return out
```

58 American Football Score Combinations

MEDIUM

Template fit: choices are the scoring plays from index i onward (the same play may repeat); complete when the remaining score is 0

QUESTION

Bryan's DP question was: enumerate all the ways a football team can accumulate a given total number of points. In American football a single scoring play is worth one of: 2 (safety), 3 (field goal), 6 (touchdown), 7 (touchdown + extra point), or 8 (touchdown + 2-point conversion). Given a target total , return every distinct combination of scoring plays whose values sum to total . Two combinations are the same if they use the same multiset of play values (order does not matter). Each combination should be returned as a list of play values sorted in non-decreasing order. The overall list of combinations may be in any order. A play value may be used any number of times.

Function signature

```
def football_combinations(total: int) -> list[list[int]]:
    """Return all distinct multisets of {2,3,6,7,8} that sum to total, each sorted
ascending."""
    ...
```

Follow-up (as reported): If you only need the count of combinations (not the lists), do it with an O(total) DP table (unbounded coin-change counting).

EXAMPLES

Example 1	
Input	`total = 6`
Output	`[[2, 2, 2], [3, 3], [6]]`
Why	`2+2+2 = 3+3 = 6 = 6.`

Example 2	
Input	`total = 7`
Output	`[[2, 2, 3], [7]]`
Why	`2+2+3 = 7 and the single play 7.`

Example 3	
Input	`total = 0`
Output	`[[]]`
Why	`The empty combination is the only way to score 0.`

CONSTRAINTS

- 0 <= total <= 60
- Allowed play values are exactly [2, 3, 6, 7, 8] .
- If total == 0 , the only combination is the empty one, so return [[]] .

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Backtracking in increasing
# order
# Idea: Choosing plays in non-decreasing order lists every
# multiset exactly once.
def football_combinations(total):
    plays = (2, 3, 6, 7, 8)
    out = []

    def dfs(start, left, path):
        if left == 0:
            out.append(path[:])
            return
```

```
        for i in range(start, len(plays)):
            if plays[i] <= left:
                path.append(plays[i])
                # the same play may be used again, so i
                # stays
                dfs(i, left - plays[i], path)
                path.pop()

    dfs(0, total, [])
    return out
```

59 Permutations — MEDIUM

Template fit: the template as written

QUESTION

Given an array of distinct integers, return all possible permutations of the array. A permutation of an array is a rearrangement of its elements. Example Input: nums = [1, 2, 3]
Output: [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Backtracking with a used array
# Idea: Fill positions left to right. At each position try
# every unused value, mark it, recurse, and unmark. Only
# valid prefixes are ever extended, so every step leads to
# a permutation.
def permute(nums):
    # backtracking: choose an unused element, recurse,
    # undo
    out, cur, used = [], [], [False] * len(nums)

    def go():
        if len(cur) == len(nums):
            out.append(cur[:])
            return
        for i, x in enumerate(nums):
            if not used[i]:
                used[i] = True
                cur.append(x)
                go()
                cur.pop()
                used[i] = False

    go()
    return out
```

60 Determine if a 14-tile hand is winning — MEDIUM

Template fit: the same try-triple-then-run recursion over a count array, after choosing the pair

QUESTION

You are given an integer array **`tiles`** of length 14 representing a Mahjong-like hand. Each integer is a tile value from 1 to 9 (single suit). You may reorder the tiles. A hand is **winning** if it can be partitioned into:

- **1 pair**: two identical tiles, and - **4 melds** (each meld uses 3 tiles), where each meld is either: - a **triplet**: three identical tiles (e.g., 7,7,7), or - a **sequence**: three consecutive values (e.g., 2,3,4). Each tile can be used at most once. Return **`true`** if the hand is winning, otherwise return **`false`**. Assumptions/constraints:
- **`tiles.length == 14`**
- **`1 <= tiles[i] <= 9`**

QUESTION

- Each value can appear multiple times (as in a real tile set).

MAIN CODE | helper functions first, then the answer

```
from collections import Counter


# HELPERS for the base question
def melds_ok(cnt, v=1):
    # can the tiles in cnt (value -> count) form only
    # melds?
    while v <= 9 and cnt[v] == 0:
        v += 1
    if v > 9:
        return True
    # the smallest tile is either in a triplet ...
    if cnt[v] >= 3:
        cnt[v] -= 3
        ok = melds_ok(cnt, v)
        cnt[v] += 3
        if ok:
            return True
    # ... or the start of a run v, v + 1, v + 2
    if v <= 7 and cnt[v + 1] and cnt[v + 2]:
        for t in (v, v + 1, v + 2):
            cnt[t] -= 1
        ok = melds_ok(cnt, v)
        for t in (v, v + 1, v + 2):
            cnt[t] += 1
        return ok
    return False

# BASE QUESTION (optimal): Counts, try each pair, then
# always group the smallest remaining tile
# Idea: Keep counts of 1..9. For each value with at least
# two tiles, take it as the pair.
def is_winning_hand(tiles):
    # one pair plus four melds, single suit, values 1..9
    cnt = Counter({v: 0 for v in range(1, 12)})
    for t in tiles:
        cnt[t] += 1
    for pair in range(1, 10):
        if cnt[pair] >= 2:
            cnt[pair] -= 2
            ok = melds_ok(cnt)
            cnt[pair] += 2
            if ok:
                return True
    return False
```

61 Count Passcode Patterns on a Keypad — MEDIUM

Template fit: the choice is the next adjacent unused key; complete at k keys; count instead of storing

QUESTION

A keypad is a rows x cols grid of keys (the classic phone dial is 3x3):
1 2 3 4 5 6 7 8 9 A passcode pattern is a path across keys where each move goes only to an orthogonally adjacent key (horizontal or vertical — no diagonals), and no key is visited more than once. The order matters: 1 -> 2 -> 3 -> 6 and 6 -> 3 -> 2 -> 1 are different patterns. Given the grid dimensions and an integer k , return the number of distinct passcode patterns that use exactly k keys.

QUESTION

Function signature

def count_patterns(rows: int, cols: int, k: int) -> int:

"""Return the number of distinct passcode patterns using exactly k keys,
moving only to orthogonally adjacent, unvisited keys on a rows x cols grid."""

...

Follow-up (as reported): Count patterns of length at least k instead of exactly k . Hint: sum the DFS counts over all lengths from k to rows*cols .

EXAMPLES

Example 1	
Input	`rows = 3, cols = 3, k = 1`
Output	`9`
Why	`Each of the 9 keys alone is one pattern.`

Example 2	
Input	`rows = 3, cols = 3, k = 2`
Output	`24`
Why	`There are 12 adjacent key pairs; each can be traversed in 2 directions, giving 24 patterns.`

Example 3	
Input	`rows = 3, cols = 3, k = 3`
Output	`44`
Why	`Summing over each possible middle key b, deg(b)*(deg(b)-1) directed 3-key paths: corners give 4*2=8, edge-mids 4*6=24, center 12; total 44.`

CONSTRAINTS

- 1 <= rows, cols <= 4 .
- 1 <= k <= rows * cols .
- A single key (k = 1) is a valid pattern of length 1.
- Count directed patterns (a path and its reverse are counted separately).

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): DFS with backtracking from
# every key
# Idea: Extend the path one adjacent unused key at a time
# and undo the choice afterwards.
def count_patterns(rows, cols, k):
    def dfs(r, c, used, seen):
        if used == k:
            return 1
        total = 0
        for dr, dc in (
            (1, 0),
            (-1, 0),
            (0, 1),
            (0, -1),
        ):
            nr, nc = r + dr, c + dc
            if (
                0 <= nr < rows
                and 0 <= nc < cols
                and (nr, nc) not in seen
```

```
            ):
                seen.add((nr, nc))
                total += dfs(nr, nc, used + 1, seen)
                seen.remove((nr, nc))
        return total

    return sum(
        dfs(r, c, 1, {(r, c)})
        for r in range(rows)
        for c in range(cols)
    )
```

62 Match people to questions using tags and priorities — MEDIUM

Template fit: try each question of a person; if it is taken, recursively ask its owner to move (augmenting path); undo by not recording on failure

QUESTION

Overview: This question evaluates competencies in bipartite matching and combinatorial optimization, along with practical data-structure design for scalable tag-based assignment and priority-aware selection.

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Augmenting paths (Kuhn's
# algorithm)
# Idea: For each person run a DFS that tries each
# acceptable question: if it is free take it, otherwise
# try to move its current owner to another question
# (recursively), using a visited set so no question is
# tried twice in one attempt.
def max_matching(adj):
    # the plain version: adj[p] = list of questions for
    # person p
    owner = {}

    def try_assign(p, seen):
        for q in adj[p]:
            if q in seen:
                continue
            seen.add(q)
            free = q not in owner
            if free or try_assign(owner[q], seen):
                owner[q] = p
                return True
        return False

    people = range(len(adj))
    return sum(try_assign(p, set()) for p in people)
```

63 Matrix Flower Placement with House Adjacency — HARD

Template fit: place row by row; reject a row early when the houses two rows above are still not next to a flower

QUESTION

EXAMPLES

```
H 0 0 0
H 0 0 0
0 0 0 H
0 0 0 0
```

The task is to fill some 0 cells with F while satisfying both the row/column constraint and each house's adjacency requirement.

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))

# BASE QUESTION (optimal): Row-by-row backtracking with
# used-column set and delayed house checks
# Idea: Place at most one flower per row, tracking used
# columns, and only on empty cells ('0').
def place_flowers(grid, exactly_one=False,
                  find_all=False):
    # 'H' house, '0' empty. At most one F per row and
    # column; every house needs an F on a side.
    R, C = len(grid), len(grid[0])
    cells = [list(row) for row in grid]
    houses = [(r, c) for r in range(R) for c in range(C)
              if cells[r][c] == "H"]
    found = []

    def house_ok(r, c):
        # is some neighbour of this house a flower?
        for dr, dc in STEPS:
            a, b = r + dr, c + dc
            if 0 <= a < R and 0 <= b < C \
                    and cells[a][b] == "F":
                return True
        return False

    def row_houses_ok(r):
        return all(house_ok(hr, hc)
                   for hr, hc in houses if hr == r)

    def solve(r, used):
        if found and not find_all:
            return
        # rows up to r-1 are decided, so row r-2 is final
        if r >= 2 and not row_houses_ok(r - 2):
            return
        if r == R:
            full = len(used) == C
            if row_houses_ok(R - 1) and (full or
                                         not exactly_one):
                found.append(["".join(x) for x in cells])
            return
        # try a flower in every free column of this row
        for c in range(C):
            if c not in used and cells[r][c] == "0":
                cells[r][c] = "F"
                solve(r + 1, used | {c})
                cells[r][c] = "0"
        # or leave the row empty (not allowed for exactly
        # one)
        if not exactly_one:
            solve(r + 1, used)

    solve(0, frozenset())
    if find_all:
        return found
    return found[0] if found else None
```

GROUP 8 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE | 2 E 7 M

Runs and groups: extend while it fits, restart or close when it does not

64 Group Timestamps Into Close Intervals — EASY

Template fit: close the group when the gap to the next timestamp is at least k; otherwise extend it

QUESTION

You are given an array of positive integer timestamps in ascending order and an integer k . Partition the timestamps into maximal runs such that within each run, the difference between every pair of adjacent values is strictly less than k . A run breaks wherever an adjacent difference is >= k . Return each run as an interval [start, end] (its first and last value), in order.

Function signature

def group_into_intervals(timestamps: list[int], k: int) -> list[list[int]]:

    ...

Follow-up (as reported): One pass, O(n) time and O(1) extra space (besides the output).

EXAMPLES

Example 1	
Input	`timestamps = [1,2,3,4,11,14,15,17,21,22,23], k = 2`
Output	`[[1,4],[11,11],[14,15],[17,17],[21,23]]`
Why	`Adjacent diffs of 1 stay in a run; a diff of >=2 (e.g. 4->11, 11->14, 15- >17, 17->21) starts a new run.`

Example 2	
Input	`timestamps = [5], k = 100`
Output	`[[5,5]]`
Why	`A single timestamp forms a run by itself.`

Example 3	
Input	`timestamps = [1,2,3], k = 1`
Output	`[[1,1],[2,2],[3,3]]`
Why	`A diff of 1 is not strictly less than k=1, so every element stands alone.`

CONSTRAINTS

- 1 <= len(timestamps) <= 10^5 , values strictly increasing.
- k >= 1 .

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): One pass with a run start
# Idea: Close the run whenever the gap to the previous
# value reaches k.
def group_into_intervals(timestamps, k):
    if not timestamps:
        return []
    runs, start = [], timestamps[0]
    for prev, cur in zip(timestamps, timestamps[1:]):
        if cur - prev >= k:
            runs.append([start, prev])
            start = cur
    runs.append([start, timestamps[-1]])
    return runs
```

65 First Out-of-Order Character

EASY

Template fit: fits = s[i] >= s[i-1]; return i at the first item that does not fit

QUESTION

You are given a string of lowercase English letters. Scanning left to right, the string is considered "in order" as long as each character is greater than or equal to the character before it (a non-decreasing sequence; equal adjacent characters are allowed). Return the index of the FIRST character that breaks this rule — that is, the first index i >= 1 where s[i] < s[i-1] . If the entire string is non-decreasing, return -1 .

Function signature

def first_out_of_order(s: str) -> int:

...

Follow-up (as reported): Solve in a single O(N) pass with O(1) extra space; state the time/space complexity as the interviewer expected.

EXAMPLES

Example 1	
Input	`"abcda"`
Output	`4`
Why	`a<=b<=c<=d holds, but at index 4 'a' < 'd' (s[3]), so 4 is returned.`

Example 2	
Input	`"aabbz"`
Output	`-1`
Why	`Every character is >= the previous one; fully non-decreasing.`

Example 3	
Input	`"cba"`
Output	`1`
Why	`At index 1, 'b' < 'c', the first violation.`

Example 4	
Input	`""`
Output	`-1`
Why	`An empty string is trivially in order.`

CONSTRAINTS

- 0 <= len(s) <= 10^5 s consists only of lowercase English letters.
- An empty string or a single-character string is trivially in order -> return -1 .

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): One pass, compare with the
# previous letter
# Idea: Order is broken exactly where a letter is smaller
# than the one before it.
def first_out_of_order(s):
    for i in range(1, len(s)):
        if s[i] < s[i - 1]:
            return i
    return -1
```

66 Longest Non-Decreasing Subarray with One Replacement

MEDIUM

Template fit: left[i] and right[i] run lengths; the best answer joins the run before and after one changed element

QUESTION

Find the maximum length of a contiguous non-decreasing subarray if at most one element may be changed to any value. Some rounds start from the plain longest-increasing-subarray question and add the one-replacement follow-up; those versions ask for an increasing run, so clarify strict vs non-strict.

Input: an integer array nums.
You may change at most one element to any value.
Return the maximum length of a contiguous subarray that is non-decreasing (nums[i] <= nums[i + 1] for every adjacent pair inside it) after the change.

Variant: other rounds ask for a strictly increasing run and open with the no-replacement version (longest increasing subarray) before adding the one-replacement follow-up — clarify strict vs non-strict before coding.

Expect to discuss and compare the complexity of several approaches; a brute-force first pass is usually followed by a request to optimize.

EXAMPLES

```
nums = [1, 0, 3, 4, 5, 2, 3]
Output: 5
Change nums[1] from 0 to 1 -> [1, 1, 3, 4, 5, 2, 3]
[1, 1, 3, 4, 5] is a non-decreasing subarray of length 5.
```

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
def run_lengths(a, ok):
    n = len(a)
    # left[i]: run ending at i, right[i]: starting at i
    left, right = [1] * n, [1] * n
    for i in range(1, n):
        if ok(a[i - 1], a[i]):
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if ok(a[i], a[i + 1]):
            right[i] = right[i + 1] + 1
    return left, right

# BASE QUESTION (optimal): Left and right run lengths
# Idea: left[i] and right[i] are the non-decreasing run
# lengths ending and starting at i. Changing a[i] gives
# left[i-1] + 1 + right[i+1] if a[i-1] <= a[i+1],
# otherwise max(left[i-1], right[i+1]) + 1.
def longest_with_one_change(a, strict=False):
    n = len(a)
    # nothing to fix in arrays of length 0, 1 or 2
    if n <= 2:
        return n
    def ok(x, y):
        return x < y if strict else x <= y

    # can a value fit between the two neighbours?
    gap = 2 if strict else 0
    left, right = run_lengths(a, ok)
    best = 1
    # try changing each element a[i]
    for i in range(n):
        L = left[i - 1] if i else 0
        R = right[i + 1] if i + 1 < n else 0
        # at an end, or when a value fits between the
        # neighbours, the change joins both runs
```

```
        if not L or not R or a[i + 1] - a[i - 1] >= gap:
            best = max(best, L + 1 + R)
        # otherwise it joins only the longer run
        else:
            best = max(best, max(L, R) + 1)
    return best
```

67 Piano Hand Movement Grouping

MEDIUM

Template fit: start a new group when adding the note would make max - min exceed k

QUESTION

EXAMPLES

```
With notes = [1, 3, 5, 7, 9, 11] and k = 4, the stated output is 2.
Earlier examples use [1,2,3,4,5] -> 0 and [5,9,1] -> 1, without stating the reach parameter.
```

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
def hand_positions(notes, k):
    # split the notes into groups that one hand position
    # (a span of at most k) can play, as early as possible
    groups = []
    for i, x in enumerate(notes):
        if groups:
            start, lo, hi = groups[-1]
            # does the note fit the current span?
            lo2, hi2 = min(lo, x), max(hi, x)
            if hi2 - lo2 <= k:
                groups[-1] = (start, lo2, hi2)
                continue
        # otherwise the hand must move: a new group starts
        groups.append((i, x, x))
    return groups

# BASE QUESTION (optimal): Greedy scan, tracking the
# group's smallest and largest note
# Idea: Keep lo and hi of the current group. Extend while
# hi - lo stays within k, otherwise close the group and
# start a new one at this note.
def groups(notes, k):
    # how many hand placements are needed
    return len(hand_positions(notes, k))
```

68 Find longest increasing contiguous subarray

MEDIUM

Template fit: fits = a[i] > a[i-1]; also keep a left run and a right run per index so one change can join two runs

QUESTION

Overview: This question evaluates proficiency in array processing and algorithmic optimization, focusing on recognition of contiguous subarray properties and handling a single-value modification to achieve strict monotonicity.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Left and right run lengths, try
# every position as the changed element
# Idea: left[i] is the length of the strictly increasing
# run ending at i and right[i] the length of the one
# starting at i.
def longest_one_change(a, integers=True):
```

```
    # strictly increasing after changing AT MOST one value
    n = len(a)
    if n == 0:
        return 0
    # left[i]: run ending at i; right[i]: run starting at
    # i
    left, right = [1] * n, [1] * n
    for i in range(1, n):
        if a[i] > a[i - 1]:
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if a[i] < a[i + 1]:
            right[i] = right[i + 1] + 1
    # the gap needed between two neighbours: 2 for whole
    # numbers (one number must fit between), > 0 for reals
    best = max(left)
    for i in range(n):
        # change a[i] to join the run on its left ...
        if i > 0:
            best = max(best, left[i - 1] + 1)
        # ... or the run on its right ...
        if i < n - 1:
            best = max(best, right[i + 1] + 1)
        # ... or both, when a value fits between the
        # neighbours
        if 0 < i < n - 1:
            room = a[i + 1] - a[i - 1]
            fits = room >= 2 if integers else room > 0
            if fits:
                joined = left[i - 1] + 1 + right[i + 1]
                best = max(best, joined)
    return min(best, n)
```

69 Sum of Good Subarrays (Adjacent Diff ±1) MEDIUM

Template fit: keep the running length and running sum of the current up-run and down-run; add both to the total at every index

QUESTION

MLE L5 coding round: sum over all contiguous subarrays whose elements form a strictly monotonic ±1 step sequence (every adjacent pair differs by exactly +1, OR every adjacent pair differs by exactly -1). Required time complexity is O(N).

Input: integer array arr of length N.
A subarray is good if its consecutive differences are either all +1 (monotone increasing by 1) or all -1 (monotone decreasing by 1).
Singleton subarrays are always good.
Output: the sum of element-sums across all good subarrays (each good subarray contributes the sum of its elements once).
Required: O(N) time.

EXAMPLES

```
arr = [3, 5, 6, 7, 6]
```

Good subarrays

```
  singletons:           [3], [5], [6], [7], [6]
  monotone +1 runs:     [5, 6], [6, 7], [5, 6, 7]
  monotone -1 runs:     [7, 6]
Not good: [3, 5] (diff 2), [6, 7, 6] (mixes +1 then -1).
```

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Two running run totals
# Idea: For an increasing run of length L ending at i let
# S be the sum of all suffix sums of the run. When the
# next element x extends it, S becomes S + (L + 1) * x and
# L becomes L + 1; a break restarts at S = x, L = 1.
def good_subarray_sum(a):
    # a good subarray has all steps +1 or all steps -1.
    # For each index keep, per direction, the sum of the
    # sums of all good subarrays that END here.
    total = 0
    up_sum = up_len = down_sum = down_len = 0
    for i, x in enumerate(a):
        step = x - a[i - 1] if i else None
        if step == 1:
            # extend the +1 run: every old subarray grows
            # by
            # x, and [x] alone is new: (len + 1) * x in
            # all
            up_sum += (up_len + 1) * x
            up_len += 1
        else:
            up_sum, up_len = x, 1
        if step == -1:
            down_sum += (down_len + 1) * x
            down_len += 1
        else:
            down_sum, down_len = x, 1
        # [x] is in both totals: count it once
        total += up_sum + down_sum - x
    return total
```

70 Sustained-Error Time Window — MEDIUM

Template fit: from the given second extend left and right while errors >= baseline; compare the time span with the interval

QUESTION

Reconstructing the "two arrays - Error and Time" question. You are given two parallel arrays: times (timestamps in seconds, strictly increasing) and errors (the error reading recorded at each timestamp), where errors[i] was recorded at times[i] . You are also given three integers: second , interval , and baseline . Determine whether there exists a contiguous run of readings that:

1. Includes the reading taken at timestamp second (guaranteed to exist exactly once in times), 2. Has every reading in the run satisfy error >= baseline , and 3. Spans a duration of at least interval seconds — that is, times[right] - times[left] >= interval , where left and right are the endpoints of the run. Return True if such a run exists, otherwise False . Note that if the reading at second is itself below baseline , the answer is immediately False .

Function signature

```
def has_sustained_error_window(
    times: list[int],
    errors: list[int],
    second: int,
    interval: int,
    baseline: int,
) -> bool:
    """Return True if a run of readings all >= baseline includes second and spans >=
interval seconds."""
    ...
```

EXAMPLES

QUESTION

Example 1	
Input	`times = [1, 2, 3, 4, 5], errors = [10, 20, 5, 30, 40], second = 1, interval = 1, baseline = 8`
Output	`True`
Why	`Starting at time 1 (err 10 >= 8) the run extends to time 2 (err 20 >= 8) but stops at time 3 (err 5 < 8). The run covers times 1..2, a span of 1 >= interval 1.`

Example 2	
Input	`times = [1, 2, 3, 4, 5], errors = [10, 20, 5, 30, 40], second = 4, interval = 2, baseline = 8`
Output	`False`
Why	`Time 4 (err 30 >= 8) can only extend right to time 5 (err 40 >= 8); time 3 (err 5) blocks going left. The run covers times 4..5, span 1 < interval 2.`

Example 3	
Input	`times = [1, 2, 3], errors = [10, 4, 10], second = 2, interval = 0, baseline = 8`
Output	`False`
Why	`The reading at second=2 is 4 < baseline 8, so no valid run includes it.`

CONSTRAINTS

- 1 <= len(times) == len(errors) <= 10^5 times is strictly increasing; 0 <= times[i] <= 10^9
- -10^9 <= errors[i] <= 10^9 , baseline in the same range second is one of the values in times ; 0 <= interval <= 10^9

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Expand around the reading
# Idea: The longest valid run around the index is the best
# one, so only its span matters.
def has_sustained_error_window(
    times, errors, second, interval, baseline
):
    at = times.index(second)
    if errors[at] < baseline:
        return False
    left = right = at
    while left > 0 and errors[left - 1] >= baseline:
        left -= 1
    while (
        right + 1 < len(times)
        and errors[right + 1] >= baseline
    ):
        right += 1
    return times[right] - times[left] >= interval
```

71 Longest Non-Decreasing Subarray From Two Arrays — MEDIUM

Template fit: two run lengths per index, one for each array you may take the value from

QUESTION

You are given two integer arrays nums1 and nums2 , both of length n . You must construct a third array nums3 of length n where, for each index i , nums3[i] is chosen to be either nums1[i] or nums2[i] (chosen independently per index). Return the length of the longest non-decreasing subarray (a contiguous run) achievable in nums3 over all valid choices.

QUESTION

Function signature

def longest_non_decreasing_subarray(nums1: list[int], nums2: list[int]) -> int:

...

Follow-up (as reported): (Reported) The interviewer first asked for the longest contiguous non-decreasing subarray of a single array, then generalized to picking per index from two arrays; also asked for the time complexity and to write test cases.

EXAMPLES

Example 1	
Input	`nums1 = [2, 3, 1], nums2 = [1, 2, 1]`
Output	`2`
Why	`Choosing nums3 = [2, 3, ...] gives the run [2, 3] of length 2. No choice yields a non-decreasing run of length 3.`

Example 2	
Input	`nums1 = [1, 3, 2, 1], nums2 = [2, 2, 3, 4]`
Output	`4`
Why	`nums3 = [1, 2, 3, 4] (pick nums1[0], nums2[1], nums2[2], nums2[3]) is non-decreasing over all 4 positions.`

Example 3	
Input	`nums1 = [1, 1], nums2 = [2, 2]`
Output	`2`
Why	`Any choice, e.g. [1, 1], is non-decreasing across both.`

CONSTRAINTS

- 1 <= n <= 10^5
- nums1 and nums2 have the same length n .
- 1 <= nums1[i], nums2[i] <= 10^9
- Target complexity: O(n) time, O(1) extra space.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Two-state DP
# Idea: dp1 and dp2 are the best run ending at i using
# nums1[i] or nums2[i].
def longest_non_decreasing_subarray(nums1, nums2):
    dp1 = dp2 = best = 1
    for i in range(1, len(nums1)):
        n1 = n2 = 1
        for prev_val, prev_len in (
            (nums1[i - 1], dp1),
            (nums2[i - 1], dp2),
        ):
            if prev_val <= nums1[i]:
                n1 = max(n1, prev_len + 1)
            if prev_val <= nums2[i]:
                n2 = max(n2, prev_len + 1)
        dp1, dp2 = n1, n2
        best = max(best, dp1, dp2)
    return best
```

72 Count Super Streaks

MEDIUM

Template fit: sort by time; the same type within T continues the streak; when it breaks, count it if size >= N and span >= X

QUESTION

A player produces a list of events, each (event_id, event_type, event_time) . Sort events by event_time . A streak is a maximal run of events, consecutive in time order, in which every event has the same event_type AND the gap between each event and the previous one in the run is at most T seconds. A streak ends when the next event has a different event_type or its gap from the previous event exceeds T . A streak is a Super Streak if it contains at least N events AND spans at least X seconds (span = last event time minus first event time within the streak). Return the number of Super Streaks.

Function signature

def count_super_streaks(events: list[tuple[int, str, int]],
N: int, T: int, X: int) -> int:
...

Follow-up (as reported): Follow-up 1: events are tagged with user_id ; return a per-user count of Super Streaks. Follow-up 2: after processing, support count_streaks(user_id, time_A, time_B) returning how many of that user's Super Streaks completed entirely within [time_A, time_B] . Store each user's Super-Streak end-times sorted and answer with two binary searches in O(log n) per query.

EXAMPLES

Example 1	
Input	`events = [(1,"A",0),(2,"A",10),(3,"A",20),(4,"A",30),(5,"A",40)], N=4, T=10, X=30`
Output	`1`
Why	`One streak of 5 A-events, gaps all 10 (<=T), span 40 (>=30), size 5 (>=4). Super.`

Example 2	
Input	`events = [(1,"A",0),(2,"A",10),(3,"A",20),(4,"A",30),(5,"A",40), (6,"A",55),(7,"A",56),(8,"A",57)], N=4, T=10, X=30`
Output	`1`
Why	`The jump 40->55 (gap 15 > T) breaks the run. The first streak [0..40] is Super; the second [55,56,57] has only 3 events and span 2, so not Super.`

Example 3	
Input	`events = [(1,"A",0),(2,"B",5),(3,"A",10)], N=2, T=10, X=1`
Output	`0`
Why	`Type changes each step, so every streak has length 1 — none reaches N=2.`

CONSTRAINTS

- 0 <= len(events) <= 10^5 .
- N >= 1 , T >= 0 , X >= 0 ; times are non-negative integers, possibly with duplicate timestamps.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): One pass over the sorted events
# Idea: Keep only the open streak's start, last time, size
# and type.
def count_super_streaks(events, N, T, X):
    count = 0
    start = prev = kind = None
    size = 0
    for _, etype, etime in sorted(
        events, key=lambda e: e[2]
    ):
```

```
        if kind == etype and etime - prev <= T:
            size += 1
        else:
            if size >= N and prev - start >= X:
                count += 1
            start, kind, size = etime, etype, 1
        prev = etime
    if size >= N and prev - start >= X:
        count += 1
    return count
```

Union-find: merge the pairs, then ask who is together

73 Count Islands in a Grid and Track the Count as Land Cells Are Added

MEDIUM

Template fit: each new land cell: groups += 1, then union it with every land neighbour (groups -= 1 per real merge); report groups after each addition

QUESTION

A map is a grid with **`m`** rows and **`n`** columns in which every cell is land (**`1`**) or water (**`0`**). An island is a maximal group of land cells connected horizontally or vertically. Cells that touch only at a corner are not connected. You are given the initial map and a sequence of additions, each of which turns one cell into land. Report the number of islands before any addition, and again after each addition.

Function Signature

```
def island_counts(grid: list[list[int]], additions: list[tuple[int, int]]) -> list[int]:
```

Rules

- **`grid[r][c]`** is **`1`** for land and **`0`** for water, where **`r`** is the row index and **`c`** is the column index.
- An addition **`(r, c)`** turns cell **`(r, c)`** into land. If that cell is already land, either initially or because of an earlier addition, the map does not change.
- Additions are applied in the given order. Land never turns back into water.
- Return a list of length **`len(additions) + 1`**. Element **`0`** is the number of islands in the initial grid. For **`i >= 1`**, element **`i`** is the number of islands after the first **`i`** additions have been applied.

Constraints

- **`1 <= m <= 1000`**, **`1 <= n <= 1000`** and **`m * n <= 200000`**
- Every row of **`grid`** has exactly **`n`** entries, each **`0`** or **`1`**.
- **`0 <= len(additions) <= 200000`**
- Every addition **`(r, c)`** satisfies **`0 <= r < m`** and **`0 <= c < n`**. The same cell may appear more than once.
- Every count is at most **`m * n`**, so all values fit in a 32-bit signed integer.

Examples

Example 1

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = []
Output: [3]
```

The islands are {(0, 0), (0, 1)}, {(1, 3), (2, 3)} and {(2, 0)}. With no additions, the only element is the initial count.

Example 2

```
Input: grid = [[0, 0, 0], [0, 0, 0], [0, 0, 0]], additions = [(0, 0), (0, 1), (1, 2), (2, 1), (1, 1), (1, 1)]
Output: [0, 1, 1, 2, 3, 1, 1]
```

The grid starts with no land. (0, 0) creates an island and (0, 1) joins it. (1, 2) and (2, 1) each touch no land, so each creates a new island. (1, 1) touches all three islands and merges them into one. The second (1, 1) is already land, so nothing changes.

Example 3

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = [(0, 2), (0, 3), (1, 0)]
Output: [3, 3, 2, 1]
```

QUESTION

(0, 2) joins the top-left island. It touches (1, 3) only at a corner, so the count stays 3. (0, 3) connects the top-left island to the island at (1, 3) and (2, 3), giving 2. (1, 0) connects the merged island to (2, 0), giving 1.

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))

# BASE QUESTION (optimal): Union-Find updated cell by cell
# Idea: Build the initial islands with unions (count
# them), then for each addition: ignore it if the cell is
# already land; otherwise make it a new set (count + 1)
# and union it with each land neighbour, subtracting one
# from the count every time two different sets merge.
class Islands:
    def __init__(self):
        # land cell -> parent cell, and sizes of the roots
        self.parent, self.size = {}, {}
        self.count, self.largest = 0, 0

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add(self, r, c):
        if (r, c) in self.parent:
            return
        self.parent[(r, c)] = (r, c)
        self.size[(r, c)] = 1
        self.count += 1
        self.largest = max(self.largest, 1)
        # join with every land neighbour
        for dr, dc in STEPS:
            nb = (r + dr, c + dc)
            if nb in self.parent:
                self.union((r, c), nb)

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return
        # union by size: the small island joins the big
        # one
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]
```

```
        self.count -= 1
        self.largest = max(self.largest, self.size[a])

# BASE QUESTION (optimal): Union-Find updated cell by cell
# Idea: Build the initial islands with unions (count
# them), then for each addition: ignore it if the cell is
# already land; otherwise make it a new set (count + 1)
# and union it with each land neighbour, subtracting one
# from the count every time two different sets merge.
def island_counts(grid, additions):
    # counts before any addition, then after each one
    isl = Islands()
    for r, row in enumerate(grid):
```

```
        for c, v in enumerate(row):
            if v == 1:
                isl.add(r, c)
    out = [isl.count]
    for r, c in additions:
        isl.add(r, c)
        out.append(isl.count)
    return out
```

74 2D Points Connected Components by Radius — MEDIUM

Template fit: pairs = points within distance r, found by bucketing points into cells of size r and checking only the 9 nearby cells; answer = groups

QUESTION

Given 2D points and radius **r**, connect two points when their distance is at most **r**; connectivity is transitive, and the output is the final number of clusters. The interviewer accepted or preferred a brute-force neighbor scan after discussing optimization ideas.

Input: a list of 2D points and a radius r.
Two points are connected if their Euclidean distance is <= r.
Connectivity is transitive: if A connects to B and B connects to C, all three belong to the same cluster.
Return the number of final clusters.

MAIN CODE | helper functions first, then the answer

```
from collections import defaultdict


# HELPERS for the base question
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)

def near(p, q, r):
    # squared distance: no square root, exact for ints
    dx, dy = p[0] - q[0], p[1] - q[1]
    return dx * dx + dy * dy <= r * r

# BASE QUESTION (optimal): Union-find with a spatial grid
# Idea: Put each point in the cell (x // r, y // r). Two
# points within distance r are in the same or adjacent
# cells, so only those 9 cells are compared.
def count_clusters(points, r):
    # grid of cells r wide: only 9 cells need a look
    size = r if r > 0 else 1
    cells = defaultdict(list)
    for i, (x, y) in enumerate(points):
        cells[(x // size, y // size)].append(i)
    uf = UnionFind(len(points))
    for i, (x, y) in enumerate(points):
        cx, cy = x // size, y // size
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
```

```
                cell = cells.get((cx + dx, cy + dy), ())
                for j in cell:
                    q = points[j]
                    if j > i and near((x, y), q, r):
                        uf.union(i, j)
    return len({uf.find(i) for i in range(len(points))})
```

75 Count Clusters of 2D Points Within a Radius

MEDIUM

Template fit: identical to the line above: cell buckets give the candidate pairs, answer = groups

QUESTION

Overview: This question tests graph traversal skills, specifically identifying connected components among a set of 2D points using distance-based adjacency. It assesses the ability to model a geometric problem as a graph and apply DFS, BFS, or Union-Find, a common evaluation of algorithmic thinking in coding interviews. It also probes awareness of efficiency trade-offs when scaling to large point sets.

MAIN CODE | helper functions first, then the answer

```
from collections import Counter, defaultdict


# HELPERS for the base question
class Clusters:
    """Points arrive one by one; count the clusters."""

    def __init__(self, r):
        self.r = r
        # grid cell (side r) -> indices of points inside
        self.cells = defaultdict(list)
        self.points, self.parent = [], []
        self.components = 0

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def near(self, p, q):
        dx, dy = p[0] - q[0], p[1] - q[1]
        return dx * dx + dy * dy <= self.r * self.r

    def add(self, x, y):
        i = len(self.points)
        self.points.append((x, y))
        self.parent.append(i)
        self.components += 1
        size = self.r or 1
        cx, cy = x // size, y // size
        self.cells[(cx, cy)].append(i)
        # look only at the 9 cells around the new point
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                key = (cx + dx, cy + dy)
                cell = self.cells.get(key, ())
                for j in cell:
                    q = self.points[j]
                    if j != i and self.near((x, y), q):
                        a, b = self.find(i), self.find(j)
                        if a != b:
                            self.parent[a] = b
                            self.components -= 1
```

```
        return self.components
```

```
    def largest(self):
        sizes = Counter(self.find(i)
                        for i in range(len(self.points)))
        return max(sizes.values(), default=0)

# BASE QUESTION (optimal): Grid hashing with cells of side
# r, then Union-Find
# Idea: Put each point in the cell (x // r, y // r). A
# point within distance r of another can only be in the
# same cell or one of the 8 neighbouring cells, so compare
# only with those.
def count_clusters(points, r):
    c = Clusters(r)
    for x, y in points:
        c.add(x, y)
    return c.components
```

76 Find A Threshold-Limited Path With Minimum Required Safety MEDIUM

Template fit: pairs = edges sorted from the highest safety down; union until find(s) == find(t); the answer is the safety of that last edge

QUESTION

Overview: Practice a graph path interview problem that asks for the minimum required safety value along a threshold-limited path. The prompt targets minimax path reasoning, filtered edges, Dijkstra-style relaxation, disconnected graphs, and edge-case handling.

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)

# BASE QUESTION (optimal): Union-Find adding edges from
# strongest to weakest
# Idea: Sort the edges by safety, strongest first, and
# union their endpoints one by one.
def best_path_safety(n, edges, s, t, threshold=None):
    # edges are (u, v, safety). The safety of a path is
    # its
    # weakest edge; return the best safety, or -1 if s and
    # t
    # are not connected through edges >= threshold
    if s == t:
        return float("inf")
    uf = UnionFind(n)
    # add the safest edges first: the edge that joins s
    # and t
    # is the weakest edge of the best path
    for u, v, w in sorted(edges, key=lambda e: -e[2]):
        if threshold is not None and w < threshold:
```

```
            break
        uf.union(u, v)
        if uf.find(s) == uf.find(t):
            return w
    return -1
```

77 Movie Recommendations via Transitive Similarity

MEDIUM

Template fit: pairs = similar movies (use a dict instead of a list); group = every movie with the same root as the target, sorted by rating

QUESTION

You are given a target movie name, a dictionary ratings mapping each movie to its numeric rating, an integer n , and a list similarities of [movieX, movieY] pairs meaning the two movies are directly similar. Similarity is transitive: if A is similar to B and B is similar to C, then A, B, and C all belong to the same similarity group. Return up to n recommended movies from the target's similarity group, excluding the target itself, ordered by rating descending (break ties alphabetically by name). If n == 1 , this returns the single highest- rated similar movie. If the group has fewer than n other movies, return all of them.

Function signature

def recommend(target: str, ratings: dict[str, float], n: int,
similarities: list[list[str]]) -> list[str]:
...

EXAMPLES

Example 1	
Input	`target = "A", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 1, similarities = [["A", "B"], ["B", "C"]]`
Output	`["C"]`
Why	`A, B, C form one group. Excluding A, the highest rated is C (9).`

Example 2	
Input	`target = "A", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 2, similarities = [["A", "B"], ["B", "C"]]`
Output	`["C", "B"]`
Why	`Similar to A are {B (5), C (9)}; sorted by rating descending -> [C, B].`

Example 3	
Input	`target = "D", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 3, similarities = [["A", "B"], ["B", "C"]]`
Output	`[]`
Why	`D is in no similarity pair, so its group is just {D}; excluding D leaves nothing.`

CONSTRAINTS

- Every movie mentioned appears as a key in ratings .
- 1 <= n ; ratings are numbers, names are distinct strings.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Union-find and a sort
# Idea: The target's component is its group; rank it by
# rating.
def recommend(target, ratings, n, similarities):
    parent = {}
```

```
    def find(x):
        parent.setdefault(x, x)
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for x, y in similarities:
        parent[find(x)] = find(y)
    root = find(target)
    group = [
        m
        for m in list(parent)
        if m != target and find(m) == root
    ]
    group.sort(key=lambda m: (-ratings[m], m))
    return group[:n]
```

78 Determine Reporting Relationships

MEDIUM

Template fit: pairs = peer links; keep the boss at the root and carry it over when two groups merge

QUESTION

Build an in-memory organization model that supports three operations on employee IDs:

1. **`add_manager(a, b)`**: employee **a** is the direct manager of employee **b**.
2. **`add_peer(a, b)`**: employees **a** and **b** share the same direct manager.
3. **`is_manager(a, b)`**: return whether employee **a** is a manager of employee **b**. The answer should account for relationships that can be inferred from prior operations. For example, if **`add_manager(m, x)`** and **`add_peer(x, y)`** were previously recorded, then **`is_manager(m, y)`** should return **`true`**. You may assume the input is logically consistent and does not create invalid cycles in the org chart. Explain how you would design the data structures and implement these operations efficiently.

Implement an in-memory organization model that processes operations on employee IDs in order. Each operation is a tuple **`(op, a, b)`** where:

- **`("add_manager", a, b)`**: employee **a** becomes the direct manager of employee **b**.
- **`("add_peer", a, b)`**: employees **a** and **b** share the same direct manager.
- **`("is_manager", a, b)`**: ask whether employee **a** is a direct or indirect manager of employee **b**. Relationships can be inferred from earlier operations. For example, after **`add_manager(m, x)`** and **`add_peer(x, y)`**, **`is_manager(m, y)`** should be **`True`**. Process the operations sequentially and return a list of answers for all **`is_manager`** queries, in order. Notes:
- The input is logically consistent.
- No operation creates a cycle in the org chart.
- Repeating the same valid relationship is allowed.
- An employee is not considered their own manager.

Constraints
0 <= len(operations) <= 20000Employee IDs are integers in the range [-10^9, 10^9]At most 10000 distinct employee IDs appearThe sequence of operations is logically consistent and does not create cycles

EXAMPLES

```
Input: ([("add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager", 1, 3)])
Expected Output: [True]
```

Explanation: Employee 2 reports to 1. Since 2 and 3 are peers, 3 also reports to 1.

QUESTION

```
Input: [("add_manager", 2, 4), ("is_manager", 1, 4), ("add_manager", 1, 2), ("is_manager", 1, 4),
    ("is_manager", 2, 4), ("is_manager", 4, 2)])
Expected Output: [False, True, True, False]
```

Explanation: Before 2 gets a manager, 1 is not a manager of 4. After adding 1 as manager of 2, 1 becomes an indirect manager of 4. Hints**`add_peer(a, b)`** means **a** and **b** share one direct manager, but they are not interchangeable as managers. Group such employees with Union-Find.Once a peer group learns its direct manager, every current member of that group gets the same parent pointer. To answer **`is_manager(a, b)`**, walk upward from **b** through parent pointers.

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Union-Find for peer groups with
# a manager stored on each group
# Idea: Each employee starts in its own group. add_peer
# unions the two groups (and keeps the manager, if either
# has one). add_manager sets the manager of b's group to
# a.
class Org:
    def __init__(self):
        # union-find over peers (people with the same
        # manager)
        self.parent = {}
        # peer-group root -> the manager of that group
        self.boss = {}

    def find(self, x):
        self.parent.setdefault(x, x)
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add_peer(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return
        # two groups with different managers contradict
        if ra in self.boss and rb in self.boss \
                and self.boss[ra] != self.boss[rb]:
            raise ValueError("conflicting managers")
        self.parent[rb] = ra
        # the merged group keeps whichever manager is
        # known
        if rb in self.boss and ra not in self.boss:
            self.boss[ra] = self.boss[rb]

    def add_manager(self, a, b):
        # a is the direct manager of b (and b's peers)
        self.find(a)
        root = self.find(b)
        if root in self.boss and self.boss[root] != a:
            raise ValueError("conflicting managers")
        self.boss[root] = a

    def manager_of(self, x):
        return self.boss.get(self.find(x))
```

```python
    def is_manager(self, a, b):
        # climb b's chain of managers; a visited set stops
        # cycles
        seen, cur = set(), b
        while cur is not None and cur not in seen:
```

```
            seen.add(cur)
            cur = self.manager_of(cur)
            if cur == a:
                return True
        return False
```

79 Determine whether two nodes are related

MEDIUM

Template fit: pairs = the given edges; the query is find(a) == find(b)

QUESTION

Overview: This question evaluates understanding of graph representations, reachability and connectivity concepts for ancestry relationships, including handling disconnected components and potential cycles.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Union-Find built once,
# constant-time queries
# Idea: Union the endpoints of every relationship; two
# nodes are related exactly when they have the same root.
class Relations:
    """Related = joined by a chain of relations."""

    def __init__(self, n, edges=()):
        self.parent = list(range(n))
        self.size = [1] * n
        for u, v in edges:
            self.union(u, v)

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        # relations can arrive over time: near-constant
        # time
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return
        # the smaller group joins the bigger one
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]

    def related(self, a, b):
        return self.find(a) == self.find(b)
```

80 Friend Graph with Timeline (Friend / Unfriend Events)

HARD

Template fit: pairs = friend events sorted by time; the answer is the time of the union that makes groups == 1

QUESTION

EXAMPLES

```
Base: N=3, events [(1,A,B,friend), (5,B,C,friend)] -> answer 5.
Follow-up 1: after the above events, query (0, A, C) -> false, (5, A, C) -> true.
```

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
def by_time(events):
    # events are (time, a, b, "friend" or "unfriend")
    return sorted(events, key=lambda e: e[0])

class TimedUnionFind:
    """Union by size, no path compression, so every
    link keeps the time it was made."""

    def __init__(self, n):
        # every person starts alone in a group of one
        self.parent = list(range(n))
        # sizes decide which group hangs under which
        self.size = [1] * n
        # time a node was linked under its parent
        self.link_time = [0] * n
        # how many separate groups exist right now
        self.groups = n

    def find(self, x):
        # walk up to the leader of x's group
        while self.parent[x] != x:
            x = self.parent[x]
        return x

    def union(self, a, b, t):
        # find the leaders of both groups
        ra, rb = self.find(a), self.find(b)
        # already in one group: nothing changes
        if ra == rb:
            return False
        # make ra the bigger group
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        # hang the smaller group under the bigger one
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        # remember when this link appeared
        self.link_time[rb] = t
        # two groups became one
        self.groups -= 1
        return True

    def connected_at(self, a, b, t):
        # a person is always connected to themselves
        if a == b:
            return True
        # climb from a, noting the newest link
        seen, newest, x = {a: 0}, 0, a
        while self.parent[x] != x:
            newest = max(newest, self.link_time[x])
            x = self.parent[x]
            seen[x] = newest
        # climb from b until we reach an ancestor of a
        newest, y = 0, b
        while y not in seen:
            # b's leader reached without meeting a
            if self.parent[y] == y:
                return False
            newest = max(newest, self.link_time[y])
            y = self.parent[y]
        # connected at t if all path links are <= t
        return max(newest, seen[y]) <= t

# BASE QUESTION (optimal): earliest time everyone is
# connected (Union-Find on sorted events)
```

```python
# Idea: Sort by time and union the two people for each
# friend event. The first time the component count reaches
# 1 is the answer; -1 if it never does.
def earliest_all_connected(n, events):
    # zero or one person is connected from the start
    if n <= 1:
        return 0
    uf = TimedUnionFind(n)
    # handle friendships in time order
    for t, a, b, kind in by_time(events):
        # the base question has friend events only
        if kind == "friend":
            uf.union(a, b, t)
            # one group left: everyone is connected now
            if uf.groups == 1:
                return t
    # the groups never merged into one
    return -1
```

A dict of per-key state that you update, expire, or rank

81 Robot Status Message Deduplication Window

EASY

Template fit: the template as written: show a message only if it was not shown in the last 10 seconds

QUESTION

Process timestamped robot messages and display a message only when it has not been displayed within the prior 10 seconds. Hidden arrivals do not refresh the last-displayed timestamp, and the exact 10-second boundary must be clarified.

Process a stream of (time, message) status events from a robot.
If the same message was displayed within the past 10 seconds, hide the new event.
Otherwise display it and update that message's last-displayed time.
Track the last time each message was displayed, not the last time it was received. A hidden duplicate must not refresh the window.
Clarify whether an event exactly 10 seconds after the prior display is allowed: the comparison may be > 10 or >= 10.

Logger API variant

Implement shouldPrintMessage(timestamp, message), returning true when the message has not been printed in the past 10 seconds and false otherwise.
Handle timestamps arriving out of order, including timestamp 12 arriving before timestamp 10.

EXAMPLES

```
time=10, message="solar panel activated" -> show
time=13, message="solar panel activated" -> hide
time=21, message="solar panel activated" -> show
```

The event at 21 compares against the display at 10, not the hidden arrival at 13.

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Hash map of last displayed time
# Idea: If the message was displayed less than `window`
# seconds ago hide it without changing the stored time;
# otherwise display it and store the new time.
class Dedup:
    """Chronological stream: remember the last display."""

    def __init__(self, window=10, boundary_allowed=True):
        self.window, self.last = window, {}
        # True: exactly `window` seconds later may show
        self.boundary_allowed = boundary_allowed

    def too_soon(self, gap):
        # is this gap still inside the hiding window?
        if self.boundary_allowed:
            return gap < self.window
        return gap <= self.window

    def show(self, t, msg):
        # only a displayed event starts a new window
        last = self.last.get(msg)
        if last is not None and self.too_soon(t - last):
            return False
        self.last[msg] = t
        return True
```

82 Active Communication Top Users

MEDIUM

Template fit: a set of seen pairs, a Counter of partners per user, and buckets by count to read the top users in O(1)

QUESTION

Process a continuous stream of user pairs such as **`A-B`** and **`B-C`**; each pair creates an active communication edge, and the system should output users ordered by current communication count. Follow-ups add self-communication handling, duplicate communication dedupe, and top-K users by count. An additional API variant exposes register and mostActiveUser(), then asks for constant-time most-active lookup and a changing-K ranking extension.SWEhashmapsetsortingtop-kstreamingmediumFrequencyLowLast asked2026-09-22Stagephone-screen

MAIN CODE | helper functions first, then the answer

```python
import heapq
from collections import Counter, defaultdict


# BASE QUESTION (optimal): Edge set, counts per user,
# count buckets for the maximum
# Idea: A new edge increments the count of both users.
# Buckets map each count to the set of users with it, so a
# user moves from bucket c to bucket c + 1.
class Active:
    def __init__(self, count_self=False):
        # distinct edges seen so far (for the dedupe rule)
        self.edges = set()
        # user -> how many different partners
        self.count = Counter()
        # count -> users having exactly that count
        self.bucket = defaultdict(set)
        # the highest count right now (it never drops)
        self.top = 0
        self.count_self = count_self

    def bump(self, user):
        c = self.count[user]
        # move the user from bucket c to bucket c + 1
        if c:
            self.bucket[c].discard(user)
        self.count[user] = c + 1
        self.bucket[c + 1].add(user)
        self.top = max(self.top, c + 1)

    def add(self, a, b):
        # A-B and B-A are the same edge
        pair = (min(a, b), max(a, b))
        # follow-ups: skip self pairs (unless counted)
        # and skip a repeated pair
        skip_self = a == b and not self.count_self
        if skip_self or pair in self.edges:
            return
        self.edges.add(pair)
        self.bump(a)
        # a self pair counts once, not twice
        if a != b:
            self.bump(b)

    def most_active(self):
```

```python
        # O(1): read the top bucket
        if not self.top:
            return set()
        return set(self.bucket[self.top])

    def top_k(self, k):
```

```
        # k users by count, ties broken by name
        def rank(u):
            return (-self.count[u], u)
        return heapq.nsmallest(k, self.count, key=rank)
```

83 Filter Repeated Robot Status Messages

MEDIUM

Template fit: the same rule applied to a stream of (timestamp, message) pairs; keep what is accepted

QUESTION

Overview: Filter a timestamped robot status stream so a message reappears only when strictly more than ten seconds have passed since its last displayed occurrence. Preserve input order, track each exact case-sensitive message independently, and ensure hidden repeats do not reset the interval.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Dictionary from message to the
# time it was last shown
# Idea: For each message in order: if it was never shown
# or now - last > 10, emit it and set last = now;
# otherwise drop it and leave last unchanged. One
# dictionary lookup per message.
def filter_status(stream, gap=10):
    # show a message only if MORE than `gap` seconds
    # passed
    # since it was last shown; hidden repeats change
    # nothing
    last, shown = {}, []
    for ts, msg in stream:
        if msg not in last or ts - last[msg] > gap:
            last[msg] = ts
            shown.append((ts, msg))
    return shown
```

84 Next-Word Predictor

MEDIUM

Template fit: defaultdict(Counter): counts[a][b] += 1 for neighbouring words; remember the best next word per word

QUESTION

Process training data and predict the most frequent next word for a supplied word. Maintain prediction state during processing so prediction can run in constant time.SWEstringhashmapdata-structureFrequencySingle reportLast asked2026-09-02Stageonsite-coding

MAIN CODE | helper functions first, then the answer

```
from collections import Counter, defaultdict


# BASE QUESTION (optimal): Counts plus a maintained best
# successor
# Idea: counts[word][next] is incremented for every
# adjacent pair. After the increment compare that one
# count with the current best for the word (higher count
# wins, then the smaller word).
class Predictor:
    def __init__(self, orders=(2, 1)):
        # orders: how many previous words are the context
        self.orders = sorted(orders, reverse=True)
        # order -> context tuple -> Counter of next words
        self.counts = {k: defaultdict(Counter)
```

```
                    for k in orders}
        # order -> context tuple -> best next word so far
        self.best = {k: {} for k in orders}

    def train(self, sentence):
        words = sentence.split()
        for k in self.orders:
            for i in range(len(words) - k):
                ctx = tuple(words[i:i + k])
                nxt = words[i + k]
                self.update(k, ctx, nxt)

    def update(self, k, ctx, nxt):
        table = self.counts[k][ctx]
        table[nxt] += 1
        cur = self.best[k].get(ctx)
        # more counts win; a tie goes to the smaller word
        better = cur is None or table[nxt] > table[cur]
        tie = cur is not None and table[nxt] == table[cur]
        if better or (tie and nxt < cur):
            self.best[k][ctx] = nxt

    def predict(self, *context):
        # O(1): try the longest context, then back off
        for k in self.orders:
            if len(context) >= k:
                ctx = tuple(context[len(context) - k:])
                if ctx in self.best[k]:
                    return self.best[k][ctx]
        return None
```

85 Batch and Streaming Log Deduplication

MEDIUM

Template fit: dict message -> first (or latest) timestamp depending on the policy; sort the result at the end

QUESTION

Process **`(timestamp, message)`** logs by deduplicating on message and sorting retained entries by timestamp, first keeping the first-seen entry and then the latest timestamp. The streaming extension adds **`add(timestamp, message)`** and **`get_next()`**, requiring a map plus min-heap, lazy deletion, and a discussion of watermarks and bounded state.SWEdedupstreamingheapsortingmediumFrequencySingle reportLast asked2026-07-23Stageonsite-coding

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Dictionary on the message, then
# sort
# Idea: One pass with a dict from message to the retained
# timestamp (first seen: keep the first; latest: replace
# when the new timestamp is larger), then sort the
# retained entries for the output.
def dedupe(logs, policy="first"):
    # logs: (timestamp, message) pairs; keep one per
    # message
    kept = {}
    for ts, msg in logs:
        # "first" keeps the first seen, "latest" the
        # newest
        if msg not in kept or (
                policy == "latest" and ts > kept[msg]):
            kept[msg] = ts
    # retained entries sorted by time, then message
    return sorted((ts, msg) for msg, ts in kept.items())
```

86 Design a logger that returns top-K messages

MEDIUM

Template fit: dict message -> list of timestamps; top_k counts each list inside the time range with bisect, then takes the k largest

QUESTION

Overview: This question evaluates skills in designing in-memory data structures and algorithms for frequency counting and interval querying, including extracting top-K elements with a heap, within the Coding & Algorithms domain.

MAIN CODE | helper functions first, then the answer

```python
import heapq
from bisect import bisect_left, bisect_right
from collections import defaultdict


# BASE QUESTION (optimal): Per-message sorted timestamps
# plus a heap for the top K
# Idea: log appends the timestamp to the message's list
# (timestamps arrive in order, so lists stay sorted). A
# window count for one message is bisect_right(end) -
# bisect_left(start).
class Logger:
    def __init__(self):
        # message -> its timestamps, in order of arrival
        self.times = defaultdict(list)

    def log(self, ts, msg):
        self.times[msg].append(ts)

    def top_k(self, k, start=None, end=None):
        # the k most frequent messages in [start, end]
        counts = {}
        for msg, ts in self.times.items():
            lo, hi = 0, len(ts)
            if start is not None:
                lo = bisect_left(ts, start)
            if end is not None:
                hi = bisect_right(ts, end)
            if hi > lo:
                counts[msg] = hi - lo
        # ties are broken alphabetically
        def rank(m):
            return (-counts[m], m)
        return heapq.nsmallest(k, counts, key=rank)

    def expire(self, before):
        # bounded memory: forget timestamps older than
        # `before`
        for msg in list(self.times):
            ts = self.times[msg]
            self.times[msg] = ts[bisect_left(ts, before):]
            if not self.times[msg]:
                del self.times[msg]
```

87 Hit Counter — MEDIUM

Template fit: one deque of hit times; drop the ones older than 300 seconds; the answer is the length

QUESTION

Design a hit counter that counts the number of hits received in the past 5 minutes (300 seconds). hit(timestamp) records a hit; getHits(timestamp) returns the number of hits in the past 300 seconds.

EXAMPLES

Example 1	
Input	`hit(1), hit(2), hit(3), getHits(4), hit(300), getHits(300), getHits(301)`
Output	`[null, null, null, 3, null, 4, 3]`
Why	`At 301, hit at timestamp 1 expires.`

CONSTRAINTS

- 1 <= timestamp <= 2 * 10^9
- Timestamps are monotonically increasing

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# HELPERS used by the code below
WINDOW = 300   # seconds

# BASE QUESTION (optimal): Queue of timestamps with
# eviction
# Idea: Keep a deque of timestamps in arrival order.
# getHits pops from the front while the oldest is outside
# the window, and the answer is the deque length. Each
# timestamp is pushed and popped once.
class QueueCounter:
    """One entry per hit; old entries are dropped when
    asked. Timestamps must not go backwards."""

    def __init__(self):
        self.hits = deque()

    def hit(self, ts):
        self.hits.append(ts)

    def get_hits(self, ts):
        # keep only the hits in (ts - 300, ts]
        while self.hits and self.hits[0] <= ts - WINDOW:
            self.hits.popleft()
        return len(self.hits)
```

88 Website Activity Analytics: Unique Users + Session Time — MEDIUM

Template fit: dict activity -> set of users for unique counts; dict user -> sorted events to add up login-to-logout session time

QUESTION

Given website action records with **`user_id`**, timestamp, and activity, compute per-activity unique-user counts and average time spent on the site per user in seconds. The screen paired this Python task with a lighter SQL section and emphasized edge-case discussion over perfect final code.

Input is a collection of user-activity records. Each record contains:

QUESTION

user_id: numeric user identifier
timestamp: timestamp string such as 2024-07-26 10:00:00
activity: action name such as login, view, purchase, or logout

Implement two analytics tasks

Return a mapping from each activity type to the number of unique users who performed that activity.
Calculate the average time spent on the website per user in seconds.

Clarify session semantics before coding. The prompt uses login/logout-style events, but the exact treatment of users without an explicit logout, multiple sessions per user, and out-of-order records must be agreed with the interviewer.

EXAMPLES

```
user_activity = [
    {'user_id': 1, 'timestamp': '2024-07-26 10:00:00', 'activity': 'login'},
    {'user_id': 2, 'timestamp': '2024-07-26 10:05:00', 'activity': 'login'},
    {'user_id': 1, 'timestamp': '2024-07-26 10:10:00', 'activity': 'view'},
    {'user_id': 1, 'timestamp': '2024-07-26 10:15:00', 'activity': 'purchase'},
    {'user_id': 2, 'timestamp': '2024-07-26 10:20:00', 'activity': 'view'},
    {'user_id': 3, 'timestamp': '2024-07-26 10:25:00', 'activity': 'login'},
    {'user_id': 1, 'timestamp': '2024-07-26 10:30:00', 'activity': 'logout'},
]
```

MAIN CODE | helper functions first, then the answer

```python
from collections import defaultdict


# HELPERS for the base question
FORMAT = "%Y-%m-%d %H:%M:%S"

# BASE QUESTION (optimal): One pass to group, then
# sessions per user
# Idea: Parse each record once. unique[activity] collects
# user ids in a set.
def unique_users_by_activity(records):
    users = defaultdict(set)
    for r in records:
        # a set counts every user once per activity
        users[r["activity"]].add(r["user_id"])
    return {a: len(u) for a, u in sorted(users.items())}

# BASE QUESTION (optimal): One pass to group, then
# sessions per user
# Idea: Parse each record once. unique[activity] collects
# user ids in a set.
def session_seconds(events):
    # events: one user's (time, activity), any order
    total, start = 0, None
    for t, act in sorted(events):
        if act == "login" and start is None:
            # a second login before a logout is ignored
            start = t
        elif act == "logout" and start is not None:
            total += (t - start).total_seconds()
            start = None
    # a login with no logout ends at the last event seen
    if start is not None:
        last = max(t for t, _ in events)
        total += (last - start).total_seconds()
    return total
```

Binary search on a yes/no condition

89 First Index Reaching a Threshold (Binary Search) EASY

Template fit: x = an index; ok(x) = arr[x] >= k on a sorted array

QUESTION

Given a non-decreasing (sorted ascending, duplicates allowed) array arr and a target value k , return the smallest index i such that arr[i] >= k — the first position where the array "reaches" the value k . If no element is >= k , return -1 . Solve in O(log n) time and O(1) extra space.

Function signature

def first_index_at_least(arr: list[int], k: int) -> int:

    ...

Follow-up (as reported): The interviewer probed space complexity; note the solution is already O(1) space. If arr is a huge sorted stream/file, describe reading via random access rather than loading it all.

EXAMPLES

Example 1	
Input	`arr = [1, 2, 2, 3, 5, 7], k = 2`
Output	`1`
Why	`arr[1] = 2 is the first element >= 2.`

Example 2	
Input	`arr = [1, 2, 2, 3, 5, 7], k = 4`
Output	`4`
Why	`The first element >= 4 is 5 at index 4.`

Example 3	
Input	`arr = [1, 2, 2, 3, 5, 7], k = 8`
Output	`-1`
Why	`No element is >= 8.`

CONSTRAINTS

- 0 <= len(arr) <= 10**7
- Values and k are integers.

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Binary search for the lower
# bound
# Idea: Halve the range, keeping the boundary between
# values below k and values at least k.
def first_index_at_least(arr, k):
    lo, hi = 0, len(arr)
    while lo < hi:
        mid = (lo + hi) // 2
        if arr[mid] >= k:
            hi = mid
        else:
            lo = mid + 1
    return lo if lo < len(arr) else -1
```

90 Cutting Ribbons (LC 1891) MEDIUM

Template fit: x = the piece length; ok(x) = the ribbons give at least k pieces; you want the LARGEST ok, so search the first x that fails and subtract 1

QUESTION

LC 1891 Cutting Ribbons: given wood lengths and K, maximize the equal piece length that yields at least K pieces. Follow-ups cover the search bound, floating-point lengths, and why complexity depends on the value range rather than the number of elements.SWEarraybinary-searchcomplexity-analysismediumFrequencySingle reportLast asked2026-08-16Stageonsite-coding

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
def pieces(ribbons, length):
    # how many pieces of this length can we cut?
    return sum(r // length for r in ribbons)

def upper_bound(ribbons, k):
    # no piece is longer than the longest ribbon, and
    # k pieces of length L need k * L total length
    return min(max(ribbons), sum(ribbons) // k)

# BASE QUESTION (optimal): Binary search the answer
# (integer lengths)
# Idea: If you can cut k pieces of length L you can cut k
# pieces of any shorter length, so feasibility is
# monotone.
def max_length(ribbons, k):
    # the best whole-number length (0 if impossible)
    lo, hi, best = 1, upper_bound(ribbons, k), 0
    while lo <= hi:
        mid = (lo + hi) // 2
        if pieces(ribbons, mid) >= k:
            # mid works: remember it and try longer
            best, lo = mid, mid + 1
        else:
            hi = mid - 1
    return best
```

91 First Bad Version with Parallel Search Follow-up MEDIUM

Template fit: x = the version number; ok(x) = is_bad(x)

QUESTION

Phone-screen R1 coding round. The base problem is the standard first-bad-version monotonic search — given an **`isBadVersion(v)`** oracle over versions **`1..n`**, find the first bad version with a single binary search. The graded follow-up requires parallelizing the search: partition the range into buckets and search them concurrently (bucket + binary search), with working code expected.SWEbinary-searchparallelismconcurrencymediumFrequencySingle reportLast asked2026-06-15Stagephone-screen

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): problem: binary search
# Idea: isBad is monotone (all good, then all bad), so
# keep the invariant 'the first bad version is in [lo,
# hi]' and halve the interval. Use lo + (hi - lo) // 2 in
# languages with fixed-width integers.
def first_bad_version(n, is_bad):
    # the base question: binary search on versions 1..n
    lo, hi = 1, n
```

```
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if is_bad(mid):
            # the first bad version is mid or earlier
            hi = mid
        else:
            lo = mid + 1
    return lo
```

92 Solve Banana Speed and Interval Merge

MEDIUM

Template fit: x = the eating speed; ok(x) = total hours (ceil of pile / x summed) <= h; first_true over 1..max pile

QUESTION

During two technical phone screens for a new graduate software engineering role, the candidate was asked to solve two coding problems: minimum eating speed and merging overlapping intervals.

Constraints & Assumptions

- Provide clear algorithms, correctness reasoning, complexity, and edge cases.
- For minimum eating speed, each hour can be spent on only one pile.
- For interval merging, intervals overlap when the next start is less than or equal to the current merged end.
- Use sorted or binary-search approaches where appropriate.

Clarifying Questions

to Ask - Are **`piles`** and **h** always positive?

- Is **h** at least the number of piles?
- Are interval endpoints inclusive?
- Can intervals be empty or unsorted?

Part 1

- Minimum Eating Speed Given **`piles`** and **h**, return the minimum integer speed **k** such that all piles can be eaten within **h** hours.

What This Part Should Cover

- Binary search on answer from 1 to **`max(piles)`**.
- Feasibility function using sum of **`ceil(pile / k)`**.
- Why feasibility is monotonic.
- Complexity and edge cases.

Part 2

- Merge Overlapping Intervals Given intervals **`[start, end]`**, merge all overlapping intervals and return non-overlapping intervals sorted by start.

What This Part Should Cover

- Sort by start.
- Maintain current merged interval.
- Extend on overlap, otherwise append and start a new interval.
- Complexity and edge cases.

What a Strong

Answer Covers - Recognizes monotonic binary search for eating speed.

- Uses integer-safe ceiling division.
- Correctly defines interval overlap.
- Gives clean code and complexity for both problems.

QUESTION

Follow-up Questions

- How would you avoid floating-point division in the banana problem?
- What if **h** is smaller than the number of piles?
- How do you merge touching intervals like **`[1,3]`** and **`[3,5]`**?
- How would you handle streaming intervals?Minimum Eating Speed (Koko Eating Bananas)Koko loves to eat bananas. There are **n** piles of bananas; the i-th pile has **`piles[i]`** bananas. The guards will return in **h** hours. Koko can decide her constant bananas-per-hour eating speed **k**. Each hour she chooses one pile and eats **k** bananas from it. If that pile has fewer than **k** bananas, she eats all of them and does not eat more during that hour. An hour is spent on only one pile. Return the minimum integer speed **k** such that she can finish all the bananas within **h** hours. Example:
- **`piles = [3, 6, 7, 11]`**, **`h = 8`** -> **`4`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 5`** -> **`30`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 6`** -> **`23`** Constraints: **`1 <= piles.length <= h`**, **`1 <= piles[i] <= 10^9`**.

Constraints

1 <= piles.length <= h1 <= piles[i] <= 10^9h can be as large as 10^9

EXAMPLES

```
Input: ([3, 6, 7, 11], 8)
Expected Output: 4
Explanation: At k=4: ceil(3/4)+ceil(6/4)+ceil(7/4)+ceil(11/4) = 1+2+2+3 = 8 hours. k=3 needs 1+2+3+4 =
    10 > 8, so 4 is minimal.
Input: ([30, 11, 23, 4, 20], 5)
Expected Output: 30
```

Explanation: With exactly 5 piles and 5 hours, each pile must be eaten in one hour, so k must be at least max(piles) = 30.

Hints

If speed k finishes in time, any larger speed also finishes in time. This monotonicity lets you binary search on the answer.The answer lies in [1, max(piles)]: speed 1 is the slowest, and max(piles) finishes every pile in at most one hour.Hours needed at speed k is sum of ceil(pile / k). Use integer-safe ceiling: (pile + k - 1) // k to avoid floating point.Merge Overlapping IntervalsGiven an array of intervals where **`intervals[i] = [start_i, end_i]`**, merge all overlapping intervals and return an array of the non-overlapping intervals that cover all the intervals in the input, sorted by start. Two intervals overlap when the next start is less than or equal to the current merged end

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
def hours_needed(piles, speed):
    # each hour she works on ONE pile, so every pile
    # rounds up
    return sum(-(-p // speed) for p in piles)

# BASE QUESTION (optimal): Bananas: binary search the
# speed
# Idea: Hours needed are non-increasing in the speed, so
# keep lo = 1 and hi = max(piles) and halve: if speed mid
# finishes within h, try slower, otherwise faster. Use
# ceiling division (an hour finishes at most one pile).
def min_eating_speed(piles, h):
    # fewer hours than piles can never work
    if h < len(piles):
        return -1
    lo, hi = 1, max(piles)
    # more speed never takes more hours: binary search
    while lo < hi:
```

```
        mid = (lo + hi) // 2
        if hours_needed(piles, mid) <= h:
            hi = mid
        else:
            lo = mid + 1
    return lo
```

93 Given a list with response time for each server, calculate how long time it will take to process n requests. MEDIUM

Template fit: x = the elapsed time; ok(x) = sum(x // time_i) >= n requests

QUESTION

Given a list with response time for each server, calculate how long time it will take to process n requests.

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
def done_by(times, t, starts=None):
    # how many requests all servers finish within time t.
    # starts[i] is a startup delay of server i (default
    # 0).
    total = 0
    for i, r in enumerate(times):
        begin = starts[i] if starts else 0
        total += max(0, (t - begin) // r)
    return total

# BASE QUESTION (optimal): Binary search on the total time
# Idea: In time T server i completes T // r_i requests, so
# the servers finish at least n requests exactly when
# sum(T // r_i) >= n, which is monotone in T.
def time_for(times, n, starts=None):
    # smallest t with done_by(t) >= n. done_by never goes
    # down as t grows, so binary search works.
    if n <= 0:
        return 0
    lo = 0
    first = starts[0] if starts else 0
    hi = first + times[0] * n
    for i, r in enumerate(times):
        begin = starts[i] if starts else 0
        hi = min(hi, begin + r * n)
    while lo < hi:
        mid = (lo + hi) // 2
        if done_by(times, mid, starts) >= n:
            hi = mid
        else:
            lo = mid + 1
    return lo
```

94 Median of Two Sorted Arrays (LC 4) with a Follow-up Variant — HARD

Template fit: x = how many items come from the shorter array's left side; ok(x) = the left part's largest <= the right part's smallest

QUESTION

L5 R1 coding: LC 4 (Median of Two Sorted Arrays). Heading toward the LC 295 streaming two-heap approach drew a redirect from the interviewer. Follow-up: extend to many sorted inputs, in the spirit of LC 23 (Merge k Sorted Lists).

Per the candidate, the base question is LC 4 (Median of Two Sorted Arrays): given two sorted arrays, return the median of their combined elements.
Follow-up: the input becomes many sorted arrays instead of two, in the spirit of LC 23 (Merge k Sorted Lists).

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
INF = float("inf")

# BASE QUESTION (optimal): Binary search on the partition
# Idea: Make a the shorter array. Binary search i in a,
# set j = half - i in b so the left side holds (m + n + 1)
# // 2 elements.
def median_two(a, b):
    """Both arrays are sorted: binary search the cut. The
    left parts must hold half of all the elements."""
    if len(a) > len(b):
        a, b = b, a
    m, n = len(a), len(b)
    if m + n == 0:
        raise ValueError("no elements")
    lo, hi, half = 0, m, (m + n + 1) // 2
    while lo <= hi:
        i = (lo + hi) // 2
        j = half - i
        # values just left and right of each cut
        a_left = a[i - 1] if i > 0 else -INF
        a_right = a[i] if i < m else INF
        b_left = b[j - 1] if j > 0 else -INF
        b_right = b[j] if j < n else INF
        if a_left <= b_right and b_left <= a_right:
            # a valid cut: read the median off its edges
            if (m + n) % 2:
                return max(a_left, b_left)
            return (max(a_left, b_left)
                    + min(a_right, b_right)) / 2
        if a_left > b_right:
            hi = i - 1
        else:
            lo = i + 1
```

95 Find the median of two sorted arrays. — HARD

Template fit: the same partition search on the shorter array

QUESTION

Find the median of two sorted arrays.

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
INF = float("inf")

# BASE QUESTION (optimal): Binary search on the partition
# Idea: Binary search how many elements of the shorter
```

```python
# array go in the left half. A partition is right when
# every left element is <= every right element, which only
# needs the four border values to be checked.
def kth_of_two(a, b, k):
    if len(a) > len(b):
        a, b = b, a
    lo, hi = max(0, k - len(b)), min(k, len(a))
    while lo <= hi:
        i = (lo + hi) // 2
        j = k - i
        a_left = a[i - 1] if i > 0 else -INF
        a_right = a[i] if i < len(a) else INF
        b_left = b[j - 1] if j > 0 else -INF
        b_right = b[j] if j < len(b) else INF
        if a_left > b_right:
            hi = i - 1
        elif b_left > a_right:
            lo = i + 1
        else:
            return max(a_left, b_left)
    raise ValueError("k is out of range")

# BASE QUESTION (optimal): Binary search on the partition
# Idea: Binary search how many elements of the shorter
# array go in the left half. A partition is right when
# every left element is <= every right element, which only
# needs the four border values to be checked.
def median_of_two(a, b):
    # the median is the middle item, or the mean of the
    # two
    # middle items when the total is even
    total = len(a) + len(b)
    if total == 0:
        raise ValueError("no data")
    if total % 2:
        return kth_of_two(a, b, total // 2 + 1)
    low = kth_of_two(a, b, total // 2)
    high = kth_of_two(a, b, total // 2 + 1)
    return (low + high) / 2
```

96 Find longest duplicated substring

HARD

Template fit: x = the substring length; ok(x) = some substring of that length appears twice (rolling hash); search the largest ok

QUESTION

You are given a string **s** consisting of lowercase English letters. A **substring** of **s** is any contiguous sequence of characters, i.e., **`s[i..j)`** for **`0 <= i < j <= len(s)`**. Design an algorithm to find **any one** of the longest substrings that appears **at least twice** in **s**. The two occurrences may overlap. If no substring appears at least twice, return the empty string.

Requirements

- Input: a string **s** of length **n** (e.g., **`1 <= n <= 2 × 10^5`**).
- Output: a string representing one of the longest duplicated substrings.
- Aim for an algorithm substantially better than the naive **`O(n^2)`** substring comparison approach.

Examples

- **`s = "banana"`**
- Possible duplicated substrings: "a", "an", "ana" - Longest length is 3 ("ana"), so acceptable outputs include **`"ana"`**.
- **`s = "abcd"`**

QUESTION

- No duplicated substring, so output should be **""** (empty string). Describe your algorithm, its time and space complexity, and how it scales for large **n**.

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
MOD = (1 << 61) - 1

BASE = 131

def find_length(s, length):
    # a repeated window of this length? (rolling hash)
    codes = [ord(c) for c in s]
    power = pow(BASE, length, MOD)
    h = 0
    for i in range(length):
        h = (h * BASE + codes[i]) % MOD
    seen = {h: [0]}
    for i in range(1, len(s) - length + 1):
        # slide the window: drop the left, add the right
        h = (h * BASE - codes[i - 1] * power
             + codes[i + length - 1]) % MOD
        # a hash match may be a collision: compare the
        # text
        for j in seen.get(h, ()):
            if s[j:j + length] == s[i:i + length]:
                return s[i:i + length]
        seen.setdefault(h, []).append(i)
    return None

# BASE QUESTION (optimal): Binary search on the length
# with a rolling hash (Rabin-Karp)
# Idea: For a fixed length L compute the polynomial hash
# of every window in O(1) each by sliding it, store hashes
# with their start positions, and when a hash repeats
# compare the actual substrings (to rule out a collision).
def longest_dup(s):
    # binary search the length: if length L repeats, so do
    # all shorter lengths
    lo, hi, best = 1, len(s) - 1, ""
    while lo <= hi:
        mid = (lo + hi) // 2
        found = find_length(s, mid)
        if found:
            best, lo = found, mid + 1
        else:
            hi = mid - 1
    return best
```

GROUP 12 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE | 3 E 5 M

Greedy: scan once, carry the best choice so far, act only when forced

97 Maximum Profit — EASY

Template fit: pick the cheapest and the dearest city with a fixed tie-break (min and max once)

QUESTION

You and your friend have decided to start an international trading business. You want to earn a profit through some ol' fashioned arbitrage: You'll buy widgets in one country and sell them in another at a higher price. Your friend has already found the current market prices for widgets in cities around the world, and you'd like to use this data to pick which cities to buy and sell widgets in. Write a function max_profit(prices) that efficiently finds the two cities that maximize your profit (i.e. largest difference in prices).
Input: A dictionary with cities as keys and prices as values Output: An array containing the names of the cities (min, max) If the prices dictionary is empty, return None/null (depending on the programming language used). If there is 0 profit to be earned, return None/null as well. If you are using C++, return {"", ""} in such cases. Example Pythonprices = {'London': 72, 'New York': 70, 'Tokyo': 67, 'Miami': 62} max_profit(prices)

=> ['Miami', 'London']Hint 1You can think of this as two sub-problems

: Finding the lowest price, and finding the highest price.Hint 2While you could compare every pair of cities, it's more efficient to find the min and max values.Hint 3You can keep track of the lowest and highest values and update them as you iterate through the list.Challenge 1Does your solution work in O(N) time and O(1) space? If not, you can be more efficient!Challenge 2What happens if the price is the same in every city? Does your function still return

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): One pass for the cheapest and
# the dearest city
# Idea: Find the city with the lowest price and the city
# with the highest price in one scan (ties by name). They
# are the best pair because profit is sell minus buy and
# the order of cities does not matter.
def max_profit_prices(prices):
    # a dictionary city -> price: buy at the cheapest city
    # and
    # sell at the dearest, order does not matter
    if len(prices) < 2:
        return 0
    values = list(prices.values())
    return max(values) - min(values)
```

98 Alarm for the Next Upcoming Birthday — EASY

Template fit: minimum over people of the circular gap (their day - today) modulo 365, ties by name

QUESTION

You maintain a list of people and their birthdays. Given today's date and a list of (name, month, day) records, return the name of the person whose birthday comes soonest starting from today. Birthdays recur every year, so if no birthday remains later this calendar year, the soonest one is the earliest birthday next year. A birthday that falls exactly on today counts as 0 days away (soonest). Ignore leap years; treat every date simply as a (month, day) pair. If two people share the soonest birthday, return the one that is alphabetically first by name.

Function signature

```
def next_birthday(today: tuple[int, int], people: list[tuple[str, int, int]]) -> str:
    ...
```

EXAMPLES

QUESTION

Example 1	
Input	today = (10, 7), people = [("Alice", 10, 9), ("Bob", 1, 15), ("Carol", 12, 25)]
Output	"Alice"
Why	Alice's birthday is 2 days away; Carol's is later this year; Bob's is next January. Alice is soonest.

Example 2	
Input	today = (12, 31), people = [("Alice", 1, 1), ("Dan", 12, 31)]
Output	"Dan"
Why	Dan's birthday is today (0 days away), which beats Alice's tomorrow.

Example 3	
Input	today = (6, 1), people = [("Eve", 5, 30), ("Finn", 5, 30)]
Output	"Eve"
Why	Both birthdays already passed this year, so both are ~next-year and tie; alphabetical order picks "Eve".

CONSTRAINTS

- 1 <= len(people) <= 10**4
- 1 <= month <= 12 , 1 <= day <= 31 (assume valid calendar dates). today is a valid (month, day) pair.

MAIN CODE | helper functions first, then the answer

```python
# HELPERS used by the code below
MONTH_DAYS = [
    31,
    28,
    31,
    30,
    31,
    30,
    31,
    31,
    30,
    31,
    30,
    31,
]

# BASE QUESTION (optimal): Day of the year and a cyclic
# minimum
# Idea: Days until the birthday is (doy - today) mod 365;
# take the smallest, ties by name.
def day_of_year(month, day):
    return sum(MONTH_DAYS[: month - 1]) + day

# BASE QUESTION (optimal): Day of the year and a cyclic
# minimum
# Idea: Days until the birthday is (doy - today) mod 365;
# take the smallest, ties by name.
def next_birthday(today, people):
    now = day_of_year(*today)
    return min(
        people,
        key=lambda p: (
            (day_of_year(p[1], p[2]) - now) % 365,
            p[0],
        ),
```

```
    )[0]
```

99 Coins on the Table — EASY

Template fit: scan the events with one counter: add on '+', subtract on '-' but never below 0

QUESTION

There is a table that starts with 0 coins on it. You are given a string events , where each character is a single event applied left to right: '+' places one coin on the table. '-' removes one coin from the table. The table can never hold a negative number of coins: a '-' event when the table is empty has no effect (it is ignored). After processing every event, return the number of coins on the table.

Function signature

def coins_on_table(events: str) -> int:

    ...

Follow-up (as reported): Generalize so each event is an integer delta (e.g. "+3" , "-2"), still clamping the running total at 0. Hint: parse tokens instead of single characters.

EXAMPLES

Example 1	
Input	events = "+++-"
Output	2
Why	+1, +1, +1, then -1 -> 2 coins remain.

Example 2	
Input	events = "----"
Output	0
Why	Every removal happens on an empty table and is ignored.

Example 3	
Input	events = "+-+-+"
Output	1
Why	1, 0, 1, 0, 1 -> 1 coin remains.

CONSTRAINTS

- 0 <= len(events) <= 10^5
- Every character of events is either '+' or '-' .
- An empty string returns 0 .

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): One clamped counter
# Idea: total = max(0, total + delta) for every event.
def coins_on_table(events):
    total = 0
    for e in events:
        total = (
            total + 1 if e == "+" else max(0, total - 1)
        )
    return total
```

100 Visible People in a Line (Last Person, Then Everyone) MEDIUM

Template fit: scan from the end carrying the tallest person seen between; count a person when no taller one blocks the view

QUESTION

Given heights of people in a line, A can see B if nobody between them is taller than both. Part 1: count how many people the last person can see. Part 2: return that count for every person. Written in a plain editor with no compilation.

Input: an array of heights representing a line of people.
Visibility rule: person A can see person B if no one standing between them is taller than both A and B.
Part 1: return how many people the last person in line can see.
Part 2: for the same array, return, for every person in the line, how many people they can see.

EXAMPLES

```
[1, 10, 6, 7, 9, 8, 2, 4, 3, 5] -> 6 (the last person, height 5, can see 3, 4, 2, 8, 9, 10).
```

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
INF = float("inf")

# BASE QUESTION (optimal): One pass with a running maximum
# (last person)
# Idea: Walk from the person next to the last one towards
# the front, keeping the largest height seen strictly
# between the last person and the current one.
def last_sees(a):
    """Part 1: walk away from the last person keeping the
    tallest height seen strictly between us and them."""
    if not a:
        return 0
    last, between, count = a[-1], -INF, 0
    for k in range(len(a) - 2, -1, -1):
        # nobody between is taller than both of us
        if between <= max(last, a[k]):
            count += 1
        between = max(between, a[k])
    return count
```

101 Bounded Knapsack Variant MEDIUM

Template fit: sort items by value, highest first; take min(available count, remaining capacity) of each

QUESTION

Each item type has a value and an available count; every item weighs 1. Given capacity W, maximize total value. DP or DFS + memo works at O(n·W), greedy by value after sorting is O(n log n), and the best answer uses quickselect for O(n).

Input: for each item type, its value and the number of copies available. Every item has weight 1.
Input: a maximum total weight W.
Output: the maximum total value achievable without exceeding W.

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Sort by value and take greedily
# Idea: Sort the item types by value, highest first. For
# each type take min(copies, slots left) and add value
# times that. Stop when the bag is full.
def max_value_sorted(items, W):
    """Unit weights: every slot is worth the same, so the
    best value always goes in first."""
    total = 0
    for value, count in sorted(items, reverse=True):
```

```
        # take as many copies as still fit in the bag
        take = min(count, W)
        total += value * take
        W -= take
        if W == 0:
            break
    return total
```

102 Find the Largest Monotone Increasing Number

MEDIUM

Template fit: scan the digits right to left; when a digit is smaller than the one before it, lower the earlier digit and fill 9s after it

QUESTION

Overview: Find the largest number no greater than n whose decimal digits are monotone increasing. Develop a linear-time greedy correction that revisits earlier decreasing boundaries and fills the remaining suffix optimally without introducing leading zeros.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Right-to-left scan, then fill
# with nines
# Idea: Walk i from the last digit to the first. When
# digit[i - 1] > digit[i], decrease digit[i - 1] by one
# and remember i as the start of the nines.
def largest_monotone(n):
    # largest number <= n whose digits never decrease
    d = list(str(n))
    mark = len(d)
    # scan from the right; a drop forces the left digit
    # down by
    # one and everything after it up to 9
    for i in range(len(d) - 1, 0, -1):
        if d[i - 1] > d[i]:
            d[i - 1] = str(int(d[i - 1]) - 1)
            mark = i
    for i in range(mark, len(d)):
        d[i] = "9"
    return int("".join(d))
```

103 Min boards to cover roof holes

MEDIUM

Template fit: the template as written

QUESTION

Overview: This question evaluates competency in combinatorial optimization and problem modeling for coverage constraints, testing algorithmic reasoning, complexity analysis, and efficient representation of grid-based data.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Greedy: start a board at the
# leftmost uncovered hole
# Idea: Sort the holes. Place a board starting at the
# first uncovered hole, which covers every hole below
# start + L; skip those and repeat.
def min_boards(holes, length):
    # greedy: the leftmost uncovered hole starts a board,
    # which
    # then covers `length` positions
    count, covered_to = 0, float("-inf")
    for p in sorted(set(holes)):
```

```
        if p > covered_to:
            count += 1
            covered_to = p + length - 1
    return count
```

104 Jump Game II (Minimum Jumps) MEDIUM

Template fit: scan the indices carrying the farthest reach; when you hit the end of the current jump, you must jump again: jumps += 1, end = farthest

QUESTION

You are given a 0-indexed array of non-negative integers nums . You start at index 0. From any index i , you can jump to any index in the range [i + 1, i + nums[i]] (i.e., nums[i] is the maximum forward jump length from i). Return the minimum number of jumps required to reach the last index len(nums)

- 1 . The test cases guarantee that the last index is always reachable.

Function signature

def jump(nums: list[int]) -> int:
    """Return the minimum number of jumps to reach the last index."""
    ...

Follow-up (as reported): Solve it in O(n) time and O(1) extra space with a greedy "current reach vs. farthest reach" scan rather than O(n) DP.

EXAMPLES

Example 1	
Input	`nums = [2, 3, 1, 1, 4]`
Output	`2`
Why	`Jump 1 step from index 0 to 1, then 3 steps to the last index.`

Example 2	
Input	`nums = [2, 3, 0, 1, 4]`
Output	`2`
Why	`0 -> 1 -> 4.`

Example 3	
Input	`nums = [0]`
Output	`0`
Why	`Already at the last index.`

CONSTRAINTS

- 1 <= len(nums) <= 10^4
- 0 <= nums[i] <= 1000
- If the array has length 1 you are already at the last index, so the answer is 0.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Greedy by ranges
# Idea: Each jump extends the covered range to the
# farthest index seen so far.
def jump(nums):
    jumps, current_end, farthest = 0, 0, 0
    for i in range(len(nums) - 1):
        farthest = max(farthest, i + nums[i])
```

```
        if i == current_end:
            # the range of the current jump is used up:
            # take another
            jumps += 1
            current_end = farthest
    return jumps
```

GROUP 13 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE | 4 E 3 M 1 H

Sets and counters: remember the first collection, check the second against it

105 Exclude First-K-Unique From Another List

EASY

Template fit: the set holds the first k distinct values of list 1; keep list 2's values that are not in it

QUESTION

You are given two lists list1 and list2 and an integer k . Scan list1 left to right and collect the first k distinct values it contains (in order of first appearance; if list1 has fewer than k distinct values, take all of them). Then return list2 with every occurrence of any of those collected values removed, preserving the order of the remaining elements.

Function signature

def exclude_first_k_unique(list1: list[int], list2: list[int], k: int) -> list[int]:

...

EXAMPLES

Example 1	
Input	`list1 = [3, 3, 1, 4, 1, 5], list2 = [1, 2, 3, 4, 3, 5], k = 2`
Output	`[2, 4, 5]`
Why	`First 2 distinct values of list1 are {3, 1}. Removing all 3s and 1s from list2 leaves [2, 4, 5].`

Example 2	
Input	`list1 = [7], list2 = [7, 8, 9], k = 3`
Output	`[8, 9]`
Why	`list1 has only one distinct value (7); remove all 7s from list2.`

CONSTRAINTS

- 0 <= len(list1), len(list2) <= 10**5 , 1 <= k .
- Values are integers.

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): A set of the first k distinct
# values
# Idea: Membership in a set makes the filter linear.
def exclude_first_k_unique(list1, list2, k):
    blocked = set()
    for x in list1:
        if len(blocked) == k:
            break
        blocked.add(x)
    return [y for y in list2 if y not in blocked]
```

106 Find the Duplicates

EASY

Template fit: values that appear in both lists, each reported once (set intersection, or two pointers on sorted lists)

QUESTION

Given two sorted arrays arr1 and arr2 of passport numbers, implement a function findDuplicates that returns an array of all passport numbers that are both arrays. Note that the output array should be sorted in ascending order. Let N and M be the lengths of arr1 and arr2, respectively. Solve for two cases and analyze the time & space complexities of your solutions: M ~ N - the array lengths are approximately the same M >> N - arr2 is much bigger than arr1. Example input: arr1 = [1, 2, 3, 5, 6, 7], arr2 = [3, 6, 7, 8, 20] output: [3, 6, 7]

since only these three values

are both in arr1 and arr2Hint 1Start with a brute force solution and optimize from there.Hint 2Note that the question asks for a sorted array. What techniques work well on sorted arrays?Hint 3If M >> N (case #2) is giving you trouble, consider how you might use the values of N, the shorter array, to increase the efficiency of your answer.Challenge 1Have you taken care to avoid array out-of-bound runtime errors?First, the brute force solution. Try two nested loops. For every value arr1[i] in the outer loop, we look it up in arr2 in the inner loop. The time complexity of this algorithm in the worst case scenario is O(N*M). Not very efficient. We can do better than that. Case 1 (M ~ N) We can use the fact the arrays are sorted to traverse both of them in an in-order manner at the same time. The general idea of the algorithm is to use two indices, i and j, for arr1 and arr2, respectively. Every time one of indices,

MAIN CODE | helper functions first, then the answer

```
from bisect import bisect_left
from math import log2


# CASE 1: M close to N (two pointers)
def common_two_pointers(a, b):
    # M close to N: walk both sorted arrays together and
    # advance the smaller side. O(N + M) time, O(1) space
    # besides the output.
    out, i, j = [], 0, 0
    while i < len(a) and j < len(b):
        if a[i] < b[j]:
            i += 1
        elif a[i] > b[j]:
            j += 1
        else:
            if not out or out[-1] != a[i]:
                out.append(a[i])
            i += 1
            j += 1
    return out

# CASE 2: M much bigger than N (binary search)
def common_binary_search(small, big):
    # M much bigger than N: binary search every item of
    # the
    # short array in the long one. O(N log M); the search
    # starts where the previous one stopped.
    out, lo = [], 0
    for x in small:
        lo = bisect_left(big, x, lo)
        if lo == len(big):
            break
        if big[lo] == x and (not out or out[-1] != x):
            out.append(x)
    return out

# BASE QUESTION (optimal): Case 1 (M close to N): two
# pointers
```

```
# Idea: Advance whichever pointer points at the smaller
# value; when equal, record the value once (skip repeats)
# and advance both. Each element is passed at most once.
def common(a, b):
    # choose by cost: N log M against N + M
    small, big = (a, b) if len(a) <= len(b) else (b, a)
```

```
    if len(small) * log2(len(big) + 2) < len(a) + len(b):
        return common_binary_search(small, big)
    return common_two_pointers(a, b)
```

107 Positional Matches Between Two Lists — EASY

Template fit: no set needed: zip both lists and keep a where a == b

QUESTION

You are given two lists of equal length, first and second . Return, in order, the values at every position where the two lists agree — that is, all first[i] such that first[i] == second[i] .

Function signature

def positional_matches(first: list[int], second: list[int]) -> list[int]:

...

EXAMPLES

Example 1	
Input	`first = [3, 1, 4, 1, 5, 5, 7, 9], second = [1, 2, 3, 4, 5, 6, 7, 8]`
Output	`[5, 7]`
Why	`Positions 4 (5 == 5) and 6 (7 == 7) match. (The report described "matches" between a digits-of-pi list and an index list.)`

Example 2	
Input	`first = [1, 2, 3], second = [4, 5, 6]`
Output	`[]`
Why	`No position matches.`

CONSTRAINTS

- len(first) == len(second) , 0 <= length <= 10**6 .
- Values are integers.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): zip and a filter
# Idea: Walk both lists together and keep the agreeing
# values.
def positional_matches(first, second):
    return [a for a, b in zip(first, second) if a == b]
```

108 Contains Duplicate — EASY

Template fit: add values to a set as you go; the first value already in the set means a duplicate

QUESTION

Given an integer array nums , return True if any value appears at least twice in the array, and False if every element is distinct.

Function signature

```
def contains_duplicate(nums: list[int]) -> bool:
    """Return True if any value appears more than once."""
    ...
```

EXAMPLES

Example 1	
Input	`nums = [1, 2, 3, 1]`
Output	`True`

Example 2	
Input	`nums = [1, 2, 3, 4]`
Output	`False`

Example 3	
Input	`nums = [1, 1, 1, 3, 3, 4, 3, 2, 4, 2]`
Output	`True`

CONSTRAINTS

- 1 <= len(nums) <= 10^5
- -10^9 <= nums[i] <= 10^9

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): A set of seen values
# Idea: Add each value to a set; seeing one again means a
# duplicate.
def contains_duplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return True
        seen.add(x)
    return False
```

109 Filter Common Elements Per Window (Two Lists) — MEDIUM

Template fit: build a set per window of the first list; keep the matching window of the second list minus that set

QUESTION

You are given two lists a and b of equal length n and a window size k that divides n . Split both lists into consecutive, non-overlapping windows of size k (window 0 is indices 0..k-1 , window 1 is k..2k-1 , and so on). For each window, remove from b 's window every value that also appears in the corresponding window of a . Concatenate the surviving elements of b , preserving their original order, and return that list.

Function signature

```
def filter_windows(a: list[int], b: list[int], k: int) -> list[int]:
    ...
```

QUESTION

Follow-up (as reported): The interviewer asked to do the removal in place / O(1) extra space. Naive in-place removal becomes O(n*k); discuss compaction with a write pointer to keep it O(n) time.

EXAMPLES

Example 1	
Input	`a = [1, 2, 3, 4, 5, 6], b = [3, 7, 8, 6, 9, 5], k = 3`
Output	`[7, 8, 9]`
Why	`Window 0: a={1,2,3}, b=[3,7,8] -> remove 3 -> [7,8]. Window 1: a={4,5,6}, b=[6,9,5] -> remove 6 and 5 -> [9]. Concatenate -> [7,8,9].`

Example 2	
Input	`a = [1, 1], b = [1, 2], k = 2`
Output	`[2]`
Why	`Single window: a={1}, b=[1,2] -> remove 1 -> [2].`

CONSTRAINTS

- 0 <= n <= 10**5 , n % k == 0 , 1 <= k <= n (for n > 0).
- Values are integers.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): A set per window
# Idea: Membership in the window of a is a set lookup.
def filter_windows(a, b, k):
    out = []
    for i in range(0, len(a), k):
        seen = set(a[i : i + k])
        out.extend(
            x for x in b[i : i + k] if x not in seen
        )
    return out
```

110 Reconstruct original array from doubled shuffle

MEDIUM

Template fit: Counter of the values; go from the smallest, pairing each x with 2x and using both counts up

QUESTION

Overview: This question evaluates array manipulation, multiset/frequency reasoning, pairing logic, and careful handling of edge cases such as zeros and negative numbers.

MAIN CODE | helper functions first, then the answer

```
from collections import Counter


# BASE QUESTION (optimal): Counter processed in order of
# absolute value
# Idea: Count the values. For each value x in order of
# absolute value with a remaining count, it needs count[x]
# copies of 2x: if count[2x] is smaller, fail; otherwise
# consume them and append x that many times.
def find_original(changed):
    # `changed` holds every original x and its double 2x,
    # in
    # any order. Smallest absolute value first: it can
    # only be
    # an original, never someone's double.
```

```
    if len(changed) % 2:
        return []
    count, out = Counter(changed), []
    for x in sorted(count, key=lambda v: (abs(v), v)):
        if count[x] == 0:
            continue
        if x == 0:
            # zeros pair with each other
            if count[0] % 2:
                return []
            out += [0] * (count[0] // 2)
            count[0] = 0
            continue
        # every x needs its own 2x
        if count[2 * x] < count[x]:
            return []
        out += [x] * count[x]
        count[2 * x] -= count[x]
        count[x] = 0
    return out
```

111 Min deletions to avoid overlap in first k

MEDIUM

Template fit: Counter of the left part and of the right part; for each value in both, delete min(left count, right count)

QUESTION

Overview: This question evaluates array and sequence manipulation skills, set-based reasoning about value overlap, and the ability to compute minimal edits while preserving relative order constraints.

MAIN CODE | helper functions first, then the answer

```
from collections import Counter


# BASE QUESTION (optimal): Sum of min(count in the first
# k, count in the rest) over shared values
# Idea: Count each value in the first k elements and in
# the rest. For every value with a positive count on both
# sides, one side has to lose all of its copies, so add
# the smaller count.
def min_deletions(a, k):
    # the values in a[:k] and a[k:] must not overlap; for
    # each
    # shared value delete the copies on the cheaper side
    left, right = Counter(a[:k]), Counter(a[k:])
    return sum(min(left[v], right[v])
               for v in left.keys() & right.keys())
```

112 Smallest Missing Positive Integer in Linear Time and Constant Extra Space

HARD

Template fit: use the array itself as the set: swap each value x into slot x - 1, then the first slot with the wrong value is the answer

QUESTION

You are given an unsorted list of integers **nums**, which may contain negative numbers, zeros, duplicates, and values far larger than the length of the list. Return the smallest positive integer, that is the smallest integer that is at least **1**, that does not appear in **nums**. In the interview, a first solution that ran in linear time but used extra memory proportional to the input was accepted as a starting point, and the interviewer then asked for the same result using only constant extra space. Aim for that tighter target.

Function Signature

QUESTION

```
def first_missing_positive(nums: list[int]) -> int:
```

Rules

- Target complexity: **`O(n)`** time and **`O(1)`** auxiliary space, where **`n = len(nums)`**. The list may be rearranged or overwritten in place; its contents after the call do not matter.
- Return only the missing value.

Constraints

- **`1 <= len(nums) <= 10^5`**
- **`-2147483648 <= nums[i] <= 2147483647`**
- The answer is a positive integer that fits in a signed 32-bit integer.

Examples

Example 1

- Input: **`nums = [4, -2, 1, 2]`**
- Output: **`3`**
- Explanation: **`1`** and **`2`** are present and **`3`** is not.

Example 2

- Input: **`nums = [5, 6, 100, -3]`**
- Output: **`1`**
- Explanation: **`1`** does not appear, so it is the answer, however large the other values are.

Example 3

- Input: **`nums = [1, 2, 2, 3]`**
- Output: **`4`**
- Explanation: **`1`**, **`2`** and **`3`** all appear (**`2`** twice), so the smallest missing positive integer is **`4`**.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Cyclic placement: put each
# value v at index v - 1
# Idea: For each position, while nums[i] is in 1..n and
# not already at its home index, swap it to index nums[i]
# - 1. Each swap puts one value in its final place, so the
# total number of swaps is at most n.
def first_missing_positive(nums):
    # put each value v in 1..n at index v - 1 by swapping
    a, n = list(nums), len(nums)
    for i in range(n):
        while 1 <= a[i] <= n and a[a[i] - 1] != a[i]:
            j = a[i] - 1
            a[i], a[j] = a[j], a[i]
    # the first index that does not hold index + 1
    for i in range(n):
        if a[i] != i + 1:
            return i + 1
    return n + 1
```

GROUP 14 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE | 1 E 5 M 2 H

bisect on a sorted list: find a position, count a range, keep it sorted

113 Count People in a Postal Code Range

EASY

Template fit: sort the codes once; the answer is the template as written, never below 0

QUESTION

You are given a list of people, each as a tuple (name, postal_code) . Build a structure that can answer many queries of the form: "how many people have a postal code within the inclusive range [low, high] ?" The interviewer explicitly wanted the optimal query implementation — not a linear scan of the people (or a scan over the range and a hash-map lookup for each value). Preprocess once, then answer each query in O(log n) . Implement a class with: a constructor that accepts the list of people, and count_in_range(low, high) returning the number of people whose postal code is in [low, high] inclusive.

Function signature

```
from typing import List, Tuple
class PostalRegistry:
    def __init__(self, people: List[Tuple[str, int]]) -> None:
        """Preprocess the (name, postal_code) pairs for fast range counts."""
        ...
    def count_in_range(self, low: int, high: int) -> int:
        """Return how many people have postal_code in [low, high] inclusive."""
        ...
```

Follow-up (as reported): Also return the list of matching names. Hint: sort by (postal_code, name) and slice between bisect_left(low) and bisect_right(high) .

EXAMPLES

Example 1	
Input	`people = [("Alice", 90210), ("Bob", 10001), ("Cara", 90211), ("Dan", 30301)]` `count_in_range(90000, 99999)`
Output	`2`
Why	`Alice (90210) and Cara (90211) fall in the range.`

Example 2	
Input	`same people; count_in_range(10001, 30301)`
Output	`2`
Why	`Bob (10001) and Dan (30301); range is inclusive on both ends.`

Example 3	
Input	`same people; count_in_range(50000, 60000)`
Output	`0`
Why	`No postal codes in that range.`

CONSTRAINTS

- 0 <= number of people <= 10^5 ; the same postal code may appear for multiple people (duplicates allowed).
- Postal codes are integers, 0 <= postal_code <= 10^9 .
- 0 <= low <= high <= 10^9 .
- Each count_in_range call must run in O(log n) after an O(n log n) preprocessing step.

MAIN CODE | helper functions first, then the answer

```python
from bisect import bisect_left, bisect_right


# BASE QUESTION (optimal): Sorted codes and two bisects
# Idea: The codes in the range form one block of the
# sorted list.
class PostalRegistry:
    def __init__(self, people):
        self.codes = sorted(code for _, code in people)

    def count_in_range(self, low, high):
        return max(
            0,
            bisect_right(self.codes, high)
            - bisect_left(self.codes, low),
        )
```

114 Sliding-Window Mean After Removing the Largest K Values

MEDIUM

Template fit: keep the window in a sorted list; insort the new value, remove the old one with bisect_left, and track the sum of the top-K region

QUESTION

For every contiguous window of a fixed size, remove the largest **k** values by multiplicity and compute the mean of the remaining values. Process sliding updates incrementally rather than sorting each complete window again. Implement **`trimmed_window_means(nums: int[], window: int, k: int) -> string[]`**. Return means in window order as reduced fractions **`numerator/denominator`**, using a positive denominator. This exact fraction encoding is an explicit practice representation that avoids cross-language floating-point rounding differences.

Constraints & Assumptions

- **`1 <= len(nums) <= 200000`**, **`1 <= window <= len(nums)`**, and **`0 <= k < window`**.
- Values range from -1,000,000 through 1,000,000. Use a sufficiently wide integer for window sums.
- Remove exactly **k** occurrences, not **k** distinct values. Equal values on the partition boundary may be represented in either group as long as the multiset counts are correct.
- For **`k == 0`**, retain the whole window. A window that would retain zero values is excluded by the input contract.
- Reduce each fraction by the greatest common divisor of the absolute numerator and denominator. Zero is **`0/1`**; an integer mean such as 5 is **`5/1`**.
- Aim for **`O(n log window)`** processing with **`O(window)`** maintained state, apart from the returned results.

Examples

```
nums = [1,9,2,8], window = 3, k = 1 result = ["3/2","5/1"]
```

```
nums = [-1,1,3], window = 2, k = 0 result = ["0/1","2/1"]
```

Explain how insertions, removals, duplicate values, and rebalancing preserve the retained sum. For the reported large-data/stream follow-up, describe which state must remain in memory and why a new window does not require a full sort.

```
Maintain a size and an ordering invariant Partition the window into the removed largest values and the ret
ained values. Keep their sizes correct and ensure no retained value is larger than a removed value.
```

MAIN CODE | helper functions first, then the answer

```python
from bisect import bisect_left, bisect_right
from fractions import Fraction
```

```python
# HELPERS used by the code below
def as_text(fr):
    return f"{fr.numerator}/{fr.denominator}"

# HELPERS for the base question
class Window:
    """A sorted window that keeps the sum of its k largest
    and k smallest values while values come and go."""

    def __init__(self, k):
        self.k, self.s = k, []
        self.total = self.top = self.bottom = 0

    def insert(self, x):
        k, s = self.k, self.s
        old_len = len(s)
        p = bisect_right(s, x)
        s.insert(p, x)
        self.total += x
        # top-k region is the last k places (new list)
        if old_len < k:
            self.top += x
        elif p >= old_len + 1 - k:
            # x entered it: the value at old_len - k left
            # it
            self.top += x - s[old_len - k]
        # bottom-k region is the first k places
        if old_len < k:
            self.bottom += x
        elif p < k:
            self.bottom += x - s[k]

    def remove(self, x):
        k, s = self.k, self.s
        old_len = len(s)
        p = bisect_left(s, x)
        self.total -= x
        if old_len <= k:
            self.top -= x
            self.bottom -= x
        else:
```

```python
            if p >= old_len - k:
                # the value just below the region joins it
                self.top += s[old_len - k - 1] - x
            if p < k:
                self.bottom += s[k] - x
        del s[p]

# BASE QUESTION (optimal): Sorted window with an
# incrementally maintained top-K sum
# Idea: Keep the window values in a sorted list, the total
# window sum, and top = the sum of its K largest elements.
def trimmed_window_means(nums, window, k, both=False):
    # mean of each window after dropping the k largest
    # (and, with both=True, the k smallest as well)
    w, out = Window(k), []
    for i, x in enumerate(nums):
        w.insert(x)
        if i >= window:
            w.remove(nums[i - window])
        if i >= window - 1:
            kept = window - (2 * k if both else k)
            drop = w.top + (w.bottom if both else 0)
            mean = Fraction(w.total - drop, kept)
            out.append(as_text(mean))
```

```
    return out
```

115 Russian Doll Envelopes — MEDIUM

Template fit: sort by width ascending and height descending, then the longest increasing run of heights using a tails list and bisect_left

QUESTION

LeetCode 354. Maximum chain of envelopes that strictly fit inside each other. Sort + LIS reduction.

Input: list of (width, height) envelopes.
Output: longest chain where each envelope strictly fits in the next.
Trick: sort by width ascending, height descending (when widths tie); run LIS (Longest Increasing Subsequence) on heights.
LIS with patience-sort gives O(n log n).

EXAMPLES

```
[[5,4],[6,4],[6,7],[2,3]] -> 3.
```

MAIN CODE | helper functions first, then the answer

```
from bisect import bisect_left


# BASE QUESTION (optimal): Sort, then LIS with binary
# search
# Idea: Sort by (width ascending, height descending).
def max_envelopes(envelopes):
    # sort by width; for equal widths put TALLER first so
    # a
    # strictly increasing height run can use only one of
    # them
    ordered = sorted(envelopes,
                     key=lambda e: (e[0], -e[1]))
    # tails[k]: smallest ending height of a chain of k + 1
    tails = []
    for _, h in ordered:
        i = bisect_left(tails, h)
        if i == len(tails):
            tails.append(h)
        else:
            tails[i] = h
    return len(tails)
```

116 Count Sorted Strings With a Given Prefix — MEDIUM

Template fit: lo = bisect_left(words, prefix); hi = bisect_left(words, prefix + a very large character); the answer is hi - lo

QUESTION

Given a lexicographically sorted array of strings words and a query string prefix , return the number of strings in words that start with prefix . Because words is sorted, all matching strings occupy a contiguous range; find that range with binary search in O(log n * L) time (where L is the prefix length), not by scanning.

Function signature

def count_with_prefix(words: list[str], prefix: str) -> int:
    ...

Follow-up (as reported): Many repeated queries: cache counts in a hash map keyed by prefix. All queries unique: build a trie of all words storing subtree sizes, so each query is O(L) .

EXAMPLES

QUESTION

Example 1	
Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "app"`
Output	`3`
Why	`"app", "apple", "application" all start with "app".`

Example 2	
Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "ba"`
Output	`2`
Why	`"banana" and "bat".`

Example 3	
Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "cat"`
Output	`0`
Why	`No string starts with "cat".`

CONSTRAINTS

- 0 <= len(words) <= 10**6 , sorted in non-decreasing lexicographic order.
- 0 <= len(prefix) <= 100 . An empty prefix matches every string.

MAIN CODE | helper functions first, then the answer

```
from bisect import bisect_left


# BASE QUESTION (optimal): Two binary searches
# Idea: The matching words form one contiguous block of
# the sorted list; find both ends.
def count_with_prefix(words, prefix):
    lo = bisect_left(words, prefix)
    hi = bisect_left(words, prefix + "\uffff")
    return hi - lo
```

117 Streaming Insert + Find K-th Largest — MEDIUM

Template fit: insort every new value; the k-th largest is sorted_list[-1 - k]

QUESTION

EXAMPLES

```
insert(3)
insert(3)
insert(2)
findLargest(0) -> 3
findLargest(1) -> 3
findLargest(2) -> 2
```

MAIN CODE | helper functions first, then the answer

```
from bisect import insort


# BASE QUESTION (optimal): Sorted list with bisect (k
# chosen per query)
# Idea: Keep all values in an ascending list with insort.
# The k-th largest (0-indexed as in the examples) is a[-1
# - k].
```

```
class StreamKth:
    """Any k, with deletes: a sorted list."""

    def __init__(self):
        self.a = []

    def insert(self, x):
        # keep the list sorted: O(log n) search, O(n)
        # shift
        insort(self.a, x)


    def find_largest(self, k):
        # k = 0 is the largest value
        if 0 <= k < len(self.a):
            return self.a[-1 - k]
        return None
```

118 Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller. MEDIUM

Template fit: for each kid bisect into the sorted sellers and take the nearer of the two neighbours; the answer is the largest of those distances

QUESTION

Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.

MAIN CODE | helper functions first, then the answer

```
from bisect import bisect_left


# BASE QUESTION (optimal): Sort the sellers, binary search
# each kid
# Idea: With the sellers sorted, bisect_left gives the
# first seller at or after the kid; the nearest seller is
# that one or the one before it. E is the largest of these
# nearest distances.
def smallest_e(kids, sellers):
    # each kid goes to the nearest seller; the answer is
    # the
    # worst such distance. The nearest seller is next to
    # kid's place in the sorted list of sellers.
    sellers = sorted(sellers)
    worst = 0
    for k in kids:
        i = bisect_left(sellers, k)
        near = INF = float("inf")
        if i < len(sellers):
            near = sellers[i] - k
        if i > 0:
            near = min(near, k - sellers[i - 1])
        worst = max(worst, near)
    return worst
```

119 Find Maximum Envelope Nesting

HARD

Template fit: exactly the same sort and tails list

QUESTION

Overview: This question evaluates algorithmic problem-solving skills, particularly reasoning about multi-dimensional ordering, sequence optimization, and time-complexity analysis.

MAIN CODE | helper functions first, then the answer

```python
from bisect import bisect_left


# BASE QUESTION (optimal): Sort by width ascending and
# height descending, then LIS by binary search
# Idea: Sorting equal widths by decreasing height means
# two envelopes of the same width can never both appear in
# a strictly increasing height sequence.
def max_envelopes(envelopes):
    # sort by width, taller first on ties, then find the
    # longest
    # strictly increasing run of heights
    ordered = sorted(envelopes,
                     key=lambda e: (e[0], -e[1]))
    tails = []
    for _, h in ordered:
        i = bisect_left(tails, h)
        if i == len(tails):
            tails.append(h)
        else:
            tails[i] = h
    return len(tails)
```

120 Count of Smaller Numbers After Self

HARD

Template fit: go from the right, insort each number into a sorted list; the count is bisect_left of the number before inserting (a Fenwick tree is the fast version)

QUESTION

Given an integer array nums, return an array counts where counts[i] is the number of smaller elements to the right of nums[i].

EXAMPLES

Example 1	
Input	`nums = [5, 2, 6, 1]`
Output	`[2, 1, 1, 0]`
Why	`To the right of 5 there are 2 smaller: [2, 1]. To the right of 2 there is 1 smaller: [1]. To the right of 6 there is 1 smaller: [1]. To the right of 1 there are 0 smaller.`

Example 2	
Input	`nums = [1, 2, 3, 4]`
Output	`[0, 0, 0, 0]`

Example 3	
Input	`nums = [4, 3, 2, 1]`
Output	`[3, 2, 1, 0]`

CONSTRAINTS

- 1 <= nums.length <= 10^5

QUESTION

- -10^4 <= nums[i] <= 10^4

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
class Fenwick:
    """Counts of seen values; prefix(i) = how many seen
    values have a rank below i."""

    def __init__(self, size):
        self.size = size
        self.tree = [0] * (size + 1)

    def add(self, i):
        i += 1
        while i <= self.size:
            self.tree[i] += 1
            i += i & -i

    def prefix(self, i):
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i
        return total

def ranks(nums):
    # coordinate compression: value -> 0, 1, 2 ... by size
    return {v: i for i, v in enumerate(sorted(set(nums)))}

# BASE QUESTION (optimal): Fenwick tree over compressed
# values, scanning right to left
# Idea: Compress values to ranks 1..m. Scan from the
# right: the answer for x is the number of already
# inserted values with rank less than rank(x) (a prefix
# sum query), then insert rank(x). Both operations are
# O(log m).
def count_smaller_after(nums):
    # go from the right; before inserting x ask how many
    # already-seen values are strictly smaller than x
    rank = ranks(nums)
    tree = Fenwick(len(rank))
    out = []
    for x in reversed(nums):
        out.append(tree.prefix(rank[x]))
        tree.add(rank[x])
    return out[::-1]
```

GROUP 15 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE | 3 E 5 M

Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

121 Parity Alternating Subarray Query

EASY

Template fit: pre counts neighbouring pairs of equal parity; a range alternates when pre[r] - pre[l] == 0

QUESTION

Given an array nums and multiple queries, each query asks whether a subarray nums[l:r] satisfies the condition that adjacent elements alternate in parity (odd-even-odd-even or even-odd-even-odd). Preprocess the array to answer each query in O(1) time.

EXAMPLES

Example 1	
Input	`nums = [1, 2, 3, 4, 5], queries = [[0, 4], [1, 3]]`
Output	`[true, true]`
Why	`All adjacent pairs alternate in parity.`

Example 2	
Input	`nums = [1, 3, 2, 4], queries = [[0, 1], [0, 3]]`
Output	`[false, false]`
Why	`nums[0]=1 and nums[1]=3 are both odd.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= queries.length <= 10^5
- 0 <= l <= r < nums.length

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Prefix count of equal-parity
# neighbour pairs
# Idea: bad[i] is the number of positions j < i with
# nums[j] and nums[j + 1] of the same parity. The pairs
# inside nums[l..r] are j = l .. r - 1, so the subarray
# alternates when bad[r] - bad[l] == 0.
class ParityQueries:
    """Static array. bad[i] = number of neighbouring pairs
    with equal parity among the first i pairs. A range
    alternates when it contains no such pair."""

    def __init__(self, nums):
        self.bad = [0]
        for a, b in zip(nums, nums[1:]):
            same = a % 2 == b % 2
            self.bad.append(self.bad[-1] + same)

    def alternates(self, l, r):
        # pairs l .. r - 1 are inside the range [l, r]
        return self.bad[r] - self.bad[l] == 0
```

122 Cake Balance Line

EASY

Template fit: the same balance test on the cake heights

QUESTION

A cake is described by a list heights , where heights[i] is the amount of cake sitting at position i (positions are left to right). You want to place a single vertical cut exactly at some position i so that the total amount of cake strictly to the left of the cut equals the total amount strictly to the right of the cut (the cake at position i itself is consumed by the cut and counts for neither side). Return the smallest index i where sum(heights[:i]) == sum(heights[i+1:]) , or -1 if no such position exists.

Function signature

def cake_balance_index(heights: list[int]) -> int:

...

Follow-up (as reported): Do it in one pass and O(1) extra space by tracking a running left sum against total - left - heights[i] .

EXAMPLES

Example 1	
Input	`heights = [1, 7, 3, 6, 5, 6]`
Output	`3`
Why	`Left of index 3 is 1+7+3=11; right of index 3 is 5+6=11.`

Example 2	
Input	`heights = [1, 2, 3]`
Output	`-1`
Why	`No cut position balances the two sides.`

Example 3	
Input	`heights = [2, 1, -1]`
Output	`0`
Why	`Left of index 0 is empty (0); right is 1+(-1)=0.`

CONSTRAINTS

- 1 <= len(heights) <= 10^5 .
- -10^4 <= heights[i] <= 10^4 (heights may be negative in this abstracted version).

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Running left sum
# Idea: The right side is total - left - heights[i].
def cake_balance_index(heights):
    total, left = sum(heights), 0
    for i, h in enumerate(heights):
        if left == total - left - h:
            return i
        left += h
    return -1
```

123 Find the Pivot Index (Balanced Split) in O(n) EASY

Template fit: the template as written: left total equals total - left - x

QUESTION

Given an integer array nums , return the leftmost index i such that the sum of all elements strictly to the left of i equals the sum of all elements strictly to the right of i . The element at i itself is not counted on either side. For the first index, the left sum is 0 ; for the last index, the right sum is 0 . If no such index exists, return -1 . Your solution must run in O(n) time and O(1) extra space.

Function signature

def pivot_index(nums: list[int]) -> int:

...

Follow-up (as reported): If several pivots exist, return them all. Hint: keep scanning after the first match, maintaining the running left sum.

EXAMPLES

Example 1	
Input	`nums = [1, 7, 3, 6, 5, 6]`
Output	`3`
Why	`Left of index 3: 1+7+3 = 11. Right of index 3: 5+6 = 11.`

Example 2	
Input	`nums = [1, 2, 3]`
Output	`-1`
Why	`No index balances the two sides.`

Example 3	
Input	`nums = [2, 1, -1]`
Output	`0`
Why	`Left of index 0 is 0; right is 1 + (-1) = 0.`

CONSTRAINTS

- 1 <= len(nums) <= 10^4
- -1000 <= nums[i] <= 1000

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Running left sum
# Idea: The right sum is total - left - nums[i], so one
# pass is enough.
def pivot_index(nums):
    total, left = sum(nums), 0
    for i, x in enumerate(nums):
        if left == total - left - x:
            return i
        left += x
    return -1
```

124 Range Duplicate Check with Constant-Time Queries

MEDIUM

Template fit: next[i] = next occurrence of the same value; reach[l] = smallest next over l..end; the range l..r is all distinct when reach[l] > r

QUESTION

Decide whether the elements in a range of an array are all unique. Progression: set-based duplicate check -> sliding window -> many range queries where reads dominate, requiring O(1) per query after linear-time preprocessing / updates with linear space.

Base: given an array and a range [l, r], determine whether the elements in that range are all unique.
The problem is staged:

Easy: duplicate detection with a set.
Sliding-window duplicate checking.
Many queries over different ranges: optimize for a read-heavy, write-light workload.

Target for the final stage: O(1) per query, preprocessing (and updates, in one version) in linear time, linear space.

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
INF = float("inf")

# BASE QUESTION (optimal): Next-occurrence array with
# suffix minimum
# Idea: Scan right to left with a map from value to its
# latest index to fill next[i] (infinity if none). Then
# reach[i] = min(next[i], reach[i + 1]). Query(l, r) is
# reach[l] > r.
class RangeUnique:
    """O(n) build, then O(1) per query."""

    def __init__(self, a):
        self.a = list(a)
        self.rebuild()

    def rebuild(self):
        a, n = self.a, len(self.a)
        # nxt[i] = next index holding the same value
        nxt, last = [INF] * n, {}
        for i in range(n - 1, -1, -1):
            nxt[i] = last.get(a[i], INF)
            last[a[i]] = i
        # reach[i] = earliest repeat from index i onward
        self.reach = [INF] * (n + 1)
        for i in range(n - 1, -1, -1):
            self.reach[i] = min(nxt[i], self.reach[i + 1])

    def unique(self, l, r):
        # no repeat starts in [l, r] and ends inside it
        return self.reach[l] > r
```

125 Find a Horizontal Cut That Bisects Rectangle Area

MEDIUM

Template fit: events by height with a running width; accumulate area strip by strip until half the total, then solve inside that strip

QUESTION

Overview: Solve a computational-geometry problem that asks for the horizontal line dividing the combined area of many disjoint rectangles in half. The exercise emphasizes numerical precision, large coordinate ranges, performance at 200,000 rectangles, and careful treatment of partially intersected shapes.

MAIN CODE | helper functions first, then the answer

```python
from collections import defaultdict
from fractions import Fraction


# HELPERS for the base question
def total_area(rects):
    # rects are disjoint (x1, y1, x2, y2)
    return sum((x2 - x1) * (y2 - y1)
               for x1, y1, x2, y2 in rects)

# BASE QUESTION (optimal): Sweep the y events and
# interpolate inside one slab
# Idea: Make events (y, +width) at each rectangle's bottom
# and (y, -width) at its top and sort them. Between
# consecutive event heights the slope is the running sum
# of widths.
def cut_height(rects, share=Fraction(1, 2)):
    want = share * total_area(rects)
    # width gained when the sweep passes each height
    events = defaultdict(int)
    for x1, y1, x2, y2 in rects:
        events[y1] += x2 - x1
        events[y2] -= x2 - x1
    ys = sorted(events)
    width, so_far = 0, Fraction(0)
    for y0, y1 in zip(ys, ys[1:]):
        # inside this slab the covered width is constant
        width += events[y0]
        gain = width * (y1 - y0)
        if width and so_far + gain >= want:
            # the cut is inside this slab: interpolate
            return y0 + (want - so_far) / width
        so_far += gain
    return Fraction(ys[-1])
```

126 Check pivot after removing one element

MEDIUM

Template fit: prefix sums plus first/last position maps: look up the value whose removal would balance both sides

QUESTION

Overview: This question evaluates array-manipulation skills, cumulative-sum/prefix-sum reasoning, and robustness in handling negative values and duplicates, testing algorithmic problem-solving competency in the Coding & Algorithms domain and focusing on practical application rather than purely conceptual understanding.

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Closed-form value to remove,
# found in a dictionary of positions
# Idea: Let P be the prefix sum before i and S the total.
def pivot_after_removal(a):
    # remove ONE element, then is there a pivot: an
    # element
    # whose left sum equals its right sum? Returns
    # (removed
    # index, pivot index) or None. O(n) with two scans.
    n, total = len(a), sum(a)
    prefix = [0]
    for x in a:
        prefix.append(prefix[-1] + x)
    # key[j]: left + left + a[j] when nothing is removed
    key = [2 * prefix[j] + a[j] for j in range(n)]
    # pivot j AFTER the removed index i:
    #   2 * prefix[j] - a[i] = total - a[j]  -> a[i] = key
    # - total
```

```
    where, ok = {}, None
    for j in range(n):
        i = where.get(key[j] - total)
        if i is not None:
            return i, j
        where[a[j]] = j
    # pivot j BEFORE the removed index i:
    #   2 * prefix[j] + a[j] = total - a[i]  -> a[i] =
    # total - key
    where = {}
    for j in range(n - 1, -1, -1):
        i = where.get(total - key[j])
        if i is not None:
            return i, j
        where[a[j]] = j
    return ok
```

127 Gold Chain Split into Two Equal Halves

MEDIUM

Template fit: prefix sums with first/last position maps: find two cut points that leave equal halves around one removed link

QUESTION

Onsite coding interview 1: given a gold chain represented by an array of segment weights, remove exactly one segment and check whether the remaining segments — when reconnected — can be cut once more to give two halves of equal total weight. Follow-up: enumerate all valid (remove, cut) pairs.SWEarrayhashmapmediumFrequencySingle reportLast asked2026-04-21Stageonsite-coding

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
def prefix_sums(a):
    # P[j] = sum of the first j segments
    P = [0]
    for x in a:
        P.append(P[-1] + x)
    return P

# BASE QUESTION (optimal): Prefix sums with first and last
# occurrence maps
# Idea: Let P[j] be the sum of the first j segments.
def gold_chain(a):
    # remove exactly one segment, then cut once into
    # two non-empty halves of equal weight
    n = len(a)
    if n < 3:
        return False
    P = prefix_sums(a)
    total = P[-1]
    # first and last position of every prefix sum (cuts
    # sit between segments, so positions 1..n-1)
    first, last = {}, {}
    for j in range(1, n):
        first.setdefault(P[j], j)
        last[P[j]] = j
    for r in range(n):
        left_over = total - a[r]
        if left_over % 2:
            continue
        half = left_over // 2
        # cut before the removed segment: P[j] == half
        if half in first and first[half] <= min(r, n - 2):
            return True
        # cut after it: P[j] - a[r] == half
```

```
        if half + a[r] in last and last[half + a[r]] > r:
            return True
    return False
```

128 Count the number of ways to split a string. MEDIUM

Template fit: positions of the ones; the answer multiplies the gap sizes around the k-th and 2k-th one (zero ones: choose two cut points)

QUESTION

Count the number of ways to split a string.

MAIN CODE | helper functions first, then the answer

```
from math import comb


# HELPERS for the base question
MOD = 10 ** 9 + 7

# BASE QUESTION (optimal): Count the ones, multiply the
# gap sizes
# Idea: Let ones be the total number of ones. If it is not
# divisible by 3 the answer is 0; if it is 0, any two of
# the n - 1 cut positions work, C(n - 1, 2).
def ways_to_split(s, parts=3, mod=MOD):
    # a binary string cut into `parts` non-empty pieces
    # that
    # hold the same number of 1s (LeetCode 1573 for
    # parts=3)
    n = len(s)
    if n < parts:
        return 0
    ones = [i for i, ch in enumerate(s) if ch == "1"]
    if len(ones) % parts:
        return 0
    if not ones:
        # no ones: any choice of parts - 1 cut points
        # works
        total = comb(n - 1, parts - 1)
        return total % mod if mod else total
    each = len(ones) // parts
    answer = 1
    for cut in range(1, parts):
        # the cut lies in the run of zeros between the
        # last
        # 1 of piece `cut` and the first 1 of the next
        # piece
        gap = ones[cut * each] - ones[cut * each - 1]
        answer *= gap
        if mod:
            answer %= mod
    return answer
```

GROUP 16 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE | 1 E 5 M 2 H

Sorted intervals: merge while they touch, then read the gaps or search

129 Check an Interval Against Existing Intervals for Conflicts — EASY

Template fit: bisect on the starts; a new booking conflicts if the previous one ends after its start or the next one starts before its end

QUESTION

Overview: Check whether one half-open candidate interval conflicts with any interval in a large unsorted collection. Respect strict start and end semantics so ranges that only touch at an endpoint do not conflict.

MAIN CODE | helper functions first, then the answer

```python
from bisect import bisect_right


# BASE QUESTION (optimal): Sorted starts with bisect:
# check only two neighbours
# Idea: Keep the booked intervals sorted by start.
class Calendar:
    """A sorted calendar: O(log n) per check."""

    def __init__(self):
        self.starts, self.ends = [], []

    def book(self, start, end):
        i = bisect_right(self.starts, start)
        # the booking before must end by `start`
        if i > 0 and self.ends[i - 1] > start:
            return False
        # the booking after must start at or after `end`
        if i < len(self.starts) and self.starts[i] < end:
            return False
        self.starts.insert(i, start)
        self.ends.insert(i, end)
        return True
```

130 Find Containing Range — MEDIUM

Template fit: merge once, keep the starts in a list, and bisect to find the one interval that could contain x

QUESTION

Overview: This question evaluates the ability to design appropriate data structures and apply algorithmic reasoning for efficient interval containment queries, emphasizing preprocessing, correctness, and time-space trade-offs.

MAIN CODE | helper functions first, then the answer

```python
from bisect import bisect_left, bisect_right


# BASE QUESTION (optimal): Merge once, then binary search
# the starts
# Idea: After sorting and merging, the starts are
# increasing, so bisect_right(starts, x) - 1 is the only
# candidate; it contains x if its end is at least x.
# Preprocessing is paid once.
class RangeIndex:
    """Merged, sorted, disjoint closed ranges."""

    def __init__(self, ranges=()):
```

```
        self.starts, self.ends = [], []
        self.original = sorted(ranges)
        for s, e in self.original:
            self.insert(s, e)

    def insert(self, s, e):
        # first range ending at or after s ...
        i = bisect_left(self.ends, s)
        # ... up to the last range starting at or before e
        j = bisect_right(self.starts, e)
        if i < j:
            s = min(s, self.starts[i])
            e = max(e, self.ends[j - 1])
        # replace everything the new range touches
        self.starts[i:j] = [s]
        self.ends[i:j] = [e]

    def find(self, x):
        # the merged range containing x, or None
        i = bisect_right(self.starts, x) - 1
        if i >= 0 and self.ends[i] >= x:
            return self.starts[i], self.ends[i]
        return None
```

131 Union of Sorted Interval Lists — MEDIUM

Template fit: walk both sorted lists with two indices, always taking the smaller start next, then the same merge test

QUESTION

Reverse of LC 986: given two individually sorted, internally non-overlapping interval lists, return their union. Follow-ups extend to K lists with a heap, inputs too large for memory, and lists that may overlap internally.SWEintervaltwo-pointerheapstreamingmediumFrequencySingle reportLast asked2026-08-16Stageonsite-coding

MAIN CODE | helper functions first, then the answer

```
import heapq


# HELPERS for the base question
def union_stream(lists, touching=True):
    # lists: sorted iterables of (start, end)
    merged = heapq.merge(*lists)
    cur = None
    for start, end in merged:
        if cur is None:
            cur = [start, end]
            continue
        # touching: [1, 3] and [3, 5] share the point 3
        joined = start < cur[1]
        if touching:
            joined = start <= cur[1]
        if joined:
            cur[1] = max(cur[1], end)
        else:
            # finished one block: emit it as we go
            yield tuple(cur)
            cur = [start, end]
    if cur is not None:
        yield tuple(cur)

# BASE QUESTION (optimal): Two-pointer merge
# Idea: Always take the interval with the smaller start
```

```
# next. If it touches or overlaps the last output interval
# (start <= last end, closed intervals) extend that one,
# otherwise append it.
def union_two(a, b, touching=True):
    return list(union_stream([a, b], touching))
```

132 Range Module (Insert and Point Query) MEDIUM

Template fit: keep sorted lefts and rights; adding bisects to the first interval that touches and the first that starts after, and replaces that block by one merged interval

QUESTION

Design a data structure that tracks a set of half-open ranges [left, right) over the number line and supports two operations: add_range(left, right) : add the half-open interval [left, right) to the tracked set. Overlapping or touching intervals should be merged so the structure always stores a minimal set of disjoint, non- adjacent intervals. query_point(x) : return True if the point x is contained in any currently tracked interval, otherwise False . Because intervals are half-open, a point equal to a right endpoint is NOT contained.

Function signature

class RangeModule:
    def __init__(self) -> None:
        """Initialize an empty set of tracked ranges."""
        ...
    def add_range(self, left: float, right: float) -> None:
        """Add the half-open interval [left, right), merging with any that overlap or
touch."""
        ...
    def query_point(self, x: float) -> bool:
        """Return True iff x lies inside some tracked half-open interval [left,
right)."""
        ...

Follow-up (as reported): First implement with a plain list of intervals scanned linearly. Then keep the merged intervals sorted and use binary search (e.g. bisect) on the left endpoints so query_point runs in O(log N). Discuss inclusive vs exclusive endpoint conventions and float precision.

CONSTRAINTS

- Endpoints and query points are integers or floats with -10^9 <= value <= 10^9 .
- For every add_range call, left < right .
- Up to 10^5 total operations.
- Touching intervals merge: add_range(1, 5) followed by add_range(5, 10) yields one interval [1,
- 10) .

MAIN CODE | helper functions first, then the answer

```
from bisect import bisect_left, bisect_right


# BASE QUESTION (optimal): Merged intervals with bisect
# Idea: Disjoint sorted intervals mean one bisect locates
# the only candidate for a point.
class RangeModule:
    def __init__(self):
        self.lefts = []
        self.rights = []
```

```
    def add_range(self, left, right):
        i = bisect_left(self.rights, left)
        j = bisect_right(self.lefts, right)
        if i < j:
            left = min(left, self.lefts[i])
            right = max(right, self.rights[j - 1])
        self.lefts[i:j] = [left]
        self.rights[i:j] = [right]

    def query_point(self, x):
        i = bisect_right(self.lefts, x) - 1
        return i >= 0 and x < self.rights[i]
```

133 Meetings Minus a Do-Not-Schedule Block

MEDIUM

Template fit: merge first, then for each merged meeting keep the part before the blocked window and the part after it

QUESTION

You are given a list of meetings , each a (start, end) interval with start < end , and one "do-not- schedule" (DNS) block (dns_start, dns_end) . Consider the set of all time points covered by at least one meeting. Return the smallest list of non-overlapping, sorted intervals that covers exactly those points except any point lying inside the DNS block (treat the DNS block as removing the open coverage on (dns_start, dns_end) ; equivalently split coverage at those bounds). Meetings may overlap and are given in any order. Return the result sorted by start time.

Function signature

def subtract_dns(meetings: list[tuple[int, int]],
dns: tuple[int, int]) -> list[tuple[int, int]]:
...

Follow-up (as reported): Generalize to a list of non-overlapping DNS blocks: merge the meetings first, then walk both sorted lists together subtracting each block.

EXAMPLES

Example 1	
Input	meetings = [(1,7),(5,10),(12,30),(22,40),(50,65)], dns = (20,35)
Output	[(1,10),(12,20),(35,40),(50,65)]
Why	Merged coverage is [1,10],[12,40],[50,65]. Removing (20,35) splits [12,40] into [12,20] and [35,40].

Example 2	
Input	meetings = [(0,10)], dns = (0, 10)
Output	[]
Why	The DNS block removes the entire covered range.

Example 3	
Input	meetings = [(0,5),(6,9)], dns = (100, 200)
Output	[(0,5),(6,9)]
Why	The DNS block covers nothing that was scheduled.

CONSTRAINTS

- 1 <= len(meetings) <= 10^5 .
- Coordinates are integers; 0 <= start < end <= 10^9 .

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Merge, then cut
# Idea: After merging, every interval keeps at most a left
# and a right piece around the block.
def merge_intervals(meetings):
    merged = []
    for s, e in sorted(meetings):
        if merged and s <= merged[-1][1]:
            merged[-1] = (
                merged[-1][0],
                max(merged[-1][1], e),
            )
        else:
            merged.append((s, e))
    return merged

# BASE QUESTION (optimal): Merge, then cut
# Idea: After merging, every interval keeps at most a left
# and a right piece around the block.
def subtract_dns(meetings, dns):
    lo, hi = dns
    out = []
    for s, e in merge_intervals(meetings):
        if s < lo:
            out.append((s, min(e, lo)))
        if e > hi:
            out.append((max(s, hi), e))
    return out
```

134 Merge Intervals

MEDIUM

Template fit: the template as written

QUESTION

Merge Intervals. An interval is a representation of a range of values along a number line. For example, [1,3] represents all numbers between 1 and 3, inclusive. Given a collection of intervals, the task is to merge all overlapping intervals. For example, consider the intervals [1,3],[2,6],[8,10],[15,18]. The function should return merged intervals where overlapping intervals are combined. In this case, intervals [1,3] and [2,6] overlap, so they should be merged into [1,6]. Examples intervals = [[1,3], [2,6], [8,10], [15,18]] output: [[1,6], [8,10], [15,18]] intervals = [[1,4], [4,5]] output: [[1,5]] intervals = [[1,4], [2,3]] output: [[1,4]] intervals = [[1,2], [3,4], [5,6], [7,8]] output: [[1,2], [3,4], [5,6], [7,8]]The mergeIntervals function takes an array of intervals and merges all overlapping intervals into a single interval. The process involves sorting the intervals by their start times and then iterating through them to merge where necessary. Here is a step-by-step breakdown: Initial Check: If the input array is empty, the function immediately returns an empty array, as there are no intervals to merge. Sorting: The intervals are sorted by their start times. This ensures that intervals can be processed in a linear sweep, merging adjacent intervals when they overlap. Merging Intervals: Initialize a result list with the first interval. Then, iterate through the sorted intervals starting from the second interval. For each interval, compare it with the last interval in th

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Sort by start and sweep
# Idea: Sort by start. If an interval starts at or before
# the end of the last output interval, extend that end
# with max; otherwise append it. The sort guarantees
# nothing earlier can overlap the new interval.
def merge_intervals(intervals, touching_ints=False):
    out = []
    # sort by start, then extend the last interval while
    # they overlap
    for s, e in sorted(intervals):
        gap = 1 if touching_ints else 0
        if out and s <= out[-1][1] + gap:
            out[-1][1] = max(out[-1][1], e)
```

```
        else:
            out.append([s, e])
    return out
```

135 Find safe travel intervals between planet influences HARD

Template fit: turn each planet into (centre - radius, centre + radius), merge, and read the gaps (including the infinite ends)

QUESTION

You are planning a space route along a one-dimensional line (the x-axis). You are given a list of planets. Each planet is represented by an integer pair ((c, r)):

- **`c`** is the coordinate of the planet's center on the x-axis.
- **`r`** is a non-negative integer radius of its gravitational influence. A point **x** on the x-axis is **unsafe** for a space traveler if it lies within the gravitational influence of at least one planet: [|x - c_i| \le r_i \quad \text{for some planet } i] Equivalently, each planet makes the interval ([c_i - r_i,\; c_i + r_i]) unsafe. Task:
- Consider the entire real line as the possible travel path.
- Compute all **maximal continuous intervals** on this line where the traveler is **safe**, i.e., points not covered by any planet's unsafe interval.
- Return these safe intervals as a sorted list of disjoint intervals. Representation details:
- Represent each safe interval as a pair **`(start, end)`** with **`start < end`**.
- Intervals may be unbounded on the left or right:
- Use **`-inf`** for an interval unbounded to the left (e.g., **`(-inf, a)`**).
- Use **`+inf`** for an interval unbounded to the right (e.g., **`(b, +inf)`**). You may assume the input list of planets can be in any order and may contain overlapping or nested gravitational ranges. Return the safe intervals in increasing order of **`start`**.

You are planning a route along the real x-axis. Each planet is given as a pair (c, r), where c is its center and r is a non-negative radius of gravitational influence. A point x is unsafe if it lies inside at least one planet's influence, meaning |x - c| <= r. So each planet creates an unsafe closed interval [c - r, c + r]. Your task is to return all maximal continuous safe intervals on the real line: regions not covered by any unsafe interval. Represent each safe interval as a tuple (start, end), where the interval means start < x < end. Because JSON/Python literals do not have a direct infinity literal, use None to represent an unbounded side:

- (None, a) means (-infinity, a)
- (b, None) means (b, +infinity)
- (None, None) means the entire real line Return the safe intervals in increasing order. Overlapping, nested, and touching unsafe intervals should be treated as one merged unsafe region.

Constraints
0 <= len(planets) <= 200000-10^9 <= c <= 10^90 <= r <= 10^9

EXAMPLES

```
Input: []
Expected Output: [(None, None)]
```

Explanation: With no planets, no point is unsafe, so the entire real line is safe.

```
Input: [(3, 1), (10, 2)]
Expected Output: [(None, 2), (4, 8), (12, None)]
Explanation: The unsafe intervals are [2, 4] and [8, 12]. Their complement is (-infinity, 2), (4, 8),
    and (12, +infinity).
```

QUESTION

Hints

First convert every planet (c, r) into the unsafe interval [c - r, c + r].Sort the unsafe intervals and merge all overlapping or touching ones. The safe intervals are the gaps before, between, and after the merged intervals.

MAIN CODE | helper functions first, then the answer

```
# HELPERS used by the code below
INF = float("inf")

# HELPERS for the base question
def unsafe_blocks(planets):
    # planet (c, r) makes [c - r, c + r] unsafe; merge
    # overlaps
    merged = []
    spans = sorted((c - r, c + r) for c, r in planets)
    for a, b in spans:
        # sharing even one point joins two blocks
        if merged and a <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], b)
        else:
            merged.append([a, b])
    return merged

# BASE QUESTION (optimal): Merge the unsafe intervals,
# then read off the gaps
# Idea: Sort the unsafe intervals by start and merge any
# that overlap or touch (they share a point, which is
# unsafe).
def safe_intervals(planets):
    # the gaps between unsafe blocks (open intervals)
    merged = unsafe_blocks(planets)
    if not merged:
        return [(-INF, INF)]
    out = [(-INF, merged[0][0])]
    for prev, nxt in zip(merged, merged[1:]):
        out.append((prev[1], nxt[0]))
    out.append((merged[-1][1], INF))
    return out
```

136 Implement a function that processes a stream of interval data. HARD

Template fit: the Range Module add: bisect, merge the block, splice in one interval; intervals() returns the pairs

QUESTION

Implement a function that processes a stream of interval data.

MAIN CODE | helper functions first, then the answer

```
from bisect import bisect_left, bisect_right


# BASE QUESTION (optimal): Sorted disjoint intervals with
# binary search
# Idea: Keep the merged intervals sorted by start. A new
# interval only interacts with the intervals found by two
# binary searches; replace that slice by the single merged
# interval. Touching closed intervals merge.
class IntervalSet:
    """Keeps closed intervals merged and sorted.
    integer=True: [1, 3] and [4, 5] cover 1..5 and merge.
    integer=False: only overlap or a shared end merges."""
```

```
    def __init__(self, integer=True):
        self.starts, self.ends = [], []
        self.gap = 1 if integer else 0
        # length of [a, b] is b - a + extra
        self.extra = 1 if integer else 0
        self.covered = 0

    def length(self, start, end):
        return end - start + self.extra

    def add(self, start, end):
        if end < start:
            raise ValueError("bad interval")
        # first interval that touches the new one, and the
        # first one that is completely to its right
        i = bisect_left(self.ends, start - self.gap)
        j = bisect_right(self.starts, end + self.gap)
        for k in range(i, j):
            # the swallowed intervals leave the total ...
            self.covered -= self.length(self.starts[k],
                                        self.ends[k])
        if i < j:
            start = min(start, self.starts[i])
            end = max(end, self.ends[j - 1])
        # ... and the merged one joins it
        self.starts[i:j] = [start]
        self.ends[i:j] = [end]
        self.covered += self.length(start, end)


    def intervals(self):
```

```
        return list(zip(self.starts, self.ends))
```

GROUP 17 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE | 7 M 1 H

Trie: nested dicts, one character per level

137 URL Request Handler Router

MEDIUM

Template fit: one trie level per URL segment; a node has literal children and one wildcard child; try the literal first

QUESTION

EXAMPLES

```
Register /users/{userId}/photo/{photoId}. A lookup for /users/123/photo/456 matches that registered
    handler.
```

MAIN CODE | helper functions first, then the answer

```python
# HELPERS used by the code below
def segments(path):
    # "/users/7/photo" -> ["users", "7", "photo"]
    return [s for s in path.split("/") if s]

# HELPERS for the base question
class Node:
    def __init__(self):
        # fixed segment -> child node
        self.literal = {}
        # child for a {placeholder} segment
        self.wild = None
        self.wild_name = None
        # handler when the path ends here
        self.handler = None
        # handler for a trailing "*" (rest of the path)
        self.rest = None

# BASE QUESTION (optimal): Segment trie with literal-first
# fallback
# Idea: Each trie node has literal children and one
# wildcard child; the pattern is stored at its last node.
# Lookup tries the literal child first, and if that whole
# branch fails it backs up and tries the wildcard child.
class Router:
    def __init__(self):
        self.root = Node()

    def add(self, pattern, handler=None, method="GET"):
        node = self.root
        for seg in [method] + segments(pattern):
            if seg == "*":
                # all deeper paths use this handler
                node.rest = handler or pattern
                return
            if seg.startswith("{") and seg.endswith("}"):
                node.wild = node.wild or Node()
                node.wild_name = seg[1:-1]
                node = node.wild
            else:
                lit = node.literal
                node = lit.setdefault(seg, Node())
        node.handler = handler or pattern

    def match(self, url, method="GET"):
```

```python
        segs = [method] + segments(url)
        params = {}
```

```
        def walk(node, i):
            if i == len(segs):
                return node.handler
            # 1) exact segment first
            child = node.literal.get(segs[i])
            if child is not None:
                found = walk(child, i + 1)
                if found is not None:
                    return found
            # 2) then a placeholder, remembering its value
            if node.wild is not None:
                params[node.wild_name] = segs[i]
                found = walk(node.wild, i + 1)
                if found is not None:
                    return found
                # undo the value when backtracking
                params.pop(node.wild_name, None)
            # 3) finally a trailing wildcard
            return node.rest

        handler = walk(self.root, 0)
        return handler, dict(params)
```

138 Trie Prefix Search / Autocomplete — MEDIUM

Template fit: walk to the prefix node; return the words under it (autocomplete)

QUESTION

LeetCode 208 + 1268. Build a trie from a dictionary and return all words sharing a query prefix. Standard Google onsite coding.

Build a trie from a dictionary / array of words.
Support searchPrefix(prefix) -> list[str]: return all words that start with the query prefix, lexicographically sorted unless ranking is specified.
Some interviewers phrase the query as "a few letters" or "substring"; clarify whether they truly mean arbitrary substring search or the standard prefix/autocomplete contract before coding.
Common follow-ups: deletion, fuzzy match (one-character edit), autocompletion ranking by frequency, and returning only the best K completions.

EXAMPLES

```
Dict ["apple","app","apricot","banana"], prefix "ap" -> ["app","apple","apricot"].
```

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
class Node:
    def __init__(self):
        self.children = {}
        # how often the word ending here was inserted
        self.freq = 0
        # best (negative freq, word) pairs under this node
        self.top = []

# BASE QUESTION (optimal): Trie with a DFS over sorted
# children
# Idea: Insert each word, marking the last node. For a
# query walk the prefix; if a step is missing return an
# empty list.
class Trie:
    def __init__(self, k=3):
```

```
        self.root, self.k = Node(), k

    def insert(self, word, freq=1):
        node = self.root
        path = [node]
        for ch in word:
            node = node.children.setdefault(ch, Node())
            path.append(node)
        node.freq += freq
        # refresh the top-k list on every node of the path
        for n in path:
            n.top = [x for x in n.top if x[1] != word]
            n.top.append((-node.freq, word))
            n.top.sort()
            del n.top[self.k:]

    def find(self, prefix):
        # walk down the prefix; None if it is missing
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def search_prefix(self, prefix):
        # every word with this prefix, in alphabetical
        # order
        node = self.find(prefix)
```

```
        out = []
        if node is None:
            return out
        stack = [(node, prefix)]
        while stack:
            n, text = stack.pop()
            if n.freq:
                out.append(text)
            for ch in n.children:
                stack.append((n.children[ch], text + ch))
        return sorted(out)

    def suggest(self, prefix):
        # best k completions, read from one node: O(L + k)
        node = self.find(prefix)
        return [w for _, w in node.top] if node else []

    def fuzzy(self, prefix):
        # words whose first letters differ in at most one
        # place
        out, stack = [], [(self.root, "", 0)]
        while stack:
            n, text, wrong = stack.pop()
            if len(text) == len(prefix):
                out += self.search_below(n, text)
                continue
            for ch, child in n.children.items():
                miss = wrong + (ch != prefix[len(text)])
                if miss <= 1:
                    stack.append((child, text + ch, miss))
        return sorted(out)

    def search_below(self, node, text):
        out, stack = [], [(node, text)]
        while stack:
            n, t = stack.pop()
```

```
            if n.freq:
                out.append(t)
            for ch, child in n.children.items():
                stack.append((child, t + ch))
        return out
```

139 Build Prefix Lookup with a Trie

MEDIUM

Template fit: build_trie, then walk to the prefix node and collect every word below it

QUESTION

Overview: Practice a Google coding interview problem focused on build prefix lookup with a trie. The prompt emphasizes edge cases, clean implementation, and verifiable test behavior without revealing the solution.

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
class Node:
    def __init__(self):
        self.children = {}
        # does a word end here?
        self.is_word = False
        # how many words pass through this node
        self.count = 0

# BASE QUESTION (optimal): Trie with a DFS below the
# prefix node
# Idea: Insert every word into nested dicts, marking word
# ends. To answer a prefix, walk down one character at a
# time (no match means no completions), then DFS the
# subtree collecting words.
class PrefixTrie:
    def __init__(self, words=()):
        self.root = Node()
        for w in words:
            self.insert(w)

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, Node())
            node.count += 1
        # a repeated word is only counted as one word
        if not node.is_word:
            node.is_word = True
        else:
            self.undo_count(word)

    def undo_count(self, word):
        node = self.root
        for ch in word:
            node = node.children[ch]
            node.count -= 1

    def walk(self, prefix):
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return None
        return node
```

```
    def root_count(self):
        kids = self.root.children.values()
        return sum(c.count for c in kids)

    def suggest(self, prefix, k=3):
        # first k completions in alphabetical order; the
        # DFS
        # visits children in order and stops after k words
        node = self.walk(prefix)
        out = []
        if node is None:
            return out
        stack = [(node, prefix)]
        while stack and len(out) < k:
            n, text = stack.pop()
            if n.is_word:
                out.append(text)
            # reversed so the smallest letter is popped
            # first
            for ch in sorted(n.children, reverse=True):
                stack.append((n.children[ch], text + ch))
        return out

    def completions(self, prefix):
        # every word with the prefix
        return self.suggest(prefix, k=10 ** 9)
```

140 Greedy Longest-Match Dictionary Tokenizer — MEDIUM

Template fit: the same longest-match walk, with a priority rule to choose between equal-length matches

QUESTION

Phone-screen coding round (R1). Tokenize a text against a dictionary of **`key:id`** entries using greedy longest-match; emit the matched id when a key matches, otherwise output the unmatched character as a literal. One follow-up on changing the tie-breaking / priority rule for which token wins. A second follow-up appears when the dictionary is very large: discuss how to keep the matcher memory- and lookup-efficient at scale.

Input: a text string and a dictionary mapping key (string) -> id.
Sweep the text left-to-right. At each position, find the longest dictionary key that matches a prefix of the remaining text; emit its id. If no key matches at the current position, emit the single character as a literal and advance by one.
Output the concatenation (or list) of ids and literal characters in order.
Follow-up: the interviewer changes the tie-breaking / priority rule for which key wins when multiple match (e.g., not longest but highest-priority by dictionary order). Adapt the data structure so the rule swap is cheap.
Follow-up: the dictionary becomes very large. Discuss how to keep memory and per-position lookup efficient — a compressed / radix trie (or sharding the key space) instead of holding every key in a flat map.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Trie walk keeping the deepest
# (or best-priority) terminal
# Idea: Insert every key into a trie with its id at the
# last node. From each text position walk down while the
# next character exists, remembering the last terminal
# passed.
class Tokenizer:
    def __init__(self, table, priority=None):
        # table: key -> id. priority(key): lower wins;
        # with
        # None the longest match wins
        self.priority = priority
```

```
        self.root = {}
        for key, ident in table.items():
            node = self.root
            for ch in key:
                node = node.setdefault(ch, {})
            # the end of a key stores the key and its id
            node["$"] = (key, ident)

    def better(self, new, old):
        # should the match `new` replace the match `old`?
        if old is None:
            return True
        if self.priority is None:
            # deeper (longer) matches come later: keep
            # them
            return True
        rank = self.priority
        return rank(new[0]) < rank(old[0])

    def longest_at(self, text, i):
        # walk the trie from position i
        node, best, j = self.root, None, i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            end = node.get("$")
            if end and self.better(end, best):
                best = end
        return best

    def tokenize(self, text):
        out, i = [], 0
        while i < len(text):
```

```
            match = self.longest_at(text, i)
            if match is None:
                # no key starts here: keep the character
                out.append(text[i])
                i += 1
            else:
                key, ident = match
                out.append(ident)
                i += len(key)
        return out
```

141 Check if all substrings are dictionary words — MEDIUM

Template fit: for every start walk the trie; a substring of length 3 or more that leaves the trie or lacks END fails the check

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **`>= 3`** is a valid dictionary word (i.e., for all **`i, j`** with **`0 <= i <= j < n`** and **`j - i + 1 >= 3`**, **`s[i..j] in dict`**). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**.
- You may assume dictionary lookups are available (e.g., **`dict.contains(word)`**), or you can preprocess the dictionary. Example:
- If **`s = "cats"`**, you must check **`"cat"`**, **`"ats"`**, and **`"cats"`** (and any other substrings of length >= 3).

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
def first_failing(s, words, min_len=3):
    # the SHORTEST substring (length >= min_len) that is
    # not a
    # word, or None. Short lengths first gives the
    # smallest
    # counterexample, and cheap checks run before
    # expensive ones.
    words = set(words)
    longest = max(map(len, words), default=0)
    for length in range(min_len, len(s) + 1):
        # longer than every word: the first window already
        # fails
        if length > longest:
            return s[:length]
        for i in range(len(s) - length + 1):
            piece = s[i:i + length]
            if piece not in words:
                return piece
    return None

# BASE QUESTION (optimal): Walk a trie from each start
# position
# Idea: For each start i descend the trie along s[i:]. At
# depth 1 and 2 only walk; from depth 3 on the current
# node must be marked as a word end.
def all_substrings_words(s, words, min_len=3):
    return first_failing(s, words, min_len) is None
```

142 Return dictionary words matching a prefix

MEDIUM

Template fit: no trie needed: sort the words and bisect to the first match, then read forward while startswith(prefix)

QUESTION

You are given a list of strings **`words`** (a dictionary) and a string **`prefix`**. Return all words in **`words`** that start with **`prefix`**. Clarifications/constraints:

- Matching is case-sensitive.
- If no words match, return an empty list.
- If **`words`** contains duplicates, include duplicates in the output.
- You may return the result in any order unless the interviewer asks for a specific ordering (e.g., lexicographic).

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Sort once, binary search the
# range
# Idea: With the words sorted, the matches are the words
# from bisect_left(prefix) until the first word that no
# longer starts with the prefix.
class PrefixIndex:
    def __init__(self, words):
        # a sorted copy: all words with a prefix sit
        # together
        self.words = sorted(words)
```

143 Implement Trie Insert and Exact-Word Search

MEDIUM

Template fit: insert is build_trie; search is walk and then END in node

QUESTION

Problem

Implement a trie that supports **`insert(word)`** and **`search(word)`**. Insert adds a lowercase word. Search returns true only when the complete word was previously inserted; a stored word merely sharing the requested prefix is not a match. Process a sequence of operations and return the results of all searches.

Constraints & Assumptions

- There are at most 200,000 operations.
- Every word has length between 1 and 1,000.
- The total number of characters across all operations is at most 1,000,000.
- Words contain lowercase English letters.

Clarifications

- Inserting the same word repeatedly is allowed and has no additional effect.
- A node must distinguish an inserted word ending there from a prefix only.
- Return booleans in the same order as search operations appear.

Examples

```
operations = [insert("apple"), search("apple"), search("app"), insert("app"), search("app")] output = [tru
e, false, true]
```

Hints

```
Represent prefixes Each edge consumes one character and each node represents the prefix along its path.
```

```
Mark complete words Reaching the final character is not enough unless the terminal node records an inserti
on.
```

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
class Node:
    def __init__(self):
        self.children = {}
        self.end = False

# BASE QUESTION (optimal): Trie of nodes with an end-of-
# word flag
# Idea: Each node has children by character and a flag.
# insert creates missing nodes and marks the last one.
# search walks the word and requires the flag at the end;
# startsWith only requires that the walk succeeds.
class Trie:
    def __init__(self):
        self.root = Node()

    def walk(self, text):
        # follow the letters; None if the path breaks
        node = self.root
        for ch in text:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def insert(self, word):
```

```
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, Node())
        node.end = True

    def search(self, word):
        # the whole word must have been inserted
        node = self.walk(word)
        return node is not None and node.end

    def starts_with(self, prefix):
        return self.walk(prefix) is not None
```

144 Implement Longest-Match Text Replacement

HARD

Template fit: at each text position walk the trie as far as it goes and remember the last END; replace the longest match and jump past it

QUESTION

You are given a text string and a dictionary of replacement entries. Each dictionary entry has the form **`token:id`**, where **`token`** is a string pattern and **`id`** is the value that should be emitted when that token is matched. Implement a function that scans the text from left to right and produces an output sequence using greedy longest-prefix matching:

1. At the current text position, find all dictionary tokens that match the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its corresponding **`id`** to the output, and advance by the length of that token.
3. If no token matches, append the original character at the current position to the output and advance by one character.
4. Continue until the entire text has been processed. Example:

```
text = "abcdxyz" dictionary = { "ab": "1", "abc": "2", "xy": "3" }
```

Output:

```
["2", "d", "3", "z"]
```

Explanation: At index **`0`**, both **`"ab"`** and **`"abc"`** match, so choose the longest token **`"abc"`** and output **`"2"`**. Then **`"d"`** has no match, so output it literally. Then **`"xy"`** matches and outputs **`"3"`**. Finally, **`"z"`** is output literally. Follow-up: How would you adapt your design if the matching priority changed from longest match to a configurable priority rule, such as dictionary insertion order, highest score, or a custom comparator?

Part 1: Longest-Match Text ReplacementGiven a text string and a dictionary that maps token strings to replacement id strings, scan the text from left to right. At each position:

1. Find every token that matches the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its replacement id to the output, and advance by the token length.
3. If no token matches, append the original character at that position as a one-character string and advance by 1. Return the final output as a list of strings.

Constraints

0 <= len(text) <= 1000000 <= len(dictionary) <= 10000Each token in dictionary is non-empty.The sum of all token lengths is at most 100000.Matching is case-sensitive.

EXAMPLES

```
Input: ("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})
Expected Output: ["2", "d", "3", "z"]
```

Explanation: Both 'ab' and 'abc' match at index 0, so the longer token 'abc' is chosen.

QUESTION

```
Input: ("aaaaa", {"a": "X", "aa": "Y"})
Expected Output: ["Y", "Y", "X"]
```

Explanation: Greedy longest matching picks 'aa' twice, then the final 'a'.

Hints

A trie lets you test all token prefixes starting at one position without checking every token from scratch.While walking forward from a position, remember the last trie node that represented a complete token.

Part 2: Configurable-Priority Text ReplacementImplement the same left-to-right text replacement process, but the match-selection rule is configurable. You are given: - a text string - a list of replacement entries in insertion order - a priority rule Each replacement entry is a tuple: (token, replacement_id, score). At each text position, find all entries whose token matches the substring starting there. If one or more entries match, choose exactly one using the selected priority rule:

- 'longest': choose the longest matching token; if tied, choose the earlier entry in the list - 'insertion': choose the earliest matching entry in the input list - 'score': choose the matching entry with the highest score; if tied, choose the longer token; if still tied, choose the earlier entry in the list Append the chosen replacement_id to the output and advance by the chosen token length. If nothing matches, append the original character and advance by 1. Return the final output as a list of strings.

Constraints

0 <= len(text) <= 1000000 <= len(entries) <= 10000Each token is non-empty.Tokens in entries are unique.The sum of all token lengths i

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Trie walk that remembers the
# last token end
# Idea: Build a trie of the tokens (each token's last node
# stores its id). At position i walk down the trie along
# the text; whenever the current node stores an id, record
# it and its length.
class Replacer:
    def __init__(self, mapping):
        # mapping: token -> id. A trie shares the
        # prefixes.
        self.root = {}
        for token, ident in mapping.items():
            node = self.root
            for ch in token:
                node = node.setdefault(ch, {})
            node["$"] = (token, ident)

    def candidates(self, text, i):
        # every token that matches at position i, shortest
        # first (all ends along the trie path, not just
        # the last)
        out, node, j = [], self.root, i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            if "$" in node:
                out.append(node["$"])
        return out


    @staticmethod
    def edge(text, i, size):
        # letters must not touch the match on either side
        before = i == 0 or not text[i - 1].isalnum()
        end = i + size
```

```
        after = end == len(text)
        after = after or not text[end].isalnum()
        return before and after
```

GROUP 18 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE | 1 E 6 M 1 H

A dict for lookup plus an ordered container for who is next (lazy deletion)

145 Implement Batched Undo/Redo Layer

EASY

Template fit: two stacks (undo, redo) of batches; a batch records (old value, new value) per property; undo applies the old values and moves the batch

QUESTION

You are implementing a simplified document layer for a design tool. The layer stores properties as a **`map<string, string>`**. Implement a class that supports the following operations:

- **`beginBatch()`**: start a batch of edits.
- **`setProperty(key, value)`**: set or overwrite a property inside the current batch.
- **`endBatch()`**: commit the batch as one undoable action.
- **`undo()`**: revert the most recently committed batch.
- **`redo()`**: reapply the most recently undone batch. Requirements:

1. A batch may update the same key multiple times.
2. **`undo()`** must restore the exact document state from before that batch.
3. **`redo()`** must restore the exact document state after that batch.
4. If a property did not exist before the batch, then **`undo()`** should remove that property.
5. If **`undo()`** has been called and then a new batch is committed, the redo history must be cleared.
6. Follow-up: make the history memory-efficient. Instead of storing every intermediate edit in a batch, compact the batch so that each key appears at most once in history while still preserving enough information for correct **`undo()`** and **`redo()`**. A useful compact record for one key is:

```
Change { string key; string value; // final value after the batch string old_value; // value before the ba
tch bool has_old_value; // whether the key existed before the batch }
```

Explain how you would build and use such a compact history so that repeated updates to the same key inside one batch do not waste memory.

Part 1: Implement Batched Undo/Redo for Document PropertiesYou are given a sequence of operations for an initially empty document represented as a map<string, string>. A batch starts with ('begin',), contains zero or more ('set', key, value) operations, and ends with ('end',). Each committed batch must be undoable and redoable as one unit. Operation ('snapshot',) asks you to append the current document to the answer; snapshots may appear while a batch is open and should reflect the live document state. If a key is set multiple times in one batch, undo must restore the value from before the batch began. If a key did not exist before the batch, undo must remove it. If a new non-empty batch is committed after an undo, the redo history is cleared. If undo or redo is impossible, do nothing. If end closes an empty batch, treat it as a no-op.
Constraints
0 <= len(operations) <= 200000The total number of ('set', key, value) operations is at most 200000Keys and values are strings of length 1 to 50Batches are not nestedIf a batch contains no set operations, committing it does not create history

EXAMPLES

```
Input: [('begin',), ('set', 'color', 'red'), ('end',), ('snapshot',), ('undo',), ('snapshot',),
    ('redo',), ('snapshot',)]
Expected Output: [{'color': 'red'}, {}, {'color': 'red'}]
```

Explanation: A single batch is committed, undone, and redone.

QUESTION

```
Input: [('begin',), ('set', 'a', '1'), ('end',), ('begin',), ('set', 'a', '2'), ('set', 'a', '3'),
    ('set', 'b', '9'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]
Expected Output: [{'a': '3', 'b': '9'}, {'a': '1'}, {'a': '3', 'b': '9'}]
```

Explanation: The second batch updates 'a' twice and adds 'b'. Undo must restore the exact pre-batch state.

Hints

For each key touched in the current batch, record its old value only the first time that key is edited in the batch.Use two stacks: one for committed batches that can be undone, and one for undone batches that can be redone. Clear the redo stack when a new non-empty batch is committed.

Part 2: Compact a Batch into Memory-Efficient Change RecordsYou are given the document state before a batch and the raw setProperty edits that happened inside that batch. Multiple edits may target the same key. Build a compact history so that each key appears at most once, while still storing enough informa

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# HELPERS for the base question
class Change:
    """One key in one batch: value before and after."""

    def __init__(self, key, old, has_old, new):
        self.key, self.old, self.has_old, self.new = (
            key, old, has_old, new)

# BASE QUESTION (optimal): Store only the changed keys per
# batch
# Idea: While a batch is open keep {key: (old, new)} with
# the old value recorded only the first time a key is
# touched. endBatch pushes it on the undo stack and clears
# the redo stack.
class Layer:
    def __init__(self, history=None):
        self.props = {}
        self.undo_stack = deque(maxlen=history)
        self.redo_stack = []
        # open batches (a stack, so batches can nest):
        # each is
        # a dict key -> Change, so a key appears only once
        self.open = []

    def begin_batch(self):
        self.open.append({})

    def set_property(self, key, value):
        if not self.open:
            # outside a batch: a batch of one change
            self.begin_batch()
            self.set_property(key, value)
            self.end_batch()
            return
        batch = self.open[-1]
        if key in batch:
            # touched again: keep the ORIGINAL old value
            # and
            # only update the final value
            batch[key].new = value
        else:
            batch[key] = Change(key, self.props.get(key),
```

```
                            key in self.props, value)
        self.props[key] = value

    def end_batch(self):
        batch = self.open.pop()
        if self.open:
            self.merge(self.open[-1], batch)
        elif batch:
            # only the outermost batch is one undoable
            # action;
            # a new action clears the redo history
            self.undo_stack.append(list(batch.values()))
            self.redo_stack.clear()

    @staticmethod
    def merge(parent, child):
        for key, change in child.items():
            if key in parent:
                parent[key].new = change.new
            else:
                parent[key] = change

    def undo(self):
        if not self.undo_stack:
            return False
        changes = self.undo_stack.pop()
        for c in changes:
            if c.has_old:
                self.props[c.key] = c.old
            else:
                # the key did not exist before: remove it
                self.props.pop(c.key, None)
        self.redo_stack.append(changes)
        return True

    def redo(self):
        if not self.redo_stack:
            return False
        changes = self.redo_stack.pop()
        for c in changes:
            self.props[c.key] = c.new
        self.undo_stack.append(changes)
        return True
```

146 Design a restaurant waitlist system

MEDIUM

Template fit: a segment tree of party sizes gives the first entry that fits in O(log n); the dict maps ids to slots

QUESTION

You are implementing the waitlist system for a restaurant. Parties arrive over time, can cancel, and get seated when a table becomes available.

Rules

- Each party has a unique **`partyId`**, a **`name`**, and a **`size`** (number of people).
- Parties are seated in **arrival order**, but only if they **fit** the available table.
- When a table of capacity **`c`** becomes available, you must seat the **earliest-arrived** party whose **`size <= c`**.
- If no party fits, the table remains unused for that event.

Operations to support Design a

data structure / class that supports the following operations efficiently:

1. **`addParty(name, size) -> partyId`**

- Adds a new party to the end of the waitlist and returns its id.

QUESTION

2. **`cancelParty(partyId) -> bool`**

- Removes the party from the waitlist if present.
- Returns **`true`** if removed, otherwise **`false`**.

3. **`seatTable(capacity) -> partyId or null`**

- Finds and removes the earliest party with **`size <= capacity`**.
- Returns that party's id, or **`null`** if nobody fits.

4. **`getPosition(partyId) -> int`**

- Returns how many parties are ahead of this party in the current waitlist order (0-based).
- If the party is not in the waitlist, return **`-1`**.

Constraints

- Up to **`2 * 10^5`** operations.
- Party size is a small positive integer (e.g., 1–20).
- Aim for better-than-linear time per operation (especially for **`seatTable`** and **`getPosition`**).

Example

(one possible interaction) - addParty("A", 4) -> id1 - addParty("B", 2) -> id2 - seatTable(2) -> id2 (B fits and is earliest among those that fit) - seatTable(4) -> id1

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
INF = float("inf")

# BASE QUESTION (optimal): Segment tree of minimum size
# over arrival order
# Idea: Each arrival gets the next position. The tree
# stores, for each range, the smallest waiting party size
# (infinity for empty or seated slots).
class Waitlist:
    """Earliest party that fits: a segment tree of the
    smallest size over arrival slots."""

    def __init__(self, capacity=1024):
        self.cap = capacity
        self.tree = [INF] * (2 * capacity)
        self.count = 0
        self.slot, self.who = {}, {}

    def update(self, i, value):
        i += self.cap
        self.tree[i] = value
        while i > 1:
            i //= 2
            self.tree[i] = min(self.tree[2 * i],
                               self.tree[2 * i + 1])

    def add(self, party_id, name, size):
        self.slot[party_id] = self.count
        self.who[self.count] = party_id
        self.update(self.count, size)
        self.count += 1

    def cancel(self, party_id):
        i = self.slot.get(party_id)
        if i is None or self.tree[i + self.cap] == INF:
            return False
        self.update(i, INF)
        return True
```

```
    def seat(self, capacity):
        # no party small enough: nobody is seated
        if self.tree[1] > capacity:
            return None
        i = 1
        # go left whenever the left half holds a party
```

```
        # that fits
        while i < self.cap:
            left = self.tree[2 * i] <= capacity
            i = 2 * i if left else 2 * i + 1
        self.update(i - self.cap, INF)
        return self.who[i - self.cap]
```

147 Implement Python's itertools.tee

MEDIUM

Template fit: one deque per consumer; whoever runs out pulls the next value from the source and appends it to every queue

QUESTION

Implement the behavior of Python's **`tee`**: accept one iterator and an integer **n**, then return **n** independent iterators over the same source sequence. Define the required function and classes yourself and improve a naive design toward an efficient shared-state implementation.SWEiteratorqueuedata-structureobject-designmediumFrequencySingle reportLast asked2026-08-20Stageonsite-coding

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# BASE QUESTION (optimal): One deque per consumer, filled
# lazily
# Idea: When a consumer's deque is empty it pulls the next
# item from the source and appends it to every consumer's
# deque, then pops its own.
def tee_queues(iterable, n=2):
    source = iter(iterable)
    # one waiting line of values per consumer
    queues = [deque() for _ in range(n)]

    def consumer(mine):
        while True:
            if not mine:
                try:
                    value = next(source)
                except StopIteration:
                    return
                # give the new value to everybody
                for q in queues:
                    q.append(value)
            yield mine.popleft()

    return tuple(consumer(q) for q in queues)
```

148 Store With Cheapest-Item Removal

MEDIUM

Template fit: the order is a heap keyed by (price, timestamp); remove_cheapest pops it

QUESTION

Design a Store that holds items. Each item has a string value , an integer price in cents, and an integer timestamp . Support: add(value, price, timestamp) : insert an item. remove_cheapest() : remove and return the value of the item with the smallest price. If several items share the smallest price, remove the one with the smallest timestamp . Return None if the store is empty. A heap keyed by (price, timestamp) gives O(log n) per operation.

Function signature

```
class Store:
    def __init__(self) -> None:
        ...
    def add(self, value: str, price: int, timestamp: int) -> None:
        ...
    def remove_cheapest(self) -> str | None:
        """Remove and return the value of the lowest-priced item (ties broken by
earliest timestamp)."""
        ...
```

Follow-up (as reported): Improve remove_cheapest from O(log n) to O(1) amortized using the price bound: keep a length-10001 array indexed by price (each slot a queue/count of items) and track the smallest non-empty price index seen so far.

EXAMPLES

Example 1	
Input	`add("a",5,1); add("b",2,2); add("c",8,3); remove_cheapest()`
Output	`"b"`
Why	`"b" has the smallest price (2 cents).`

Example 2	
Input	`add("x",7,10); add("y",7,4); remove_cheapest()`
Output	`"y"`
Why	`Prices tie at 7; "y" wins on the earlier timestamp (4 < 10).`

Example 3	
Input	`remove_cheapest() # on an empty store`
Output	`None`

CONSTRAINTS

- Prices are integers with 1 <= price <= 10000 (i.e. 1 cent up to $100.00).
- Up to 10^6 operations.

MAIN CODE | helper functions first, then the answer

```python
import heapq


# BASE QUESTION (optimal): A heap keyed by price and
# timestamp
# Idea: The smallest (price, timestamp) is always at the
# top of the heap.
class Store:
    def __init__(self):
        self.heap = []
```

```
    def add(self, value, price, timestamp):
        heapq.heappush(
            self.heap, (price, timestamp, value)
        )

    def remove_cheapest(self):
        if not self.heap:
            return None
        return heapq.heappop(self.heap)[2]
```

149 Round-Robin Job Scheduler

MEDIUM

Template fit: a deque of [job, time left]; pop, run for min(quantum, left); push back at the end if unfinished, else record the finish time

QUESTION

The third question was "not really leetcode style" but was a job-scheduling problem solved with a round- robin approach. Reconstruct it as follows. You are given a list of jobs, each with a unique string id and a positive integer burst (the number of time units of CPU it needs). All jobs are available at time 0 and are placed in a ready queue in the given order. The CPU runs jobs round-robin with a fixed time quantum : it takes the job at the front of the queue, runs it for min(quantum, remaining) time units, and then — if the job still has work left — puts it at the back of the queue; otherwise the job completes at that moment. Return the list of (id, completion_time) pairs in the order the jobs complete.

Function signature

def round_robin(jobs: list[tuple[str, int]], quantum: int) -> list[tuple[str, int]]:
    """Simulate round-robin scheduling; return (id, completion_time) pairs in completion order."""
    ...

Follow-up (as reported): Report note ("a constraint was changed"): add per-job arrival times so a job only enters the queue once the clock reaches its arrival. Hint: keep a pointer over arrival-sorted jobs and admit them each time the clock advances.

EXAMPLES

Example 1

Input	`jobs = [("A", 4), ("B", 3), ("C", 2)], quantum = 2`
Output	`[("C", 6), ("A", 8), ("B", 9)]`
Why	`t=0-2 A (rem 2), t=2-4 B (rem 1), t=4-6 C completes at 6, t=6-8 A completes at 8, t=8-9 B completes at 9.`

Example 2

Input	`jobs = [("X", 5)], quantum = 2`
Output	`[("X", 5)]`
Why	`One job, run in slices 2+2+1, finishing at time 5.`

Example 3

Input	`jobs = [("P", 1), ("Q", 1)], quantum = 3`
Output	`[("P", 1), ("Q", 2)]`
Why	`Each finishes within a single quantum: P at 1, Q at 2.`

CONSTRAINTS

- 1 <= len(jobs) <= 10^4
- 1 <= burst <= 10^6 , 1 <= quantum <= 10^6

QUESTION

- Job ids are unique, non-empty strings.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# BASE QUESTION (optimal): A queue of jobs
# Idea: Run the front job for one slice and send it to the
# back if it is not finished.
def round_robin(jobs, quantum):
    queue = deque([jid, burst] for jid, burst in jobs)
    clock, done = 0, []
    while queue:
        job = queue.popleft()
        run = min(quantum, job[1])
        clock += run
        job[1] -= run
        if job[1] > 0:
            # not finished: back of the line
            queue.append(job)
        else:
            done.append((job[0], clock))
    return done
```

150 Aggregate FizzBuzz Offer Matches

MEDIUM

Template fit: a deque of offers per type; a new offer matches against the front of the opposite queue, partial quantities stay at the front

QUESTION

You are given a list of offers in priority order (earliest = highest priority). Each offer is a dict with keys type (either "Fizz" or "Buzz"), name (a string), and quantity (a positive integer). Fizz offers match against Buzz offers on a first-come, first-served basis, and offers may be partially fulfilled. Process the offers in order, maintaining a FIFO queue of unmatched Fizz offers and a FIFO queue of unmatched Buzz offers. When a new offer arrives, match it against the front of the opposite queue repeatedly, each time consuming min(remaining quantities) and recording a match (fizz_name, buzz_name, matched_quantity) ; drop an offer from the queue once its quantity hits 0. If the arriving offer still has quantity left when the opposite queue empties, enqueue it in its own queue. Aggregate matches so the same (fizz_name, buzz_name) pair is reported once with its summed quantity. Return the aggregated matches, in the order each pair was first produced, formatted as: "FizzBuzz! <quantity> <FizzName>:<BuzzName>" .

Function signature

def fizzbuzz_matches(offers: list[dict]) -> list[str]:

...

EXAMPLES

Example 1	
Input	`offers = [ {"type": "Fizz", "name": "A", "quantity": 5}, {"type": "Buzz", "name": "X", "quantity": 3}, {"type": "Buzz", "name": "Y", "quantity": 4}, ]`
Output	`["FizzBuzz! 3 A:X", "FizzBuzz! 2 A:Y"]`
Why	`A(5) waits. X(3) matches A -> (A,X,3), A now 2. Y(4) matches A's remaining 2 -> (A,Y,2); Y's leftover 2 waits with no Fizz to match.`

QUESTION

Example 2	
Input	`offers = [ {"type": "Fizz", "name": "A", "quantity": 3}, {"type": "Buzz", "name": "X", "quantity": 1}, {"type": "Buzz", "name": "X", "quantity": 2}, ]`
Output	`["FizzBuzz! 3 A:X"]`
Why	A matches the first X for 1 and the second X for 2. Both are the pair (A, X), so they aggregate to a single "FizzBuzz! 3 A:X".

CONSTRAINTS

- 0 <= len(offers) <= 10**4 ; quantities are positive integers.

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# BASE QUESTION (optimal): Two FIFO queues and an ordered
# total
# Idea: Match against the opposite queue's front until the
# offer or the queue is empty.
def fizzbuzz_matches(offers):
    queues = {"Fizz": deque(), "Buzz": deque()}
    totals = {}
    for offer in offers:
        mine = queues[offer["type"]]
        other = queues[
            (
                "Buzz"
                if offer["type"] == "Fizz"
                else "Fizz"
            )
        ]
        left = offer["quantity"]
        while left and other:
            front = other[0]
            m = min(left, front[1])
            fizz_name = (
                offer["name"]
                if offer["type"] == "Fizz"
                else front[0]
            )
            buzz_name = (
                front[0]
                if offer["type"] == "Fizz"
                else offer["name"]
            )
            totals[(fizz_name, buzz_name)] = (
                totals.get((fizz_name, buzz_name), 0)
                + m
            )
            left -= m
            front[1] -= m
            if front[1] == 0:
                other.popleft()
        if left:
            mine.append([offer["name"], left])
    return [
        f"FizzBuzz! {q} {f}:{b}"
        for (f, b), q in totals.items()
    ]
```

151 Restaurant Waitlist: join / delete / find_first_match

MEDIUM

Template fit: a deque per party size with tickets; delete only removes the name from the dict; find_first_match skips dead tickets

QUESTION

Onsite coding round: design and implement a restaurant waitlist data structure supporting join, delete, and find_first_match(table_size). The find call returns the earliest-joined user whose party size fits the given table.

Implement a class with these methods

join(user) — append user (with their party_size) to the waitlist.
delete(user) — remove user from the waitlist (could be anywhere in the queue).
find_first_match(table_size: int) — return the earliest user still on the waitlist whose party_size <= table_size. Do not remove on find.

Implicit constraints

Order of joins matters (FIFO within those that match).
Users may be deleted out of order, so a plain array/linked list with O(1) join cannot give O(1) delete and find together.

EXAMPLES

```
join(Alice, size=2)
join(Bob, size=4)
join(Carol, size=6)
find_first_match(3) -> Alice   // only Alice fits
find_first_match(5) -> Alice   // Alice still earliest matching
delete(Alice)
find_first_match(5) -> Bob
find_first_match(2) -> null     // no one fits
```

MAIN CODE | helper functions first, then the answer

```python
from collections import defaultdict, deque


# BASE QUESTION (optimal): Queue per party size with lazy
# deletion
# Idea: join stores (sequence number, name) in the queue
# for its size and in a map from name to its record.
# delete just removes the record from the map.
class Waitlist:
    def __init__(self):
        # arrival counter: lower means joined earlier
        self.seq = 0
        # person -> their arrival number (people still
        # waiting)
        self.live = {}
        # party size -> line of (arrival number, person)
        self.queues = defaultdict(deque)

    def join(self, name, size):
        self.seq += 1
        self.live[name] = self.seq
        self.queues[size].append((self.seq, name))

    def delete(self, name):
        # lazy: the entry in its queue is skipped later
        self.live.pop(name, None)

    def front(self, queue):
        # drop deleted people from the front of a queue
        while queue:
            seq, name = queue[0]
            if self.live.get(name) == seq:
                break
            queue.popleft()
```

```
        return queue[0] if queue else None

    def find_first_match(self, table):
        best = None
        for size, queue in self.queues.items():
            if size > table:
                continue
            head = self.front(queue)
            # the earliest head among all sizes that fit
            if head and (not best or head[0] < best[0]):
                best = head
```

```
        return best[1] if best else None
```

152 Implement LRU Cache. HARD

Template fit: dict key -> node plus a doubly linked list; every get/put moves the node to the front; evict from the back (or use OrderedDict.move_to_end)

QUESTION

Your program is running slowly because it's accessing data from disk over and over again. To improve the performance, you want to build a simple key-value store to cache this data in memory, but you also want to limit the amount of memory used. You decide to build a caching system that only keeps the N most recently used items—also known as a least recently used (LRU) cache. Write a class LRUCache(n) that accepts a size limit n. It should support a set(key, value) method for inserting or updating items and a get(key) method for retrieving items. Can you implement a solution where both of these methods run in O(1) time? Pythonclass LRUCache(n): set(key, value) get(key) Examples To make room for new items, the least recently used item should be removed. An item is 'used' whenever it is set, retrieved, or updated. Pythoncache = LRUCache(2)

Limit of 2 items cache.set('user1',

'Alex') cache.set('user2', 'Brian') cache.set('user3', 'Chris') Here user1 is empty because it was the least recently used and thus removed to make room for user3: Pythoncache.get('user1')

=> None cache.get('user2')

=> 'Brian' cache.get('user3')

=> 'Chris'Hint 1If you're having

trouble getting started, think about which data structure(s) you can use to solve this problem. Which one is best for a key-value store? Which one can help you store items in order of recency?Hint 2You'll need to be able to find and remove the least recent item in O(1) time, so some sort of ordered list is necessary. An array

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
class Node:
    __slots__ = ("key", "value", "prev", "next")

    def __init__(self, key=None, value=None):
        self.key, self.value = key, value
        self.prev = self.next = None

# BASE QUESTION (optimal): Hash map plus doubly linked
# list (built by hand)
# Idea: Each key maps to a node of a doubly linked list
# ordered by recency, with sentinel head and tail nodes so
# no operation has to special-case an empty list.
class LRUCache:
    """get and put in O(1): a hash map plus a doubly
    linked list, most recently used item at the front."""
```

```
    def __init__(self, capacity):
        self.cap, self.map = capacity, {}
        # sentinel nodes mean no special cases for the
        # ends
        self.head, self.tail = Node(), Node()
        self.head.next = self.tail
        self.tail.prev = self.head

    def unlink(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def push_front(self, node):
        node.next, node.prev = self.head.next, self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        node = self.map.get(key)
        if node is None:
            return -1
        # reading makes it the most recently used
        self.unlink(node)
        self.push_front(node)
        return node.value

    def put(self, key, value):
        node = self.map.get(key)
```

```
        if node:
            node.value = value
            self.unlink(node)
            self.push_front(node)
            return
        if len(self.map) == self.cap:
            # full: evict the least recently used (at the
            # back)
            oldest = self.tail.prev
            self.unlink(oldest)
            del self.map[oldest.key]
        node = Node(key, value)
        self.map[key] = node
        self.push_front(node)
```

GROUP 19 OF 43 | 7 QUESTIONS, ONE CODE TEMPLATE | 2 E 4 M 1 H

Rolling state: the next state comes only from the previous one

153 Climbing Stairs — EASY

Template fit: state = the last two counts: a, b = b, a + b, repeated n - 1 times

QUESTION

Given an integer n, representing the number of steps in a staircase, write a function climbStairs that returns the number of distinct ways to climb to the top of the staircase. Each time you can either climb 1 step or 2 steps. You may assume that n is a non-negative integer. Examples Input: n = 1 Output: 1 Explanation: There is only 1 way to climb a staircase with 1 step.

Input: n = 2 Output: 2 Explanation: There are 2 ways to climb a staircase with 2 steps:

1. Climb 1 step + 1 step 2. Climb 2 steps directly Input: n = 3 Output: 3 Explanation: There are 3 ways to climb a staircase with 3 steps:
1. Climb 1 step + 1 step + 1 step 2. Climb 1 step + 2 steps 3. Climb 2 steps + 1 step Input: n = 4 Output: 5 Explanation: There are 5 ways to climb a staircase with 4 steps:
1. Climb 1 step + 1 step + 1 step + 1 step 2. Climb 1 step + 1 step + 2 steps 3. Climb 1 step + 2 steps + 1 step 4. Climb 2 steps + 1 step + 1 step 5. Climb 2 steps + 2 stepsChallenge 1Can you come up with a solution that has a O(n) time complexity?Challenge 2Can you come up with a solution that has a O(1) space complexity?Solution 1: Dynamic Programming Approach In this solution, we use dynamic programming to compute the number of distinct ways to climb n stairs. The idea is to build up the solution incrementally by solving subproblems. We initialize a list ways where ways[i] represents the number of ways to reach the ith step. We start by setting the base cases: ways[0] = 1 (1 way to stay at the ground) ways[1] =

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Two running values
# Idea: Keep ways for the previous two steps and advance:
# (a, b) = (b, a + b). Start with ways(0) = ways(1) = 1.
# No table and no recursion.
def climb(n):
    # ways(n) = ways(n - 1) + ways(n - 2): O(n), O(1)
    # space
    a, b = 1, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b if n >= 1 else 1
```

154 Fibonacci Numbers — EASY

Template fit: the same pair update starting from (0, 1)

QUESTION

Write a function fib(n) that returns the nth Fibonacci number. The Fibonacci sequence is defined as F(n) = F(n-1) + F(n-2) with F(0) = 1 and F(1) = 1, which produces the following pattern: 1, 1, 2, 3, 5, 8, 13 ... Examples Pythonfib(0)

=> 1 fib(1)

=> 1 fib(2)

=> 2 fib(3)

=> 3 ... fib(10)

=> 89 fib(20)

=> 10946Hint 1The Fibonacci sequence

QUESTION

is defined recursively since it refers to itself.Hint 2We can start with a recursive implementation and optimize it.Hint 3Each call to F(n) has two subproblems: F(n-1) and F(n-2). Can we memoize these calculations?Challenge 1Does your solution work in O(n) time?This question is a classic example of how dynamic programming, or memoization, can help us solve problems faster by reducing repetitive computations. In this first version, we recursively call the fib function twice, and then we call it twice more in every subsequent recursive call, so the total number of operations expands to O(2^n)! To improve the runtime of our program, let's save our prior calculations in a dictionary called memo and pass it into our function. Now, if we've already found fib(n) before, our function will use this stored value instead of recursing again. Time Complexity: The time complexity of the memoized version is O(n). This is because each Fibonacci number from 0 to n is computed only once and then stored in the memo dictionary. The recursive calls will now simply look up the value in the dictionary if it has already been computed, reducing redundant

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Iteration with two variables
# Idea: Keep the previous two values and advance n times.
# Memoising the recursion gives the same time with O(n)
# memory; the loop needs no memory and no recursion.
def fib(n):
    # bottom-up with two variables: O(n) time, O(1) memory
    a, b = 1, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

155 Maximize sum without choosing adjacent elements

MEDIUM

Template fit: exactly the house robber update on the array

QUESTION

Overview: This question evaluates a candidate's understanding of array-based optimization and dynamic programming concepts for computing a maximum sum under non-adjacent selection constraints.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Two running values (take, skip)
# Idea: take is the best total that includes the current
# element, skip the best that excludes it. For the next
# element: new take = skip + x (the previous element must
# have been skipped), new skip = max(take, skip).
def rob_line(nums):
    # take: best when the current element is taken,
    # skip: best when it is not
    take = skip = 0
    for x in nums:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)
```

156 Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses. MEDIUM

Template fit: state = (best if you take this house, best if you skip it): take, skip = skip + x, max(take, skip)

QUESTION

Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Two running values (take, skip)
# Idea: take is the best total that includes the current
# element, skip the best that excludes it. For the next
# element: new take = skip + x (the previous element must
# have been skipped), new skip = max(take, skip).
def rob_line(nums):
    # take = best sum if we take this house; skip = if we
    # do not
    take = skip = 0
    for x in nums:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)
```

157 Paint House II MEDIUM

Template fit: state = a whole row of costs, one per colour; new[j] = cost[j] + the smallest previous value that is not colour j (keep the two smallest)

QUESTION

There are n houses in a row, each to be painted one of k colors. costs[i][j] is the cost to paint house i with color j . No two adjacent houses may share the same color. Return the minimum total cost to paint all houses.

Function signature

def min_cost_paint_houses(costs: list[list[int]]) -> int:

...

Follow-up (as reported): Reduce the per-row transition from O(k) per cell to O(1) amortized by tracking the two smallest values of the previous row.

EXAMPLES

Example 1	
Input	`costs = [[1, 5, 3], [2, 9, 4]]`
Output	`5`
Why	`Paint house 0 color 0 (1) and house 1 color 2 (4) -> 5; or house 0 color 2 (3) and house 1 color 0 (2) -> 5.`

Example 2	
Input	`costs = [[1, 3], [2, 4]]`
Output	`5`
Why	`1 (house0 color0) + 4 (house1 color1) = 5, the cheapest valid pairing.`

Example 3	
Input	`costs = [[7]]`
Output	`7`
Why	`A single house, one color.`

CONSTRAINTS

QUESTION

- costs is an n x k grid, 0 <= n , k >= 1 (and k >= 2 whenever n >= 2 , so a valid painting exists).
- Costs are non-negative integers. Return 0 for n == 0 .
- Aim for O(n*k) time.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Two smallest values of the
# previous row
# Idea: A cell only needs the cheapest previous colour
# that differs from its own.
def min_cost_paint_houses(costs):
    prev = [0] * len(costs[0])
    for row in costs:
        best = sorted(
            range(len(prev)), key=lambda j: prev[j]
        )[:2]
        first = best[0]
        second = best[1] if len(best) > 1 else best[0]
        prev = [
            row[j]
            + (
                prev[second]
                if j == first
                else prev[first]
            )
            for j in range(len(row))
        ]
    return min(prev)
```

158 Write SQL code to publish the Fibonacci series. MEDIUM

Template fit: WITH RECURSIVE keeps (n, a, b); every new row is (n + 1, b, a + b): the same rolling pair, written as a query

QUESTION

Write SQL code to publish the Fibonacci series.

MAIN CODE | helper functions first, then the answer

```
import sqlite3


# HELPERS for the base question
# a recursive query: each row is computed from the
# previous one
FIB_SQL = """
WITH RECURSIVE fib(n, a, b) AS (
    SELECT 1, 0, 1
    UNION ALL
    SELECT n + 1, b, a + b FROM fib WHERE n < ?
)
SELECT a FROM fib
"""

# BASE QUESTION (optimal): Recursive CTE carrying (n, a,
# b)
# Idea: The anchor row is (1, 0, 1): the first term is 0
# and the next pair is (0, 1). Each recursive step
# produces (n + 1, b, a + b) while n is below the limit,
# so the term in column a is the Fibonacci number.
def fibonacci(count):
    with sqlite3.connect(":memory:") as db:
        rows = db.execute(FIB_SQL, (count,)).fetchall()
```

```
    return [r[0] for r in rows]
```

159 Constrained Dice Roll Sequences — HARD

Template fit: state = a table [value][run length]; a different value starts a run of 1, the same value extends its run (up to the limit)

QUESTION

You roll a single 6-faced die exactly n times. You are given an array roll_max of length 6, where roll_max[i] is the maximum number of times the value i + 1 may appear consecutively. That is, you may never roll the same value more than roll_max[i] times in a row. Return the number of distinct sequences of n rolls that respect every consecutive-count limit. Because the answer can be very large, return it modulo 10**9 + 7 .

Function signature

def count_dice_sequences(n: int, roll_max: list[int]) -> int:

    ...

Follow-up (as reported): The interviewer's actual twist used two dice per turn (a pair rolled together, with the consecutive limit applied to each die independently). Extend the state to track the run lengths of both dice and count the joint sequences.

EXAMPLES

Example 1	
Input	`n = 2, roll_max = [1, 1, 2, 2, 2, 3]`
Output	`34`
Why	`There are 6*6 = 36 sequences of length 2. Two of them are forbidden: rolling value 1 twice, and rolling value 2 twice (both have roll_max = 1). 36 - 2 = 34.`

Example 2	
Input	`n = 2, roll_max = [1, 1, 1, 1, 1, 1]`
Output	`30`
Why	`No value may repeat consecutively, so the 6 sequences "value v then v" are removed: 36 - 6 = 30.`

Example 3	
Input	`n = 3, roll_max = [1, 1, 1, 2, 2, 3]`
Output	`181`
Why	`Hand-verified against the memoized DP over (rolls_used, last_value, run_length).`

CONSTRAINTS

- 1 <= n <= 5000 roll_max has exactly 6 entries, each 1 <= roll_max[i] <= 15 .
- A naive brute force over all 6**n sequences is far too slow; you must memoize over (rolls_used, last_value, consecutive_count_of_last_value) .

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): DP over the last value and its
# run length
# Idea: dp[v][r] counts prefixes ending in value v
# repeated r times.
def count_dice_sequences(n, roll_max):
    mod = 10**9 + 7
    dp = [[0] * (roll_max[v] + 1) for v in range(6)]
    for v in range(6):
        dp[v][1] = 1
    for _ in range(n - 1):
```

```
        total = sum(sum(row) for row in dp) % mod
        new = [
            [0] * (roll_max[v] + 1) for v in range(6)
        ]
        for v in range(6):
            # a different previous value starts a run of
            # length one
            new[v][1] = (total - sum(dp[v])) % mod
            for r in range(1, roll_max[v]):
                new[v][r + 1] = dp[v][r]
        dp = new
    return sum(sum(row) for row in dp) % mod
```

GROUP 20 OF 43 | 7 QUESTIONS, ONE CODE TEMPLATE | 4 E 3 M

Two pointers: move i and j toward each other (or along two lists)

160 Reverse a linked list. EASY

Template fit: three pointers prev, current, next: point current.next back at prev, then move all three one step

QUESTION

Given the head of a singly linked list, reverse the list and return its head. Implement a function reverseList that takes the head of the linked list as input and returns the new head of the reversed list. Refer to the ListNode class/struct and do not comment it out. Example Input: 1 -> 2 -> 3 -> 4 Output: 4 -> 3 -> 2 -> 1Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(1)?

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Flip the pointers in one pass
# Idea: Walk with prev and cur. Save cur.next, point cur
# at prev, then move both forward. When cur is None, prev
# is the new head. Saving next before the overwrite is the
# step people forget.
def reverse_list(head):
    # point every node back at the one before it
    prev = None
    while head:
        head.next, prev, head = prev, head, head.next
    return prev
```

161 Sort a binary array in linear time. EASY

Template fit: left skips 0s, right skips 1s; when left sits on a 1 and right on a 0, swap them

QUESTION

Sort a binary array in linear time.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Two pointers swapping from both
# ends
# Idea: Keep lo at the next position for a 0 and hi at the
# next position for a 1. Advance lo past zeros, retreat hi
# past ones, and when both are stuck swap them. One pass,
# in place.
def sort_binary(arr):
    # two pointers: zeros collect on the left, ones on the
    # right; every step swaps or moves one pointer: O(n)
    lo, hi = 0, len(arr) - 1
    while lo < hi:
        if arr[lo] == 0:
            lo += 1
        elif arr[hi] == 1:
            hi -= 1
        else:
            # a 1 on the left and a 0 on the right: swap
            # them
            arr[lo], arr[hi] = arr[hi], arr[lo]
            lo += 1
            hi -= 1
    return arr
```

162 Is this a valid palindrome? EASY

Template fit: compare the two ends; skip characters that are not letters or digits; stop when the pointers cross

QUESTION

Given a string s, return true if it is a palindrome, and false otherwise. A string is considered a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Write a function isPalindrome that implements this logic. Examples Input: s = "H123!@321h" Output: true Input: s = "race a car" Output: false Constraints 1 <= s.length <= 2 * 10^5 The input string s can include any printable ASCII characters. Challenge 1Can you come up with a solution that achieves O(n) time complexity?Challenge 2Can you come up with a solution that achieves O(1) space complexity?Solution 1: Filtering and Reversing Approach This solution processes the input string by first filtering out non-alphanumeric characters and converting all remaining characters to lowercase. It then checks if this filtered string reads the same forward and backward. Filtering: Iterate through the string and collect alphanumeric characters while converting them to lowercase. Reversing and Comparing: Create a reversed version of the filtered string and compare it with the original filtered string to check for palindrome properties. Time Complexity: The time complexity is O(n), where n is the length of the input string. This is because filtering the characters and creating the reversed string both require iterating through the input once. Space Complexity: The space complexity is O(n) for storing the filtered characters and their

MAIN CODE | helper functions first, then the answer

```
import unicodedata


# HELPERS for the base question
def clean(s):
    # follow-up: unicode. Decompose accents, drop the
    # combining marks, fold case, keep letters and digits.
    text = unicodedata.normalize("NFKD", s)
    text = "".join(ch for ch in text
                   if not unicodedata.combining(ch))
    return [ch for ch in text.casefold() if ch.isalnum()]

# BASE QUESTION (optimal): Two pointers that skip ignored
# characters
# Idea: Move left and right toward each other, skipping
# characters that are not letters or digits, and compare
# the lowercase forms. The first mismatch ends it. Nothing
# is copied.
def is_palindrome(s):
    # compare the cleaned characters from both ends inward
    chars = clean(s)
    lo, hi = 0, len(chars) - 1
    while lo < hi:
        if chars[lo] != chars[hi]:
            return False
        lo += 1
        hi -= 1
    return True
```

163 Reverse a Sentence

EASY

Template fit: swap from both ends to reverse the whole array, then reverse every word the same way

QUESTION

You are given an array of characters arrthat consists of sequences of characters separated by space characters. Each space-delimited sequence of characters defines a word. Implement a function reverseWords that reverses the order of the words in the array in the most efficient manner. Explain your solution and analyze its time and space complexities. Example input: arr = ['p', 'e', 'r', 'f', 'e', 'c', 't', ' ', 'm', 'a', 'k', 'e', 's', ' ', 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e'] output: ['p', 'r', 'a', 'c', 't', 'i', 'c', 'e', ' ', 'm', 'a', 'k', 'e', 's', ' ', 'p', 'e', 'r', 'f', 'e', 'c', 't']Hint 1Even if your language of choice enables this, don't be tempted to use standard functions. Consider how inefficient it is to separately create a string, split the words, and reverse the split.Challenge 1As a general rule, if you're tempted to use a standard or library function, be sure to explain that function's time and space complexity before committing to that solution. You may find a more efficient way to solve the problem yourself. Oftentimes in coding interviews, this is the whole point of the exercise.Challenge 2How have you handled edge cases? For example, empty arrays, arrays with nothing but spaces, arrays with single words only, or with multiple spaces between words, etc.Challenge 3Were you able to achieve a linear time complexity and a constant space complexity?Challenge 4Bonus: how might the mirrorReverse function be implemented with a single index?One possible s

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
def reverse_range(arr, lo, hi):
    # reverse arr[lo..hi] in place with two pointers
    while lo < hi:
        arr[lo], arr[hi] = arr[hi], arr[lo]
        lo += 1
        hi -= 1

def each_word(arr):
    # yield the (start, end) of every word: a run of
    # characters without a space
    i = 0
    while i < len(arr):
        if arr[i] == " ":
            i += 1
            continue
        j = i
        while j + 1 < len(arr) and arr[j + 1] != " ":
            j += 1
        yield i, j
        i = j + 1

# BASE QUESTION (optimal): Reverse the whole array, then
# reverse each word in place
# Idea: Reverse every character; each word is now
# backwards but in the right place. Walk the array and
# reverse each maximal run of non-space characters.
# Everything happens inside the same array.
def reverse_words(arr):
    # reverse the whole array, so the words are in the
    # right
    # order but spelled backwards; then reverse each word
    # back. O(n) time and O(1) extra space.
    reverse_range(arr, 0, len(arr) - 1)
    for start, end in list(each_word(arr)):
        reverse_range(arr, start, end)
    return arr
```

164 Find median of merged RLE arrays

MEDIUM

Template fit: walk both sorted run lists, always taking the smaller value, adding its count until you pass the middle position

QUESTION

Overview: The question evaluates understanding of run-length encoding, order-statistics (median) computation, and efficient merging of frequency-annotated sequences, emphasizing handling large frequencies without explicit decompression.

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
def kth_smallest(runs_a, runs_b, k):
    # zero-based rank in the merged order, walking the
    # runs
    # of two sorted run-length lists; nothing is expanded
    i = j = seen = 0
    while True:
        take_a = j >= len(runs_b)
        if not take_a and i < len(runs_a):
            take_a = runs_a[i][0] <= runs_b[j][0]
        value, count = runs_a[i] if take_a else runs_b[j]
        seen += count
        if seen > k:
            return value
        if take_a:
            i += 1
        else:
            j += 1

# BASE QUESTION (optimal): Merge the runs and walk to the
# two middle ranks
# Idea: Let N be the sum of all counts.
def median_rle(a, b):
    # a and b: [(value, count)] sorted by value
    total = sum(c for _, c in a) + sum(c for _, c in b)
    if total % 2:
        return kth_smallest(a, b, total // 2)
    low = kth_smallest(a, b, total // 2 - 1)
    high = kth_smallest(a, b, total // 2)
    return (low + high) / 2
```

165 Run-Length Vector Storage + Dot Product

MEDIUM

Template fit: one pointer per run list; multiply the values and take min(remaining counts) of the two current runs

QUESTION

EXAMPLES

```
Logical A = [3, 3, 3, 0, 0, 0, 4, 4]
Logical B = [1, 1, 1, 1, 2, 2, 2, 2]
Encoded A (RLE): [(3,3),(0,3),(4,2)]
Encoded B (RLE): [(1,4),(2,4)]
Dot product = 3·1·3 + 3·1·0 + 0·1·0 + 0·2·0 + 4·2·2 = 9 + 0 + 16 = 25
```

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Two-pointer over runs
# Idea: Each run is (value, count). Compare the remaining
# lengths of the two current runs, multiply the values by
# the overlap, then advance whichever run was exhausted
# (or both).
def rle_dot(a, b):
```

```
    # two pointers over the runs: never expand anything
    i = j = 0
    left_a = a[0][1] if a else 0
    left_b = b[0][1] if b else 0
    total = 0
    while i < len(a) and j < len(b):
        # both runs overlap for min(left_a, left_b)
        # positions
        take = min(left_a, left_b)
        total += a[i][0] * b[j][0] * take
        left_a -= take
        left_b -= take
        if left_a == 0:
            i += 1
            left_a = a[i][1] if i < len(a) else 0
        if left_b == 0:
            j += 1
            left_b = b[j][1] if j < len(b) else 0
    return total
```

166 Count Unique Triplets That Sum to Zero

MEDIUM

Template fit: sort, fix the first number, then run the pair template on the rest; skip equal neighbours so each triple counts once

QUESTION

Given an integer array nums , count how many unique triplets (nums[i], nums[j], nums[k]) with three distinct indices i < j < k satisfy nums[i] + nums[j] + nums[k] == 0 . Two triplets are considered the same if they consist of the same multiset of values, regardless of the indices they came from. Return the number of distinct value-triplets (not the number of index combinations).

Function signature

def count_zero_sum_triplets(nums: list[int]) -> int:

    ...

Follow-up (as reported): Return the actual list of unique triplets instead of the count. Hint: sort the array, fix one element, and use two pointers while skipping duplicate values.

EXAMPLES

Example 1	
Input	`nums = [-1, 0, 1, 2, -1, -4]`
Output	`2`
Why	`The distinct value-triplets are [-1, -1, 2] and [-1, 0, 1]. Even though [-1, 0, 1] can be formed with different index sets, it is counted once.`

Example 2	
Input	`nums = [0, 0, 0, 0]`
Output	`1`
Why	`The only triplet is [0, 0, 0].`

Example 3	
Input	`nums = [1, 2, 3]`
Output	`0`
Why	`No three elements sum to 0.`

CONSTRAINTS

- 0 <= len(nums) <= 3000

QUESTION

- -10^5 <= nums[i] <= 10^5
- The array may contain duplicates.
- If len(nums) < 3 , the answer is 0 .

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Sort and two pointers
# Idea: Fix the smallest value and sweep the other two
# inward, skipping duplicates.
def count_zero_sum_triplets(nums):
    nums = sorted(nums)
    count = 0
    for i in range(len(nums) - 2):
        if i and nums[i] == nums[i - 1]:
            continue
        left, right = i + 1, len(nums) - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                count += 1
                left += 1
                right -= 1
                while (
                    left < right
                    and nums[left] == nums[left - 1]
                ):
                    left += 1
    return count
```

Walk two strings at once with two indices

167 Find the First Occurrence of a Substring — EASY

Template fit: try every start i; inside, j walks the needle; a full match returns i

QUESTION

Reconstructing the "find substring in array" item as substring search: given two strings haystack and needle , return the index of the first occurrence of needle in haystack , or -1 if needle is not part of haystack .

Function signature

def str_str(haystack: str, needle: str) -> int:
    """Return the index of the first occurrence of needle in haystack, or -1."""
    ...

Follow-up (as reported): Achieve O(n + m) worst-case time with the Knuth-Morris-Pratt (KMP) failure-function approach instead of the naive O(n * m) scan.

EXAMPLES

Example 1	
Input	haystack = "sadbutsad", needle = "sad"
Output	0
Why	"sad" occurs at indices 0 and 6; the first is index 0.

Example 2	
Input	haystack = "leetcode", needle = "leeto"
Output	-1
Why	"leeto" does not occur in "leetcode".

Example 3	
Input	haystack = "abc", needle = "c"
Output	2

CONSTRAINTS

- 1 <= len(haystack), len(needle) <= 10^4 haystack and needle consist of only lowercase English letters.
- If needle is the empty string it is considered to occur at index 0 (though the constraints above make it non-empty).

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Naive scan with early exit
# Idea: Compare character by character and stop at the
# first mismatch.
def str_str(haystack, needle):
    n, m = len(haystack), len(needle)
    for i in range(n - m + 1):
        j = 0
        while j < m and haystack[i + j] == needle[j]:
            j += 1
        if j == m:
            return i
    return -1
```

168 Determine if two words are sorted in lexicographic order. EASY

Template fit: walk both words; at the first differing letter compare the letters' ranks in the given alphabet; if one word ends first, it must be the shorter

QUESTION

Determine if two words are sorted in lexicographic order.

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
ALPHABET = "abcdefghijklmnopqrstuvwxyz"

# BASE QUESTION (optimal): Compare character by character,
# optionally with a custom alphabet
# Idea: Find the first position where the words differ and
# compare the two characters' ranks. If one word is a
# prefix of the other, the shorter one comes first, so
# 'app' is before 'apple' but not the reverse.
def in_order(a, b, rank):
    # compare letter by letter by rank; the first
    # difference
    # decides. If one word is a prefix of the other, the
    # shorter one comes first. A character outside the
    # alphabet is ranked after all the known ones.
    def key(ch):
        return rank.get(ch, len(rank) + ord(ch))

    for x, y in zip(a, b):
        if x != y:
            return key(x) < key(y)
    return len(a) <= len(b)
```

169 Return all unique substrings of a given string. EASY

Template fit: sort all suffixes; the unique substrings are all prefixes minus the shared prefixes of neighbouring suffixes

QUESTION

Return all unique substrings of a given string.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Count with a suffix array and
# LCP
# Idea: Sort the suffixes; the number of distinct
# substrings is sum over suffixes of (length) minus sum of
# the LCP of neighbours in sorted order, because each new
# suffix adds only the prefixes that the previous one did
# not already contribute.
def unique_substrings(s):
    # simplest: every slice into a set, then sort. O(n^3).
    return sorted({s[i:j] for i in range(len(s))
                   for j in range(i + 1, len(s) + 1)})
```

170 Can board states be transformed? MEDIUM

Template fit: skip the X squares in both strings; the remaining letters must match in order, and L may only move left, R only right

QUESTION

Overview: This question evaluates reasoning about constrained piece movement, string transformation, and invariant-based correctness in the Coding & Algorithms domain, testing skills in string manipulation and combinatorial reasoning while examining both conceptual properties and practical algorithmic implementation.

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
def pieces(board):
    # the non-blank symbols with their positions
    return [(ch, i) for i, ch in enumerate(board)
            if ch != "X"]

# BASE QUESTION (optimal): Two pointers over the pieces
# Idea: Skip blanks in both strings and compare the next
# piece in each: they must be the same letter.
def can_transform(start, end):
    # L moves left, R moves right, X is blank. Pieces
    # never pass
    # each other, so the order of the non-X symbols is
    # fixed;
    # a '#' is a wall: it never moves and no piece crosses
    # it.
    if len(start) != len(end):
        return False
    a, b = pieces(start), pieces(end)
    if len(a) != len(b):
        return False
    for (c1, i), (c2, j) in zip(a, b):
        if c1 != c2:
            return False
        # an L may only end up further left, an R further
        # right
        if c1 == "L" and i < j:
            return False
        if c1 == "R" and i > j:
            return False
        # a wall must stay exactly where it is
        if c1 == "#" and i != j:
            return False
    return True
```

171 Decide string transform with directional tokens and walls MEDIUM

Template fit: the same walk with dots as blanks and the movement rule from the statement (walls block)

QUESTION

You are given two equal-length strings start and target of length n, each consisting only of the characters 'L', 'R', '.', and 'X'. A dot '.' represents an empty slot; 'X' represents an immovable wall; 'L' and 'R' are tokens that may move across empty slots subject to these rules: (1) An 'L' token may move any number of positions left into '.', but may never move right, pass through another token, or pass through an 'X'. (2) An 'R' token may move any number of positions right into '.', but may never move left, pass through another token, or pass through an 'X'. (3) Only one token can occupy a position at any time. Determine whether target can be obtained from start by a sequence of valid moves. If yes, return true; otherwise, return false. Design an algorithm that runs in O(n) time and O(1) extra space beyond a few pointers/counters. Explain your invariants, prove correctness informally, analyze time and space complexity, and walk through a few tricky edge cases (e.g., mismatched wall layouts, blocks separated by walls, consecutive tokens with no gaps).

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
def pieces(board):
    # everything except the empty '.' slots, with
    # positions
    return [(ch, i) for i, ch in enumerate(board)
            if ch != "."]

# BASE QUESTION (optimal): Compare the non-dot sequences
# with position rules
# Idea: Walk both strings skipping dots. The next symbol
# must be equal; a wall must be at the same index; an L
# must satisfy start index >= target index; an R must
# satisfy start index <= target index.
def can_transform(start, end):
    # 'L' moves left, 'R' moves right, 'X' is an immovable
    # wall. Pieces never pass each other, so their order
    # is
    # fixed; each one only has a direction it may move.
    if len(start) != len(end):
        return False
    a, b = pieces(start), pieces(end)
    if len(a) != len(b):
        return False
    for (c1, i), (c2, j) in zip(a, b):
        if c1 != c2:
            return False
        if c1 == "X" and i != j:
            return False
        if c1 == "L" and i < j:
            return False
        if c1 == "R" and i > j:
            return False
    return True
```

172 Determine if two sentences are similar. MEDIUM

Template fit: zip the two sentences; each pair of words must be equal or listed as similar (store pairs in both directions)

QUESTION

Determine if Two Sentences Are Similar. Two sentences are similar if they have the same length and each pair of corresponding words in the two sentences is similar. The similarity between words is defined by the provided list of similar word pairs. A word is always similar to itself. For example, if we have the list of similar word pairs as [("great", "good"), ("acting","drama"), ("skills","talent")], then the sentences "You have great acting skills" and "You have good drama talent" are similar. Examples sentence1 = ["Let's", "code", "in", "Python"] sentence2 = ["Let's", "program", "in", "Python"] similarPairs = [("code", "program"),] output: true sentence1 = ["I", "love", "to", "play", "football"] sentence2 = ["I", "love", "playing", "soccer"] similarPairs = [("play", "playing"), ("football", "soccer")] output: false, different sentence lengths sentence1 = ["Do", "you", "like", "coffee"] sentence2 = ["Do", "you", "love", "coffee"] similarPairs = [("like", "enjoy"), ("coffee", "tea"),] output: false

"like" is not similar to

"love" based on the given pairs. sentence1 = ["I", "really", "love", "leetcode", "and", "apples"] sentence2 = ["I", "so", "like", "codesignal", "and", "oranges"] similarPairs = [("very", "so"), ("love", "adore"), ("really", "very"), ("leetcode", "codesignal"), ("apples", "oranges"), ("like", "adore"),] output: trueThe provided solution checks if two given sentences are similar by using a graph-based approach. It constructs a graph where words are c

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Set of pairs (not transitive)
# Idea: Put each pair into a set in both directions, so a
# word pair is similar if the words are equal or the pair
```

```
# is in the set. Sentences of different length are never
# similar.
def similar_direct(s1, s2, pairs):
    # same length, and every pair of words is equal or a
    # listed pair (in either direction)
    if len(s1) != len(s2):
        return False
    allowed = {frozenset(p) for p in pairs}
    return all(a == b or frozenset((a, b)) in allowed
               for a, b in zip(s1, s2))
```

173 Find Dictionary Matches for a Jammed String HARD

Template fit: compress every word to (letter, run length); a query matches when the letters agree and each run is equal or stretched to at least 3

QUESTION

Overview: This question evaluates string-processing and pattern-matching skills, specifically run-length grouping of consecutive characters and efficient dictionary lookup, within the Coding & Algorithms domain.

MAIN CODE | helper functions first, then the answer

```
from itertools import groupby


# HELPERS for the base question
def runs(word):
    # "heello" -> [("h", 1), ("e", 2), ("l", 2), ("o", 1)]
    return [(ch, len(list(g))) for ch, g in groupby(word)]

# BASE QUESTION (optimal): Index the dictionary by its run
# letters
# Idea: Group words by the string of their distinct run
# letters ('hello' -> 'helo'). A query can only match
# words in its own group, so compare run counts only
# there. Build the index once and reuse it for many
# queries.
class JammedIndex:
    """A query run may be longer than the word's run
    when it is at least min_stretch long."""

    def __init__(self, words):
        # words grouped by their run letters ("hello" ->
        # "helo")
        self.groups = {}
        for w in words:
            r = runs(w)
            letters = "".join(ch for ch, _ in r)
            self.groups.setdefault(letters, []).append(
                (w, [n for _, n in r]))


    @staticmethod
    def fits(q, w, min_stretch):
        # the same length, or a stretch of at least
        # min_stretch
        return q == w or (q > w and q >= min_stretch)
```

Simulate a machine: a class with state and one branch per kind of event

174 Mario-Style Score Tracker (OOP) EASY

Template fit: the template as written: an event name decides how score, lives and size change

QUESTION

Design a class that tracks the state of a platformer character across a sequence of events. The character has a score (starts at 0), a lives count (starts at 3), and a big flag (starts False). Process events with these rules: "coin" : add 100 to the score. "mushroom" : add 1000 to the score and set big = True . "enemy" : add 200 to the score (the character stomps an enemy). "damage" : if big is True , set big = False (no life lost); otherwise decrease lives by 1. Score is unchanged. The character is alive while lives > 0 . Once lives reaches 0, the game is over and any further events are ignored. Implement the class and a method to run a batch of events.

Function signature

```
class MarioGame:
    def __init__(self) -> None:
        """score=0, lives=3, big=False."""
        ...
    def process(self, event: str) -> None:
        """Apply one event, ignoring it if the game is already over."""
        ...
    def run(self, events: list[str]) -> None:
        """Apply a sequence of events in order."""
        ...
    def get_score(self) -> int:
        ...
    def is_alive(self) -> bool:
        ...
```

EXAMPLES

Example 1	
Input	`g = MarioGame(); g.run(["coin", "mushroom", "enemy", "damage", "damage"]); (g.get_score(), g.is_alive())`
Output	`(1300, True)`
Why	`coin -> 100; mushroom -> 1100, big=True; enemy -> 1300; damage -> big=False (no life lost); damage -> small, lives 3->2. Score 1300, still alive.`

Example 2	
Input	`g = MarioGame(); g.run(["damage", "damage", "damage", "coin"]); (g.get_score(), g.is_alive())`
Output	`(0, False)`
Why	`Three small-state hits drop lives 3->0 (game over). The trailing "coin" is ignored.`

CONSTRAINTS

- Event strings are always one of the four listed above.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): A small state class
# Idea: One method owns the rules; the game ends when
```

```python
# lives hit zero.
class MarioGame:
    def __init__(self):
        self.score = 0
        self.lives = 3
        self.big = False

    def process(self, event):
        if self.lives == 0:
            return
        if event == "coin":
            self.score += 100
        elif event == "mushroom":
            self.score += 1000
            self.big = True
        elif event == "enemy":
            self.score += 200
        elif event == "damage":
            if self.big:
                self.big = False
            else:
                self.lives -= 1

    def run(self, events):
        for event in events:
            self.process(event)

    def get_score(self):
        return self.score

    def is_alive(self):
        return self.lives > 0
```

175 Tic-Tac-Toe System with K Players and 3-in-a-Row Win

MEDIUM

Template fit: store the board; after each move count the run through the new cell in four directions and compare with the target length

QUESTION

Design a Tic-Tac-Toe system for **k** players on an **n x n** board. After each move, return the game status; any player wins immediately after forming 3 consecutive marks in a straight line.

Input parameters: k players and an n x n board.
Players take turns placing their marks on empty cells.
A player wins as soon as they form 3 consecutive marks in a straight line, regardless of how large n is.
After each move, return the correct game status, such as Game Over or <player> won.
Discuss implementation details and time complexity before coding.

MAIN CODE | helper functions first, then the answer

```python
# HELPERS used by the code below
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))

# BASE QUESTION (optimal): Check the four lines through
# the new mark
# Idea: Place the mark, then for each of the four
# directions walk outward both ways while the cells hold
# the same player and add the two counts plus one. A total
# of at least 3 wins immediately.
class TicTacToe:
    def __init__(self, n, k, target=3):
        self.n, self.k, self.target = n, k, target
        self.board = [[0] * n for _ in range(n)]
        self.moves = 0
```

```
        self.over = False

    def inside(self, r, c):
        return 0 <= r < self.n and 0 <= c < self.n

    def run_length(self, r, c, dr, dc):
        # marks of the same player in a line through (r,
        # c)
        player, run = self.board[r][c], 1
        for sign in (1, -1):
            a, b = r + sign * dr, c + sign * dc
            while self.inside(a, b) and \
                    self.board[a][b] == player:
                run += 1
                a, b = a + sign * dr, b + sign * dc
        return run

    def move(self, player, r, c):
        # whose turn is it? players go 1, 2, ..., k, 1,
        # ...
        turn = self.moves % self.k + 1
        if self.over or player != turn:
            raise ValueError("not this player's turn")
        if not self.inside(r, c) or self.board[r][c]:
            raise ValueError("illegal cell")
        self.board[r][c] = player
        self.moves += 1
        # only lines through the new mark can have changed
        for dr, dc in DIRECTIONS:
            run = self.run_length(r, c, dr, dc)
            if run >= self.target:
```

```
                self.over = True
                return f"player {player} wins"
        if self.moves == self.n * self.n:
            self.over = True
            return "draw"
        return "continue"
```

176 Design Tic-Tac-Toe With K Players

MEDIUM

Template fit: per player Counters for rows, columns and both diagonals; a move wins when a counter reaches n

QUESTION

Design and implement a Tic-Tac-Toe game engine. You are given:

- **k** players, each with a unique player ID or mark.
- An **`n x n`** board, initially empty.
- Players take turns placing their own mark on an empty cell. Unlike classic Tic-Tac-Toe, a player wins as soon as they form **3 consecutive marks** in a straight line, regardless of how large **n** is. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. Implement a class or set of functions that supports making moves and returns the game status after each move, such as:
- **`"Game Over"`** if a move is attempted after the game has ended.
- **`"<player> won"`** when the latest move causes that player to have 3 consecutive marks.
- A non-terminal status such as **`"Continue"`** if the game should keep going. Your implementation should handle invalid moves, such as placing a mark outside the board or on an occupied cell. Discuss the time and space complexity of your approach, especially for large **n**.

QUESTION

Design a Tic-Tac-Toe game engine for k players on an n x n board. The board starts empty, and players must play in cyclic order: player 1, player 2, ..., player k, then back to player 1. A player wins immediately when their latest valid move creates 3 consecutive marks in a straight line. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. For this problem, implement a function that processes a list of move attempts and returns the status after each attempt. Each move is given as [player, row, col] using 0-based indexing. Return one status string per move:

- "Game Over" if a move is attempted after someone has already won.
- "Invalid move" if the move is not legal. A move is invalid if: - the player ID is not in [1, k] - it is not that player's turn - the cell is outside the board - the cell is already occupied - "<player> won" if the move is valid and causes that player to have 3 consecutive marks - "Continue" if the move is valid and the game should continue Important: an invalid move does not change the board and does not advance the turn. You do not need to detect draws. If no one wins, valid moves simply return "Continue".

Constraints
1 <= n <= 10^91 <= k <= 10^50 <= len(moves) <= 2 * 10^5Each move is a list of three integers: [player, row, col]Row and column in the input may be outside the board and should then be treated as an invalid moveYour approach should be efficient for large n, so avoid scanning the whole board after every move

EXAMPLES

```
Input: (3, 2, [])
Expected Output: []
```

Explanation: No moves are made, so there are no statuses to report.

```
Input: (5, 2, [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1], [1, 0, 2], [2, 2, 2]])
Expected Output: ['Continue', 'Continue', 'Continue', 'Continue', '1 won', 'Game Over']
```

Explanation: Player 1 forms three consecutive marks horizontally at (0,0), (0,1), and (0,2). Any later move returns 'Game Over'.

Hints

Only the latest valid move can create a new 3-in-a-row, so you never need to rescan the entire board.For large boards, store only occupied cells in a hash map keyed by (row, col), then check the 4 line directions around the new move.

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))

# BASE QUESTION (optimal): Counters per player and per
# line
# Idea: rows[p][r], cols[p][c], diag[p] and anti[p] count
# each player's marks on those lines. A move increments
# the counters for its row, column and, if applicable, the
# diagonals; the player wins when one reaches n.
class TicTacToe:
    def __init__(self, n, k, target=3):
        self.n, self.k, self.target = n, k, target
        self.cells = {}
        self.history = []
        self.winner = None

    def turn(self):
        # players go 1, 2, ..., k, then 1 again
        return len(self.history) % self.k + 1

    def run_length(self, r, c, dr, dc, player):
        # marks of `player` in a line through (r, c)
        run = 1
        for sign in (1, -1):
            a, b = r + sign * dr, c + sign * dc
```

```
            while self.cells.get((a, b)) == player:
                run += 1
                a, b = a + sign * dr, b + sign * dc
        return run

    def move(self, player, r, c):
        if self.winner is not None:
            return "Game Over"
        # an invalid move changes nothing
        if not 1 <= player <= self.k:
            return "Invalid move"
        if player != self.turn():
            return "Invalid move"
        if not (0 <= r < self.n and 0 <= c < self.n):
            return "Invalid move"
        if (r, c) in self.cells:
            return "Invalid move"
        self.cells[(r, c)] = player
        self.history.append((r, c))
        # only the four lines through the new mark can win
```

```
        for dr, dc in DIRECTIONS:
            run = self.run_length(r, c, dr, dc, player)
            if run >= self.target:
                self.winner = player
                return f"{player} won"
        return "Continue"
```

177 Infinite Deck Card Duel

MEDIUM

Template fit: an infinite deck is just a random draw; play rounds, compare the highest cards, and decide the champion by the lead in wins

QUESTION

Design an InfiniteDeck that can be drawn from forever: each call to draw() returns a uniformly random integer card value in [1, 13] (cards are never exhausted). Make the deck seedable so games are reproducible. Then simulate a duel between two players. In each round, each player draws hand_size cards. The round winner is the player holding the single highest card that round; if both players' highest cards are equal, the round is a draw (nobody scores). Play num_rounds rounds and tally round wins per player. Finally, declare a champion: the player is champion only if they lead the other in round wins by at least 2; otherwise there is no champion. Implement: InfiniteDeck.draw() returning the next card. round_winner(hand1, hand2) -> 1 , 2 , or 0 (0 meaning draw), based on the highest card in each hand. decide_champion(wins1, wins2) -> 1 , 2 , or 0 (0 meaning no champion), applying the "lead by at least 2" rule. play_game(...) tying it together (uses the deck for randomness).

QUESTION

Function signature

```
import random
class InfiniteDeck:
    def __init__(self, seed: int | None = None) -> None:
        """Create a deck backed by a (optionally seeded) RNG."""
        ...
    def draw(self) -> int:
        """Return a uniformly random card value in [1, 13]."""
        ...
def round_winner(hand1: list[int], hand2: list[int]) -> int:
    """Return 1 if player 1's max card is higher, 2 if player 2's is higher, 0 on a
tie."""
    ...
def decide_champion(wins1: int, wins2: int) -> int:
    """Return the champion (1 or 2) only if they lead by >= 2 round wins, else 0."""
    ...
def play_game(deck: InfiniteDeck, hand_size: int = 5, num_rounds: int = 100) -> int:
    """Play the duel and return the champion via decide_champion, or 0 if none."""
    ...
```

Follow-up (as reported): Extend to n players: replace the two win counters with a dictionary keyed by player id and pick the highest card across all hands each round. Introduce multiple independent InfiniteDeck instances (or one shared deck with per-card metadata) to support probabilistic house rules.

EXAMPLES

Example 1	
Input	`round_winner([3, 7, 2], [7, 1, 9])`
Output	`2`
Why	`Player 2's highest card (9) beats player 1's highest (7).`

Example 2	
Input	`round_winner([5, 5], [5, 4])`
Output	`0`
Why	`Both players' highest card is 5, so the round is a draw.`

CONSTRAINTS

- 1 <= hand_size <= 20 , 1 <= num_rounds <= 100000 , card values in [1, 13] .

MAIN CODE | helper functions first, then the answer

```python
import random


# BASE QUESTION (optimal): A seedable deck and pure game
# rules
# Idea: The deck owns the Random; the rules are plain
# functions of hands and win counts.
class InfiniteDeck:
    def __init__(self, seed=None):
        self.rng = random.Random(seed)

    def draw(self):
        return self.rng.randint(1, 13)
```

```python
# BASE QUESTION (optimal): A seedable deck and pure game
# rules
# Idea: The deck owns the Random; the rules are plain
# functions of hands and win counts.
def round_winner(hand1, hand2):
    if max(hand1) == max(hand2):
        return 0
    return 1 if max(hand1) > max(hand2) else 2

# BASE QUESTION (optimal): A seedable deck and pure game
# rules
# Idea: The deck owns the Random; the rules are plain
# functions of hands and win counts.
def decide_champion(wins1, wins2):
    if wins1 - wins2 >= 2:
        return 1
    return 2 if wins2 - wins1 >= 2 else 0

# BASE QUESTION (optimal): A seedable deck and pure game
# rules
# Idea: The deck owns the Random; the rules are plain
# functions of hands and win counts.
def play_game(deck, hand_size=5, num_rounds=100):
    wins = {1: 0, 2: 0}
    for _ in range(num_rounds):
        h1 = [deck.draw() for _ in range(hand_size)]
        h2 = [deck.draw() for _ in range(hand_size)]
        winner = round_winner(h1, h2)
        if winner:
            wins[winner] += 1
    return decide_champion(wins[1], wins[2])
```

178 Implement iterator merging lists with filtering

MEDIUM

Template fit: keep a position per list and a turn counter; advance() looks for the next unblocked, unseen value and stores it for next()/has_next()

QUESTION

Overview: This question evaluates iterator/generator design, sequence merging, stateful filtering and deduplication, checking a candidate's ability to maintain input ordering while excluding blocked IDs and preventing duplicate yields.

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Lazy iterator with one pointer
# per list and a lookahead
# Idea: Keep a pointer per list and a rotation index.
# _advance() scans round robin for the next value that is
# not blocked and not already yielded and stores it as the
# lookahead (or None when everything is exhausted).
class FilteredZigzag:
    """Take one item from each list in turn; skip blocked
    ids and ids already returned. Has hasNext / next."""

    def __init__(self, lists, blocked=()):
        self.lists = lists
        self.blocked = set(blocked)
        self.pos = [0] * len(lists)
        self.turn = 0
        self.seen = set()
        # one item of lookahead makes hasNext easy
        self.ahead = None
        self.advance()

    def advance(self):
        self.ahead = None
```

```
        misses = 0
        # stop when every list had nothing left in a full
        # round
        while misses < len(self.lists):
            i = self.turn
            self.turn = (self.turn + 1) % len(self.lists)
            if self.pos[i] >= len(self.lists[i]):
                misses += 1
                continue
            value = self.lists[i][self.pos[i]]
            self.pos[i] += 1
            misses = 0
            blocked = value in self.blocked
            skip = blocked or value in self.seen
            if not skip:
                self.ahead = value
                return

    def has_next(self):
        return self.ahead is not None

    def next(self):
        value = self.ahead
```

```
        self.seen.add(value)
        self.advance()
        return value
```

179 Design an elevator control system

MEDIUM

Template fit: each car keeps floor, direction and stops; a request goes to the car with the lowest cost; step() moves every car one floor

QUESTION

Design the software for a multi-elevator control system in a high-rise building. Specify classes, scheduling algorithms for car assignments, handling peak traffic, fairness, fault tolerance, and safety constraints. Consider real-time constraints and a simulation and testing strategy.

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
INF = float("inf")

class Request:
    def __init__(self, floor, direction):
        # direction: +1 up, -1 down
        self.floor, self.direction = floor, direction

# BASE QUESTION (optimal): Elevator (SCAN) per car and
# cost-based assignment of calls
# Idea: Each car keeps a set of stops and a direction.
# step() moves one floor in its direction and removes a
# stop it reaches; when no stops remain ahead it reverses,
# and when none remain at all it goes idle.
class Elevator:
    def __init__(self, car_id, floor=0):
        self.id, self.floor = car_id, floor
        # 0 = idle, +1 = going up, -1 = going down
        self.direction = 0
        self.stops = set()
        self.in_service = True

    def cost(self, req):
        # how bad is it to give this car the request?
```

```
        if not self.in_service:
            return INF
        dist = abs(self.floor - req.floor)
        if self.direction == 0:
            return dist
        # on the way: the car is moving toward the floor
        # and
        # the call goes the same way
        gap = req.floor - self.floor
        ahead = gap * self.direction >= 0
        if ahead and req.direction == self.direction:
            return dist
        # otherwise it finishes its sweep first
        span = max(self.stops) - min(self.stops) + 1
        return dist + 2 * span + 10

    def step(self):
        # one tick: serve a stop here, then move one floor
        served = None
        if self.floor in self.stops:
            self.stops.discard(self.floor)
```

```
            served = self.floor
        if not self.stops:
            self.direction = 0
            return served
        up = [s for s in self.stops if s > self.floor]
        down = [s for s in self.stops if s < self.floor]
        # keep the direction while there are stops ahead
        # (SCAN)
        if (self.direction >= 0 and up) or (
                self.direction < 0 and not down):
            self.direction = 1
        else:
            self.direction = -1
        self.floor += self.direction
        return served

# BASE QUESTION (optimal): Elevator (SCAN) per car and
# cost-based assignment of calls
# Idea: Each car keeps a set of stops and a direction.
# step() moves one floor in its direction and removes a
# stop it reaches; when no stops remain ahead it reverses,
# and when none remain at all it goes idle.
class Controller:
    def __init__(self, cars, lobby=0):
        self.cars, self.lobby = cars, lobby

    def assign(self, req):
        # the cheapest in-service car takes the call
        car = min(self.cars, key=lambda c: c.cost(req))
        if car.cost(req) == INF:
            return None
        car.stops.add(req.floor)
        return car.id

    def tick(self):
        return {c.id: c.step() for c in self.cars}
```

180 Simulate In-Place Cellular State Updates HARD

Template fit: encode old and new state in two bits of each cell so every cell is computed from the old state; shift at the end

QUESTION

Overview: This question evaluates a candidate's ability to implement grid-based state transitions and manage in-place updates with strict space constraints, assessing skills in matrix manipulation, neighbor traversal, and correctness under simultaneous update rules.

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
AROUND = [(dr, dc) for dr in (-1, 0, 1)
          for dc in (-1, 0, 1) if (dr, dc) != (0, 0)]

def live_neighbors(board, r, c, wrap=False):
    m, n = len(board), len(board[0])
    count = 0
    for dr, dc in AROUND:
        a, b = r + dr, c + dc
        if wrap:
            # a torus: the edges are glued together
            a, b = a % m, b % n
        elif not (0 <= a < m and 0 <= b < n):
            continue
        # bit 0 is the OLD state, even for updated cells
        count += board[a][b] & 1
    return count

# BASE QUESTION (optimal): Two-bit encoding in the same
# board
# Idea: Bit 0 holds the current state and bit 1 holds the
# next state. First pass: count neighbors using only bit 0
# (value & 1) and set bit 1 by the rules.
def step_in_place(board, wrap=False):
    m, n = len(board), len(board[0])
    for r in range(m):
        for c in range(n):
            live = live_neighbors(board, r, c, wrap)
            alive = board[r][c] & 1
            # bit 1 stores the NEW state
            if live == 3 or (alive and live == 2):
                board[r][c] |= 2
    for r in range(m):
        for c in range(n):
            # shift the new state into place
            board[r][c] >>= 1
    return board
```

GROUP 23 OF 43 | 6 QUESTIONS, ONE CODE TEMPLATE | 6 M

Top-K by frequency: Counter plus nsmallest

181 Top-K Frequent in Large Logs — MEDIUM

Template fit: count the first word of every line (the speaker); return the names of the k most frequent

QUESTION

EXAMPLES

```
Input:  chatLog = ["A says hi", "B says hi", "A says bye", "C says hi", "A says yo"], k=2
Counts: A=3, B=1, C=1
Output: [A, B]  // or [A, C] — ties allowed
```

MAIN CODE | helper functions first, then the answer

```python
import heapq
from collections import Counter


# HELPERS for the base question
def speaker(line):
    # "A says hi" -> "A"
    return line.split(" ", 1)[0]

def rank_key(item):
    # more messages first; ties by name
    return (-item[1], item[0])

# BASE QUESTION (optimal): Count, then keep a size-K heap
# Idea: Parse the speaker from each line and count in a
# hash map. nsmallest keeps a heap of size k, so it never
# sorts all d distinct users. Ties break by user name so
# the output is deterministic.
def top_k_speakers(chat_log, k):
    counts = Counter(speaker(line) for line in chat_log)
    # a heap of size k: O(n + d log k)
    items = counts.items()
    best = heapq.nsmallest(k, items, key=rank_key)
    return [user for user, _ in best]
```

182 Rearrange Tasks With Cooldown — MEDIUM

*Template fit: only the biggest count matters: answer = max(len(tasks), (biggest - 1) * (n + 1) + how many tasks share the biggest count)*

QUESTION

Overview: This question evaluates task-scheduling and constraint-handling skills, testing algorithmic reasoning about arranging repeated jobs with cooldown constraints along with analysis of schedule optimality and algorithmic complexity.

MAIN CODE | helper functions first, then the answer

```python
from collections import Counter


# BASE QUESTION (optimal): Closed form from the most
# frequent task
# Idea: Let m be the highest frequency and k the number of
# tasks that have that frequency. The skeleton needs (m -
# 1) * (n + 1) + k slots.
def least_interval(tasks, n):
    # closed form: the most frequent task sets the frame
```

```
    counts = Counter(tasks).values()
    most = max(counts, default=0)
    # how many tasks are tied for the most
    tied = sum(1 for c in counts if c == most)
    if not tasks:
        return 0
    # (most - 1) full frames of n + 1 slots, plus the last
    # row
    return max(len(tasks), (most - 1) * (n + 1) + tied)
```

183 Implement frequency + distance top■K queries

MEDIUM

Template fit: count the point tuples; the tie-break is distance from the origin, then x, then y

QUESTION

You will implement solutions for two coding interview questions.

Question

1: Return the top-K points by frequency (with distance tie-break) You are given an array **`points`** of 2D integer points, where each point is represented as **`[x, y]`**. The array may contain duplicates (the same **`[x, y]`** can appear multiple times). Define:

- **`freq(p)`** = number of times point **p** appears in the array.
- **`dist(p)`** = squared Euclidean distance to the origin, **`x^2 + y^2`**. Return **k distinct points** sorted by the following priority:

1. Higher frequency first (**`freq`** descending)
2. If frequencies tie, smaller distance first (**`dist`** ascending)
3. If still tied, sort by **`x`** ascending, then **`y`** ascending

Input

- **`points`**: list of length **n**, each element **`[x, y]`**
- **`k`**: integer, **`1 <= k <= (# of distinct points)`**

Output

- A list of **k** distinct points following the ranking above.

Constraints (typical interview assumptions)

- **`1 <= n <= 2 * 10^5`**
- **`-10^4 <= x, y <= 10^4`** ---

Question

2: Time-based key-value store with “closest timestamp” lookup Design a data structure supporting the following operations:

- **`set(key, value, timestamp)`**: store the string **`value`** for string **`key`** at integer **`timestamp`**.
- Assume timestamps for the same **`key`** are inserted in **non-decreasing** order.
- **`getClosest(key, timestamp) -> value or ""`**:
- If **`key`** has no stored values, return **`""`**.
- Otherwise, return the **`value`** whose stored timestamp is **closest** to the query **`timestamp`**.
- If there is a tie (equal distance to two timestamps), return the value with the **smaller** timestamp.

Example

If for key **`"a"`** you stored timestamps **`[2, 10, 15]`** with values **`["x", "y", "z"]`**:

- **`getClosest("a", 11)`** returns **`"y"`** (10 is closest)
- **`getClosest("a", 14)`** returns **`"z"`** (15 is closest)

QUESTION

- **`getClosest("a", 12)`** returns **`"y"`** (10 and 15 are equally far; choose smaller timestamp)

What to discuss

- Time and space complexity of your approach - How you would improve a “naive” solution (e.g., linear scan) to a more optimal one

Constraints (typical interview assumptions)

- Up to **`2 * 10^5`** total operations - Keys and values are strings; timestamps are integers within typical 32-bit range

Part 1: Top-K Points by Frequency with Distance Tie-BreakGiven a list of 2D integer points, where duplicate points may appear, return the top **k** distinct points ranked by these rules: (1) higher frequency first, (2) if frequencies tie, smaller squared distance to the origin first, and (3) if still tied, smaller **x** first, then smaller **y**. The result must contain exactly **k** distinct points in ranked order.

Constraints

1 <= len(points) <= 2 * 10^5-10^4 <= x, y <= 10^41 <= k <= number of distinct points

EXAMPLES

```
Input: ([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)
Expected Output: [[2, 2], [1, 1]]
Explanation: Point [2,2] appears 3 times and [1,1] appears 2 times, so they are the top 2 distinct
    points.
Input: ([[1,2],[0,2],[1,2],[2,1],[0,2],[2,1],[3,0]], 3)
Expected Output: [[0, 2], [1, 2], [2, 1]]
Explanation: The points [0,2], [1,2], and [2,1] each appear twice. Their squared distances are 4, 5, and
    5, so [0,2] comes first. Between [1,2] and [2,1], x is smaller for [1,2].
```

Hints

First count how many times each point appears. After that, each distinct point only needs to be ranked once.Python can sort tuples directly, so think about building a sort key like **`(-frequency, distance, x, y)`**.

Part 2: Time-Based Key-Value Store with Closest Timestamp QueriesProcess a sequence of operations for a time-based key-value store. Each operation is either **`['set', key, value, timestamp]`** or **`['getClosest', key, timestamp]`**. For every **`getClosest`** operation, return the stored value whose timestamp is closest to the query tim

MAIN CODE | helper functions first, then the answer

```python
import heapq
from collections import Counter


# BASE QUESTION (optimal): Q1: heap with the full key
# Idea: Count with a Counter, then heapq.nsmallest(k, ...)
# with the key (-frequency, squared distance, x, y).
# Squared distances avoid square roots and are exact for
# integers.
def top_points(points, k):
    # more frequent first, then closer to the origin, then
    # x, y
    counts = Counter(map(tuple, points))

    def rank(p):
        dist = p[0] ** 2 + p[1] ** 2
        return (-counts[p], dist, p[0], p[1])

    best = heapq.nsmallest(k, counts, key=rank)
    return [list(p) for p in best]
```

184 Solve chat, grid paths, and car rentals

MEDIUM

Template fit: the top-K part is the template as written; the other parts are a rolling-column path count and a greedy car assignment

QUESTION

You are given three independent programming tasks. For each, describe and implement an efficient algorithm, and be prepared to analyze its time and space complexity. ---

Q1.

Find top-K most active chat users You are given a log of chat messages from a messaging application.

Input:

- An array **`messages`** of length **`n`**, where each element **`messages[i]`** is the **`user_id`** (string or integer) of the sender of the i-th message.
- An integer **`k`** (1 <= k <= number of distinct users).

Output:

- Return a list of **`k`** user IDs corresponding to the users who have sent the most messages.

Details & requirements:

- The "activity" of a user is defined as the number of messages they have sent (i.e., their frequency in **`messages`**).
- If there are fewer than **`k`** distinct users, return all distinct users.
- If multiple users have the same frequency, you may break ties arbitrarily, or (for determinism) sort tied users by **`user_id`** in ascending order.
- Aim for an algorithm that is more efficient than sorting all users by count when **`k`** is much smaller than the number of distinct users. Design and implement a function, for example:
- **`List<UserId> topKActiveUsers(List<UserId> messages, int k)`** Also explain:
- At least two different approaches (e.g., based on sorting, heaps, or selection algorithms).
- The time and space complexity of each approach. ---

Q2.

Count paths in a 2D grid with constrained moves You have a 2D grid with **`m`** rows and **`n`** columns. Index rows from top to bottom as **`0..m-1`** and columns from left to right as **`0..n-1`**.

- The **start** cell is the bottom-left corner: **`(m-1, 0)`**.
- The **end** cell is the bottom-right corner: **`(m-1, n-1)`**. From any cell **`(r, c)`**, you may move **only** to the next column to the right, in one of up to three ways (if within bounds):
- Right-up: **`(r-1, c+1)`**
- Right: **`(r, c+1)`**
- Right-down: **`(r+1, c+1)`** You cannot move left or stay in the same column. All cells are passable (no obstacles).

Task:

- Given integers **`m`** and **`n`**, compute the total number of distinct paths from the start cell **`(m-1, 0)`** to the end cell **`(m-1, n-1)`** using only the allowed moves.

Requirements:

- Implement a function such as **`long countPaths(int m, int n)`**.
- The result may be large; assume it fits in a 64-bit integer, or, if you prefer, you may return the count modulo a large prime (e.g., 1,000,000,007) — specify which convention you are using.
- First design a straightforward dynamic programming solution using O(m·n) time and O(m·n) space.
- Then describe how to optimize the space usage (e.g., to O(m)) while keeping the same time complexity, and explain why the optimization is correct. ---

Q3.

QUESTION

Minimum cars needed for rentals and assignment You work at a car rental company. You are given a list of rental bookings, each with a pickup and return time.

Input:

- An array of **`R`** rental records. Each record has:
- **`rental_id`**: a unique identifier (e.g., integer or string).
- **`pickup_time`**: an integer start time.
- **`return_time`**: an integer end time, with **`pickup_time < return_time`**. Assume times are on a single timeline (e.g., minutes from some epoch). A car can be reassigned immediately after it is returned, i.e., if one rental ends at time **`t`** and another starts at time **`t`**, the same car can serve both.

Tasks: 1.

Capacity: Compute the **minimum number of cars** required so that all rentals can be fulfilled without time conflicts on any single car. 2.

Assignment: Produce one valid assignment of rentals to cars that uses exactly that minimum number of cars.

Output:

- An integer **`C`** = minimal number of cars needed.
- A mapping from each **`rental_id`** to a **`car_id`** in the range **`1..C`**, such that no two rentals assigned to the same **`car_id`** overlap in time.

Requirements & clarifications:

- Overlap definition: two rentals with intervals **`[s1, e1)`** and **`[s2, e2)`** overlap if their time intervals intersect with non-empty duration. Using half-open intervals, if **`e1 == s2`** they do **not** over

MAIN CODE | helper functions first, then the answer

```
import heapq
from collections import Counter


# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def top_k_users(messages, k):
    # Q1, heap: O(n + d log k); sorted order breaks ties
    counts = Counter(messages)
    best = heapq.nsmallest(
        k, counts.items(), key=lambda kv: (-kv[1], kv[0]))
    return [user for user, _ in best]

# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def count_paths(m, n, blocked=(), mod=None):
    # Q2: start (m-1, 0), end (m-1, n-1), moves right-up,
    # right, right-down. One column of ways is enough:
    # O(m).
    wall = set(blocked)
    ways = [0] * m
    ways[m - 1] = 1
    for c in range(1, n):
        nxt = [0] * m
        for r in range(m):
```

```
            if (r, c) in wall:
                # a blocked cell contributes no paths
                continue
            total = ways[r]
            if r > 0:
                total += ways[r - 1]
            if r + 1 < m:
                total += ways[r + 1]
            nxt[r] = total % mod if mod else total
        ways = nxt
    return ways[m - 1]
```

```
# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def min_cars(rentals):
    # Q3: rentals are (pickup, return); how many cars are
    # needed?
    # A heap holds the return times of the cars on the
    # road.
    cars = []
    for pickup, back in sorted(rentals):
        if cars and cars[0] <= pickup:
            # a car came back in time: reuse it
            heapq.heapreplace(cars, back)
        else:
            heapq.heappush(cars, back)
    return len(cars)
```

185 Top-K Chatters by Word Count

MEDIUM

Template fit: count len(body.split()) per speaker instead of lines; ties by name

QUESTION

You are given a chat transcript as a list of message lines. Each line has the form "Name: message text" , where everything before the first ": " is the author's name and everything after it is the message body. Count the number of words each person typed across the entire transcript (a word is a maximal run of non- whitespace characters in the body). A person may appear on many lines; sum their words. Return the k people who typed the most words, as a list of (name, word_count) tuples sorted by word_count descending. Break ties by name in ascending (lexicographic) order.

Function signature

def top_k_chatters(lines: list[str], k: int) -> list[tuple[str, int]]:

    ...

Follow-up (as reported): The interviewer's real ask was to implement the parsing inside a class alongside stubbed API methods. Wrap this in a ChatAnalyzer class that ingests lines incrementally via add_line(line) and answers top_k(k) on demand.

EXAMPLES

Example 1	
Input	`lines = ["Alice: hello there friend", "Bob: hi", "Alice: how are you"], k = 1`
Output	`[("Alice", 6)]`
Why	`Alice typed 3 + 3 = 6 words, Bob typed 1. Top 1 is Alice.`

QUESTION

Example 2	
Input	`lines = ["Ann: a b", "Bob: c d", "Cara: e"], k = 2`
Output	`[("Ann", 2), ("Bob", 2)]`
Why	`Ann and Bob both have 2 words; tie broken alphabetically, so Ann precedes Bob. Cara (1 word) is excluded.`

Example 3	
Input	`lines = [], k = 3`
Output	`[]`

CONSTRAINTS

- 0 <= len(lines) <= 10^5
- Every line contains exactly one ": " separating name from body; the name is non-empty and contains no ": " .
- The body may be empty (0 words).
- 1 <= k ; if k exceeds the number of distinct people, return all of them.

MAIN CODE | helper functions first, then the answer

```
import heapq
from collections import Counter


# BASE QUESTION (optimal): Counter and a bounded selection
# Idea: Sum the words per author, then pick the k best
# with a heap keyed by (-count, name).
def top_k_chatters(lines, k):
    counts = Counter()
    for line in lines:
        name, body = line.split(": ", 1)
        counts[name] += len(body.split())
    return heapq.nsmallest(
        k,
        counts.items(),
        key=lambda kv: (-kv[1], kv[0]),
    )
```

186 Find top/bottom-k words in list or stream — MEDIUM

Template fit: the same Counter; run nsmallest once with -count and once with +count for the bottom-k

QUESTION

You are given words (strings) either as a finite list or as an unbounded stream. 1)
List version: Given an array **`words`** and an integer **k**, return: - the **k** most frequent distinct words, and - the **k** least frequent distinct words. Specify how you break ties (e.g., lexicographically, or any order). 2)
Stream version (online): Design a data structure that supports:

- **`ingest(word)`**: process the next word in the stream - **`topK(k)`**: return the **k** most frequent distinct words seen so far - **`bottomK(k)`**: return the **k** least frequent distinct words seen so far Discuss time/space complexity and how you would implement these operations efficiently. 3)

Variant (discuss/design): Extend the stream design to support returning the **first k** and/or **last k** words that are currently **non-repeating** (i.e., words with frequency exactly 1) according to stream arrival order.

QUESTION

Part 1: Top and Bottom K Frequent Words in a ListGiven an array of strings words and a non-negative integer k, return two lists: the k most frequent distinct words and the k least frequent distinct words. Use deterministic tie-breaking: if two words have the same frequency, the lexicographically smaller word comes first. If k is larger than the number of distinct words, return all distinct words.

Constraints

0 <= len(words) <= 2 * 10^50 <= k <= 2 * 10^5Each word is a non-empty stringComparison is case-sensitive unless otherwise stated

EXAMPLES

```
Input: (['i', 'love', 'leetcode', 'i', 'love', 'coding'], 2)
Expected Output: [['i', 'love'], ['coding', 'leetcode']]
Explanation: `i` and `love` both appear twice, so lexicographical order gives `['i', 'love']`. `coding`
    and `leetcode` both appear once, so lexicographical order gives `['coding', 'leetcode']`.
Input: (['apple', 'banana', 'apple', 'cherry', 'banana', 'date'], 3)
Expected Output: [['apple', 'banana', 'cherry'], ['cherry', 'date', 'apple']]
```

Explanation: Frequencies are: apple=2, banana=2, cherry=1, date=1. Top 3 are ordered by frequency descending, then lexicographically: **`apple`**, **`banana`**, **`cherry`**. Bottom 3 are ordered by frequency ascending, then lexicographically: **`cherry`**, **`date`**, **`apple`**.

Hints

Count how many times each distinct word appears before deciding the answer.You can sort the distinct words twice: once by (-frequency, word) and once by (frequency, word).

Part 2: Online Top/Bottom K Words from a StreamYou are given a sequence of stream operations. Each operation is a 2-element list of strings: ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]. Process the operations in order. For each query, return the k most frequent or k least frequent distinct words seen so far. Ties are broken lexicographically ascending. If k is larger than the number of distinct words seen so far, return all distinct words.

Constraints

1 <= len(operations) <= 2 * 10^5Each operation is one of ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]0 <= k <= 2 * 10^5Queries may appear before any word is ingested

Examples

Input: [['ingest', 'apple'], ['ingest', 'banana'], ['ingest', 'apple'], ['topK', '2'], ['bottomK', '2']]

Expected Output: [['apple', 'banana'], ['banana', 'apple']]

Explanation: After the ingests, counts are apple=2 and banana=1. topK(2) returns the most frequent first, while bottomK(2) returns the least frequent first.

Input: [['topK', '3'], ['ingest', 'dog'], ['ingest', 'cat'], ['ingest', 'cat'], ['topK', '2'], ['bottomK', '2'], ['topK', '5']]

Expected Output: [[], ['cat', 'dog'], ['dog', 'cat'], ['cat', 'dog']]

Explanation: The first query happens before any words are seen, so it returns []. Later, cat has frequency 2 and dog has frequency 1.

Hints

Keep an exact frequency map for all words seen so far.A max-heap and a min-heap with lazy deletion let you support online updates without fully resorting all words

MAIN CODE | helper functions first, then the answer

```python
import heapq
from collections import Counter


# BASE QUESTION (optimal): Counter plus two heaps of size
# K
# Idea: One pass counts the words. heapq.nsmallest with
# key (-count, word) gives the K most frequent and with
# key (count, word) the K least frequent, each in O(d log
# k) without a full sort.
```

```
def top_bottom(words, k):
    # ties are broken alphabetically
    c = Counter(words)
    top = heapq.nsmallest(k, c, key=lambda w: (-c[w], w))
    low = heapq.nsmallest(k, c, key=lambda w: (c[w], w))
    bottom = low
    return top, bottom
```

GROUP 24 OF 43 | 6 QUESTIONS, ONE CODE TEMPLATE | 1 E 3 M 2 H

Digits and characters: peel one at a time and carry

187 Caesar Cipher: Recover Key, Encrypt, and Decrypt — EASY

Template fit: per letter: (ord(ch) - 97 + k) % 26 + 97; the key comes from one letter pair; decrypt uses -k

QUESTION

A message was scrambled with a Caesar cipher: every lowercase letter was shifted forward by the same secret amount k (0 <= k < 26) with wraparound (z +1 -> a); all non-lowercase characters were left unchanged. You are given one example pair — a known plaintext and its corresponding ciphertext — from which you must recover the secret shift, and then implement the cipher in both directions. Specifically implement: find_shift(plaintext, ciphertext) -> int : return the shift k (0..25) that maps plaintext to ciphertext . The two strings have equal length and contain at least one lowercase letter that pins down k . encrypt(text, k) -> str : shift each lowercase letter forward by k with wraparound; leave other characters unchanged. decrypt(text, k) -> str : the inverse of encrypt — shift each lowercase letter backward by k with wraparound.

Function signature

```
def find_shift(plaintext: str, ciphertext: str) -> int:
    ...
def encrypt(text: str, k: int) -> str:
    ...
def decrypt(text: str, k: int) -> str:
    ...
```

Follow-up (as reported): Given only a ciphertext (no key and no plaintext), how would you break the cipher? Discuss trying all 26 shifts (brute force) or English letter-frequency analysis.

EXAMPLES

Example 1	
Input	`find_shift("abc", "def")`
Output	`3`
Why	`'a'->'d', 'b'->'e', 'c'->'f' is a uniform forward shift of 3.`

Example 2	
Input	`encrypt("hello world", 3)`
Output	`"khoor zruog"`
Why	`Each letter moves forward 3 ('h'->'k', 'o'->'r', 'w'->'z'); the space is unchanged.`

Example 3	
Input	`decrypt("khoor zruog", 3)`
Output	`"hello world"`
Why	`decrypt inverts encrypt, so the original message is recovered.`

Example 4	
Input	`encrypt("xyz", 3)`
Output	`"abc"`
Why	`Wraparound: 'x'->'a', 'y'->'b', 'z'->'c'.`

CONSTRAINTS

- 1 <= len(text) <= 10^5 . text may contain lowercase letters, spaces, digits, and punctuation; only lowercase letters a – z are shifted.

QUESTION

- For find_shift , len(plaintext) == len(ciphertext) and they align character by character.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Key from one letter pair,
# shared shift helper
# Idea: The difference of one letter pair is the key; the
# same helper shifts forward or backward.
def find_shift(plaintext, ciphertext):
    for p, c in zip(plaintext, ciphertext):
        if "a" <= p <= "z":
            return (ord(c) - ord(p)) % 26
    return 0

# BASE QUESTION (optimal): Key from one letter pair,
# shared shift helper
# Idea: The difference of one letter pair is the key; the
# same helper shifts forward or backward.
def shift_text(text, k):
    out = []
    for ch in text:
        if "a" <= ch <= "z":
            ch = chr((ord(ch) - 97 + k) % 26 + 97)
        out.append(ch)
    return "".join(out)

# BASE QUESTION (optimal): Key from one letter pair,
# shared shift helper
# Idea: The difference of one letter pair is the key; the
# same helper shifts forward or backward.
def encrypt(text, k):
    return shift_text(text, k)

# BASE QUESTION (optimal): Key from one letter pair,
# shared shift helper
# Idea: The difference of one letter pair is the key; the
# same helper shifts forward or backward.
def decrypt(text, k):
    return shift_text(text, -k)
```

188 Add two big integers from digit lists

MEDIUM

Template fit: the template as written

QUESTION

You are given two non-empty singly linked lists (or arrays) representing two non-negative integers. The digits are stored in **reverse order**, and each node contains a single digit from 0 to 9. Your task is to add the two numbers and return the sum as a new linked list, also in reverse order. You may assume:

- Each list has at least one node.
- The integers do not contain leading zeros, except the number 0 itself.

Example

Input:

- List 1: 2 -> 4 -> 3 (represents the integer 342)
- List 2: 5 -> 6 -> 4 (represents the integer 465)

Output:

- 7 -> 0 -> 8 (represents the integer 807, since 342 + 465 = 807)

Another example

QUESTION

Input:

- List 1: 9 -> 9 -> 9 (represents 999)
- List 2: 1 (represents 1)

Output:

- 0 -> 0 -> 0 -> 1 (represents 1000)

Constraints

- Let n be the length of the first list and m be the length of the second list.
- 1 <= n, m <= 10^5.
- Each node value is an integer in the range [0, 9].

Task: Describe an algorithm to perform this addition and construct the resulting linked list in O(n + m) time and O(1) additional space (excluding the space for the output list).

You are given two non-empty arrays of digits representing two non-negative integers. The digits are stored in **reverse order** (least significant digit first), and each element is a single digit from 0 to 9. Add the two numbers and return the sum as a new array of digits, also in reverse order.

Assumptions

- Each array has at least one element.
- The integers do not contain leading zeros, except the number 0 itself.

Example 1

```
Input: l1 = [2, 4, 3], l2 = [5, 6, 4] (342 + 465)
Output: [7, 0, 8] (807)
```

Example 2

```
Input: l1 = [9, 9, 9], l2 = [1] (999 + 1)
Output: [0, 0, 0, 1] (1000)
```

Implement **`add_two_numbers(l1, l2)`** (camelCase **`addTwoNumbers`** in Java/C++/JS) returning the digit array of the sum in reverse order. Aim for O(n + m) time and O(1) extra space beyond the output.

Constraints

1 <= n, m <= 10^5 where n = len(l1), m = len(l2)Each element is an integer in the range [0, 9]Digits are stored in reverse order (least significant digit first)No leading zeros except for the number 0 itself

EXAMPLES

```
Input: ([2, 4, 3], [5, 6, 4])
Expected Output: [7, 0, 8]
Explanation: 342 + 465 = 807, stored reversed as [7, 0, 8].
Input: ([9, 9, 9], [1])
Expected Output: [0, 0, 0, 1]
```

Explanation: 999 + 1 = 1000; carry propagates through all three digits and creates a new leading digit.

Hints

Walk both arrays from index 0 (the least significant digit) toward the end, just like grade-school column addition.Maintain a single carry variable. At each position, sum the available digits plus the carry; append (sum % 10) and set carry = sum // 10.When one array runs out, keep going with just the carry and the remaining longer array. Don't forget a final carry can add one extra digit (e.g. 999 + 1 -> [0,0,0,1]).

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Digit by digit with a carry
# Idea: Walk both lists from the right with indices i and
# j; at each step add a[i] (or 0 once exhausted), b[j] (or
```

```python
# 0) and the carry, append sum % 10, and carry = sum //
# 10.
def add_digits(a, b):
    # arrays with the most significant digit FIRST
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry
        total += a[i] if i >= 0 else 0
        total += b[j] if j >= 0 else 0
        out.append(total % 10)
        carry = total // 10
        i, j = i - 1, j - 1
    return out[::-1] or [0]
```

189 Solve several coding interview problems

MEDIUM

Template fit: the big-number addition on strings, with int(ch) per digit; the other parts are list-meeting and overlap sweeps

QUESTION

Overview: This set of problems evaluates proficiency in string processing and combinatorial parsing, arbitrary-precision arithmetic using string representations, pointer/ancestor-chain intersection detection, and computational geometry for overlap counting, emphasizing algorithmic design, correctness, and performance.

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Big numbers: digit by digit
# from the right with a carry
# Idea: Walk both strings from the right, add the two
# digits and the carry, append total % 10, carry total //
# 10; after the loop add a final carry digit if needed and
# reverse.
def add_strings(a, b):
    # big numbers kept as digit strings, added from the
    # right
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry
        if i >= 0:
            total += int(a[i])
        if j >= 0:
            total += int(b[j])
        out.append(str(total % 10))
        carry = total // 10
        i, j = i - 1, j - 1
    return "".join(out[::-1])

# BASE QUESTION (optimal): Big numbers: digit by digit
# from the right with a carry
# Idea: Walk both strings from the right, add the two
# digits and the carry, append total % 10, carry total //
# 10; after the loop add a final carry digit if needed and
# reverse.
def intersection(head_a, head_b):
    # two chains meeting: each pointer walks its chain and
    # then
    # the other one, so both walk the same total length
    a, b = head_a, head_b
    while a is not b:
        a = a.next if a else head_b
        b = b.next if b else head_a
    return a

# BASE QUESTION (optimal): Big numbers: digit by digit
```

```
# from the right with a carry
# Idea: Walk both strings from the right, add the two
# digits and the carry, append total % 10, carry total //
# 10; after the loop add a final carry digit if needed and
# reverse.
def overlap_depth(rects):
    # rects (x1, y1, x2, y2), half-open: the largest
```

```
    # number of
    # rectangles sharing one point. The best point can be
    # a
    # rectangle's lower-left corner, so test those.
    best = 0
    for x, y, _, _ in rects:
        inside = [r for r in rects if r[0] <= x < r[2]
                  and r[1] <= y < r[3]]
        best = max(best, len(inside))
    return best
```

190 Check Whether Digits Are Monotone Increasing

MEDIUM

Template fit: peel digits from the right; each digit must not exceed the previous one you saw

QUESTION

Overview: Check whether a non-negative integer's decimal digits are non-decreasing from left to right. Cover zero, single-digit values, repeated digits, and an O(d) implementation using either arithmetic digit extraction or string traversal.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Peel digits from the right and
# compare with the previous one
# Idea: Take the last digit with n % 10, then the next
# one, and so on. Because we move right to left, the digit
# just read must not exceed the one read before it. A
# single digit and zero are monotone.
def is_monotone_digits(n):
    # non-decreasing digits, read from the right with n %
    # 10
    prev = 9
    while n:
        d = n % 10
        # a digit bigger than the one to its right: a
        # descent
        if d > prev:
            return False
        prev = d
        n //= 10
    return True
```

191 Check Digits and Combine Ranges

HARD

Template fit: palindrome number: reverse only the second half with n % 10 and n // 10 and compare with the first half

QUESTION

You are asked to solve two independent coding tasks.

Task 1:

Check Whether an Integer Reads the Same Backward Given an integer **`x`**, return **`true`** if its decimal representation reads the same from left to right and from right to left; otherwise return **`false`**. Requirements and edge cases:

- Negative integers should return **`false`** because of the leading minus sign.

QUESTION

- You may assume **x** fits in a signed 32-bit integer.
- Try to solve it without converting the integer to a string. Example:

```
Input: x = 121
Output: true
```

```
Input: x = -121
Output: false
```

```
Input: x = 10
Output: false
```

Task 2:

Combine Overlapping Ranges Given an array of intervals **`intervals`**, where each interval is represented as **`[start, end]`**, combine all overlapping intervals and return an array of non-overlapping intervals that covers exactly the same ranges. Requirements and edge cases:

- Intervals may be provided in any order.
- Two intervals overlap if they share at least one point. For example, **`[1, 4]`** and **`[4, 5]`** should be combined into **`[1, 5]`**.
- Return the intervals in increasing order by start value. Example:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
```

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
```

Part 1: Check Whether an Integer Reads the Same BackwardGiven an integer x, return True if its decimal representation reads the same from left to right and from right to left. Otherwise, return False. Negative integers are not palindromes because of the leading minus sign. Try to solve the problem using arithmetic operations instead of converting the integer to a string.
Constraints-2^31 <= x <= 2^31 - 1Negative integers must return FalseAn arithmetic-based solution is preferred over string conversion

EXAMPLES

```
Input: 121
Expected Output: True
```

Explanation: 121 reads the same forward and backward.

```
Input: -121
Expected Output: False
```

Explanation: Negative numbers are not palindromes.

QUESTION

Hints

A negative number can never be a palindrome because the minus sign only appears on the left.Try reversing only the second half of the digits instead of the whole number.

Part 2: Combine Overlapping RangesGiven a list of intervals, where each interval is represented as [start, end], merge all overlapping intervals and return a new list of non-overlapping intervals that covers exactly the same ranges. Intervals may appear in any order. Two intervals overlap if they share at least one point, so [1, 4] and [4, 5] should be merged into [1, 5]. Return the result sorted by starting value.

Constraints

0 <= len(intervals) <= 10^4-10^9 <= start <= end <= 10^9Intervals may be unsorted

Examples

Input: [[1, 3], [2, 6], [8, 10], [15, 18]]

Expected Output: [[1, 6], [8, 10], [15, 18]]

Explanation: The first two intervals overlap and merge into [1, 6].

Input: [[1, 4], [4, 5]]

Expected Output: [[1, 5]]

Explanation: The intervals share the point 4, so they overlap.

Hints

Sort the intervals by their start value before trying to merge them.Keep track of the last interval in the output. If the next interval overlaps, extend the end; otherwise, start a new interval.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): reverse only the second half
# arithmetically
# Idea: Reject negatives and positive numbers ending in 0.
def is_palindrome(x, base=10):
    # no string: build the reversed second half
    if x < 0 or (x % base == 0 and x != 0):
        return False
    rev = 0
    # stop when the reversed half catches up with the rest
    while x > rev:
        rev = rev * base + x % base
        x //= base
    # an odd number of digits has a middle digit to drop
    return x == rev or x == rev // base
```

192 Permutation Sequence — HARD

Template fit: factorial number system: divide k - 1 by (n - 1)!, (n - 2)!, ... to pick each digit from the list of unused digits

QUESTION

The set [1, 2, ..., n] has n! distinct permutations. Listing them in strictly increasing (lexicographic) order and numbering them from 1 , return the k -th permutation as a string. Aim for an O(n) -ish construction using the factorial number system rather than generating all permutations.

Function signature

def get_permutation(n: int, k: int) -> str:

...

Follow-up (as reported): A classmate asked whether a brute-force "generate all and index" is acceptable; discuss why the factorial-base method avoids O(n!) .

EXAMPLES

QUESTION

Example 1	
Input	n = 3, k = 3
Output	"213"
Why	Order is 123, 132, 213, 231, 312, 321; the 3rd is "213".

Example 2	
Input	n = 4, k = 9
Output	"2314"
Why	The 9th permutation of 1234 in lexicographic order is "2314".

Example 3	
Input	n = 3, k = 1
Output	"123"
Why	The first permutation is the sorted sequence.

CONSTRAINTS

- 1 <= n <= 9
- 1 <= k <= n!

MAIN CODE | helper functions first, then the answer

```python
from math import factorial


# BASE QUESTION (optimal): Factorial number system
# Idea: Each digit is chosen by dividing the remaining
# index by the size of a block of permutations.
def get_permutation(n, k):
    digits = list(range(1, n + 1))
    k -= 1
    out = []
    for i in range(n, 0, -1):
        block = factorial(i - 1)
        out.append(digits.pop(k // block))
        k %= block
    return "".join(map(str, out))
```

GROUP 25 OF 43 | 6 QUESTIONS, ONE CODE TEMPLATE | 4 M 2 H

Look at the input once and keep only a small summary (and count your calls)

193 Two-Day Log Intersection under Memory Limit

MEDIUM

Template fit: dict object -> set of event codes for each day; keep the objects that appear on both days with at least two distinct codes

QUESTION

PhD intern coding round 1: two log files with **`(timestamp, obj_id, client_id)`** entries. Find **`obj_id`**s that (a) appear in both files and (b) have at least two distinct **`client_id`**s. Follow-up imposes a memory limit, expecting external sort + two-pointer.

Input: two log files; each line is (timestamp, obj_id, client_id).
Find every obj_id such that:

(a) appears in both files, and
(b) is associated with at least 2 distinct client_ids (aggregated across both files).

Follow-up 1: complexity of set-intersection on the file contents.
Follow-up 2: memory cannot hold either file fully — design a streaming solution.

EXAMPLES

```
File A:  (t1, X, c1), (t2, X, c2), (t3, Y, c1)
File B:  (t4, X, c1), (t5, Z, c2)
-> Only X is in both files; X has 2 distinct clients {c1, c2}.
Output: [X]
```

MAIN CODE | helper functions first, then the answer

```python
from collections import defaultdict


# BASE QUESTION (optimal): Hash map of client sets
# (everything fits in memory)
# Idea: Build object -> set of clients for each file, keep
# the objects that appear in both, and count the union of
# their client sets.
def log_objects(day1, day2):
    # everything fits in memory: object -> set of clients
    c1, c2 = defaultdict(set), defaultdict(set)
    for _, obj, client in day1:
        c1[obj].add(client)
    for _, obj, client in day2:
        c2[obj].add(client)
    # in both files, with 2+ clients over both days
    both = c1.keys() & c2.keys()
    return sorted(o for o in both
                  if len(c1[o] | c2[o]) >= 2)
```

194 Find shared objects across two log files — MEDIUM

Template fit: load only the smaller file's object ids into a set, then stream the big file and keep ids found in the set

QUESTION

You are given two large log files representing activity on two different days. Each line of each log has three fields:

- **`timestamp`**: a time value (you may treat it as an opaque string or integer)
- **`obj_id`**: identifier of an object (string or integer)
- **`client_id`**: identifier of a client (string or integer) The logs are not necessarily sorted by any field. Define an object as **interesting** if:

1. Its **`obj_id`** appears at least once in day 1's log **and** at least once in day 2's log; and 2. Across all of its appearances in both days combined, it is associated with at least **two distinct `client_id`** values.

Tasks

1. Design and implement a function that, given file handles (or streaming iterators) for the two log files, returns the set of all such interesting **`obj_id`**s.
2. For your initial, straightforward in-memory solution (you may assume both logs fit in memory), state the time and space complexity in terms of the total number of log records.
3. Follow-up (memory constrained): Now assume the logs are very large and **cannot** both fit into memory at once.

- You may still sequentially scan each file and you may perform external sorting.
- Describe how you would modify your approach to work under this memory constraint.
- Analyze the time and space complexity of this memory-efficient solution.

Return sorted obj_ids that appear in both day logs and have at least two distinct client_ids across both days combined.
Constraints
Rows are [timestamp, obj_id, client_id] or dicts with those fields

EXAMPLES

```
Input: ([['t1', 'o1', 'c1'], ['t2', 'o2', 'c1']], [['t3', 'o1', 'c2'], ['t4', 'o2', 'c1']])
Expected Output: ['o1']
Input: ([], [['t', 'o', 'c']])
Expected Output: []
```

Hints

Track which days each object appears in and its set of clients.

MAIN CODE | helper functions first, then the answer

```python
from collections import defaultdict


# BASE QUESTION (optimal): Hash set of the smaller file's
# ids
# Idea: Load the object ids of the smaller file into a
# set, then stream the other file and keep ids that are in
# the set. Each line is read once; memory depends only on
# the smaller file.
def shared_objects(day1, day2):
    # rows are [timestamp, obj_id, client_id]. Remember,
    # per
    # object, which days it appeared on and its clients.
    days, clients = defaultdict(set), defaultdict(set)
    for number, rows in ((1, day1), (2, day2)):
        for _, obj, client in rows:
            days[obj].add(number)
            clients[obj].add(client)
    return sorted(o for o in days
                  if len(days[o]) == 2
                  and len(clients[o]) >= 2)
```

195 Streaming Points: Emit K Within Distance

MEDIUM

Template fit: keep the points in a sorted list; each new point is inserted with bisect and only windows around it are re-checked

QUESTION

Process a stream of float positions. Whenever three points satisfy the distance condition, output and remove those three points; a follow-up generalizes from 3 points to K points. The important ambiguity is whether the condition means a span of length **d** or pairwise distance within **d**.SWEdata-structuresortingsliding-windowmediumambiguous-specFrequencySingle reportLast asked2026-06-23Stageonsite-coding

MAIN CODE | helper functions first, then the answer

```python
from bisect import bisect_left, insort


# BASE QUESTION (optimal): Sorted list, check only windows
# touching the change
# Idea: Insert with bisect. Because no window existed
# before, a new window must contain the new point, so test
# the K windows that include its position.
class PointStream:
    def __init__(self, k, d):
        # emit when k points fit in a span of d
        self.k, self.d = k, d
        # all points still waiting, kept sorted
        self.points = []

    def window_start(self, pos):
        # first window of k neighbours that includes pos
        lo = max(0, pos - self.k + 1)
        hi = min(pos, len(self.points) - self.k)
        for i in range(lo, hi + 1):
            last = self.points[i + self.k - 1]
            span = last - self.points[i]
            if span <= self.d:
                return i
        return None

    def add(self, x):
        # insert in order, then emit every window made by
        # x
        insort(self.points, x)
        out, pos = [], bisect_left(self.points, x)
        while True:
            i = self.window_start(pos)
            if i is None:
                return out
            # remove the group so it is emitted only once
            out.append(self.points[i:i + self.k])
            del self.points[i:i + self.k]
            pos = i
```

196 Minimize calls to find all bad test pairs

MEDIUM

Template fit: the same group testing: put a whole group in one call and split only the groups that fail

QUESTION

You have (n) atomic tests (t_1, t_2, \dots, t_n). You are given access to a black-box function:

```
bool runTest(set<Test> S)
```

which behaves as follows:

- **`runTest(S) == True`** if the subset (S) contains **no** incompatible (bad) pair of tests.
- **`runTest(S) == False`** if the subset (S) contains **at least one** incompatible (bad) pair of tests. There is some unknown set of unordered **bad pairs** ((t_i, t_j)), where (i\ne j). You do **not** know in advance which pairs are bad or how many bad pairs there are. Your goal is to discover **all** bad pairs while making as few calls to **`runTest`** as possible. You may adaptively choose each subset (S) based on the results of previous calls.

Tasks

1. Design an algorithm that, using only calls to **`runTest`**, finds **all** bad pairs ((t_i, t_j)).
2. Analyze the number of calls to **`runTest`** your algorithm requires, as a function of:

- (n): the total number of tests, and - (k): the total number of bad pairs.

3. Argue why your algorithm is correct (i.e., why it will eventually find every bad pair and not report any good pair as bad). You do not need to provide full production code, but describe the algorithm clearly enough that it could be implemented.

You have n atomic tests numbered 0 to n - 1. Some unknown unordered pairs of tests are incompatible, called bad pairs. A black-box oracle runTest(S) returns True if the subset S contains no bad pair, and False if S contains at least one bad pair. In this non-interactive version, you are given bad_pairs only so your code can simulate the oracle. Implement the following deterministic oracle-discovery strategy and return both the bad pairs it discovers and the number of oracle calls it makes. Strategy:

1. Process vertices from right to left: v = n - 1, n - 2, ..., 0.
2. Maintain all bad pairs already discovered among vertices greater than the current v.
3. For the suffix vertices v + 1 through n - 1, greedily partition them into color classes such that no discovered bad pair appears inside one color class. Since all suffix-suffix bad pairs have already been discovered, every color class is truly compatible internally.
4. For each color class C, find all bad partners of v inside C using recursive group testing:

- Call runTest({v} union C).
- If the oracle returns True, v has no bad pair with any test in C.
- If C has size 1 and the oracle returns False, that single pair is bad.
- Otherwise split C into two halves and recurse on both halves. Return all discovered bad pairs sorted lexicographically, where each pair is represented as [i, j] with i < j, along with the exact number of runTest calls made by this strategy.

Constraints
0 <= n <= 5000 <= len(bad_pairs) <= n * (n - 1) / 2Each bad pair contains two distinct test indices in the range [0, n - 1]bad_pairs may be given in any endpoint order

EXAMPLES

```
Input: (0, [])
Expected Output: ([], 0)
```

Explanation: There are no tests and therefore no oracle calls are needed.

```
Input: (4, [])
Expected Output: ([], 3)
```

QUESTION

Explanation: For v = 2, 1, and 0, the algorithm queries the compatible suffix once and finds no bad pairs. v = 3 has an empty suffix.

Hints

If a candidate set C is internally compatible, then runTest({v} union C) tells you whether v conflicts with at least one element of C.Process tests from right to left so that when handling v, all bad pairs inside the suffix v + 1 ... n - 1 are already known and can be used to build compatible groups.

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
def shortest_failing_prefix(items, extra, run_test):
    # binary search: the smallest prefix of `items` that
    # fails
    # together with `extra`; its last item is a partner
    lo, hi = 1, len(items)
    while lo < hi:
        mid = (lo + hi) // 2
        if run_test(set(items[:mid]) | extra):
            lo = mid + 1
        else:
            hi = mid
    return items[lo - 1]

def partners(group, t, run_test):
    # every member of the group that is bad with t
    rest, found = list(group), []
    while rest and not run_test(set(rest) | {t}):
        x = shortest_failing_prefix(rest, {t}, run_test)
        found.append(x)
        rest.remove(x)
    return found

# BASE QUESTION (optimal): Independent groups plus binary
# search for partners
# Idea: Process tests one by one. Keep groups of tests
# that pass runTest.
def find_bad_pairs(tests, run_test):
    # keep groups of tests that pass together; a new test
    # is
    # checked against each group and joins one with no
    # partner
    groups, pairs = [], set()
    for t in tests:
        placed = False
        for group in groups:
            found = partners(group, t, run_test)
            for x in found:
                pairs.add((min(x, t), max(x, t)))
            if not found and not placed:
                group.append(t)
                placed = True
        if not placed:
            groups.append([t])
    return pairs
```

197 Find All Bad Pairs Using runTest Oracle

HARD

Template fit: for each test, binary search the prefix of earlier tests where runTest first fails; that test is a bad partner; repeat until none is left

QUESTION

PhD intern coding round 2: given a black-box oracle **`runTest(S)`** that returns **`true`** iff a subset **`S`** contains no bad pair, identify all bad pairs across N test items with as few oracle calls as possible.

Items t_1, ..., t_N are given.
Oracle: runTest(S: Set[Item]) -> bool returns true iff S contains zero bad pairs; otherwise false.
Output: every bad pair (t_i, t_j).
Minimize total oracle calls.

EXAMPLES

```
N=4, bad pairs = {(1,2)}.
runTest({1,2,3,4}) -> false -> at least one bad pair somewhere.
runTest({1,3,4})   -> true  -> 2 is in every bad pair containing items from this set.
... binary search to isolate.
```

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
def first_partner(group, t, run_test):
    # binary search: the shortest prefix of the group that
    # fails together with t; its last member is a partner
    lo, hi = 1, len(group)
    while lo < hi:
        mid = (lo + hi) // 2
        if run_test(set(group[:mid]) | {t}):
            lo = mid + 1
        else:
            hi = mid
    return group[lo - 1]

def partners(group, t, run_test):
    # all members of the group that are bad with t
    rest, found = list(group), []
    while rest and not run_test(set(rest) | {t}):
        x = first_partner(rest, t, run_test)
        found.append(x)
        # take it out and test the others again
        rest.remove(x)
    return found

# BASE QUESTION (optimal): Group the tests and binary
# search for partners
# Idea: Keep groups of tests that pass together.
def find_bad_pairs(items, run_test):
    # groups of items that pass together (no bad pair
    # inside)
    groups, pairs = [], set()
    for t in items:
        placed = False
        for group in groups:
            found = partners(group, t, run_test)
            for x in found:
                pairs.add((min(x, t), max(x, t)))
            # t joins the first group with no partner for
            # it
            if not found and not placed:
                group.append(t)
                placed = True
        if not placed:
            groups.append([t])
    return pairs
```

198 Find maximum value in huge text file

HARD

Template fit: the template as written; the file is never loaded as a whole

QUESTION

You are given the path to a **very large** text file on disk. The file has the following properties:

- Each line contains exactly one signed 64-bit integer (in decimal).
- The file size can be tens or hundreds of gigabytes, so it **cannot** be fully loaded into memory.
- You have standard file I/O APIs that allow you to read the file sequentially (e.g., line by line or in chunks).

Tasks

1.

Single-machine solution Design an algorithm to compute the **maximum integer value** in this file.

- Assume you have limited RAM (much smaller than the file size).
- Discuss how you would read the file and track the result.
- Give high-level pseudocode.
- Analyze time and space complexity. 2.

Performance improvement / parallelization Suppose you now have a multi-core machine or multiple machines available and want to speed up the computation.

- How would you modify your approach to take advantage of parallelism (e.g., processing the file in chunks)?
- Explain how you would combine partial results to get the global maximum.
- Mention any edge cases or failure scenarios you would consider (e.g., very large values, empty file). Assume you do not have a database; you are working directly with the file system and your programming language's standard I/O libraries.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Single machine: stream the
# file, keep one running maximum
# Idea: Never load the file. Read it through a buffered
# iterator and keep one variable, the maximum so far,
# started at None so that an all-negative file or a one-
# line file is handled correctly (do not start at 0).
def max_in_file(path):
    # one sequential pass, one number in memory
    best = None
    with open(path, "r", buffering=1 << 20) as f:
        for line in f:
            line = line.strip()
            # blank lines are ignored (state this
            # contract)
            if not line:
                continue
            value = int(line)
            # start from None, not 0: all numbers may be
            # negative
            if best is None or value > best:
                best = value
    return best
```

GROUP 26 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 3 M 2 H

Grid DP: each cell from its neighbours above and to the left

199 Grid Path Count with Diagonal Moves and Checkpoints — MEDIUM

Template fit: advance column by column keeping the number of ways in each row; new[r] = old[r-1] + old[r] + old[r+1]; zero out rows that miss a checkpoint

QUESTION

Count paths in an n x m matrix from (n-1, 0) to (n-1, m-1) moving up-right, right, or down-right. Follow-ups: count paths visiting all checkpoints; then checkpoints must be visited in order, with the observation that each move changes column, and optimize space.

Matrix of size n x m. Start at (n-1, 0), end at (n-1, m-1).
Allowed moves from (r, c): up-right (r-1, c+1), right (r, c+1), down-right (r+1, c+1); stay inside the matrix.
Return the total number of distinct paths from start to end.
Follow-up 1: the matrix contains checkpoints; count paths that visit all checkpoints.
Follow-up 2: the checkpoints must be visited in order. Since every move changes the column, each column can contain at most one visited checkpoint. Count the paths and optimize space.

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
def step(ways):
    # next column from the current column only
    n = len(ways)
    return [(ways[r - 1] if r else 0) + ways[r]
            + (ways[r + 1] if r + 1 < n else 0)
            for r in range(n)]

def keep_row(ways, row):
    # a checkpoint forces the path through one row
    return [x if r == row else 0
            for r, x in enumerate(ways)]

def checkpoint_rows(checkpoints, ordered):
    # column -> required row; None if impossible
    rows = {}
    for r, c in checkpoints:
        # two different checkpoints in one column: no path
        if rows.setdefault(c, r) != r:
            return None
    if len(rows) != len(checkpoints):
        return None
    cols = [c for _, c in checkpoints]
    # ordered: they must be listed left to right
    if ordered and cols != sorted(cols):
        return None
    return rows

# BASE QUESTION (optimal): Column DP with a rolling array
# and checkpoint filters
# Idea: ways[r] is the number of paths that are in row r
# at the current column. The next column is ways'[r] =
# ways[r - 1] + ways[r] + ways[r + 1]. At a checkpoint
# column keep only the checkpoint's row.
def count_paths(n, m, checkpoints=(), ordered=False):
    rows = checkpoint_rows(list(checkpoints), ordered)
    if rows is None:
        return 0
    # one rolling column is enough: O(n) space
```

```
    ways = [0] * n
    ways[n - 1] = 1
    if 0 in rows:
        ways = keep_row(ways, rows[0])
    for c in range(1, m):
        ways = step(ways)
```

```
        if c in rows:
            ways = keep_row(ways, rows[c])
    return ways[n - 1]
```

200 Longest Same-Color Grid Sequence — MEDIUM

Template fit: keep four run lengths per cell (horizontal, vertical, two diagonals) from the previous row and the left cell; the answer is the largest

QUESTION

Given a grid of colors, return the longest consecutive same-color sequence along a horizontal, vertical, or diagonal line.

Input is a two-dimensional grid of colors such as R, G, and B.
Return the length of the longest consecutive sequence of a single color horizontally, vertically, or diagonally.

EXAMPLES

```
R R R G
R R B B
G R R R
B G B R

Answer: 4
```

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Four run lengths per cell
# Idea: Keep, for the previous row, the run length ending
# at each column in the vertical and both diagonal
# directions, and a running value for the horizontal run
# in the current row.
def longest_same_color_line(g):
    if not g or not g[0]:
        return 0
    n, best = len(g[0]), 0
    # run lengths ending in the previous row:
    # straight down, down-left, down-right
    up, dl, dr = [0] * n, [0] * n, [0] * n
    for i, row in enumerate(g):
        nu, nl, nr = [1] * n, [1] * n, [1] * n
        left = 0
        for j, color in enumerate(row):
            # horizontal run from the cell on the left
            same = j and row[j - 1] == color
            left = left + 1 if same else 1
            if i:
                # extend the run from the cell above
                if g[i - 1][j] == color:
                    nu[j] = up[j] + 1
                # from above-left (down-right diagonal)
                if j and g[i - 1][j - 1] == color:
                    nr[j] = dr[j - 1] + 1
                # from above-right (down-left diagonal)
                if j + 1 < n and g[i - 1][j + 1] == color:
                    nl[j] = dl[j + 1] + 1
            best = max(best, left, nu[j], nl[j], nr[j])
        up, dl, dr = nu, nl, nr
    return best
```

201 Count Same-Color Squares in Matrix — MEDIUM

Template fit: the template as written: a square ends here if the three neighbours match; its size is 1 + min of their sizes

QUESTION

LeetCode 221 variant — count maximal same-color square sub-grids. DP on each cell as the bottom-right corner.

Input: m × n grid of colored cells.
Output: count of square sub-grids whose cells are all the same color (or maximum side length, depending on prompt).
DP: dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) if grid[i][j] == grid[i-1][j] == grid[i][j-1] == grid[i-1][j-1] else 1.
Sum dp[i][j] across the grid for the count variant; max for the side variant.

EXAMPLES

The plain "count all-1 squares" DP generalizes to same-color directly.

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
def square_sides(g):
    # side[i][j]: the biggest same-colour square whose
    # bottom-right corner is (i, j); one row at a time
    if not g:
        return []
    prev, out = [0] * len(g[0]), []
    for i, row in enumerate(g):
        cur = [1] * len(row)
        for j, c in enumerate(row):
            # all three neighbours must match this cell
            if i and j and g[i - 1][j] == c \
                    and g[i][j - 1] == c \
                    and g[i - 1][j - 1] == c:
                cur[j] = 1 + min(prev[j], cur[j - 1],
                                 prev[j - 1])
        out.append(cur)
        prev = cur
    return out

# BASE QUESTION (optimal): dp on the bottom-right corner
# Idea: dp[j] holds the best square side ending in the
# current row at column j.
def count_same_color_squares(g):
    # every cell contributes one square per side up to its
    # side
    return sum(sum(row) for row in square_sides(g))
```

202 Find the Longest Straight Run of One Color — HARD

Template fit: the same four run lengths; extend a run when the neighbour in that direction has the same colour

QUESTION

Given a rectangular grid of colors, return the length of the longest consecutive sequence of cells with the same color along a horizontal, vertical, or diagonal straight line. A sequence cannot change direction or skip a cell. Both diagonal orientations count. The length is the number of cells in the sequence.

Input

- **`grid`**: an array of strings of equal length. Each character represents one cell's color.

Output

Return the maximum length as an integer.

Constraints

QUESTION

and Edge Cases - For this practice version, the grid contains at least one row and one column, with at most **1000000** cells in total.

- Colors are uppercase English letters; equal letters mean equal colors.
- A one-cell grid has answer **1**.
- Diagonal lines may run down-right or down-left. Reading the same line in reverse does not change its length.
- A connected region that bends is not necessarily one valid sequence.

Example

1

```
grid = ["RRRG", "RRBB", "GRRR", "BGBR"] output = 4
```

The main diagonal contains four consecutive **R** cells.

Example

2

```
grid = ["RBG", "BGR", "GRB"] output = 3
```

The down-left diagonal from the top-right cell contains three **G** cells. No horizontal or vertical sequence has that length.

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))

def runs(grid):
    # yield (length, start, direction) for every maximal
    # run;
    # a run is walked only from its first cell, so the
    # total
    # work is linear in the number of cells
    rows, cols = len(grid), len(grid[0])
    for r in range(rows):
        for c in range(cols):
            for dr, dc in DIRECTIONS:
                pr, pc = r - dr, c - dc
                before = 0 <= pr < rows and 0 <= pc < cols
                if before and grid[pr][pc] == grid[r][c]:
                    # not the first cell of its run
                    continue
                n, a, b = 0, r, c
                while 0 <= a < rows and 0 <= b < cols \
                        and grid[a][b] == grid[r][c]:
                    n += 1
                    a, b = a + dr, b + dc
                yield n, (r, c), (dr, dc)

# BASE QUESTION (optimal): DP per direction with a rolling
# row
# Idea: run[d][i][j] is the length of the same-color run
# ending at (i, j) in direction d. It is the previous
# cell's run plus one when the colors match, else 1.
def longest_run(grid):
    if not grid or not grid[0]:
        return 0
    return max(n for n, _, _ in runs(grid))
```

203 Count same-color squares in a character grid HARD

Template fit: exactly the same recurrence on characters

QUESTION

You are given a 2D grid (matrix) of characters. Each character represents a color: cells with the same character are considered the same color. Formally:

- The grid has **`m`** rows and **`n`** columns.
- **`grid[i][j]`** is an alphabetic character (e.g., **`'a'`**–**`'z'`** or **`'A'`**–**`'Z'`**). A **square** in this grid is defined as a contiguous **`k × k`** submatrix aligned with the grid axes, for some integer **`k >= 1`**. A square is **monochromatic** if **all** of its cells contain the **same** character (i.e., the same color). Task:
- Count and return the total number of monochromatic squares in the grid.
- Count all sizes of squares (1×1, 2×2, ..., up to the largest possible that fits in the grid). Example (just for clarity, not necessarily exhaustive): If the grid is:

```
a a b a a b b b b
```

Then some monochromatic squares include:

- All 1×1 cells (each single cell is a monochromatic square).
- The 2×2 square in the top-left corner formed by **`'a'`**. Your function should return the total count of such monochromatic squares for a given input grid.

You are given a 2D grid of characters, where each character represents a color. A square is a contiguous k × k submatrix aligned with the grid axes, for some k >= 1. A square is monochromatic if all of its cells contain the same character. Return the total number of monochromatic squares of all possible sizes and positions in the grid. Every 1 × 1 cell is considered a monochromatic square.

Constraints

0 <= len(grid) <= 1000If grid is non-empty, 0 <= len(grid[0]) <= 1000All rows in grid have the same lengthgrid[i][j] is an alphabetic characterThe total number of cells is at most 1,000,000

EXAMPLES

```
Input: ([])
Expected Output: 0
```

Explanation: An empty grid contains no squares.

```
Input: (["aab", "aab", "bbb"])
Expected Output: 10
```

Explanation: There are 9 single-cell squares and one 2x2 monochromatic square of 'a' in the top-left corner.

Hints

Try defining dp[i][j] as the side length of the largest monochromatic square whose bottom-right corner is cell (i, j).A square larger than 1 can end at (i, j) only if the current cell matches its top, left, and top-left neighboring cells.

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
def square_table(grid):
    # side[i][j]: the biggest same-colour square with its
    # bottom-right corner at (i, j)
    prev, rows = [0] * len(grid[0]), []
    for i, row in enumerate(grid):
        cur = [1] * len(row)
        for j, ch in enumerate(row):
            # the up, left and up-left cells must match
            # too
            if i and j and ch == grid[i - 1][j] \
                    and ch == grid[i][j - 1] \
```

```
                    and ch == grid[i - 1][j - 1]:
                cur[j] = 1 + min(prev[j], cur[j - 1],
                                 prev[j - 1])
        rows.append(cur)
        prev = cur
    return rows

# BASE QUESTION (optimal): DP: side of the largest square
# ending at each cell, summed
# Idea: If the cell matches its top, left and top-left
# neighbours then dp = 1 + min(dp top, dp left, dp top-
# left); otherwise 1. Summing dp over all cells counts
# every square exactly once, at its bottom-right corner.
def count_same_color_squares(grid):
    # each cell adds one square for every size up to its
    # side
    return sum(sum(row) for row in square_table(grid))
```

GROUP 27 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 5 M

Nested structures: a stack of contexts, or recursion with a cache

204 Decode String — MEDIUM

Template fit: the template as written

QUESTION

EXAMPLES

```
"3[a]2[bc]" -> "aaabcbc".
"2[abc]3[cd]ef" -> "abcabccdcdcdef".
"3[a2[c]]" -> "accaccacc".
```

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Stack of (prefix, repeat)
# frames
# Idea: On '[' push the text built so far and the repeat
# count, then start fresh. On ']' pop and append the
# current text repeated count times to the saved prefix.
def decode(s):
    # stack of (text so far, repeat count) for each open [
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":
            # remember what we have, start the inner text
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":
            prev, times = stack.pop()
            prev.append("".join(cur) * times)
            cur = prev
        else:
            cur.append(ch)
    return "".join(cur)
```

205 Template Variable Expansion with Follow-Ups — MEDIUM

Template fit: resolve(name) with a cache and a 'being resolved' set; meeting a name that is being resolved is a cycle

QUESTION

Replace **%KEY%** placeholders using a key-value map whose values may contain other placeholders. Clarify malformed delimiters and literal-percent escaping, flatten nested values, and detect cyclic dependencies before expansion.

Input: a key-value map and a string containing placeholders delimited by %, such as %USER%. Replace every placeholder with the corresponding value.
Map values can themselves contain placeholders, so resolve or flatten dependencies before expanding the final string. Define behavior for malformed delimiters such as unmatched percent signs, for a literal % in output, and for nested-looking forms before coding; the prompt intentionally leaves these contracts open.
Follow-up: detect cyclic dependencies among map values. Model keys as a dependency graph and use indegrees plus a queue to determine whether a complete topological resolution is possible.

EXAMPLES

```
Given {USER: "admin", HOME: "/%USER%/home"}:
```

Input: I am %USER% My home is %HOME% Output: I am admin My home is /admin/home

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
def tokens(s):
    # split text into ("text", char) and ("var", name)
    i = 0
    while i < len(s):
        if s[i] != "%":
            yield "text", s[i]
            i += 1
        elif s[i:i + 2] == "%%":
            # %% is a literal percent sign
            yield "text", "%"
            i += 2
        else:
            j = s.find("%", i + 1)
            if j < 0:
                raise ValueError("unmatched %")
            yield "var", s[i + 1:j]
            i = j + 1

# BASE QUESTION (optimal): Depth-first expansion with memo
# and a visiting set
# Idea: expand(text) scans the text once: %% becomes %,
# %NAME% becomes the resolved value of NAME.
def expand_template(text, mapping):
    done, visiting = {}, set()

    def resolve(name):
        # values may hold placeholders: expand them first
        if name in done:
            return done[name]
        if name in visiting:
            raise ValueError("cycle at " + name)
        if name not in mapping:
            raise KeyError(name)
        visiting.add(name)
        done[name] = scan(mapping[name])
        visiting.discard(name)
        return done[name]

    def scan(s):
        return "".join(resolve(x) if kind == "var" else x
                       for kind, x in tokens(s))

    return scan(text)
```

206 Expand Recursive Key-Value Placeholders in a Template — MEDIUM

Template fit: regex-find {key}, expand each key recursively with a cache and a visiting set to stop cycles

QUESTION

Overview: Expand %KEY% placeholders in a template, recursively resolving placeholders inside mapped values. Use memoized dependency traversal to preserve literal text, avoid repeated work, and explain cycle, unknown-key, and escaping policies.

MAIN CODE | helper functions first, then the answer

```
import re


# HELPERS for the base question
PATTERN = re.compile(r"%%|%(\w+)%")

# BASE QUESTION (optimal): Memoised recursive expansion
```

```
# with a visiting set
# Idea: expand(key) substitutes every placeholder in the
# key's value by expand(inner), caches the result, and
# uses a visiting set to raise on a cycle. Unknown keys
# stay as written.
def expand(template, mapping, unknown="keep",
           limit=10 ** 6):
    # unknown: "keep" the text, "empty" it, or "error"
    cache, visiting = {}, set()

    def expand_text(text):
        def one(m):
            key = m.group(1)
            if key is None:
                # %% is a literal percent sign
                return "%"
            if key not in mapping:
                if unknown == "error":
                    raise KeyError(key)
                if unknown == "empty":
                    return ""
                return m.group(0)
            return expand_key(key)
        out = PATTERN.sub(one, text)
        if len(out) > limit:
            raise ValueError("expansion too large")
        return out

    def expand_key(key):
        # each key is expanded once and then cached
        if key in cache:
            return cache[key]
        if key in visiting:
            raise ValueError(f"cycle at {key}")
        visiting.add(key)
        cache[key] = expand_text(mapping[key])
        visiting.discard(key)
        return cache[key]
```

```
    return expand_text(template)
```

207 Nested List Weight Sum (LC 339) with Follow-Ups

MEDIUM

*Template fit: depth counter: '[' adds 1, ']' subtracts 1; each number adds value * depth (read digits and a minus sign as you go)*

QUESTION

EXAMPLES

```
"[3, 8, [2, 14], [2, [91]]]" -> 3×1 + 8×1 + 2×2 + 14×2 + 2×2 + 91×3 = 320.
Use [2, [91]] as a manual trace for bracket depth and the final-number flush.
```

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
def with_depths(nested):
    # every integer with its depth (top level = 1),
    # using a stack so deep input cannot overflow
    stack = [(nested, 1)]
    while stack:
        item, depth = stack.pop()
        if isinstance(item, list):
            for child in item:
                stack.append((child, depth + 1))
```

```
        else:
            yield item, depth - 1

# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
# Idea: Every integer is weighted by the number of '['
# currently open, so no tree needs to be built. Accumulate
# digits in a buffer and flush the buffer when a non-digit
# arrives or the string ends.
def weight_sum(nested):
    # deeper integers weigh more (LeetCode 339)
    return sum(v * d for v, d in with_depths(nested))

# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
# Idea: Every integer is weighted by the number of '['
# currently open, so no tree needs to be built. Accumulate
# digits in a buffer and flush the buffer when a non-digit
# arrives or the string ends.
def weight_sum_string(s):
    # one pass over the text, O(1) extra space
    depth = total = 0
    num, sign, have = 0, 1, False
    # a closing sentinel flushes a trailing number
    for ch in s + "]":
        if ch.isdigit():
            num, have = num * 10 + int(ch), True
            continue
        # any other character ends the number being read
        if have:
            total += sign * num * depth
            num, sign, have = 0, 1, False
        if ch == "[":
            depth += 1
        elif ch == "]":
```

```
            depth -= 1
        elif ch == "-":
            sign = -1
    return total
```

208 Nested Function Expression Evaluator — MEDIUM

Template fit: tokenise, then a recursive parse(i): a number returns itself; a name reads its arguments until ')' and applies the function

QUESTION

EXAMPLES

```
add(1, sub(1, 0)) -> 2
add(2, mul(3, pow(4, 5))) -> 3074
```

MAIN CODE | helper functions first, then the answer

```
import math
import re


# HELPERS for the base question
TOKEN = re.compile(r"-?\d+|[A-Za-z_]\w*|[(),]")

def divide(a, b):
    # integer division that truncates toward zero
    if b == 0:
        raise ZeroDivisionError("division by zero")
```

```
    return int(a / b)

OPS = {
    "add": lambda *a: sum(a),
    "sub": lambda a, b: a - b,
    "mul": lambda *a: math.prod(a),
    "pow": lambda a, b: a ** b,
    "div": divide,
    "min": min,
    "max": max,
}

# BASE QUESTION (optimal): Tokenizer + recursive descent
# over name(arg, arg, ...)
# Idea: Tokenize into identifiers, integers, commas and
# parentheses.
def evaluate(expr, env=None):
    env = env or {}
    tokens = TOKEN.findall(expr)

    def parse(i):
        # returns (value, index after this expression)
        tok = tokens[i]
        if re.fullmatch(r"-?\d+", tok):
            return int(tok), i + 1
        if i + 1 >= len(tokens) or tokens[i + 1] != "(":
            if tok not in env:
                raise ValueError(f"undefined {tok!r}")
            return env[tok], i + 1
        if tok not in OPS:
            raise ValueError(f"unknown function {tok!r}")
        i += 2
        args = []
        # read arguments until the closing bracket
```

```
        while tokens[i] != ")":
            value, i = parse(i)
            args.append(value)
            if tokens[i] == ",":
                i += 1
        return OPS[tok](*args), i + 1

    value, end = parse(0)
    if end != len(tokens):
        raise ValueError("trailing tokens")
    return value
```

GROUP 28 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 5 M

Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

209 Shortest Path Nodes and Escaping a Pursuer — MEDIUM

Template fit: run it from a, from t and from b; a node is on a shortest a-to-t path when da + dt == da[t]; compare with the pursuer's distance db

QUESTION

Unweighted undirected graph: shortest path from Alice's node to a target (BFS). Follow-up 1: all nodes that lie on at least one shortest path. Follow-up 2: Bob starts at another node at the same time; decide whether Alice can follow some shortest path to the target without being caught.

Input: an undirected, unweighted graph; Alice starts at one node and must reach a target node.
Return the shortest path (length) from Alice to the target.
Follow-up 1: find all nodes that can appear on some shortest path from Alice to the target.
Follow-up 2: Bob is at another node. Alice picks one shortest path and starts moving; Bob starts moving at the same time. Decide whether Alice can reach the target safely without being caught by Bob.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# HELPERS used by the code below
def bfs(adj, s):
    # distance from s to every reachable node
    d = {s: 0}
    q = deque([s])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if v not in d:
                d[v] = d[u] + 1
                q.append(v)
    return d

# HELPERS for the base question
FAR = 10 ** 9

# BASE QUESTION (optimal): Three BFS runs and a safe
# shortest-path DAG
# Idea: dA, dT and dB come from BFS. The nodes on some
# shortest path are those with dA[v] + dT[v] == dA[t].
def shortest_length(adj, a, t):
    # the base question: edges on the shortest path
    return bfs(adj, a).get(t)
```

210 Radio Broadcast Reachability — MEDIUM

Template fit: neighbors(u) = every radio within the radius, worked out with squared distances; goal = the target radio, answer = whether it was reached

QUESTION

You are given a list of radios placed on a 2D plane, each described by its (x, y) coordinates. Every radio can broadcast to any other radio within a fixed Euclidean distance radius (inclusive). A message can travel from one radio to another either directly (within radius) or through a chain of intermediate radios that each relay it to a next radio within radius . Given the radios, the broadcast radius , a source index, and a target index, determine whether a message originating at source can reach target through zero or more hops. Return True or False .

Function signature

```
from typing import List, Tuple
def can_reach(radios: List[Tuple[float, float]],
              radius: float,
              source: int,
              target: int) -> bool:
    ...
```

Follow-up (as reported): Model the radios as nodes and connect any two within radius , then run BFS (or DFS / union-find) from source . To avoid floating-point issues, compare squared distances against radius * radius .

EXAMPLES

Example 1	
Input	`radios = [(0, 0), (1, 0), (2, 0), (5, 0)], radius = 1.5, source = 0, target = 2`
Output	`True`
Why	`0->1 (dist 1) and 1->2 (dist 1) are each within 1.5, so 2 is reachable.`

Example 2	
Input	`radios = [(0, 0), (1, 0), (2, 0), (5, 0)], radius = 1.5, source = 0, target = 3`
Output	`False`
Why	`Radio 3 is 3 units from radio 2 (> 1.5) and farther from the rest; no chain of hops reaches it.`

Example 3	
Input	`radios = [(0, 0), (3, 4)], radius = 5.0, source = 0, target = 1`
Output	`True`
Why	`The Euclidean distance is exactly 5.0, which is within radius.`

CONSTRAINTS

- 1 <= len(radios) <= 2000
- Coordinates are integers or floats with -10^4 <= x, y <= 10^4 . radius >= 0 .
- 0 <= source, target < len(radios) .
- Two radios a and b are directly connected when sqrt((ax-bx)^2 + (ay-by)^2) <= radius .
- If source == target , the answer is True .

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# BASE QUESTION (optimal): BFS over the implicit graph
# Idea: Compare squared distances; every radio enters the
# queue once.
def can_reach(radios, radius, source, target):
    limit = radius * radius
    seen = [False] * len(radios)
    seen[source] = True
```

```
    queue = deque([source])
    while queue:
        u = queue.popleft()
        if u == target:
            return True
        for v, (x, y) in enumerate(radios):
            dx, dy = radios[u][0] - x, radios[u][1] - y
            if (
                not seen[v]
                and dx * dx + dy * dy <= limit
            ):
                seen[v] = True
                queue.append(v)
    return False
```

211 Minimum Direction Violations in a Directed Graph

MEDIUM

Template fit: an edge in its own direction costs w = 0, the same edge walked backwards costs w = 1; free steps use appendleft

QUESTION

EXAMPLES

```
n = 5
edges = [[0, 1], [2, 1], [2, 3], [4, 3]]
start = 0
end = 4

path: 0 -> 1 -> 2 -> 3 -> 4
cost: 0 + 1 + 0 + 1 = 2
output: 2
```

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# HELPERS used by the code below
INF = float("inf")

# BASE QUESTION (optimal): 0-1 BFS on the doubled graph
# Idea: Add every edge twice: forward with weight 0,
# backward with weight 1. Shortest path with weights in
# {0, 1} is a deque BFS: push a zero-weight neighbour to
# the front and a one-weight neighbour to the back.
def min_violations(n, edges, start, end):
    # 0-1 BFS: following an edge is free, reversing costs
    # 1
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append((v, 0))
        graph[v].append((u, 1))
    dist = [INF] * n
    dist[start] = 0
    dq = deque([start])
    while dq:
        u = dq.popleft()
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                # cost 0 goes to the front, cost 1 to the
                # back
                if w == 0:
                    dq.appendleft(v)
                else:
```

```
                dq.append(v)
    return -1 if dist[end] == INF else dist[end]
```

212 Compute shortest paths with blocked nodes — MEDIUM

Template fit: neighbors skip every blocked node; if the source itself is blocked the answer is all -1; unreachable nodes stay -1

QUESTION

Given a graph with nodes and edges and a designated source node s, compute the shortest distance from s to every other node. Some nodes are inaccessible (blocked) and cannot be traversed or reached; treat them as removed from the graph. Return a distance for each node where unreachable nodes (including blocked ones) are -1. Describe your algorithm, data structures, and time/space complexity, and note any differences for directed vs. undirected graphs.

Given n nodes labeled 0..n-1, edges, a source, and blocked nodes, return shortest unweighted distances from the source. Blocked and unreachable nodes must be -1. Set directed=True to treat edges as directed.
Constraints
Nodes are labeled 0 through n-1Edges are unweighted

EXAMPLES

```
Input: (5, [[0, 1], [1, 2], [0, 3], [3, 4]], 0, [3], False)
Expected Output: [0, 1, 2, -1, -1]
Input: (4, [[0, 1], [1, 2], [2, 3]], 0, [], True)
Expected Output: [0, 1, 2, 3]
```

Hints

Treat blocked nodes as removed before BFS.For directed graphs, only add the given edge direction.

MAIN CODE | helper functions first, then the answer

```python
from collections import defaultdict, deque


# BASE QUESTION (optimal): BFS that never enters a blocked
# node
# Idea: Build adjacency lists (one direction only if
# directed). If the source is blocked return all -1.
def blocked_distances(n, edges, source, blocked,
                      directed=False):
    # BFS that treats blocked nodes as removed;
    # unreachable
    # and blocked nodes get -1
    bad = set(blocked)
    if source in bad:
        return [-1] * n
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        if not directed:
            adj[v].append(u)
    dist = [-1] * n
    dist[source] = 0
    queue = deque([source])
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if v not in bad and dist[v] == -1:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist
```

213 Shortest Directed Cycle Through a Node

MEDIUM

Template fit: start = v; the goal is an edge that leads back to v, so return dist[u] + 1 the moment a neighbour equals v

QUESTION

EXAMPLES

```
Edges [(0,1),(1,2),(2,0)], v = 0 -> 3.
Edges [(0,1),(1,2)], v = 0 -> -1.
```

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# HELPERS for the base question
def make_graph(n, edges):
    adj = [[] for _ in range(n)]
    for a, b, *w in edges:
        # edges may carry a weight (default 1)
        adj[a].append((b, w[0] if w else 1))
    return adj

# BASE QUESTION (optimal): BFS from v, close the cycle on
# the first edge back to v
# Idea: A directed cycle through v is a path v to u
# followed by an edge u to v. BFS visits nodes in
# nondecreasing distance, so the first time we see an edge
# back to v its length is dist[u] + 1 and is the shortest.
def shortest_cycle_through(n, edges, v):
    # BFS from v: the first edge that comes back to v
    # closes
    # the shortest cycle through v
    adj = make_graph(n, edges)
    dist, queue = {v: 0}, deque([v])
    while queue:
        u = queue.popleft()
        for w, _ in adj[u]:
            if w == v:
                return dist[u] + 1
            if w not in dist:
                dist[w] = dist[u] + 1
                queue.append(w)
    return -1
```

Topological sort (Kahn) and cycle detection

214 Course Scheduling and Dependency Variants

MEDIUM

Template fit: build the vertex set from the edges too; return the order or None

QUESTION

Topological-ordering family. Variants: course ordering with validity checks and a nested-list extension; merging two arrays whose internal order defines precedence (return false on a conflict cycle); and a damaged-node dependency-path variant that minimizes broken nodes on a route.

Course-ordering variant

Work through the course-ordering problem, including validity checks, complexity analysis, and test cases.
Extend the result to a nested-list format. The grouping rule is unspecified; clarify what each inner list represents before implementing the extension.

Damaged-node and path variant

Remove damaged nodes and their downstream effects, then find a valid path.
A follow-up minimizes broken nodes on a required route. Clarify the graph direction, path endpoints, and meaning of broken nodes before choosing an algorithm.

Merge-two-ordered-sequences variant

Input: two integer arrays a and b. The order of elements inside each array is an ordering constraint — a = [1, 3, 5] means 1 must come before 3, and 3 before 5.
Determine whether the two arrays can be merged into one sequence that preserves the relative order of both. Return false if the constraints conflict; otherwise return true (or any valid merged sequence).

EXAMPLES

Merge-two-ordered-sequences

```
a = [1, 2, 3]
b = [2, 4, 5]
-> true   (one valid merged order: [1, 2, 3, 4, 5])

a = [1, 6, 4]
b = [4, 1]
-> false  (a requires 1 -> 6 -> 4, b requires 4 -> 1: contradiction)
```

MAIN CODE | helper functions first, then the answer

```python
from collections import defaultdict, deque


# HELPERS for the base question
def edges_from_sequences(*sequences):
    # each neighbouring pair in a sequence is a rule
    return [(s[i], s[i + 1]) for s in sequences
            for i in range(len(s) - 1)]

def merge_two_orders(a, b):
    # shared values are the same vertex
    edges = edges_from_sequences(a, b)
    # a cycle (conflict) gives None
    return topological_order(set(a) | set(b), edges)

# BASE QUESTION (optimal): Kahn's algorithm (course order,
# or can two sequences merge)
# Idea: Build the graph (prerequisite -> dependent,
# including isolated vertices), count indegrees and keep a
```

```python
# queue of vertices with indegree 0. Pop one, append it to
# the order and lower the indegree of its dependents.
def topological_order(vertices, edges):
    # edge (u, v) means u must come before v
    adj = defaultdict(list)
    indeg = {v: 0 for v in vertices}
    for u, v in edges:
        adj[u].append(v)
        # count unmet prerequisites of v
        indeg[v] = indeg.get(v, 0) + 1
        indeg.setdefault(u, 0)
    # start with everything that has no prerequisite
    queue = deque(v for v, d in indeg.items() if d == 0)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        # finishing u frees its dependents
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                queue.append(v)
    # fewer than all vertices means a cycle: no order
    return order if len(order) == len(indeg) else None

# BASE QUESTION (optimal): Kahn's algorithm (course order,
```

```python
# or can two sequences merge)
# Idea: Build the graph (prerequisite -> dependent,
# including isolated vertices), count indegrees and keep a
# queue of vertices with indegree 0. Pop one, append it to
# the order and lower the indegree of its dependents.
def can_merge(a, b):
    # True when some order keeps both sequences in order
    return merge_two_orders(a, b) is not None
```

215 Validate Race Results (Cycle Detection) MEDIUM

Template fit: edge = winner -> loser; the results are consistent exactly when there is no cycle

QUESTION

You are given a list of pairwise race results as (winner, loser) string pairs, meaning winner beat loser . The results are consistent if they can all be true simultaneously under a strict ranking — equivalently, if the directed graph (edge winner -> loser) contains no cycle. Return True if the results are valid (acyclic) and False otherwise.

Function signature

def results_are_valid(results: list[tuple[str, str]]) -> bool:

...

Follow-up (as reported): Recursive DFS can hit Python's recursion limit on long chains; convert to an iterative DFS with an explicit stack, or use Kahn's topological-sort algorithm.

EXAMPLES

Example 1	
Input	`results = [("A","B"), ("B","C")]`
Output	`True`
Why	`A beat B beat C — a consistent ranking A > B > C, no cycle.`

QUESTION

Example 2	
Input	`results = [("A","B"), ("B","C"), ("C","A")]`
Output	`False`
Why	`A > B > C > A is a contradiction (a cycle).`

Example 3	
Input	`results = []`
Output	`True`
Why	`No results impose no contradiction.`

CONSTRAINTS

- 0 <= len(results) <= 10^5 .
- Player names are non-empty strings; duplicate result pairs may appear.
- An empty result list is valid (True).

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# BASE QUESTION (optimal): Kahn's algorithm
# Idea: Peel off nodes with no incoming edge; everything
# peels off exactly when there is no cycle.
def results_are_valid(results):
    graph, indeg = {}, {}
    for w, l in set(results):
        graph.setdefault(w, []).append(l)
        graph.setdefault(l, [])
        indeg[l] = indeg.get(l, 0) + 1
        indeg.setdefault(w, 0)
    queue = deque(u for u in graph if indeg[u] == 0)
    removed = 0
    while queue:
        u = queue.popleft()
        removed += 1
        for v in graph[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                queue.append(v)
    return removed == len(graph)
```

216 Solve tree, graph, sliding-window problems — MEDIUM

Template fit: the course-schedule part is can-finish: len(order) == n; the zigzag and window parts are separate small loops

QUESTION

Question

LeetCode 103. Binary Tree Zigzag Level Order Traversal – given a binary tree root, return its zigzag level-order traversal. LeetCode 207. Course Schedule – given course prerequisites, determine if all courses can be finished by detecting cycles with topological sort. LeetCode 340. Longest Substring with At Most K Distinct Characters – given string s and integer k, return the length of the longest substring containing at most k distinct characters.
https://leetcode.com/problems/binary-tree-zigzag-level-order-traversal/description/
https://leetcode.com/problems/course-schedule/description/
https://leetcode.com/problems/longest-substring-with-at-most-k-distinct-characters/description/

QUESTION

Part 1: Binary Tree Zigzag Level Order TraversalYou are given a binary tree in level-order array form, where each element is either an integer or None for a missing child. Return the zigzag level-order traversal of the tree's node values. In a zigzag traversal, the first level is read from left to right, the second from right to left, the third from left to right, and so on.

Constraints

0 <= len(root) <= 2000Each non-None node value is an integer in the range [-10^4, 10^4]The input list represents a valid binary tree in level-order form

EXAMPLES

```
Input: ([3, 9, 20, None, None, 15, 7],)
Expected Output: [[3], [20, 9], [15, 7]]
```

Explanation: Level 1 is read left-to-right, level 2 right-to-left, and level 3 left-to-right.

```
Input: ([1],)
Expected Output: [[1]]
```

Explanation: A single-node tree has only one level.

Hints

A breadth-first search naturally processes the tree one level at a time.You can collect each level left-to-right, then reverse every other level before appending it to the answer.

Part 2: Course ScheduleThere are num_courses courses labeled from 0 to num_courses - 1. You are given a list of prerequisite pairs [a, b], meaning you must take course b before course a. Determine whether it is possible to finish all courses. This is equivalent to checking whether the prerequisite graph contains a cycle.

Constraints

1 <= num_courses <= 10^50 <= len(prerequisites) <= 2 * 10^5Each prerequisite pair has length 20 <= course, prerequisite < num_courses

Examples

Input: (2, [[1, 0]])

Expected Output: True

Explanation: Course 0 can be taken first, then course 1.

Input: (2, [[1, 0], [0, 1]])

Expected Output: False

Explanation: The two courses depend on each other, forming a cycle.

Hints

Think of courses as nodes in a directed graph and prerequisites as directed edges.If you repeatedly take courses with indegree 0 and cannot process all nodes, a cycle exists.

Part 3: Longest Substring with At Most K Distinct CharactersGiven a string s and an integer k, return the length of the longest substring that contains at most k distinct characters.

Constraints

0 <= len(s) <= 2 * 10^50 <= k <= len(s)s may contain any standard characters

QUESTION

Examples
Input: ("eceba", 2)
Expected Output: 3
Explanation: The longest valid substring is 'ece', which has length 3.
Input: ("aa", 1)
Expected Output: 2
Explanation: The whole string contains only one distinct character.
Hints
A sliding window can maintain a valid substring while expanding to the right.Keep character counts in the current window, and shrink from the left whenever the number of distinct characters exceeds k.

MAIN CODE | helper functions first, then the answer

```
from collections import defaultdict, deque


# HELPERS for the base question
def course_order(n, prereqs):
    # (a, b): to take a you must take b first. Kahn's
    # algorithm;
    # if some course never frees up there is a cycle
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
    queue = deque(c for c in range(n) if indeg[c] == 0)
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    return order if len(order) == n else []

# BASE QUESTION (optimal): Course schedule: Kahn's
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
# of the courses that depend on it, and queue those that
# reach zero.
def zigzag(root):
    # level order, alternating direction
    out, left_to_right = [], True
    queue = deque([root] if root else [])
    while queue:
        level = deque()
        for _ in range(len(queue)):
            node = queue.popleft()
            # append for left-to-right, appendleft
            # otherwise
            if left_to_right:
                level.append(node.val)
            else:
                level.appendleft(node.val)
            for child in (node.left, node.right):
                if child:
```

```
                    queue.append(child)
        out.append(list(level))
        left_to_right = not left_to_right
    return out

# BASE QUESTION (optimal): Course schedule: Kahn's
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
# of the courses that depend on it, and queue those that
# reach zero.
def can_finish(n, prereqs):
    return len(course_order(n, prereqs)) == n

# BASE QUESTION (optimal): Course schedule: Kahn's
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
# of the courses that depend on it, and queue those that
# reach zero.
def longest_at_most_k(s, k):
    # sliding window with a count of each letter inside it
    count, left, best = defaultdict(int), 0, 0
    for right, ch in enumerate(s):
        count[ch] += 1
        # too many distinct letters: shrink from the left
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        best = max(best, right - left + 1)
    return best
```

217 Validate course catalog dependencies

MEDIUM

Template fit: first reject prerequisites naming courses that do not exist; then the same peel; report the missing names or the cycle

QUESTION

Design a function to validate an e-learning course catalog. You are given: (1) a set of course IDs, and (2) a list of prerequisite pairs (u, v) meaning course v requires course u. Determine if the catalog is valid: no prerequisite references a non-existent course, and there are no cycles in the dependency graph. If valid, return true and a possible ordering of courses; otherwise return false and report the set of missing course references and at least one cycle you detect. Specify your algorithm, time and space complexity.

MAIN CODE | helper functions first, then the answer

```
from collections import defaultdict, deque


# HELPERS for the base question
def find_cycle(known, prereqs, indeg):
    # every leftover course still has a leftover
    # prerequisite,
    # so walking backwards must eventually repeat a course
    left = {c for c in known if indeg[c] > 0}
    before = defaultdict(list)
    for u, v in prereqs:
        if u in left and v in left:
            before[v].append(u)
    x = min(left, key=str)
    walk, pos = [], {}
    while x not in pos:
        pos[x] = len(walk)
```

```
        walk.append(x)
        x = before[x][0]
    cycle = walk[pos[x]:] + [x]
    # walked against the edges: reverse to list
    # prerequisites first
    return cycle[::-1]

# BASE QUESTION (optimal): Missing references, then Kahn's
# algorithm, then a cycle from the leftovers
# Idea: Report unknown courses (any referenced id not in
# the set). If none, run Kahn's algorithm: repeatedly
# remove courses with no remaining prerequisites, building
# an order.
def validate_catalog(courses, prereqs):
    # prereqs (u, v): course v requires course u.
    # Returns (ok, missing ids, order or one cycle)
    known = set(courses)
    missing = sorted({x for pair in prereqs for x in pair
                      if x not in known}, key=str)
    if missing:
        return False, missing, None
    adj, indeg = defaultdict(list), {c: 0 for c in known}
    for u, v in prereqs:
        adj[u].append(v)
        indeg[v] += 1
    ready = [c for c in known if indeg[c] == 0]
    queue = deque(sorted(ready, key=str))
    order = []
```

```
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    if len(order) == len(known):
        return True, [], order
    return False, [], find_cycle(known, prereqs, indeg)
```

218 Course Schedule — MEDIUM

Template fit: edge = prerequisite -> course; return the order, or an empty list when there is a cycle

QUESTION

You are given numCourses and a list of prerequisites, where each element in prerequisites[i] = [a_i, b_i] means you must complete course b_i before taking course a_i. Write a function canFinish that returns True if it's possible to finish all courses, and False if it's not. You need to complete all the courses for the result to be True. Assume there are no duplicate prerequisites, and numCourses is a non-negative number. Examples Input: numCourses = 3, prerequisites = [[2, 1], [1, 0]]
Output: True Explanation: You can complete course 0 first, then course 1, and finally course 2.
Input: numCourses = 4, prerequisites = [[3, 2], [2, 1], [1, 0], [0, 3]]
Output: False Explanation: The prerequisites form a cycle, making it impossible to finish all courses.
Input: numCourses = 5, prerequisites = [[4, 2], [3, 1], [2, 0]]
Output: True Explanation: You can complete the courses in order from course 0 to course 4.Hint 1Try modeling the courses as a graph, with edges representing prerequisites. Consider how you can detect cycles to determine if all courses can be completed.

MAIN CODE | helper functions first, then the answer

```python
from collections import defaultdict, deque


# HELPERS for the base question
def build(n, prereqs):
    # (a, b): course b must be taken before course a
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
    return adj, indeg

def find_order(n, prereqs):
    # Kahn's algorithm: take courses with no unmet
    # prerequisite
    adj, indeg = build(n, prereqs)
    queue = deque(c for c in range(n) if indeg[c] == 0)
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    # fewer than n taken: a cycle blocks the rest
    return order if len(order) == n else []

# BASE QUESTION (optimal): Kahn's algorithm (topological
# sort)
# Idea: Count each course's unmet prerequisites and queue
# those with none. Take a course, reduce the counts of the
# courses that need it, and queue any that reach zero.
def can_finish(n, prereqs):
    return len(find_order(n, prereqs)) == n
```

GROUP 30 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 4 M 1 H

Counting pairs by looking up each partner in a Counter or a bucket

219 Progressive Array Partitioning — MEDIUM

Template fit: Counter of the values; stage 1 asks whether all the counts are different: len(set(counts.values())) == len(counts)

QUESTION

EXAMPLES

Stage 1

```
[1, 2, 3, 4, 2, 3, 3, 4, 4, 4] -> true because the four value-groups have different sizes.
[1, 2, 3] -> false because every value-group has size one.
```

Stage 2

```
[1, 2, 2, 3, 3, 4, 4, 5, 5, 6] -> true.
[4, 5, 6, 7, 7, 8, 9] -> false.
```

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Counter, then compare the set
# of counts with the number of values
# Idea: All counts are different exactly when the set of
# counts has as many elements as there are distinct
# values. One pass to count, one pass over the counts.
def stage1(counts):
    # true when every value occurs a different number of
    # times: the counts have no repeats
    return len(set(counts.values())) == len(counts)
```

220 K-diff Pairs in an Array — MEDIUM

Template fit: the template as written: partner = v + k; for k == 0 count values that appear twice

QUESTION

Given an integer array nums and an integer k , return the number of unique k-diff pairs in the array. A k-diff pair is a pair of integers (nums[i], nums[j]) where: i != j , and abs(nums[i] - nums[j]) == k . Two k-diff pairs (a, b) and (c, d) are considered the same (counted once) if they consist of the same pair of values. Count each distinct value pair only once.

Function signature

```
def count_k_diff_pairs(nums: list[int], k: int) -> int:
    """Return the number of unique value-pairs with absolute difference k."""
    ...
```

EXAMPLES

Example 1	
Input	`nums = [3, 1, 4, 1, 5], k = 2`
Output	`2`
Why	`The unique 2-diff value pairs are (1, 3) and (3, 5).`

Example 2	
Input	`nums = [1, 2, 3, 4, 5], k = 1`
Output	`4`
Why	`(1,2), (2,3), (3,4), (4,5).`

QUESTION

Example 3	
Input	`nums = [1, 3, 1, 5, 4], k = 0`
Output	`1`
Why	`With k = 0 count values that appear at least twice; only 1 does.`

CONSTRAINTS

- 1 <= len(nums) <= 10^4
- -10^7 <= nums[i] <= 10^7
- 0 <= k <= 10^7

MAIN CODE | helper functions first, then the answer

```
from collections import Counter


# BASE QUESTION (optimal): Count the values
# Idea: A value v starts a pair when v + k is present; for
# k = 0 it must appear twice.
def count_k_diff_pairs(nums, k):
    if k < 0:
        return 0
    counts = Counter(nums)
    if k == 0:
        return sum(1 for c in counts.values() if c >= 2)
    return sum(1 for v in counts if v + k in counts)
```

221 Count Axis-Aligned Squares Among Grid Points — MEDIUM

Template fit: a set of points; two opposite corners fix the other two, so look both up; every square is found twice, divide by 2

QUESTION

The interviewer described a diagram of squares and rectangles and asked which data structure to use and how to count the squares. Framed concretely: you are given a list of distinct integer lattice points in the plane. Count the number of axis-aligned squares whose four corners all appear in points . A square is axis-aligned when its sides are parallel to the x- and y-axes. Return the count of such squares.

Function signature

from typing import List, Tuple
def count_axis_aligned_squares(points: List[Tuple[int, int]]) -> int:
    """Return the number of axis-aligned squares whose 4 corners are all in points."""
    ...

Follow-up (as reported): A cleaner O(n^2) approach: put all points in a set, then for every pair that could be two opposite corners of a square (equal absolute x- and y-difference), check the other two corners; each square is found via both its diagonals, so divide the total by 2.

EXAMPLES

Example 1	
Input	`points = [(0, 0), (0, 1), (1, 0), (1, 1)]`
Output	`1`
Why	`The four points form exactly one unit square.`

QUESTION

Example 2	
Input	`points = [(0,0),(0,1),(1,0),(1,1),(0,2),(1,2),(2,0),(2,1),(2,2)]`
Output	`5`
Why	`A 3x3 grid of points has four 1x1 squares and one 2x2 square.`

Example 3	
Input	`points = [(0, 0), (2, 0), (0, 1), (2, 1)]`
Output	`0`
Why	`These form a 2x1 rectangle, which is not a square.`

CONSTRAINTS

- 0 <= len(points) <= 2000 ; all points are distinct.
- Each point is (x, y) with -10^9 <= x, y <= 10^9 .
- A square must have positive side length (four distinct corners); degenerate zero-size "squares" do not count.
- Only axis-aligned squares count (tilted squares are ignored).

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Opposite corners and a set
# Idea: Two opposite corners decide the other two, so one
# set lookup per pair is enough.
def count_axis_aligned_squares(points):
    seen = set(points)
    total = 0
    for i, (x1, y1) in enumerate(points):
        for x2, y2 in points[i + 1 :]:
            if abs(x1 - x2) == abs(y1 - y2) != 0:
                if (x1, y2) in seen and (
                    x2,
                    y1,
                ) in seen:
                    total += 1
    return total // 2
```

222 Sum of Two Squares in Two Ways — MEDIUM

*Template fit: enumerate every pair (a, b) with a <= b once, add a*a + b*b to a Counter, answer = sums counted exactly twice*

QUESTION

Given an integer n, find all integers s < n such that s can be expressed as the sum of two squares in exactly two distinct unordered ways. For example: 50 = 1^2 + 7^2 = 5^2 + 5^2.

EXAMPLES

Example 1	
Input	`n = 100`
Output	`[50, 65, 85]`
Why	`50 = 1^2 + 7^2 = 5^2 + 5^2, 65 = 1^2 + 8^2 = 4^2 + 7^2, 85 = 2^2 + 9^2 = 6^2 + 7^2`

Example 2	
Input	`n = 50`
Output	`[]`
Why	`No number less than 50 has exactly two representations.`

QUESTION

CONSTRAINTS

- 1 <= n <= 2^31 - 1

MAIN CODE | helper functions first, then the answer

```
from collections import Counter


# BASE QUESTION (optimal): Count every pair once in a hash
# map
# Idea: Loop a from 1 while 2a^2 < n and b from a while
# a^2 + b^2 < n, adding each sum to a Counter. There are
# about pi * n / 8 pairs. The answer is the sorted list of
# sums whose count is exactly 2.
def sums_in_k_ways(n, k=2, zero=False):
    # every s < n is counted once per pair a <= b with
    # a*a + b*b = s. Keep the sums seen exactly k times.
    ways = Counter()
    a = 0 if zero else 1
    while 2 * a * a < n:
        b = a
        while a * a + b * b < n:
            ways[a * a + b * b] += 1
            b += 1
        a += 1
    return sorted(s for s, c in ways.items()
                  if c == k and s > 0)
```

223 Maximum-Area Rectangle from Planar Points

HARD

Template fit: bucket every pair of points by (sum of coordinates, squared length); two pairs in one bucket are the diagonals of a rectangle

QUESTION

Given 2D points, find the maximum area of a rectangle whose four corners are input points; sides need not be parallel to the axes. Candidates compare it to LC 963 (Minimum Area Rectangle II) but maximizing. Interviewers ask to improve beyond an O(N^3) approach.

Input: a list of 2D coordinate points.
Return the maximum area of any rectangle whose four vertices are all input points.
Rectangle sides do not need to be parallel to the coordinate axes.
Follow-up pressure: start from an O(N^3) idea, then optimize toward a better complexity.

MAIN CODE | helper functions first, then the answer

```
from collections import defaultdict
from itertools import combinations


# HELPERS used by the code below
def cross(o, p, q):
    # twice the signed area of triangle o, p, q
    ax, ay = p[0] - o[0], p[1] - o[1]
    bx, by = q[0] - o[0], q[1] - o[1]
    return ax * by - ay * bx

# HELPERS for the base question
def all_rectangle_areas(points):
    # duplicates cannot help, so drop them
    pts = list(set(map(tuple, points)))
    buckets = defaultdict(list)
    # every pair is a possible diagonal
```

```
    for p, q in combinations(pts, 2):
        # same centre (sum of corners) and same length
        key = (p[0] + q[0], p[1] + q[1],
               (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)
        buckets[key].append((p, q))
    # two diagonals in one bucket form a rectangle
    for pairs in buckets.values():
        for (p, q), (r, s) in combinations(pairs, 2):
            # area = |cross| of two sides leaving p
            yield abs(cross(p, r, s))

# BASE QUESTION (optimal): Bucket pairs by diagonal
# midpoint and length
# Idea: For every pair (p, q) use the key (p.x + q.x, p.y
# + q.y, squared length). All pairs with the same key are
# diagonals of one circle around the same centre, so any
# two of them form a rectangle.
def max_rect_area(points):
    # best area, or 0 when no rectangle exists
    return max(all_rectangle_areas(points), default=0)
```

GROUP 31 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 3 M 2 H

Tree DP: build a parent order once, then add up from the leaves

224 Compute File System Total Size — MEDIUM

Template fit: add each file's size while walking down from the root with a stack; a shared directory is counted once (seen set)

QUESTION

EXAMPLES

```
root = Dir(
  Dir(File(10), File(20)),
  File(5),
  Dir(File(30)),
)
Total: 65
```

MAIN CODE | helper functions first, then the answer

```python
# HELPERS used by the code below
class File:
    def __init__(self, size):
        self.size, self.parent = size, None

# BASE QUESTION (optimal): Iterative traversal with a
# visited set
# Idea: An explicit stack removes the recursion limit, and
# a set of visited node ids makes each node count once, so
# shared directories are not double counted and cycles
# terminate.
def total_size(root, count_once=True):
    # iterative walk; count_once handles hard links
    total, seen, stack = 0, set(), [root]
    while stack:
        node = stack.pop()
        if count_once:
            if id(node) in seen:
                continue
            seen.add(id(node))
        if isinstance(node, File):
            total += node.size
        else:
            stack.extend(node.children)
    return total
```

225 Minimum-Cost Tree Disconnection — MEDIUM

Template fit: backwards sweep with two numbers per node: the best cost when the node is cut off, and the cost if it is free

QUESTION

EXAMPLES

Negative-weight follow-up

```
A --(-1)--> B --(-1)--> C
Minimum total cut cost: -2
```

Cutting both edges costs -2, even though cutting just the first edge already separates the root from the original leaf.

MAIN CODE | helper functions first, then the answer

```python
from collections import defaultdict
```

```python
# HELPERS for the base question
INF = float("inf")

def tree_costs(n, edges):
    # edges are (parent, child, cost); node 0 is the root
    kids = defaultdict(list)
    for u, v, w in edges:
        kids[u].append((v, w))
    order = [0]
    for u in order:
        for v, _ in kids[u]:
            order.append(v)
    # best[v]: cheapest way to cut v off from its leaves
    # free[v]: sum of every negative cost below v
    best, free = [0] * n, [0] * n
    for v in reversed(order):
        for c, w in kids[v]:
            # 1) cut this edge and the negatives below
            cut = w + free[c]
            # 2) keep it and cut deeper (a leaf cannot)
            keep = best[c] if kids[c] else INF
            best[v] += min(cut, keep)
            free[v] += min(0, w) + free[c]
    return kids, best, free

# BASE QUESTION (optimal): Tree DP: best(v) = cheapest
# cuts inside v's subtree that disconnect v from its
# leaves
# Idea: For a child c with edge weight w there are two
# options: cut the edge, paying w plus whatever extra
# negative-weight edges you can still cut below c (free(c)
# = sum of min(0, weight) over its subtree), or keep the
# edge and require best(c), which is impossible when c is
# a leaf.
def disconnect_cost(n, edges):
    # works for negative costs and needs no recursion
    return tree_costs(n, edges)[1][0]
```

226 Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.

MEDIUM

Template fit: t[v] = max over children (rank among children sorted by time, longest first, + t[child]); read t at the root

QUESTION

Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.

MAIN CODE | helper functions first, then the answer

```python
# HELPERS used by the code below
def order_children(tree, node):
    # children of `node` as (child, cost) pairs
    return tree.get(node, [])

# HELPERS for the base question
def finish_times(tree, root):
    # time[v] = rounds until v and everything below has
    # the
```

```
    # news, once v has it. v calls its children one after
    # another; the call to the i-th child costs the edges
    # so
    # far, and that child then needs time[child] more.
    # Best order (Jackson's rule): the child with the
    # LARGEST time[child] first.
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        kids = order_children(tree, node)
        stack.extend(c for c, _ in kids)
    time, plan = {}, {}
    # process children before parents
    for node in reversed(order):
        kids = sorted(order_children(tree, node),
                      key=lambda kc: -time[kc[0]])
        spent, worst = 0, 0
        for child, cost in kids:
            spent += cost
            worst = max(worst, spent + time[child])
        time[node] = worst
        plan[node] = [c for c, _ in kids]
    return time, plan

# BASE QUESTION (optimal): Post-order DP: serve the
# slowest subtree first
# Idea: Let T(v) be the rounds needed to inform the
# subtree of v once v knows. v informs its children one
# per round.
def min_rounds(tree, root):
    return finish_times(tree, root)[0][root]
```

227 Compute distance-sum from every tree node

HARD

Template fit: down[v] = sum of distances to v's subtree; then a forward sweep: ans[child] = ans[parent] - size[child] + (n - size[child])

QUESTION

Overview: This question evaluates understanding of tree and graph algorithms, including computation of pairwise distances and techniques for aggregating path lengths across nodes.

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
def build(n, edges):
    adj = [[] for _ in range(n)]
    for e in edges:
        u, v = e[0], e[1]
        w = e[2] if len(e) > 2 else 1
        adj[u].append((v, w))
        adj[v].append((u, w))
    return adj

def distances_from(adj, s):
    # distance from s to every node (a tree: one pass)
    dist, stack = {s: 0}, [s]
    while stack:
        u = stack.pop()
        for v, w in adj[u]:
            if v not in dist:
                dist[v] = dist[u] + w
                stack.append(v)
    return dist
```

```python
# BASE QUESTION (optimal): Re-rooting with subtree sizes
# Idea: Root the tree at 0 and compute size[v] (subtree
# sizes) and ans[0] = the sum of depths.
def distance_sums(n, edges):
    # sum of distances from every node; edges may be
    # weighted
    adj = build(n, edges)
    parent, order, up = [-1] * n, [0], [0] * n
    for u in order:
        for v, w in adj[u]:
            if v != parent[u]:
                parent[v], up[v] = u, w
                order.append(v)
    size = [1] * n
    for u in reversed(order):
        if parent[u] >= 0:
            size[parent[u]] += size[u]
    # the root's sum is a plain sum of distances
    ans = [0] * n
    ans[0] = sum(distances_from(adj, 0).values())
    # root moves across an edge of weight w: size[u] nodes
    # get
    # w closer, the other n - size[u] get w farther
    for u in order[1:]:
```

```python
        shift = up[u] * (n - 2 * size[u])
        ans[u] = ans[parent[u]] + shift
    return ans
```

228 Tree Distance Sum (Re-root DP) HARD

Template fit: exactly the same two sweeps (this is the re-rooting version of the question)

QUESTION

Onsite coding round 1: given an undirected tree on N nodes, return for every node the sum of distances to all other nodes. Equivalent to LC 834. Interviewer expects baseline O(n²) DFS followed by the O(n) re-root DP.

Input: undirected tree with N nodes and N-1 edges.
Output: array ans where ans[u] is the sum of shortest-path distances from u to every other node.
Walk through baseline (BFS/DFS from every node, O(n²)) before optimizing.
Final solution should be O(n) via two-pass re-root DP.

EXAMPLES

```
Edges: [[0,1],[0,2],[2,3],[2,4],[2,5]]
ans:   [8, 12, 6, 10, 10, 10]
```

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
def build(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    return adj

# BASE QUESTION (optimal): Two passes: subtree sizes, then
# re-root
# Idea: Root the tree at 0 and get a BFS order. In reverse
# order compute size[u] and the sum of distances inside
# u's subtree. For the root that sum is already the
# answer.
```

```python
def tree_distance_sums(n, edges):
    if n == 0:
        return []
    adj = build(n, edges)
    # order: parents before children (no recursion)
    parent, order = [-1] * n, [0]
    for u in order:
        for v in adj[u]:
            if v != parent[u]:
                parent[v] = u
                order.append(v)
    # size[u]: nodes in u's subtree
    # inside[u]: distances from u to its own subtree
    size, inside = [1] * n, [0] * n
    for u in reversed(order):
        p = parent[u]
        if p >= 0:
            size[p] += size[u]
            inside[p] += inside[u] + size[u]
    # the root's answer is already complete
    ans = [0] * n
    ans[0] = inside[0]
    # moving the root from p to child u: the size[u] nodes
    # below get 1 closer, the other n - size[u] get 1
    # farther
    for u in order[1:]:
        ans[u] = ans[parent[u]] + n - 2 * size[u]
    return ans
```

Counting or reaching an amount: the knapsack loop

229 Maximize Shelf Space (Unbounded Knapsack) MEDIUM

Template fit: reachable[t] = any(reachable[t - size]); the answer is the largest reachable total up to the capacity

QUESTION

A shelf has a fixed integer capacity capacity . You have boxes of various integer sizes given in sizes , and you may use any number of each box, including duplicates (each size is available infinitely). Choose boxes so their total size does not exceed capacity , maximizing the total size used. Return that maximum total.

Function signature

def max_shelf_space(capacity: int, sizes: list[int]) -> int:

...

EXAMPLES

Example 1	
Input	capacity = 10, sizes = [3, 5, 7]
Output	10
Why	3 + 7 = 10 exactly fills the shelf.

Example 2	
Input	capacity = 11, sizes = [5, 3]
Output	11
Why	5 + 3 + 3 = 11.

Example 3	
Input	capacity = 7, sizes = [4, 5]
Output	5
Why	Options within 7 are 4 or 5 (two boxes would be >= 8). The best is 5.

CONSTRAINTS

- 0 <= capacity <= 10**4 .
- 1 <= len(sizes) <= 100 , each size is a positive integer.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Reachable totals DP
# Idea: A total is reachable if removing one box leaves a
# reachable total.
def max_shelf_space(capacity, sizes):
    reachable = [True] + [False] * capacity
    for c in range(1, capacity + 1):
        reachable[c] = any(
            s <= c and reachable[c - s] for s in sizes
        )
    return max(
        c for c in range(capacity + 1) if reachable[c]
    )
```

230 Coin Change

MEDIUM

Template fit: fewest coins: start with infinity, best[t] = min(best[t], best[t - c] + 1); -1 when it stays infinity

QUESTION

You are given an integer array coins representing different coin denominations and an integer amount representing a total amount of money. Write a function coinChange that returns the fewest number of coins needed to make up that amount. If that amount cannot be made up by any combination of the coins, return -1. You may assume that you have an infinite number of each kind of coin. Examples PythonInput: coins = [1, 2, 5], amount = 11 Output: 3 Explanation: The minimum number of coins needed is 3 (11 = 5 + 5 + 1).
Input: coins = [4, 5], amount = 7 Output: -1 Explanation: It is not possible to make up the amount 7 with coins of denominations 4 and 5.Solution 1: Brute-Force approach The naive approach to solving the coin change problem involves a recursive strategy to explore all possible combinations of coins to find the minimum number needed to make up the target amount. It starts by recursively trying each coin denomination to reduce the target amount. For each coin, the algorithm subtracts the coin's value from the current amount and solves the reduced problem. If the amount becomes zero, it indicates that no more coins are needed, and the function returns zero. If the amount becomes negative, it signals that this path is not feasible, so the function returns infinity or a large number. The algorithm combines the results from each recursive call with the current coin to determine the minimum number of coins required. Finally, after exploring all possibilities, it returns the

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
INF = float("inf")

# BASE QUESTION (optimal): Bottom-up DP over amounts
# Idea: best[0] = 0, and for a from 1 to amount, best[a] =
# 1 + min(best[a - c]) over coins c <= a (infinity if none
# works). The answer is best[amount] or -1. Each amount is
# solved once.
def coin_change(coins, amount):
    # best[a] = fewest coins that make exactly a
    best = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and best[a - c] + 1 < best[a]:
                best[a] = best[a - c] + 1
    return -1 if best[amount] == INF else best[amount]
```

231 Partition an array into two sub-arrays with equal sum.

MEDIUM

Template fit: subset sum to total / 2, written with a bitset: reach |= reach << x

QUESTION

Given an array of integers, determine if it can be partitioned into two subsets such that the sum of the elements in each subset is equal. A partition is considered valid if it divides the array into two subsets where both subsets have the same sum. Examples Input: [2, 4, 11, 5]
Output: True Explanation: The array can be partitioned into two subsets [2, 4, 5] and [11] with equal sum.
Input: [2, 7, 3, 1]
Output: False Explanation: The array cannot be partitioned into two subsets with equal sums.

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Subset-sum as a bitset
# Idea: reach is an integer whose bit s is set when some
# subset sums to s; start with bit 0. For each number x do
# reach |= reach << x. The answer is whether bit total / 2
# is set.
```

```python
def can_partition(nums):
    # reachable subset sums as the bits of one integer
    total = sum(nums)
    if total % 2:
        return False
    reach = 1
    for x in nums:
        # every old sum s also gives s + x
        reach |= reach << x
    return bool(reach >> (total // 2) & 1)
```

232 Explain how to find a target sum in an array. MEDIUM

Template fit: turn +/- signs into a subset sum of (total + S) / 2, items used once (sweep downwards)

QUESTION

Determine the Number of Ways to Assign Symbols to Reach a Target Sum. You are given a list of non-negative integers, a1, a2, ..., an, and a target sum S. Your task is to assign each integer in the list a symbol + or - and calculate how many different ways there are to arrange these symbols such that the sum of the integers equals S after applying the assigned symbols. For example, if you have the list [1, 1, 1, 1, 1] and the target sum 3, one way to reach the target sum is to assign + to three 1s and - to two 1s, like so: +1 +1 +1 +1 -1. Another way could be +1 -1 +1 +1 +1. You need to return the total number of unique arrangements that meet the target sum S. Examples nums = [1, 1, 1, 1, 1] targetSum = 3 output: 5 nums = [1, 2, 1] targetSum = 2 output: 2 nums = [1, 2, 7, 1, 5, 3] targetSum = 8 output: -1 (Explanation: Since the total sum of the numbers plus the target is odd, it's not possible to reach the target sum.)The original problem asks for the number of ways to assign symbols (+ or -) to a list of numbers such that their sum equals a target sum S. This problem can be converted into a problem of finding subsets of the given numbers whose sum equals (totalSum + S) / 2. This is because if we can find subsets whose sum equals (totalSum + S) / 2, the remaining numbers will naturally form the other subset that sums to (totalSum - S) / 2, which effectively gives us the desired configuration of symbols. NoteWhy (totalSum + S) / 2?Given a list of numbers nums=[a1,a2,...,an]\te

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Subset-sum counting: choose P
# with sum (total + S) / 2
# Idea: Let need = (total + S) / 2 (return 0 if it is not
# a non-negative integer). ways[s] counts the subsets with
# sum s; for each number x update ways from high to low:
# ways[s] += ways[s - x]. The answer is ways[need].
def count_signs(nums, target):
    # +/- signs reaching `target` equal a subset with sum
    # (total + target) / 2 that gets the plus signs. So
    # the
    # question becomes: how many subsets have that sum?
    total = sum(nums)
    if abs(target) > total or (total + target) % 2:
        return -1
    goal = (total + target) // 2
    ways = [1] + [0] * goal
    for x in nums:
        # go downward so every number is used at most once
        for s in range(goal, x - 1, -1):
            ways[s] += ways[s - x]
    return ways[goal]
```

233 Reverse Coin Change

MEDIUM

Template fit: given the ways table, find the coin values: the first amount whose count is wrong must be a coin; add it and rebuild

QUESTION

In standard coin change, given a coins array, we build dp where dp[i] = number of ways to make amount i. Reverse problem: given a dp array, find the original coins array that produced it.

EXAMPLES

Example 1	
Input	`dp = [1, 1, 2, 2, 3]`
Output	`[1, 2]`
Why	`Coins [1, 2] produce this dp array.`

Example 2	
Input	`dp = [1, 1, 1, 2, 2, 2, 3]`
Output	`[1, 3]`

CONSTRAINTS

- 1 <= dp.length <= 1000
- dp[0] = 1

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Scan upward and add a coin
# whenever the table falls short
# Idea: Keep the ways table for the coins found so far.
def coins_from_dp(dp):
    # backward: rebuild the table with the coins found so
    # far. The first amount where the real dp is exactly
    # one
    # bigger must be a new coin (it adds the single way
    # "just that coin"). Anything else is impossible.
    n = len(dp) - 1
    if not dp or dp[0] != 1:
        return None
    cur, coins = [1] + [0] * n, []
    for a in range(1, n + 1):
        if dp[a] == cur[a]:
            continue
        if dp[a] != cur[a] + 1:
            return None
        coins.append(a)
        for x in range(a, n + 1):
            cur[x] += cur[x - a]
    # the table rebuilt from the coins must equal the
    # input
    return coins if cur == list(dp) else None
```

GROUP 33 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 4 M 1 H

Prefix sums, Tier 2: running total plus a dict of earlier totals

234 Maximum-Sum Subarray With Equal Endpoints — MEDIUM

Template fit: per value keep the earliest start with the smallest total before it; candidate = total now - that earlier total

QUESTION

Given an integer array nums , choose a (contiguous) subarray whose first and last elements are equal and whose sum is as large as possible. A single element is a valid subarray (its first and last elements coincide). Return the chosen subarray itself (as a list). If several subarrays tie for the maximum sum, return the one with the smallest starting index; if they still tie, return the shortest.

Function signature

def max_sum_equal_endpoints(nums: list[int]) -> list[int]:

    ...

Follow-up (as reported): Achieve O(n): for each value, you only ever care about pairing the earliest occurrence (with the best prefix sum) against later occurrences.

EXAMPLES

Example 1	
Input	`nums = [1, 3, -2, 4, 1]`
Output	`[1, 3, -2, 4, 1]`
Why	`The value 1 appears at indices 0 and 4; that whole window sums to 7, beating any single element.`

Example 2	
Input	`nums = [5, -10, 5]`
Output	`[5]`
Why	`The window from the first 5 to the last 5 sums to 0, which is worse than taking a single 5 (sum 5).`

Example 3	
Input	`nums = [-3, -1, -2]`
Output	`[-1]`
Why	`All values are distinct, so only single elements qualify; -1 is the largest.`

CONSTRAINTS

- 1 <= len(nums) <= 10^5 .
- -10^4 <= nums[i] <= 10^4 .

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Best earlier prefix per value
# Idea: For each end the best start is the same value with
# the smallest prefix sum so far.
def max_sum_equal_endpoints(nums):
    prefix = [0]
    for x in nums:
        prefix.append(prefix[-1] + x)
    best_start, best = {}, None
    for j, x in enumerate(nums):
        if (
            x not in best_start
```

```
            or prefix[j] < prefix[best_start[x]]
        ):
            best_start[x] = j
        i = best_start[x]
        key = (-(prefix[j + 1] - prefix[i]), i, j - i)
        if best is None or key < best:
            best = key
    return (
        nums[best[1] : best[1] + best[2] + 1]
        if best
        else []
    )
```

235 Maximum Subarray Sum With a Minimum Length Constraint

MEDIUM

Template fit: the smallest earlier total is only allowed from min_len positions back, so update it one step late

QUESTION

This is the classic "maximum subarray" problem with the extra constraint the report mentioned ("Max sub array question with a weird constraint"). Given an integer array nums and an integer min_len , find the contiguous subarray (containing at least min_len elements) that has the largest sum, and return that sum.

Function signature

def max_subarray_min_len(nums: list[int], min_len: int) -> int:

"""Return the maximum sum over all contiguous subarrays of length >= min_len."""

...

Follow-up (as reported): The report said each question had a follow-up where a constraint was changed. Now bound the subarray length to the range [min_len, max_len] . Hint: maintain a sliding window of prefix-sum minima with a monotonic deque.

EXAMPLES

Example 1	
Input	`nums = [1, -2, 3, 4, -1], min_len = 2`
Output	`7`
Why	`The subarray [3, 4] (length 2) has sum 7, the largest among all subarrays of length >= 2.`

Example 2	
Input	`nums = [-1, -2, -3], min_len = 2`
Output	`-3`
Why	`Every element is negative. The best length->=2 subarray is [-1, -2] with sum -3.`

Example 3	
Input	`nums = [5, 4, -1, 7, 8], min_len = 1`
Output	`23`
Why	`With min_len = 1 this reduces to ordinary Kadane; the whole array sums to 23.`

CONSTRAINTS

- 1 <= min_len <= len(nums) <= 10^5
- -10^4 <= nums[i] <= 10^4
- The array is guaranteed to be non-empty and min_len never exceeds len(nums) , so a valid subarray always exists.
- Values may be negative; the answer can therefore be negative.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Prefix sums and a running
# minimum
# Idea: Sweep the end of the subarray; the best start is
# the smallest prefix sum that leaves at least min_len
# elements.
def max_subarray_min_len(nums, min_len):
    prefix = [0]
    for x in nums:
        prefix.append(prefix[-1] + x)
    best, smallest = None, float("inf")
    for end in range(min_len, len(nums) + 1):
        # the prefix that is allowed to start a valid
        # subarray
        smallest = min(smallest, prefix[end - min_len])
        cand = prefix[end] - smallest
        best = cand if best is None else max(best, cand)
    return best
```

236 Solve array-sum and city-community problems

MEDIUM

Template fit: count subarrays summing to k with the template as written; the second task is union-find over building cells

QUESTION

Overview: This question evaluates array-processing and grid/graph connectivity skills, covering counting subarrays with a target sum and identifying connected building communities in a grid; it belongs to the Coding & Algorithms domain and involves streaming/online algorithm reasoning as well as grid traversal concepts.

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))

# BASE QUESTION (optimal): prefix sums in a dictionary
# Idea: Walk the array keeping the prefix sum.
def count_subarrays(nums, k):
    # prefix sums: a subarray ending here has sum k if
    # some
    # earlier prefix equals (prefix - k)
    seen, prefix, count = {0: 1}, 0, 0
    for x in nums:
        prefix += x
        count += seen.get(prefix - k, 0)
        seen[prefix] = seen.get(prefix, 0) + 1
    return count

# BASE QUESTION (optimal): prefix sums in a dictionary
# Idea: Walk the array keeping the prefix sum.
def communities(grid):
    # connected groups of buildings (1) in a grid
    seen, count = set(), 0
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] != 1 or (r, c) in seen:
                continue
            count += 1
            seen.add((r, c))
            stack = [(r, c)]
            while stack:
                x, y = stack.pop()
                for dx, dy in STEPS:
                    a, b = x + dx, y + dy
                    if not (0 <= a < len(grid)):
                        continue
```

```
                if not (0 <= b < len(grid[0])):
                    continue
                land = grid[a][b] == 1
                if land and (a, b) not in seen:
                    seen.add((a, b))
                    stack.append((a, b))
    return count
```

237 Find the maximum subarray sum. MEDIUM

Template fit: the dict is just one number: the smallest earlier total; the best subarray ends here with total - smallest (Kadane is the same idea)

QUESTION

Given an array of integers nums, write a function maxSubarraySum to find the maximum sum of a contiguous subarray within the array and return that maximum sum. The subarray must be contiguous, meaning that the elements must appear consecutively in the original array. Examples Input: nums = [2, 3, -2, 4]
Output: 7 Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7.
Input: nums = [1, -1, -5, -4]
Output: 1 Explanation: The maximum sum is 1, which is the single element with the highest value.Challenge 1Can you come up with a solution with a O(n) time complexity?Challenge 2Can you come up with a solution with a O(1) space complexity?

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Kadane's algorithm
# Idea: cur is the best sum of a subarray ending at the
# current element: either the element alone or the element
# added to the previous cur. best is the maximum cur seen.
def max_subarray(nums):
    # Kadane: the best sum of a run ending here either
    # extends
    # the previous run or starts again at this element
    cur = best = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
```

238 Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k. HARD

Template fit: key = total % k; store the first index; a hit at least 2 positions back means a subarray whose sum is a multiple of k

QUESTION

Determine if a given array contains a subarray of at least two elements whose sum is a multiple of a specified number k. An array is considered to have a "good subarray" if there exists at least one subarray (consisting of two or more elements) such that the sum of the elements in this subarray is a multiple of k. For example, the array [23, 2, 4, 7] with k = 6 has a "good subarray" ([2, 4]), as the sum 6 is a multiple of k = 6. The array [5, 0, 0, 0] with k = 3 also has a "good subarray" ([0, 0]), since the sum is 0, and 0 is a multiple of 3. Examples nums = [23, 2, 4, 7], k = 6 output: true nums = [5, 0, 0, 0], k = 3 output: true nums = null, k = 1 output: false In these examples, the output is true if there exists a "good subarray" in the given array nums for the specified k, and false otherwise.Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(n)?

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Prefix sums mod k with the
# first index of each remainder
# Idea: Walk the array keeping the running sum modulo k.
# If this remainder was seen before at index j and the
# current index i satisfies i - j >= 2, the elements
# between form a good subarray.
def has_good_subarray(nums, k):
    # prefix sums modulo k: two equal remainders at
    # indexes
    # at least 2 apart mean a block of 2 or more items
    # whose
    # sum is a multiple of k. Keep only the FIRST index of
    # each remainder: a later one only gives a shorter
    # block.
    first = {0: -1}
    total = 0
    for i, x in enumerate(nums):
        total += x
        # k == 0 means "equal to zero": compare the sums
        key = total % k if k else total
        if key not in first:
            first[key] = i
        elif i - first[key] >= 2:
            return True
    return False
```

Group by a canonical key

239 Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard. EASY

Template fit: key = the keyboard row of each letter; a word qualifies when its letters map to one row

QUESTION

Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
QWERTY = ["qwertyuiop", "asdfghjkl", "zxcvbnm"]

# LAYOUT table (change it for other keyboards)
def row_map(rows):
    # letter -> index of its row; built once and reused
    return {ch: i for i, row in enumerate(rows)
            for ch in row}

# BASE QUESTION (optimal): Map each letter to its row and
# compare sets
# Idea: Build a dict from letter to row index (26
# entries). A word qualifies if all of its lowercased
# letters map to the same row, i.e. the set of rows has
# size one. Lowercase first so 'Alaska' works.
def one_row_words(words, rows=QWERTY):
    row_of = row_map(rows)
    out = []
    for w in words:
        letters = w.lower()
        # a letter that is on no row makes the word
        # invalid
        if any(c not in row_of for c in letters):
            continue
        # all letters must come from the same row
        if len({row_of[c] for c in letters}) <= 1:
            out.append(w)
    return out
```

240 Check if all substrings are anagrams of words MEDIUM

Template fit: key = the tuple of letter counts; keep the keys of the allowed words in a set and test every substring of length 3 or more against it

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **`>= 3`** can be **rearranged (anagrammed)** into a valid dictionary word. Formally, for every substring **`t = s[i..j]`** with **`|t| >= 3`**, there exists some word **`w in dict`** such that **`w`** is a permutation of **`t`** (same multiset of characters). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**. - “Can be rearranged into a word” means the substring and the dictionary word have exactly the same character counts. Example:
- If **`t = "act"`**, it passes if the dictionary contains any of **`{ "act", "cat", "tac" }`**.

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
ALPHABET = "abcdefghijklmnopqrstuvwxyz"

# BASE QUESTION (optimal): Set of signatures, incremental
# count vectors
# Idea: Precompute the set of count-vector signatures of
# the dictionary words. For each start extend one letter
# at a time, update the count vector in O(1), and look up
# its tuple (once the length is at least 3).
def all_anagram_words(s, words, min_len=3):
    # every substring needs a dictionary word with the
    # same
    # letters. Count vectors are updated as the window
    # grows.
    index = {c: i for i, c in enumerate(ALPHABET)}
    sigs = set()
    for w in words:
        v = [0] * len(ALPHABET)
        for ch in w:
            v[index[ch]] += 1
        sigs.add(tuple(v))
    for i in range(len(s)):
        v = [0] * len(ALPHABET)
        for j in range(i, len(s)):
            v[index[s[j]]] += 1
            long_enough = j - i + 1 >= min_len
            if long_enough and tuple(v) not in sigs:
                return False
    return True
```

241 Group anagrams

MEDIUM

Template fit: key = the sorted letters (or a tuple of 26 letter counts)

QUESTION

Group anagrams

MAIN CODE | helper functions first, then the answer

```python
from collections import defaultdict


# BASE QUESTION (optimal): Dictionary keyed by the sorted
# letters
# Idea: Sort the letters of each word and use the
# resulting string as the key; words with the same key are
# anagrams. One pass, one dictionary.
def group_by_sorting(words):
    # anagrams share the same letters in sorted order
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(w))].append(w)
    return list(groups.values())
```

242 Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same. MEDIUM

Template fit: key = the differences between neighbouring letters, modulo 26 (a shifted word has the same differences)

QUESTION

Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.

MAIN CODE | helper functions first, then the answer

```
from collections import defaultdict


# HELPERS for the base question
def shift_key(word):
    # the gaps between neighbouring letters, wrapping z to
    # a.
    # Words that are shifts of each other share this key.
    return tuple((ord(b) - ord(a)) % 26
                 for a, b in zip(word, word[1:]))

# BASE QUESTION (optimal): Group by the tuple of
# successive differences
# Idea: Two strings are buddies when every letter is
# shifted by the same amount, which means the gaps between
# neighbouring letters (mod 26) are identical. Use those
# gaps as the hash key.
def group_buddies(words, key=shift_key):
    # words of different length have keys of different
    # length, so they never land in the same group
    groups = defaultdict(list)
    for w in words:
        groups[(len(w), key(w))].append(w)
    return list(groups.values())
```

243 Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second? HARD

Template fit: same letters used, and the sorted list of letter frequencies must match: compare set(s) == set(t) and the sorted Counter values

QUESTION

Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second?

MAIN CODE | helper functions first, then the answer

```
from collections import Counter


# HELPERS for the base question
ALPHABET = 26

def can_group(have, want):
    # can the counts in `have` be grouped (each used once)
    # so that the group sums are exactly the counts in
    # `want`? Letters merge, they never split.
    if sum(have) != sum(want):
        return False
    have = sorted(have, reverse=True)
```

```
    left = list(want)

    def place(i):
        if i == len(have):
            return all(x == 0 for x in left)
        tried = set()
        for j in range(len(left)):
            # equal bins behave the same: try each size
            # once
            fits = left[j] >= have[i]
            if fits and left[j] not in tried:
                tried.add(left[j])
                left[j] -= have[i]
                if place(i + 1):
                    return True
                left[j] += have[i]
        return False

    return place(0)

# BASE QUESTION (optimal): (LeetCode 1657): same letter
# set and same multiset of frequencies
# Idea: Operation 1 reorders positions freely, so only the
# multiset of letters matters. Operation 2 exchanges the
# identities of two letters that exist, which permutes the
# frequencies among existing letters.
def both_operations(s, t, alphabet=ALPHABET):
    # swaps make positions irrelevant: only the letter
    # counts matter, and a conversion can only merge or
    # rename a letter
    if len(s) != len(t):
        return False
```

```
    cs, ct = Counter(s), Counter(t)
    if cs == ct:
        return True
    mine, goal = list(cs.values()), list(ct.values())
    if not can_group(mine, goal):
        return False
    # only identical sizes mean no merge happens. With
    # every
    # letter used there is no spare letter to rename
    # safely.
    if len(cs) == len(ct) == alphabet:
        return False
    return True
```

GROUP 35 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 1 E 2 M 2 H

K-way merge: a heap of the next item from each source

244 Find k pairs with smallest sums — EASY

Template fit: seed with (a[i] + b[0], i, 0); the successor of pair (i, j) is (i, j + 1)

QUESTION

You are given two **sorted** integer arrays **`arr1`** and **`arr2`** (each sorted in non-decreasing order), and an integer **`k`**. Consider all possible pairs formed by taking one element from **`arr1`** and one element from **`arr2`**. For indices **`i`** and **`j`**, the pair is **`(arr1[i], arr2[j])`** with sum **`arr1[i] + arr2[j]`**. Your task is to return the **k pairs `(arr1[i], arr2[j])`** with the smallest sums. If there are fewer than **`k`** total possible pairs, return all possible pairs. You may return the pairs in any order. ---

Function Signature

(one possible form)

```
kSmallestPairs(arr1: List[int], arr2: List[int], k: int) -> List[Tuple[int, int]]
```

Example

```
arr1 = [1, 7, 11] arr2 = [2, 4, 6] k = 3
```

All possible pairs and their sums are:

- (1, 2) -> 3 - (1, 4) -> 5 - (1, 6) -> 7 - (7, 2) -> 9 - (7, 4) -> 11 - (7, 6) -> 13 - (11, 2) -> 13 - (11, 4) -> 15 - (11, 6) -> 17
  The 3 pairs with the smallest sums are:

```
[(1, 2), (1, 4), (1, 6)]
```

(The order among these returned pairs does not matter.) Explain an efficient algorithm to solve this problem and analyze its time and space complexity. You may assume the arrays are large, so a solution that explicitly generates and sorts all **`len(arr1) * len(arr2)`** pairs is too slow.

MAIN CODE | helper functions first, then the answer

```python
import heapq


# BASE QUESTION (optimal): Min-heap over the sorted rows
# Idea: Push (a[i] + b[0], i, 0) for the first min(k, m)
# values of a. Pop the smallest, record the pair, and push
# the next pair in the same row, (a[i] + b[j + 1], i, j +
# 1). After k pops you have the k smallest sums.
def k_smallest_pairs(a, b, k):
    # a and b sorted: a heap holds the best unused pair of
    # each row i; popping (i, j) offers (i, j + 1)
    if not a or not b or k <= 0:
        return []
    rows = range(min(k, len(a)))
    heap = [(a[i] + b[0], i, 0) for i in rows]
    heapq.heapify(heap)
    out = []
    while heap and len(out) < k:
        _, i, j = heapq.heappop(heap)
        out.append((a[i], b[j]))
        if j + 1 < len(b):
            nxt = a[i] + b[j + 1]
            heapq.heappush(heap, (nxt, i, j + 1))
    return out
```

245 K Largest Elements Across Sorted Lists

MEDIUM

Template fit: seed with the LAST item of each list (negated); the successor is the item before it; reverse the result

QUESTION

You are given a list of inner lists, where each inner list is sorted in ascending order. Return the k largest elements taken from the union of all the inner lists, and return them sorted in ascending order. Duplicate values are kept (an element counts once per position it occupies). The interviewer first accepted a simple solution, then asked for an optimized one.

Function signature

```
from typing import List
def k_largest_sorted(lists: List[List[int]], k: int) -> List[int]:
    """Return the k largest elements across all (individually ascending) inner
    lists, in ascending sorted order."""
    ...
```

Follow-up (as reported): Optimize beyond concatenate-and-sort. Hint: because each list is already sorted, seed a max-heap with the last element of every non-empty list and pop k times, pushing the predecessor within a list after each pop; collect the popped values and reverse them.

EXAMPLES

Example 1	
Input	`lists = [[1, 4, 7], [2, 5, 8], [3, 6, 9]], k = 4`
Output	`[6, 7, 8, 9]`
Why	`The four largest values overall are 9, 8, 7, 6, returned ascending.`

Example 2	
Input	`lists = [[10, 20, 30]], k = 2`
Output	`[20, 30]`

Example 3	
Input	`lists = [[5], [1, 3]], k = 5`
Output	`[1, 3, 5]`
Why	`k exceeds the total count, so all elements are returned sorted.`

Example 4	
Input	`lists = [[1, 2], [3, 4]], k = 0`
Output	`[]`

CONSTRAINTS

- Let m be the number of inner lists and T the total number of elements across all of them.
- 0 <= m <= 10^4 , 0 <= T <= 10^6 .
- Each element is an integer in [-10^9, 10^9] .
- 0 <= k . If k == 0 return [] . If k >= T , return every element sorted ascending.
- Some inner lists may be empty.

MAIN CODE | helper functions first, then the answer

```
import heapq


# BASE QUESTION (optimal): A max-heap over the ends of the
# lists
# Idea: The next largest element is always at the end of
```

```
# one of the lists.
def k_largest_sorted(lists, k):
    heap = [
        (-lst[-1], i, len(lst) - 1)
        for i, lst in enumerate(lists)
        if lst
    ]
    heapq.heapify(heap)
    out = []
    while heap and len(out) < k:
        neg, i, j = heapq.heappop(heap)
        out.append(-neg)
        if j:
            heapq.heappush(
                heap, (-lists[i][j - 1], i, j - 1)
            )
    return out[::-1]
```

246 Sort a nearly sorted (or K sorted) array. MEDIUM

Template fit: keep a heap of the first k + 1 items; pop the smallest and push the next input item (heapreplace)

QUESTION

Sort a nearly sorted (or K sorted) array.

MAIN CODE | helper functions first, then the answer

```
import heapq


# BASE QUESTION (optimal): Min-heap of size K + 1
# Idea: Push the first K + 1 elements. Then for each
# remaining element, pop the smallest into the output and
# push the new element. At the end pop the heap empty.
def sort_k_sorted(arr, k):
    # every item is at most k places from its final spot,
    # so
    # the smallest of the next k + 1 items is the next
    # answer. Keep k items in a heap, and for each new one
    # push it and pop the smallest: O(n log k).
    heap = list(arr[:k])
    heapq.heapify(heap)
    out = []
    for x in arr[k:]:
        # push the new item and pop the smallest in one go
        out.append(heapq.heappushpop(heap, x))
    while heap:
        out.append(heapq.heappop(heap))
    return out
```

247 Merge k sorted linked lists. HARD

Template fit: seed with each list head (value, list number, node); the successor of a node is node.next

QUESTION

Merge K Sorted Linked Lists. You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Your task is to merge all the linked-lists into one sorted linked-list and return it. Examples: If lists = [[1,4,5],[1,3,4],[2,6]], where each array represents a linked-list, the merged and sorted linked-list would be [1,1,2,3,4,4,5,6]. The output for the merged list should be in array form for simplicity. If lists = [], indicating an array of no linked-lists, the output should be []. If lists = [[]], representing an array with a single empty linked-list, the output should be []. If lists = [[],[1]], where the first linked-list is empty and the second contains a single element, the output should be [1]. If lists = [[-1,5,11],[],[6,10]], the merged and sorted linked-list should be [-1,5,6,10,11]. Constraints: k == lists.length 0 <= k <= 10^4 0 <= lists[i].length <= 500 10^4 <= lists[i][j] <= 10^4 lists[i] is sorted in an increasing order. In this video, Simon, Spotify Software Engineer, answers an interview question about merging k sorted linked lists.No answers contributed by the community yet.Interview experiences6 shared‹›Revolut · 2 months ago"Its good, but I was bad and dont really remember much. But all rounds was good. HR was nice. Tech guy on tech interview was nice too."Software Engineer · Entry Level / L3FFloQast · 2 months ago"Screening and the system design with director are virtual, while on-site loop in the office. The process is well organi

MAIN CODE | helper functions first, then the answer

```
import heapq


# HELPERS used by the code below
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next

# HELPERS for the base question
def _to_list(head):
    out = []
    while head:
        out.append(head.val)
        head = head.next
    return out

# BASE QUESTION (optimal): Min-heap of the current heads
# Idea: Push the head of each non-empty list as (value,
# list index, node). Pop the smallest, append it to the
# output, and push its successor. The list index is a tie-
# breaker so nodes are never compared.
def merge_k_heap(lists):
    # a heap holds the current head of every list; the
    # node
    # index breaks ties so nodes are never compared
    heap = [(n.val, i, n)
            for i, n in enumerate(lists) if n]
    heapq.heapify(heap)
    dummy = tail = ListNode()
    while heap:
        _, i, node = heapq.heappop(heap)
        tail.next = node
        tail = node
        if node.next:
            nxt = node.next
            heapq.heappush(heap, (nxt.val, i, nxt))
    return dummy.next
```

248 Return top-k subset sums in descending order HARD

Template fit: heap of (sum, index); each popped state pushes two successors: add the next item, or swap the last item for the next

QUESTION

Overview: This question evaluates proficiency in combinatorial reasoning, algorithmic optimization for large search spaces, and efficient extraction of top-k values within the coding & algorithms domain.

MAIN CODE | helper functions first, then the answer

```python
import heapq


# BASE QUESTION (optimal): Heap of the k smallest sums of
# absolute values
# Idea: Every subset sum equals (sum of positives) minus a
# subset sum of the absolute values, so the k largest sums
# are the sum of positives minus the k smallest subset
# sums of the sorted absolute values.
def top_k_sums(nums, k):
    # Idea: start from the best subset (all positives).
    # Every
    # other subset is that one with some elements
    # "flipped",
    # which costs |x|. So the k best subsets are the k
    # cheapest sets of flips: a heap over sorted |x|.
    base = sum(x for x in nums if x > 0)
    cost = sorted(abs(x) for x in nums)
    out = [0]
    heap = [(cost[0], 0)] if cost else []
    while heap and len(out) < k:
        s, i = heapq.heappop(heap)
        out.append(s)
        if i + 1 < len(cost):
            # two successors, neither is ever cheaper:
            # add the next element ...
            heapq.heappush(heap, (s + cost[i + 1], i + 1))
            # ... or move the last flipped one up by one
            moved = s - cost[i] + cost[i + 1]
            heapq.heappush(heap, (moved, i + 1))
    return [base - c for c in out[:k]]
```

GROUP 36 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 1 E 4 M

Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

249 Next Birthday (BST In-Order Successor with Wraparound) EASY

Template fit: descend(): go left after a hit (a closer candidate may exist), right otherwise; if no candidate exists wrap around to the leftmost node

QUESTION

This is the "alarm / next-birthday" problem: from a set of birthdays stored in a binary search tree, find the next one that will occur. Each birthday is stored as an integer day-of-year in [1, 365] , and the tree is a valid BST keyed on that value. Given the root of the BST and an integer today , return the next upcoming birthday: the smallest key that is strictly greater than today . If no key is strictly greater than today , the calendar wraps around, so return the smallest key in the whole tree. If the tree is empty, return None .

Function signature

```
from typing import Optional
class TreeNode:
    def __init__(self, val: int, left: "Optional[TreeNode]" = None,
                 right: "Optional[TreeNode]" = None) -> None:
        self.val = val
        self.left = left
        self.right = right
def next_birthday(root: Optional[TreeNode], today: int) -> Optional[int]:
    """Return the smallest key strictly greater than today, wrapping around
    to the smallest key if none is greater. Return None if the tree is empty."""
    ...
```

Follow-up (as reported): If the input were an unsorted binary tree (not a BST), extract all values, sort them, and binary-search for the successor. Hint: bisect_right .

EXAMPLES

Example 1	
Input	root = above, today = 25
Output	30
Why	Smallest key strictly greater than 25 is 30.

Example 2	
Input	root = above, today = 40
Output	10
Why	No key > 40, so wrap around to the smallest key, 10.

Example 3	
Input	root = above, today = 5
Output	10
Why	Smallest key greater than 5 is 10.

Example 4	
Input	root = None, today = 100
Output	None

CONSTRAINTS

QUESTION

- 0 <= number of nodes <= 10^5 ; all node values are distinct and in [1, 365] .
- 1 <= today <= 365 .
- Aim for O(h) time where h is the tree height (do not sort all nodes if you can avoid it).

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Descend the BST once
# Idea: A node above today is a candidate; its left
# subtree may hold a closer one.
def next_birthday(root, today):
    if root is None:
        return None
    best, node = None, root
    while node:
        if node.val > today:
            best, node = node.val, node.left
        else:
            node = node.right
    if best is not None:
        return best
    node = root
    while node.left:
        node = node.left
    return node.val
```

250 Lowest Common Ancestor

MEDIUM

Template fit: dfs returns the node if it is p or q; a node that gets an answer from both sides is the ancestor

QUESTION

Given a binary tree and two nodes p and q, find their lowest common ancestor (LCA): the lowest node that has both p and q as descendants (a node can be a descendant of itself). Follow-up: what if it is an n-ary tree?

EXAMPLES

Example 1	
Input	`root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1`
Output	`3`
Why	`LCA of 5 and 1 is 3.`

Example 2	
Input	`root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4`
Output	`5`
Why	`A node can be its own ancestor.`

CONSTRAINTS

- Number of nodes in tree: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique
- p != q
- p and q exist in the tree

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
class BNode:
```

```
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = (
            val,
            left,
            right,
        )

# BASE QUESTION (optimal): One recursive pass
# Idea: dfs(node) returns node if node is p or q,
# otherwise combines its children: if both children return
# something, node is the LCA; if only one does, pass that
# up.
def lca(root, p, q):
    # each call reports where p or q was found below. A
    # node
    # with hits from two different children (or that is p
    # or
    # q itself) is the answer; a single hit is passed up.
    def find(node):
        if node is None:
            return None
        if node is p or node is q:
            return node
        hits = [r for r in map(find, node.children) if r]
        if len(hits) >= 2:
            return node
        return hits[0] if hits else None

    return find(root)
```

251 Determine if a given binary tree is a binary search tree (BST). MEDIUM

Template fit: check(): pass lo and hi bounds down; fail when a value leaves its allowed window

QUESTION

Determine if a given binary tree is a valid Binary Search Tree (BST).

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Pass the allowed range (low,
# high) down
# Idea: Each node must lie strictly between low and high.
# The left child inherits (low, node.val) and the right
# child (node.val, high). The root starts with (-inf,
# +inf).
def is_bst(root, dup="none"):
    def fits(val, lo, hi):
        if lo is not None:
            if val < lo or (val == lo and dup != "right"):
                return False
        if hi is not None:
            if val > hi or (val == hi and dup != "left"):
                return False
        return True

    stack = [(root, None, None)]
    while stack:
        node, lo, hi = stack.pop()
        if node is None:
            continue
        if not fits(node.val, lo, hi):
            return False
        stack.append((node.left, lo, node.val))
        stack.append((node.right, node.val, hi))
```

```
    return True
```

252 Find mode in a trinary search tree — MEDIUM

Template fit: in-order walk (left, middle, right); equal values are neighbours, so one running count finds the mode(s)

QUESTION

You are given the root of a **trinary search tree** where each node has up to three children:

```
class Node { int val; Node left; // all values < val Node middle; // all values = val Node right; // all v
alues > val }
```

Assume the tree is intended to satisfy the ordering rules above.

Task

Return the **mode(s)** of the tree: the value(s) that occur most frequently among all nodes.

- If there is one mode, return a single-element list.
- If multiple values tie for highest frequency, return **all** such values (in any order).

Constraints (typical)

- Number of nodes **n** can be large (e.g., up to **`10^5`**).
- Values can be any 32-bit signed integers.

Follow-up

How would your approach change if the tree is **broken** (i.e., it may not respect the **<**, **=**, **>** constraints), but you still must return the correct mode(s)?

Part 1: Find Mode(s) in a Valid Trinary Search TreeYou are given a valid trinary search tree. Each node has an integer value and up to three children: left, middle, and right. For every node, all values in its left subtree are strictly less than the node's value, all values in its middle subtree are equal to the node's value, and all values in its right subtree are strictly greater than the node's value. The tree is encoded using parallel arrays instead of Node objects. Return the value or values that occur most frequently in the tree. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.
Constraints
0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a valid tree with no cyclesThe tree satisfies the trinary search ordering rules recursivelyEach value is a 32-bit signed integer

EXAMPLES

```
Input: ([], [], [], [], -1)
Expected Output: []
```

Explanation: The tree is empty, so there are no modes.

```
Input: ([42], [-1], [-1], [-1], 0)
Expected Output: [42]
```

Explanation: A single-node tree has that node's value as the only mode.

QUESTION

Hints

In a valid trinary search tree, an inorder traversal using left, node, middle, right visits values in nondecreasing order.Because equal values appear consecutively during that traversal, you can track only the current run length and the best run length so far.

Part 2: Find Mode(s) in a Broken Trinary TreeYou are given a trinary tree with integer values. Each node has up to three children: left, middle, and right. Unlike a valid trinary search tree, this tree may be broken: the left, middle, and right children do not necessarily follow any ordering rules. Return the value or values that occur most frequently among all reachable nodes. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.

Constraints

0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a tree with no cyclesNo ordering rule is guaranteed between a node and its childrenEach value is a 32-bit signed integer

Examples

Input: ([], [], [], [], -1)

Expected Output: []

Explanation: The tree is empty, so there are no modes.

Input: ([-2147483648], [-1], [-1], [-1], 0)

Expected Output: [-2147483648]

Explanation: A single-node tree has that node's value as the only mode.

Hints

Without the ordering property, equal values are not guaranteed to be adjacent in any traversal.Use a frequency table while traversing the entire tree.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): In-order traversal counting
# runs (O(height) extra space)
# Idea: Walk left, middle, right with an explicit stack.
def find_modes(root):
    # in-order: equal values are next to each other, so a
    # running count needs no dictionary
    best, modes, cur, run = 0, [], None, 0

    def visit(v):
        nonlocal best, modes, cur, run
        run = run + 1 if v == cur else 1
        cur = v
        if run > best:
            best, modes = run, [v]
        elif run == best and v not in modes:
            modes.append(v)

    stack, node = [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        visit(node.val)
        # the middle chain repeats node.val once per child
        mid = node.middle
        while mid:
            visit(mid.val)
            mid = mid.middle
        node = node.right
    return sorted(modes)
```

253 Next Upcoming Birthday in a Family Tree

MEDIUM

Template fit: walk the ancestors with a stack and a seen set, keep the best birthday by the circular distance from today

QUESTION

You are given a family tree rooted at "me". Each Person node has a name , a birthday given as (month, day) , and pointers to parent1 and parent2 (either may be None). Following parent pointers reaches the whole ancestry; treat the reachable set of Person nodes as the tree to search (each distinct person counted once, even if reachable by two paths). Given today's date (today_month, today_day) , implement alarm_next_birthday(me, today_month, today_day) that scans every person and returns the (month, day) of the soonest upcoming birthday, where a birthday landing exactly on today counts as upcoming (distance 0) and the calendar wraps around the year. If the tree is empty (me is None), return the empty string "" . Use this ordering key to compare dates: key(m, d) = (m - 1) * 31 + d , and cyclic distance (key(person) - key(today)) mod 372 . Return the person with the smallest such distance; if several people share that soonest date, return that (month, day) .

Function signature

```
from typing import Optional, Union
class Person:
    def __init__(self, name: str, month: int, day: int,
                 parent1: "Optional[Person]" = None,
                 parent2: "Optional[Person]" = None) -> None:
        self.name = name
        self.month = month
        self.day = day
        self.parent1 = parent1
        self.parent2 = parent2
def alarm_next_birthday(me: Optional[Person],
                        today_month: int,
                        today_day: int) -> Union[tuple[int, int], str]:
    """Return (month, day) of the nearest upcoming birthday, or "" if there is
nobody."""
    ...
```

Follow-up (as reported): Support repeated alarms: after reporting one birthday, the next call returns the following-soonest, and once all have been reported, return "" .

EXAMPLES

Example 1	
Input	`me = Me(12,1) with parent1 = Alice(9,25), parent2 = Bob(1,10); today = (9, 12)`
Output	`(9, 25)`
Why	`keys — today=260, Alice=273 (dist 13), Me=342 (dist 82), Bob=10 (dist (10- 260) mod 372 = 122). Alice is soonest.`

Example 2	
Input	`me = None; today = (5, 5)`
Output	`""`
Why	`No people to check.`

QUESTION

Example 3	
Input	`me = Me(3,3) alone; today = (3, 3)`
Output	`(3, 3)`
Why	`The only birthday is today, distance 0.`

CONSTRAINTS

- 1 <= month <= 12 , 1 <= day <= 31 .
- Up to 10^5 people; parent pointers form a DAG (no cycles).

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): One traversal tracking the
# minimum distance
# Idea: Walk every distinct ancestor once and keep the
# smallest cyclic distance.
class Person:
    def __init__(
        self,
        name,
        month,
        day,
        parent1=None,
        parent2=None,
    ):
        self.name = name
        self.month = month
        self.day = day
        self.parent1 = parent1
        self.parent2 = parent2

# BASE QUESTION (optimal): One traversal tracking the
# minimum distance
# Idea: Walk every distinct ancestor once and keep the
# smallest cyclic distance.
def alarm_next_birthday(me, today_month, today_day):
    if me is None:
        return ""
    today = key(today_month, today_day)
    best, answer = None, ""
    stack, seen = [me], set()
    while stack:
        p = stack.pop()
        if id(p) in seen:
            continue
        seen.add(id(p))
        dist = (key(p.month, p.day) - today) % 372
        if best is None or dist < best:
            best, answer = dist, (p.month, p.day)
        stack += [
            q for q in (p.parent1, p.parent2) if q
        ]
    return answer
```

GROUP 37 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 4 M 1 H

Random pick without repeats: swap the chosen item with the last one and pop

254 Random Generator Without Repeats

MEDIUM

Template fit: the template as written: values 0..n-1, every call returns a new one; raise when empty

QUESTION

Design a class RandomGenerator initialized with a positive integer n . Each call to generate() returns one integer chosen uniformly at random from the values in [0, n-1] that have not yet been returned. After n calls every value in [0, n-1] has been returned exactly once; a further call must raise IndexError (the pool is empty). Each generate() call must run in O(1) time. The intended approach maintains an array of the remaining values and, on each call, swaps a randomly chosen remaining value to the end of the active region and shrinks that region (partial Fisher-Yates).

Function signature

```
import random
class RandomGenerator:
    def __init__(self, n: int) -> None:
        """Prepare to yield a random permutation of [0, n-1], one value per
generate()."""
        ...
    def generate(self) -> int:
        """Return a uniformly random not-yet-returned value in O(1).
        Raise IndexError if all values have already been generated."""
        ...
```

Follow-up (as reported): Add a reset() that makes all n values available again in O(1) amortized.

EXAMPLES

Example 1	
Input	`g = RandomGenerator(3); [g.generate() for _ in range(3)]`
Output	`e.g. [2, 0, 1]`
Why	`The three calls return a permutation of {0, 1, 2} with no repeats; any of the 6 permutations is a valid output. A 4th call would raise IndexError.`

Example 2	
Input	`g = RandomGenerator(1); g.generate()`
Output	`0`
Why	`Only one value exists.`

CONSTRAINTS

- 1 <= n <= 10**6 .

MAIN CODE | helper functions first, then the answer

```python
import random


# BASE QUESTION (optimal): Partial Fisher-Yates with an
# active region
# Idea: Swap the drawn value to the end of the active
# region and shrink it.
class RandomGenerator:
```

```
    def __init__(self, n, seed=None):
        self.n = n
        self.values = list(range(n))
        self.size = n
        self.rng = random.Random(seed)

    def generate(self):
        if self.size == 0:
            raise IndexError(
                "all values have been generated"
            )
        i = self.rng.randrange(self.size)
        self.size -= 1
        self.values[i], self.values[self.size] = (
            self.values[self.size],
            self.values[i],
        )
        return self.values[self.size]
```

255 Random Song Shuffler With a No-Repeat Window — MEDIUM

Template fit: the same pool, plus a deque of the last k songs; the played song leaves the pool and returns to it when it drops off the deque

QUESTION

Design a shuffler that plays songs at random from a fixed playlist, subject to one rule: the last k songs played may not be replayed. In other words, a song becomes eligible again only after k other (distinct) plays have occurred since it was last played. Implement the class so that each call to play() runs in O(1) time (no scanning of the playlist). The intended technique is a valid_songs list of currently eligible songs plus a song_to_index dictionary, using swap-and-pop to move a just-played song out of the eligible region and to re-admit the song that is leaving the window.

Function signature

```
import random
class SongShuffler:
    def __init__(self, songs: list[str], k: int) -> None:
        """Initialize with the playlist and the no-repeat window size k."""
        ...
    def play(self) -> str:
        """Return a uniformly random eligible song in O(1); it must not be
        one of the last k songs returned. Update state so the window slides."""
        ...
```

Follow-up (as reported): The interviewer wanted the naive deque solution replaced by O(1). Show why swap-and-pop with an index map achieves constant time per play() .

EXAMPLES

Example 1	
Input	`SongShuffler(["A", "B", "C"], k=1); then play(), play(), play(), play()`
Output	`e.g. "B", "A", "C", "B"`
Why	`k=1 forbids repeating the single most recent song. "B" then "A" is fine (A != B). Then "C" (C != A). Then "B" is allowed again because the only forbidden song is the immediately previous one ("C"). Any sequence with no two adjacent equal songs is valid.`

QUESTION

Example 2	
Input	`SongShuffler(["X", "Y"], k=0); play(), play()`
Output	`e.g. "Y", "Y"`
Why	`With k=0 there is no restriction, so repeats are allowed.`

CONSTRAINTS

- 1 <= len(songs) <= 10**5 , song titles are distinct strings.
- 0 <= k < len(songs) (so at least one song is always eligible). play() returns any eligible song chosen uniformly at random; every returned song must not be among the previous k returned songs.

MAIN CODE | helper functions first, then the answer

```python
from collections import deque


# BASE QUESTION (optimal): Eligible array, index map and
# swap-and-pop
# Idea: Remove the played song from the eligible array by
# swapping it with the last element; the song leaving the
# window is appended back.
class SongShuffler:
    def __init__(self, songs, k, seed=None):
        self.valid = list(songs)
        self.index = {
            s: i for i, s in enumerate(self.valid)
        }
        self.recent = deque()
        self.k = k
        self.rng = random.Random(seed)

    def play(self):
        i = self.rng.randrange(len(self.valid))
        song = self.valid[i]
        last = self.valid[-1]
        # swap-and-pop: the last song takes the hole
        self.valid[i] = last
        self.index[last] = i
        self.valid.pop()
        del self.index[song]
        self.recent.append(song)
        if len(self.recent) > self.k:
            back = self.recent.popleft()
            self.index[back] = len(self.valid)
            self.valid.append(back)
        return song
```

256 Design set with O(1) random access — MEDIUM

Template fit: add appends and records the position; delete swaps with the last item and pops; get_random picks a random index

QUESTION

Design a data structure ("FancySet") that stores unique integers and supports the following operations, each in average O(1) time:

- **`add(x)`**: Insert value **`x`** if it is not already present. Return whether the insert happened.
- **`delete(x)`**: Remove value **`x`** if it exists. Return whether a deletion happened.
- **`getRandom()`**: Return a uniformly random element currently in the set. Requirements / follow-ups:
- All operations should be average O(1).

QUESTION

- Handle corner cases such as calling **`getRandom()`** on an empty set (define and justify behavior).
- Discuss how you would make this data structure thread-safe. Compare using a single lock for the whole set vs finer-grained locking (e.g., element-level locks), and explain trade-offs.

Design a data structure called FancySet that stores unique integers and supports these operations in average O(1) time: add(x), delete(x), and getRandom(). For this coding problem, implement a function **`solution(operations, values, seed)`** that simulates a newly created empty FancySet and returns the result of every operation. Use the standard dynamic-array + hash-map behavior:

- Store current elements in an array-like list.
- Store each element's current index in a hash map.
- When deleting an element that is not at the end of the list, move the last element into the removed slot, then pop the last slot. To make test cases deterministic, **`getRandom()`** does not use the language's built-in RNG. Instead, keep an internal 32-bit state initialized to **`seed`**. When **`getRandom()`** is called on a non-empty set, update the state with: **`state = (1664525 * state + 1013904223) mod 2^32`** and return the element at index **`state % current_size`** from the current internal list. If **`getRandom()`** is called on an empty set, return **`None`** and do not advance the state. In a real interview, picking a random index from the array gives uniform random access; the fixed generator above is only for reproducible grading. Follow-up discussion (not graded by the code runner): explain how you would make FancySet thread-safe, and compare using one lock for the whole structure versus finer-grained locking.

Constraints
1 <= len(operations) <= 2 * 10^5len(values) == len(operations)Each operation is one of 'add', 'delete', or 'getRandom'For 'add' and 'delete', values[i] contains exactly one integer; for 'getRandom', values[i] is []-10^9 <= x <= 10^9All operations should run in O(1) average time

EXAMPLES

```
Input: (['add','add','getRandom','delete','getRandom','add','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)
Expected Output: [True, True, 1, True, 2, False, 2]
```

Explanation: After inserting 1 and 2, the first deterministic random choice picks index 0, so it returns 1. Deleting 1 leaves only 2. Adding 2 again fails because it is already present.

```
Input: (['getRandom','add','add','delete','delete','getRandom'], [[],[5],[5],[7],[5],[]], 1)
Expected Output: [None, True, False, False, True, None]
```

Explanation: Calling getRandom on an empty set returns None. Adding 5 twice inserts it only once. Deleting 7 fails because it is not present. After deleting 5, the set becomes empty again.

Hints

You need both fast membership/index lookup and fast access by numeric position. What two data structures combine to give that?Deleting from the middle of an array is expensive unless you replace the removed element with the last element and update that last element's stored index.

MAIN CODE | helper functions first, then the answer

```
import random


# BASE QUESTION (optimal): Array of values plus a dict
# from value to index
# Idea: add appends to the array and records the index.
# delete finds the index from the dict, moves the last
# element into that slot, updates its index in the dict,
# pops the array and removes the key.
class FancySet:
    """add, delete and getRandom, each O(1) on average."""
```

```python
    def __init__(self, rng=None):
        # the values in a list (for random access) and a
        # map
        # from value to its index in that list
        self.items, self.pos = [], {}
        self.rng = rng or random

    def add(self, x):
        if x in self.pos:
            return False
        self.pos[x] = len(self.items)
        self.items.append(x)
        return True

    def delete(self, x):
        if x not in self.pos:
            return False
        i, last = self.pos[x], self.items[-1]
        # move the last item into the hole, then drop the
        # tail
        self.items[i] = last
        self.pos[last] = i
        self.items.pop()
        del self.pos[x]
        return True

    def get_random(self):
        if not self.items:
            # say what happens on an empty set
            raise IndexError("empty set")
        i = self.rng.randrange(len(self.items))
        return self.items[i]
```

257 Unique Random Number Generator

MEDIUM

Template fit: the lazy version for huge n: a dict 'moved' stores only the swapped slots, so memory stays O(calls)

QUESTION

Design a class UniqueRandom initialized with an integer n . Each call to generate() returns a number drawn uniformly at random from [0, n-1] that has not been returned before, so across successive calls no value ever repeats. After all n distinct values have been produced, further calls return None . Every unused value must remain equally likely on each draw (no bias), and generate() must run in O(1) time per call.

Function signature

```
import random
class UniqueRandom:
    def __init__(self, n: int, seed: int | None = None) -> None:
        ...
    def generate(self) -> int | None:
        """Return a not-yet-returned number in [0, n-1], or None once all n are
exhausted."""
        ...
```

Follow-up (as reported): Use a partial Fisher-Yates shuffle: keep a virtual array 0..n-1 , and on the i -th call swap a random unused slot into position i using a sparse dict of moved entries, giving O(1) time and O(number-of- draws) space.

EXAMPLES

QUESTION

Example 1	
Input	`g = UniqueRandom(3); [g.generate(), g.generate(), g.generate()]`
Output	`a permutation of [0, 1, 2] (e.g. [2, 0, 1])`
Why	`The three calls return all of 0,1,2 in some random order with no repeats.`

Example 2	
Input	`(continuing above) g.generate()`
Output	`None`
Why	`All 3 values already returned, so the pool is exhausted.`

Example 3	
Input	`g = UniqueRandom(1); [g.generate(), g.generate()]`
Output	`[0, None]`
Why	`The only value 0 is returned once, then the generator is exhausted.`

CONSTRAINTS

- 1 <= n <= 10^7 .
- Support an optional seed for reproducible tests.

MAIN CODE | helper functions first, then the answer

```python
import random


# BASE QUESTION (optimal): Lazy Fisher-Yates with a
# dictionary
# Idea: Swap a random unused slot into position i,
# remembering only the moved slots.
class UniqueRandom:
    def __init__(self, n, seed=None):
        self.n = n
        self.drawn = 0
        self.moved = {}
        self.rng = random.Random(seed)

    def generate(self):
        if self.drawn == self.n:
            return None
        i = self.drawn
        j = self.rng.randrange(i, self.n)
        value = self.moved.get(j, j)
        # the value that sat at slot i takes the place of
        # the one just drawn
        self.moved[j] = self.moved.get(i, i)
        self.drawn += 1
        return value
```

258 Shuffle array using random integer API

HARD

Template fit: Fisher-Yates: for i from the end, j = rand_int(0, i), swap a[i] and a[j]

QUESTION

You are given an array **nums** of length **n** representing a deck of distinct cards. You **do not** have access to a built-in shuffle function, but you are given the following API:

```
rand_int(l, r) # returns a uniformly random integer in [l, r], inclusive
```

Task

Implement a function **`shuffle(nums)`** that randomly permutes the array **in place** such that **every one of the `n!` permutations is equally likely**.

Requirements

- You may only use **`rand_int(l, r)`** as your source of randomness.
- The algorithm should run in **`O(n)`** time and **`O(1)`** extra space.
- You may assume **`rand_int`** itself is perfectly uniform and independent between calls. Describe your algorithm, argue why the resulting permutation is uniform, and give the time and space complexity.

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
def shuffle_by_keys(nums):
    keyed = sorted(
        (
            (rand_int(0, 10**9), i)
            for i in range(len(nums))
        )
    )
    return [nums[i] for _, i in keyed]

# BASE QUESTION (optimal): Fisher-Yates
# Idea: Walk i from n - 1 down to 1, pick j uniformly from
# 0..i (inclusive) and swap nums[i] with nums[j].
def shuffle(nums, rand_int):
    # Fisher-Yates: from the end, swap each position with
    # a
    # random one at or before it. All n! orders are
    # equally
    # likely because each step picks uniformly among i +
    # 1.
    for i in range(len(nums) - 1, 0, -1):
        j = rand_int(0, i)
        nums[i], nums[j] = nums[j], nums[i]
    return nums
```

GROUP 38 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 3 M 2 H

Build and serialize: write a structure out, read it back with the same walk

259 Construct a binary tree given its preorder and inorder traversals. MEDIUM

Template fit: the first preorder value is the root; its position in a value-to-index map of the inorder list splits left from right

QUESTION

Given two integer arrays, preorder and inorder, which represent the pre-order and in-order traversal of a binary tree, construct and return the binary tree. Pre-order traversal visits nodes in this order: root -> left subtree -> right subtree In-order traversal visits nodes in this order: left subtree -> root -> right subtree You need to construct the binary tree and return its root node. Examples preorder = [3, 9, 20, 15, 7], inorder = [9, 3, 15, 20, 7] output: [3, 9, 20, null, null, 15, 7] Explanation: Given the two arrays, the binary tree constructed is: 3 / \ 9 20 / \ 15 7 preorder = [1, 2], inorder = [2, 1] output: [1, 2]
Explanation: Given the two arrays, the binary tree constructed is: 1 / 2 Constraints: The number of nodes in the tree is in the range [1, 3000]. preorder and inorder consist of unique integers. The elements of inorder will always be the same as the elements of preorder but in a different order.

MAIN CODE | helper functions first, then the answer

```
# HELPERS used by the code below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

# BASE QUESTION (optimal): Hash map of inorder positions
# and index bounds
# Idea: Map each value to its inorder index.
def build_pre_in(preorder, inorder):
    # preorder: root, left, right. The root's position in
    # the
    # inorder list splits it into the left and right
    # parts.
    where = {v: i for i, v in enumerate(inorder)}
    nxt = 0

    def build(lo, hi):
        nonlocal nxt
        if lo >= hi:
            return None
        root = TreeNode(preorder[nxt])
        nxt += 1
        mid = where[root.val]
        root.left = build(lo, mid)
        root.right = build(mid + 1, hi)
        return root

    return build(0, len(inorder))
```

260 Write functions to serialize and deserialize a list of strings. MEDIUM

Template fit: write len + '#' + text for each string; read the length, then take exactly that many characters

QUESTION

Serialize and Deserialize a List of Strings. You are tasked with writing two functions: serialize and deserialize. The serialize function will take a list of strings as input and return a single string. The deserialize function will take this single string and reconstruct the original list of strings. Details The serialize function should concatenate all strings in the list into a single string, with each original string's length encoded as a single character prefix. The deserialize function should be able to take the single string and accurately split it back into the original list of strings, using the encoded length characters as guides. For example, consider a list ["hello", "world"]. The serialization of this list might be "\x05hello\x05world", where \x05 indicates that the following string has a length of 5 characters. Constraints The maximum length of any string is 65535 (0xFFFF). If a string's length exceeds the maximum, the serialize function should raise an exception. The deserialize function should accurately handle any edge cases, such as empty strings or single-character strings. Assume that all strings consist of ASCII characters only. Examples lst = ['hello', 'world'] serialized = serialize(lst)

serialized output

: "\x05hello\x05world" deserialized = deserialize(serialized)

deserialized output should be

: ['hello', 'world'] lst = [''] serialized = serialize(lst)

serialized output

: "\x00" deserialized = deserialize(serialized)

deserialized output should be

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Length-prefix every string
# Idea: Write len(s), a separator, then s itself. To
# decode read digits up to the separator to get the
# length, then take exactly that many characters.
def serialize(items):
    # length prefix: "<length>#<text>" for every string.
    # The
    # reader knows exactly how much to read, so the text
    # may
    # hold any character, even "#" and digits.
    return "".join(f"{len(s)}#{s}" for s in items)

# BASE QUESTION (optimal): Length-prefix every string
# Idea: Write len(s), a separator, then s itself. To
# decode read digits up to the separator to get the
# length, then take exactly that many characters.
def deserialize(text):
    items, i = [], 0
    while i < len(text):
        # read the digits up to the "#"
        sep = text.index("#", i)
        size = int(text[i:sep])
        start = sep + 1
        items.append(text[start:start + size])
        i = start + size
    return items
```

261 Print a Directory Tree

MEDIUM

Template fit: build nested dicts from the path parts, then print them with a recursive walk that indents by depth

QUESTION

You are given a list of file/directory paths , each a '/' -separated string (for example "dir/subdir/file.txt"). Reproduce what a Unix directory listing tool prints: build the directory hierarchy implied by the paths and return it as a single newline-separated string. Rules: Each path component is a node. A component that appears under the same parent in several paths must be printed only once. Indent each node with 2 * depth spaces, where the top-level components have depth 0 . Children are printed in the order they are first encountered across the input list (preserve first-seen order; do not sort). The output must not have a trailing newline.

Function signature

def print_directory_tree(paths: list[str]) -> str:

    ...

Follow-up (as reported): Add box-drawing connectors (, ,) like the real tree command. Hint: track, per level, whether the current node is the last child of its parent.

EXAMPLES

Example 1	
Input	`paths = ["a/b/c", "a/b/d", "a/e"]`
Output	`"a\n b\n c\n d\n e"`
Why	`Rendered: a b c d e`

Example 2	
Input	`paths = ["src/main.py", "src/util.py", "README.md"]`
Output	`"src\n main.py\n util.py\nREADME.md"`
Why	`"src" and its two files come first (first-seen order), then the top-level "README.md".`

Example 3	
Input	`paths = []`
Output	`""`

CONSTRAINTS

- 0 <= len(paths) <= 10^4
- Each path is non-empty, has no leading/trailing '/' , and no empty components.
- If paths is empty, return the empty string "" .

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): Nested dictionaries and a DFS
# Idea: Insert every path into a dict of dicts, then print
# depth first with two spaces per level.
def print_directory_tree(paths):
    root = {}
    for path in paths:
        node = root
        for part in path.split("/"):
            node = node.setdefault(part, {})
    lines = []

    def walk(node, depth):
        for name, kids in node.items():
            lines.append("  " * depth + name)
            walk(kids, depth + 1)
```

```
    walk(root, 0)
    return "\n".join(lines)
```

262 Implement a decode and encode function of a general N-ary tree. HARD

Template fit: write value and child count for each node (preorder); read the count to know how many children to build

QUESTION

Implement a decode and encode function of a general N-ary tree.

MAIN CODE | helper functions first, then the answer

```
import json


# HELPERS for the base question
class Node:
    def __init__(self, val, children=None):
        self.val = val
        self.children = children or []

def preorder_pairs(root):
    # (value, number of children) for every node in pre-
    # order,
    # iterative so a very deep tree cannot overflow the
    # stack
    out = []
    stack = [root] if root else []
    while stack:
        node = stack.pop()
        out.append((node.val, len(node.children)))
        # reversed: the first child must be popped first
        stack.extend(reversed(node.children))
    return out

def build_preorder(pairs):
    # the reverse: rebuild the tree from the same pairs
    pairs = iter(pairs)
    first = next(pairs, None)
    if first is None:
        return None
    root = Node(first[0])
    # each stack entry is a parent and the children it
    # still has to receive
    stack = [[root, first[1]]]
    while stack:
        top = stack[-1]
        if top[1] == 0:
            stack.pop()
            continue
        top[1] -= 1
        val, count = next(pairs)
        child = Node(val)
        top[0].children.append(child)
        stack.append([child, count])
    return root
```

```
# BASE QUESTION (optimal): Pre-order with child counts,
# iterative decode
# Idea: Write (value, number of children) in pre-order.
def encode(root):
    flat = []
    for val, count in preorder_pairs(root):
        flat += [val, count]
```

```
    return json.dumps(flat)

# BASE QUESTION (optimal): Pre-order with child counts,
# iterative decode
# Idea: Write (value, number of children) in pre-order.
def decode(text):
    flat = json.loads(text)
    return build_preorder(zip(flat[::2], flat[1::2]))
```

263 Serialize and Deserialize a Graph

HARD

Template fit: BFS with a seen set; write one line per node: value, then the values of its neighbours; rebuild by creating all nodes first, then linking

QUESTION

Design a codec that can serialize a connected, undirected graph to a string and deserialize that string back into an identical graph. Each node has a unique integer val and a list of neighbors . The graph may contain cycles. After a round trip, the reconstructed graph must have the same node values and the same adjacency relationships as the original. You are given the reference node class below. Implement serialize (node -> string) and deserialize (string -> node). serialize(None) must round-trip to None via deserialize .

Function signature

```
class Node:
    def __init__(self, val: int = 0, neighbors: list["Node"] | None = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
class Codec:
    def serialize(self, node: "Node | None") -> str:
        """Encode the whole graph reachable from node into a string."""
        ...
    def deserialize(self, data: str) -> "Node | None":
        """Rebuild the graph from the string produced by serialize."""
        ...
```

Follow-up (as reported): Support directed graphs (asymmetric adjacency). Hint: record each directed edge (u -> v) rather than assuming both directions during deserialization.

EXAMPLES

Example 1	
Input	`adjList = [[]] # a single node with no neighbors`
Output	`A single node with val 1 and an empty neighbor list.`

Example 2	
Input	`node = None # empty graph`
Output	`codec.deserialize(codec.serialize(None)) is None`

CONSTRAINTS

- 0 <= number of nodes <= 100
- Node values are unique.
- The graph is connected when non-empty; you receive any one node as the entry point.

MAIN CODE | helper functions first, then the answer

```
from collections import deque
```

```python

# BASE QUESTION (optimal): BFS to a text, two passes back
# Idea: One token per node with its neighbour values;
# rebuild by creating the nodes first and linking
# afterwards.
class Node:
    def __init__(self, val=0, neighbors=None):
        self.val = val
        self.neighbors = (
            neighbors if neighbors is not None else []
        )

# BASE QUESTION (optimal): BFS to a text, two passes back
# Idea: One token per node with its neighbour values;
# rebuild by creating the nodes first and linking
# afterwards.
class Codec:
    def serialize(self, node):
        if node is None:
            return ""
        seen, queue, parts = (
            {node.val},
            deque([node]),
            [],
        )
        while queue:
            cur = queue.popleft()
            parts.append(
                f"{cur.val}:"
                + ",".join(
                    str(n.val) for n in cur.neighbors
                )
            )
            for nb in cur.neighbors:
                if nb.val not in seen:
                    seen.add(nb.val)
                    queue.append(nb)
        return ";".join(parts)

    def deserialize(self, data):
        if not data:
            return None
        rows = [
```

```python
            part.split(":") for part in data.split(";")
        ]
        nodes = {int(v): Node(int(v)) for v, _ in rows}
        for v, nbs in rows:
            nodes[int(v)].neighbors = [
                nodes[int(x)]
                for x in nbs.split(",")
                if x
            ]
        return nodes[int(rows[0][0])]
```

GROUP 39 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 1 M 4 H

Games: win or lose with a cache, and guessing with feedback

264 Choose optimal guesses for green-only Wordle — MEDIUM

Template fit: bitmask per (position, letter) of the words that have it; the best guess splits the words into the most even groups

QUESTION

Overview: This question evaluates probabilistic reasoning, expected-value optimization, information partitioning, and algorithmic efficiency in selecting guesses under partial (green-only) feedback; it belongs to the Coding & Algorithms domain, specifically search and probabilistic modeling, and emphasizes practical algorithm design and implementation rather than purely theoretical concepts. It is commonly asked to assess a candidate's ability to model uncertainty with weighted distributions, minimize the expected remaining search space, and manage time/space trade-offs for inputs up to several thousand words.

MAIN CODE | helper functions first, then the answer

```
from collections import defaultdict
from fractions import Fraction


# HELPERS used by the code below
def green(guess, answer):
    # green-only feedback: which positions match exactly
    return tuple(g == a for g, a in zip(guess, answer))

def partition(guess, words):
    # group the possible answers by the feedback they
    # would give
    groups = defaultdict(list)
    for w in words:
        groups[green(guess, w)].append(w)
    return list(groups.values())

def score(guess, words, weight=None):
    # expected number of answers left after this guess:
    # sum(group^2) / total, with optional per-word weights
    weight = weight or (lambda w: 1)
    total = sum(weight(w) for w in words)
    spread = sum(sum(weight(w) for w in g) ** 2
                 for g in partition(guess, words))
    return Fraction(spread, total)

# BASE QUESTION (optimal): Bitsets per (position, letter),
# splitting groups with AND
# Idea: Index the words: bits[pos][letter] is an integer
# whose bit i is set when word i has that letter at pos.
def best_guess(words, weight=None):
    # the guess that leaves the fewest answers on average
    # (ties: alphabetical, so the result is deterministic)
    def rank(g):
        return (score(g, words, weight), g)
    return min(words, key=rank)
```

265 Wordle-Style 5-Letter Minimum-Guess Strategy HARD

Template fit: ask about the letter at a position that splits the remaining words most evenly (min of yes-count and no-count is largest)

QUESTION

Phone screen coding: given a black-box oracle that tells you whether a specific letter is at a specific position in a hidden 5-letter word, design an algorithm to identify the word in as few oracle calls as possible. Follow-up assumes a dictionary of candidate words is available.

Hidden word: a 5-letter string from a..z.
Oracle: query(pos: int, letter: char) -> bool returns whether position pos (0-indexed) holds letter.
Return the hidden word.
Primary metric: minimize the number of query calls.

Brute-force baseline: 5 × 26 = 130 calls.
Follow-up

A dictionary of candidate 5-letter words is available; design the strategy to use bigram/letter-frequency information to cut the expected number of calls.

EXAMPLES

```
HiddenWord = "GRAPE"
query(0, 'A') -> false
query(0, 'G') -> true   // first letter found
...
```

MAIN CODE | helper functions first, then the answer

```python
# HELPERS for the base question
# letters from most to least common in English words
BY_FREQUENCY = "etaoinshrdlcumwfgypbvkjxqz"

def split_score(words, i, c):
    # how evenly does asking (position i, letter c) split?
    yes = sum(w[i] == c for w in words)
    return min(yes, len(words) - yes)

# BASE QUESTION (optimal): Frequency order, or entropy
# split when a dictionary is known
# Idea: Without a dictionary: for each position ask
# letters from most to least common and stop at the first
# yes; if 25 letters said no the 26th is implied and needs
# no call.
def by_frequency(query):
    # no dictionary: common letters first, stop at the
    # first
    # yes, and skip the 26th letter (it must be the one
    # left)
    calls, word = 0, []
    for i in range(5):
        for k, c in enumerate(BY_FREQUENCY):
            if k == 25:
                word.append(c)
                break
            calls += 1
            if query(i, c):
                word.append(c)
                break
    return "".join(word), calls

# BASE QUESTION (optimal): Frequency order, or entropy
# split when a dictionary is known
# Idea: Without a dictionary: for each position ask
# letters from most to least common and stop at the first
```

```
# yes; if 25 letters said no the 26th is implied and needs
# no call.
def with_dictionary(query, dictionary):
    # keep the words that still fit every answer so far
    words, calls = list(dictionary), 0
    while len(words) > 1:
        options = {(i, w[i]) for w in words
                   for i in range(5)}
        # ask the question that cuts the list closest to
```

```
        # half
        i, c = max(options, key=lambda o: split_score(
            words, o[0], o[1]))
        calls += 1
        hit = query(i, c)
        words = [w for w in words if (w[i] == c) == hit]
    return words[0], calls
```

266 Determine winner in stack-merging game HARD

Template fit: state = the sorted tuple of stack sizes; a move merges two whose sum stays within the cap

QUESTION

Overview: This question evaluates understanding of combinatorial game theory, impartial game analysis, state-space encoding, and efficient state-space search techniques.

MAIN CODE | helper functions first, then the answer

```
from functools import lru_cache


# HELPERS for the base question
def moves(state, cap):
    # merge two stacks if the new stack fits under the cap
    seen = set()
    for i in range(len(state)):
        for j in range(i + 1, len(state)):
            if state[i] + state[j] <= cap:
                rest = [x for k, x in enumerate(state)
                        if k not in (i, j)]
                merged = state[i] + state[j]
                new = tuple(sorted(rest + [merged]))
                # equal results are the same move
                if new not in seen:
                    seen.add(new)
                    yield new

# BASE QUESTION (optimal): Memoize on the sorted state
# Idea: Merging does not depend on order, so a sorted
# tuple identifies the position. Each state is solved once
# and then looked up. This is the standard fix for 'many
# move orders reach the same position'.
def first_player_wins(stacks, cap, misere=False):
    # the player who cannot move loses (misere: wins)
    @lru_cache(None)
    def wins(state):
        options = list(moves(state, cap))
        if not options:
            return misere
        # a winning move leaves the opponent in a losing
        # state
        return any(not wins(nxt) for nxt in options)
```

```
    return wins(tuple(sorted(stacks)))
```

267 Find secret word with match-count feedback HARD

Template fit: guess the word that minimises the size of the biggest feedback group; keep the words with the same match count

QUESTION

Problem

You are given a list of **unique** lowercase words, all of the same length (e.g., length = 6). One of these words is a **secret** word. You can interact with a black-box API:

- **`matchCount(guess) -> int`**: when you submit a word **`guess`** from the list, the API returns the number of positions **`i`** such that **`guess[i] == secret[i]`** (i.e., exact character match at the same index). You are allowed to call **`matchCount`**

at most 10 times.

Task

Design/implement a function (e.g., **`findSecretWord(words)`**) that uses the API to identify the secret word within the guess limit.

Notes / Constraints

- **`words`** contains between 1 and 100 words.
- All words have equal length **`L`** (commonly **`L = 6`**).
- Every guess must be a word from **`words`**.
- After each API response, you may adapt your next guess.

What to return

Depending on the interface, you may either: - return the identified secret word, or - stop once **`matchCount(guess) == L`** (found it). (Your interviewer may treat this as an interactive problem; focus on strategy + correctness under the 10-guess constraint.)

You are given a list of unique lowercase words, and every word has the same length **`L`**. Exactly one word in the list is the secret word. In the original interview version, you interact with a black-box API:

- **`matchCount(guess) -> int`**
- It returns how many positions **`i`** satisfy **`guess[i] == secret[i]`** You may call this API at most 10 times, and every guess must be a word from the given list. In this non-interactive version, implement **`solution(words, secret)`** and simulate the API inside your function. The parameter **`secret`** is provided only so the judge can verify your strategy. Your function should repeatedly make guesses, use the returned match counts to eliminate impossible candidates, and return the discovered secret word. If your strategy does not find the secret within 10 guesses, return an empty string **`""`**.

Constraints**`1 <= len(words) <= 1001 <= len(words[i]) <= 10`**All words are unique, lowercase, and have the same length **`Lsecret`** is guaranteed to be one of the words in **`words`**Your algorithm may use at most 10 simulated **`matchCount`** callsTest data is designed so that a correct elimination strategy can identify the secret within the limit

EXAMPLES

```
Input: (["acckzz", "ccbazz", "eiowzz", "abcczz"], "acckzz")
```

Expected Output: 'acckzz'Explanation: The secret is already one of the candidates. Using match-count feedback, the candidate set shrinks until **`acckzz`** is identified.

```
Input: (["secret"], "secret")
```

Expected Output: 'secret'Explanation: With only one word in the list, the first allowed guess must be the secret.

QUESTION

Hints

After a guess returns **k**, any valid remaining candidate must also have exactly **k** matching positions with that guess.A strong strategy is to choose the next guess that splits the remaining candidates as evenly as possible, minimizing the size of the largest feedback bucket.

MAIN CODE | helper functions first, then the answer

```
from collections import Counter


# HELPERS for the base question
def match(a, b):
    # number of positions holding the same letter
    return sum(x == y for x, y in zip(a, b))

# BASE QUESTION (optimal): Guess the word that minimises
# the size of its largest match-count group
# Idea: For each candidate guess count how many other
# candidates would fall into each match-count group, and
# pick the guess whose biggest group is smallest (a
# minimax choice).
def best_guess(candidates):
    # minimax: the guess whose worst answer leaves the
    # fewest
    # candidates (ties: alphabetical)
    def worst(g):
        sizes = Counter(match(g, w) for w in candidates)
        return max(sizes.values()), g
    return min(candidates, key=worst)

# BASE QUESTION (optimal): Guess the word that minimises
# the size of its largest match-count group
# Idea: For each candidate guess count how many other
# candidates would fall into each match-count group, and
# pick the guess whose biggest group is smallest (a
# minimax choice).
def find_secret(words, match_count, limit=10):
    # match_count(guess) is the black-box API
    candidates, guesses = list(words), 0
    while candidates and guesses < limit:
        guess = best_guess(candidates)
        guesses += 1
        score = match_count(guess)
        if score == len(guess):
            return guess, guesses
        # only words with the same score against this
        # guess
        # can still be the secret
        candidates = [w for w in candidates
                      if match(guess, w) == score]
    return None, guesses
```

268 Babylon Tile-Stack Merge Game

HARD

Template fit: state = the sorted tuple of stacks; a move merges two stacks that share a height or a colour; you win if some merge leaves the opponent without a winning reply

QUESTION

Onsite NG round 1 (Munich office): implement a two-player tile-stack merge game ("Babylon") given the rules. Start state is 12 height-1 stacks of 4 colors (Y / W / G / B), 3 of each; a move places one stack on top of another under a constraint; the player who cannot move loses.

12 tiles initially, partitioned into 12 height-1 stacks.
4 colors: Y / W / G / B; exactly 3 tiles per color.
Two players alternate moves.
A move = pick two stacks and place one entirely on top of the other (merge).

Legal iff at least one of: (a) the two stacks have the same height, OR (b) their top-most tiles share the same color.
After merge: new height = sum of the two heights, new top color = color of the stack placed on top.

The player who has no legal move on their turn loses.
Implement the full game so it can be played out (legal-move detection, state transition, terminal check).

EXAMPLES

Start: 12 stacks of height 1, with tops Y Y Y W W W G G G B B B. A legal opening move: stack the first Y on the second Y (same color top), producing one height-2 stack of color Y plus 10 singletons.

MAIN CODE | helper functions first, then the answer

```python
import itertools


# HELPERS for the base question
class Babylon:
    # a stack is (height, colour of its top tile)
    def __init__(self, stacks=None):
        if stacks is None:
            # 12 tiles: Y, W, G, B three times each
            stacks = [(1, c) for c in "YWGB" * 3]
        self.stacks = list(stacks)

    def legal_moves(self):
        # put stack i on top of stack j
        moves = []
        for i, j in itertools.permutations(
                range(len(self.stacks)), 2):
            h1, c1 = self.stacks[i]
            h2, c2 = self.stacks[j]
            # same height, or the same colour on top
            if h1 == h2 or c1 == c2:
                moves.append((i, j))
        return moves

    def after(self, i, j):
        # the stacks after the move (the original is kept)
        (h1, c1), (h2, _) = self.stacks[i], self.stacks[j]
        rest = [s for k, s in enumerate(self.stacks)
                if k not in (i, j)]
        # heights add; the top colour is the moved stack's
        return rest + [(h1 + h2, c1)]

    def play(self, i, j):
        if (i, j) not in self.legal_moves():
            raise ValueError("illegal move")
        self.stacks = self.after(i, j)

    def is_over(self):
```

```
        # the player with no move loses
        return not self.legal_moves()

# BASE QUESTION (optimal): Negamax with memoisation on the
# sorted stack tuple
# Idea: legal_moves lists the ordered pairs that may
# merge, apply builds the next state, and wins(state) is
```

```
# true when some move reaches a state where the opponent
# loses.
def wins(stacks):
    # does the player to move win with best play?
    cache = {}

    def solve(state):
        if state in cache:
            return cache[state]
        game = Babylon(state)
        result = False
        for i, j in game.legal_moves():
            nxt = tuple(sorted(game.after(i, j)))
            # a move to a position the opponent loses wins
            if not solve(nxt):
                result = True
                break
        cache[state] = result
        return result

    # sorted stacks: equal positions share one cache entry
    return solve(tuple(sorted(stacks)))
```

GROUP 40 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 4 M 1 H

Sliding window: grow on the right, shrink on the left while it is too bad

269 Temperature Monitoring Data Structure — MEDIUM

Template fit: a deque of the last n readings with a running total; a second deque of decreasing values gives the maximum

QUESTION

Implement temperature insertion, a moving average, and a constant-time maximum query for a recent window. Clarify whether the window is based on reading count or elapsed days.SWEdata-structurestreamingsliding-windowFrequencySingle reportLast asked2026-09-10Stageonsite-coding

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# HELPERS for the base question
class MonotoneQueue:
    """Best window value is kept at the front."""

    def __init__(self, better):
        self.items = deque()
        # better(a, b): a is at least as good as b
        self.better = better

    def push(self, key, value):
        # values the new one beats can never be best
        while self.items and self.better(
                value, self.items[-1][1]):
            self.items.pop()
        self.items.append((key, value))

    def drop_while(self, too_old):
        # forget entries that left the window
        while self.items and too_old(self.items[0][0]):
            self.items.popleft()

    def best(self):
        return self.items[0][1]

class SlidingStats:
    def __init__(self):
        self.items = deque()
        self.total = 0
        self.maxq = MonotoneQueue(lambda a, b: a >= b)
        self.minq = MonotoneQueue(lambda a, b: a <= b)

    def add(self, key, value):
        self.items.append((key, value))
        self.total += value
        self.maxq.push(key, value)
        self.minq.push(key, value)

    def expire(self, too_old):
        # drop readings that left the window everywhere
        while self.items and too_old(self.items[0][0]):
            self.total -= self.items.popleft()[1]
        self.maxq.drop_while(too_old)
```

```
        self.minq.drop_while(too_old)
```

```
    def average(self):
        return self.total / len(self.items)

    def maximum(self):
        return self.maxq.best()

    def minimum(self):
        return self.minq.best()

# BASE QUESTION (optimal): Running sum plus a monotonic
# deque
# Idea: Keep the window in a deque and a running sum; on
# insert add the value and drop the oldest when the window
# is full.
class Monitor(SlidingStats):
    """Window = the last n readings."""

    def __init__(self, n):
        super().__init__()
        self.n, self.seen = n, 0

    def insert(self, x):
        # the reading number is its key
        self.add(self.seen, x)
        self.seen += 1
        # keep only the newest n readings
        limit = self.seen - self.n
        self.expire(lambda key: key < limit)
```

270 Shortest Subarray with K Distinct Integers

MEDIUM

Template fit: state = a count dict; while it holds at least k distinct values, record the window and shrink from the left

QUESTION

Given an array nums and an integer k, find the length of the shortest contiguous subarray that contains at least k distinct integers. If no such subarray exists, return -1.

EXAMPLES

Example 1	
Input	`nums = [1, 2, 1, 2, 3], k = 3`
Output	`3`
Why	`The shortest contiguous subarray with 3 distinct integers is [1, 2, 3].`

Example 2	
Input	`nums = [1, 1, 1, 1], k = 2`
Output	`-1`
Why	`Only 1 distinct integer exists.`

Example 3	
Input	`nums = [1, 2, 3, 4, 5], k = 3`
Output	`3`
Why	`Any 3 consecutive elements give 3 distinct integers.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= nums[i] <= 10^9

QUESTION

- 1 <= k <= nums.length

MAIN CODE | helper functions first, then the answer

```
from collections import Counter


# BASE QUESTION (optimal): Sliding window with a frequency
# map
# Idea: Expand the right end, counting each value. While
# the window has at least k distinct values, record its
# length and shrink from the left (deleting a value from
# the map when its count reaches 0).
def shortest_k_distinct(nums, k):
    # a window grows on the right; whenever it holds at
    # least k distinct values, record it and shrink from
    # the
    # left as far as it stays valid
    seen, left, best = Counter(), 0, None
    for right, x in enumerate(nums):
        seen[x] += 1
        while len(seen) >= k:
            size = right - left + 1
            if best is None or size < best:
                best = size
            seen[nums[left]] -= 1
            if seen[nums[left]] == 0:
                del seen[nums[left]]
            left += 1
    return -1 if best is None else best
```

271 Find the minimum window substring. MEDIUM

Template fit: state = how many of each needed letter are still missing; while nothing is missing, record the window and shrink

QUESTION

Find the minimum window substring.

MAIN CODE | helper functions first, then the answer

```
from collections import Counter


# HELPERS for the base question
def valid_windows(s, t):
    # sliding window: grow `right` until the window holds
    # all
    # of t, then shrink `left` while it still does. Yield
    # every (left, right) that is valid when it is seen.
    need = Counter(t)
    missing = len(t)
    left = 0
    for right, ch in enumerate(s):
        if need[ch] > 0:
            missing -= 1
        need[ch] -= 1
        while missing == 0:
            yield left, right
            # drop the leftmost character and see if it
            # hurts
            need[s[left]] += 1
            if need[s[left]] > 0:
                missing += 1
```

```
            left += 1

# BASE QUESTION (optimal): Sliding window with a 'still
# needed' counter
# Idea: need[c] is how many more of c the window requires,
# and missing counts the distinct requirements still
# unmet.
def min_window(s, t):
    # the shortest window; the first one if there is a tie
    best, width = None, 0
    for left, right in valid_windows(s, t):
        if best is None or right - left < width:
            best, width = (left, right), right - left
    return s[best[0]:best[1] + 1] if best else ""
```

272 Footpath Beauty After Repainting

MEDIUM

Template fit: bad = a block that is not the wanted colour; allowed = the repaint budget m; the longest window is the answer

QUESTION

A footpath is built from n blocks laid in a line, painted with one of k colors. block_colors[i] is the color of block i (colors are integers in 1..k). The beauty value for a given favorite color c is the length of the longest run of consecutive blocks that are all color c . You are allowed to repaint at most m blocks (each to any color you choose). Return the maximum possible beauty value for favorite color c after repainting at most m blocks.

Function signature

def max_beauty_after_repaint(block_colors: list[int], c: int, m: int) -> int:

    ...

Follow-up (as reported): (Reported) Answer q queries, each with a different favorite color, and return the beauty value (with m fixed) for each. Hint: a sliding window per query, or precompute runs. (Reported) The base version with m = 0 was asked first; the repaint version is the twist.

EXAMPLES

Example 1	
Input	`block_colors = [1, 2, 1, 1, 2, 1], c = 1, m = 1`
Output	`4`
Why	`Repaint one of the 2's. The best window has at most one non-c block, e.g. indices 2..5 [1,1,2,1] -> repaint the 2 -> 4.`

Example 2	
Input	`block_colors = [3, 3, 3], c = 3, m = 0`
Output	`3`
Why	`All blocks are already color 3; no repaint needed.`

Example 3	
Input	`block_colors = [2, 2, 2], c = 1, m = 2`
Output	`2`
Why	`Only 2 repaints allowed, so at most 2 consecutive c blocks.`

CONSTRAINTS

- 1 <= n <= 10^5 , block_colors has length n
- 1 <= k , 1 <= c <= k , 1 <= block_colors[i] <= k
- 0 <= m <= n

QUESTION

- With m = 0 , the answer is simply the longest existing run of color c .

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Sliding window with at most m
# repaints
# Idea: The longest window holding at most m other-colour
# blocks is the answer.
def max_beauty_after_repaint(block_colors, c, m):
    left, other, best = 0, 0, 0
    for right, color in enumerate(block_colors):
        other += color != c
        while other > m:
            other -= block_colors[left] != c
            left += 1
        best = max(best, right - left + 1)
    return best
```

273 Necklace Cut into Two Halves with Equal D/R Counts

HARD

Template fit: a fixed-size window of half the length slides around the doubled string, keeping a count of D

QUESTION

EXAMPLES

```
s = "DRDR"    -> cut between index 1 and 2: "DR" + "DR".
s = "DDRR"    -> 1-cut not balanced ("DD" vs "RR"); rotate, two cuts at offsets 1 and 3:
                 pieces "DR" and "DR".
```

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Fixed-length circular window
# with the discrete intermediate value argument
# Idea: If each arc has half of the D's and half of the
# R's, each arc has length n/2, and the total D and R
# counts must both be even. Slide a window of length n/2
# around the circle (use s + s), tracking its D count.
def necklace_two_cuts(s):
    n = len(s)
    d_total, r_total = s.count("D"), s.count("R")
    # each piece needs half of both letters
    if n % 2 or d_total % 2 or r_total % 2:
        return None
    half, target = n // 2, d_total // 2
    doubled = s + s
    # slide a window of length n/2 around the circle
    window = doubled[:half].count("D")
    for i in range(n):
        if window == target:
            # cuts at i and i + n/2 give two good halves
            first = doubled[i:i + half]
            second = doubled[i + half:i + n]
            return i, (i + half) % n, first, second
        # move the window one step: add the new, drop the
        # old
        window += (doubled[i + half] == "D")
        window -= (doubled[i] == "D")
    return None
```

GROUP 41 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 1 E 4 M

Parent pointers: store who is above each node and climb

274 Tree Node. EASY

Template fit: the SQL version of the same idea: p_id is the parent pointer, NULL means root, being someone's parent means inner, else leaf

QUESTION

Given a binary tree, identify which nodes are leaf nodes, root nodes or inner nodes. Leaf nodes: A node with no children Root nodes: A node with no parent Inner nodes: A node with both children and parent nodes You are given the following table tree_node_table: idp_idintegerinteger where p_id is the id of the parent node. Your output should contain the following columns: id, node_types ('Root', 'Inner', 'Leaf').No answers contributed by the community yet.Interview experiences7 shared‹›Amazon · 5 months ago"It went well and answered all questions. and did both leetcode questions asked one in each interview"Software Engineer · InternGoogle · 6 months ago"Pretty easy, just very long. For technical just need to do leetcode, no system design. Interviewers help you if you are stuck."Software Engineer · Entry Level / L3Microsoft · 6 months ago"What went well: The live coding and system architecture rounds went smoothly. I was able to articulate my design choices clearly, especially around API design, database strategy, and maintaining high availability for microservices. What didn't go well: The initial behavioral portion felt slightly rushed because we spent extra time discussing specific cross-functional project dynamics from my previous roles. How does this interview compare to other interviews you've done? It was significantly more interactive. Rather than giving static answers, the interviewers actively challenged my assumptions and asked "what if" scenarios, making the evaluat

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): One CASE expression
# Idea: CASE tests in order: no parent means Root, an id
# that appears as someone's parent means Inner, anything
# else is a Leaf.
# Root: no parent. Inner: has a parent and a child.
# Leaf: has a parent and no child. The Root test comes
# first, so a one-node tree is a Root, not a Leaf.
TYPES_SQL = """
SELECT t.id,
  CASE
    WHEN t.p_id IS NULL THEN 'Root'
    WHEN EXISTS (SELECT 1 FROM tree_node_table c
                 WHERE c.p_id = t.id) THEN 'Inner'
    ELSE 'Leaf'
  END AS node_types
FROM tree_node_table t
ORDER BY t.id
"""
```

275 Employee Hierarchy Add / Delete MEDIUM

Template fit: manager[emp] and a set of reports per manager; delete re-attaches the reports to the deleted person's manager

QUESTION

Design an employee hierarchy class that supports adding employees under managers and deleting employees by id. Follow-ups cover deleting managers, deleting the CEO, cycle handling, and choosing a replacement manager by seniority.SWEtreehashmapobject-designmediumFrequencySingle reportLast asked2026-06-10Stageonsite-coding

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Parent map plus children sets
# Idea: Keep manager[emp] and reports[emp] (a set). Add
```

```
# inserts into both. Delete reattaches each report to the
# deleted node's manager and removes the node from its
# manager's set.
class Org:
    def __init__(self, ceo):
        self.root = ceo
        self.manager = {ceo: None}
        self.reports = {ceo: set()}
        # hire order, used for "most senior" choices
        self.hired = {ceo: 0}
        self.clock = 1

    def add(self, emp, mgr):
        if emp in self.manager or mgr not in self.manager:
            return False
        self.manager[emp] = mgr
        self.reports[emp] = set()
        self.reports[mgr].add(emp)
        self.hired[emp] = self.clock
        self.clock += 1
        return True

    def senior(self, people):
        # the person hired earliest
        return min(people, key=lambda p: self.hired[p])

    def delete(self, emp, policy="grandparent"):
        if emp not in self.manager:
            return False
        mgr = self.manager[emp]
        kids = set(self.reports[emp])
        if mgr is None and not kids:
            # the CEO alone: nothing could take over
            return False
        if policy == "senior" or mgr is None:
            # the most senior report takes the place of
            # emp
            heir = self.senior(kids) if kids else None
        else:
            heir = mgr
        for r in kids:
            if r != heir:
                self.reassign(r, heir)
```

```
        if mgr is None:
            # the promoted report becomes the new CEO
            self.root = heir
            self.manager[heir] = None
        elif policy == "senior" and heir != mgr:
            self.reassign(heir, mgr)
        if mgr is not None:
            self.reports[mgr].discard(emp)
        del self.manager[emp], self.reports[emp]
        return True

    def reassign(self, emp, new_mgr):
        old = self.manager[emp]
        if old is not None and old in self.reports:
            self.reports[old].discard(emp)
        self.manager[emp] = new_mgr
        self.reports[new_mgr].add(emp)
```

276 Validate parent array forms a tree

MEDIUM

Template fit: climb from every node with three states (unseen, on this walk, verified); meeting your own walk is a cycle; there must be exactly one root

QUESTION

Problem

You are given an integer array **`parent`** of length **`n`** describing a directed parent pointer for each node **`i`** (nodes are labeled **`0..n-1`**).

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these **`n`** nodes form a **valid rooted tree**. A valid rooted tree must satisfy all of the following: 1.

Exactly one root (exactly one index **`i`** with **`parent[i] = -1`**). 2.
No cycles (following parent pointers from any node must eventually reach the root). 3.
Connectivity (every node is reachable from the root; equivalently, every node has exactly one simple path to the root).

Input

- **`parent`**: integer array of length **`n`** where each value is either **`-1`** or in **`[0, n-1]`**.

Output

- Return **`true`** if **`parent`** represents a valid rooted tree; otherwise return **`false`**.

Notes

/ Edge Cases - **`n`** can be 1 (then the only valid tree is **`parent[0] = -1`**).

- Self-parenting like **`parent[i] = i`** is invalid.
- Multiple **`-1`** entries (multiple roots) is invalid.
- A cycle among non-root nodes is invalid.
- A disconnected component (some nodes not reachable from the root) is invalid.

You are given an integer array **`parent`** of length **`n`** describing a parent pointer for each node **`i`** where nodes are labeled **`0`** to **`n-1`**.

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these nodes form a valid rooted tree. A valid rooted tree must satisfy all of the following:

1. Exactly one root.
2. No cycles.
3. Every node belongs to the same connected structure and has a path to the root. Return **`True`** if **`parent`** represents a valid rooted tree; otherwise return **`False`**.

Constraints
1 <= n <= 2 * 10^5Each **`parent[i]`** is either **`-1`** or an integer in **`[0, n-1]parent[i] = i`** is invalid

EXAMPLES

```
Input: [-1, 0, 0, 1, 1]
Expected Output: True
```

Explanation: Node 0 is the only root. All other nodes are connected under it, and there are no cycles.

```
Input: [2, -1, 1, 2]
Expected Output: True
Explanation: Node 1 is the root. The paths are 0->2->1, 2->1, and 3->2->1, so every node reaches the
    same root.
```

QUESTION

Hints

First count how many roots there are. A valid tree must have exactly one **-1** entry.Build children lists from the parent array, then traverse from the root. If you do not visit every node, the structure is disconnected or contains a cycle away from the root.

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
def reason(parent):
    # None if valid, else the first rule that fails
    n = len(parent)
    roots = parent.count(-1)
    if roots != 1:
        return f"{roots} roots (need exactly 1)"
    for i, p in enumerate(parent):
        if p != -1 and not 0 <= p < n:
            return f"node {i}: out-of-range parent {p}"
        if p == i:
            return f"node {i} is its own parent"
    # 0 = unseen, 1 = on the current walk, 2 = checked
    state = [0] * n
    for start in range(n):
        path, x = [], start
        while x != -1 and state[x] == 0:
            state[x] = 1
            path.append(x)
            x = parent[x]
        # meeting a node twice on one walk is a cycle
        if x != -1 and state[x] == 1:
            return f"cycle through node {x}"
        for v in path:
            state[v] = 2
    return None

# BASE QUESTION (optimal): One root, then colour each node
# once while walking up
# Idea: Check there is exactly one -1 and all other values
# are in range. Then walk up from each unvisited node
# marking nodes as 'on this walk'.
def validate_parent_array(parent):
    return reason(parent) is None
```

277 Lowest Common Ancestor of a Binary Tree III — MEDIUM

Template fit: two pointers climb; when one falls off the top it restarts at the other node, so both meet at the common ancestor

QUESTION

Given two nodes of a binary tree p and q, return their lowest common ancestor (LCA). Each node has a parent pointer.

EXAMPLES

Example 1	
Input	`root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1`
Output	`3`
Why	`The LCA of nodes 5 and 1 is 3.`

QUESTION

Example 2	
Input	`root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4`
Output	`5`
Why	`The LCA of nodes 5 and 4 is 5.`

Example 3	
Input	`root = [1,2], p = 1, q = 2`
Output	`1`
Why	`Node 1 is ancestor of node 2.`

CONSTRAINTS

- Number of nodes: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique
- p != q
- p and q exist in the tree

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
def lca_set(p, q):
    seen = set()
    while p:
        seen.add(id(p))
        p = p.parent
    while q:
        if id(q) in seen:
            return q
        q = q.parent
    return None

# BASE QUESTION (optimal): Two pointers that switch to the
# other node at the end
# Idea: Move a from p and b from q one step up per round.
# When a pointer falls off the root, restart it at the
# other node.
def lca_swap(p, q):
    # two pointers climb to the root; at the top each one
    # jumps to the other node's start. Both walk the same
    # total distance, so they meet at the LCA (or both
    # reach
    # None together when the nodes are in different
    # trees).
    a, b = p, q
    while a is not b:
        a = a.parent if a else q
        b = b.parent if b else p
    return a
```

278 Root a Tree So It Becomes a Binary Tree — MEDIUM

Template fit: the root has no parent so it needs degree <= 2; everyone else spends one edge on its parent; answer = the first node of degree <= 2

QUESTION

You are given a connected, undirected, acyclic graph (i.e. a tree) with n nodes labeled 0 .. n-1 , described by an edge list. It is guaranteed that every node has at most 3 neighbors. Choose a node to be the root. When rooted at node r , one neighbor of each non-root node becomes its parent and the rest become its children; the root has no parent, so all of its neighbors become children. The rooted tree is a binary tree if every node has at most 2 children. Return the smallest label of a node that can serve as such a root. If no node works, return -1 . Key observation (the trick): a non-root node with degree d has d - 1 children, so it needs d <= 3 ; the root has degree children, so the root needs degree <= 2 . Since all nodes already have degree <= 3 , any node with degree <= 2 is a valid root — and leaves (degree 1) always exist in a nonempty tree, so a valid root always exists for n >= 1 .

Function signature

```
from typing import List, Tuple
def find_binary_tree_root(n: int, edges: List[Tuple[int, int]]) -> int:
    """Return the smallest node label that can root the tree as a binary tree
    (every node has at most 2 children), or -1 if none exists."""
    ...
```

Follow-up (as reported): Now nodes must be 2-colored (black/white) so that every parent differs in color from its children and all nodes at the same depth share a color. Determine whether a valid root plus coloring exists and return one. Hint: coloring each node by depth % 2 from any chosen root always satisfies both rules in a tree, so a valid coloring exists whenever a valid root does.

EXAMPLES

Example 1	
Input	`n = 4, edges = [(0, 1), (0, 2), (0, 3)]`
Output	`1`
Why	`Node 0 has degree 3, so as root it would have 3 children (invalid). Nodes 1, 2, 3 have degree 1; the smallest valid root is 1.`

Example 2	
Input	`n = 3, edges = [(0, 1), (1, 2)]`
Output	`0`
Why	`A path; every node has degree <= 2, so the smallest label 0 works.`

Example 3	
Input	`n = 1, edges = []`
Output	`0`
Why	`A single node is trivially a binary tree.`

CONSTRAINTS

- 1 <= n <= 10^5 ; the input is a valid tree with n - 1 edges (for n == 1 , no edges).
- Every node has at most 3 neighbors.
- Return the smallest valid root label.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): First node of degree at most 2
# Idea: Only the root has as many children as its degree;
# every other node has one fewer.
def find_binary_tree_root(n, edges):
```

```
    degree = [0] * n
    for a, b in edges:
        degree[a] += 1
        degree[b] += 1
    for node in range(n):
        if degree[node] <= 2:
            return node
    return -1
```

Build or transform a grid with index arithmetic

279 Filled ASCII Diamond — EASY

*Template fit: row i has pad = |i - R| underscores on each side and 2 * (R - pad) + 1 x's in the middle*

QUESTION

Given a non-negative integer radius R , render a filled diamond on a square grid of side 2*R + 1 . The grid is indexed by (i, j) with 0 <= i, j <= 2*R , and its center is (R, R) . A cell belongs to the diamond (draw 'x') if its Manhattan distance from the center is at most R , i.e. abs(i - R) + abs(j - R) <= R . Every other cell is background (draw '_'). Return the picture as a list of 2*R + 1 strings, one per row (top to bottom), each of length 2*R + 1 .

Function signature

def render_diamond(R: int) -> list[str]:

    ...

Follow-up (as reported): The naive fill is O(R^2). Can you produce each row directly from its width instead of testing every cell? (Row i is 'x' repeated 2*(R - abs(i-R)) + 1 times, padded with '_' .)

EXAMPLES

Example 1	
Input	`R = 1`
Output	`["_x_", "xxx", "_x_"]`
Why	`Center (1,1). Row 0 has distance 1 from center vertically, so only column 1 (distance 0) is inside.`

Example 2	
Input	`R = 2`
Output	`["__x__", "_xxx_", "xxxxx", "_xxx_", "__x__"]`
Why	`Each row i allows columns within (R - abs(i-R)) of the center.`

Example 3	
Input	`R = 0`
Output	`["x"]`
Why	`A 1x1 grid whose only cell is the center.`

CONSTRAINTS

- 0 <= R <= 500 .
- For R = 0 the result is the single cell ["x"] .

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Build each row from its width
# Idea: Row i has 2 * (R - |i - R|) + 1 x's with |i - R|
# underscores on each side.
def render_diamond(R):
    rows = []
    for i in range(2 * R + 1):
        pad = abs(i - R)
        rows.append(
            "_" * pad
            + "x" * (2 * (R - pad) + 1)
            + "_" * pad
        )
```

```
    return rows
```

280 Connected Crop Arrangement on a Grid — MEDIUM

Template fit: fill the cells in snake order (left to right, then right to left) so equal crops stay in one connected strip

QUESTION

Given an N x M grid and K crop counts summing to N*M, fill every cell so each crop's cells form one 4-connected region; return any valid arrangement. Follow-up: two adjacent square gardens (N x N and M x M) with counts summing to N^2 + M^2.

A garden is an N x M grid. There are K crop types labeled 1..K; crop i must occupy exactly count[i] cells.
Guaranteed: sum(count) = N * M, so every cell is used.
For every crop type, all of its cells must form one connected component under 4-directional adjacency (up, down, left, right; diagonals do not count).
Different crops have no relationship constraints with each other.
Return any valid arrangement.

Follow-up: two square gardens

Instead of one rectangle, there are two square gardens placed directly next to each other: garden A is N x N, garden B is M x M; together they form the planting area.
Now sum(count) = N^2 + M^2. The same single-connected-region rule applies across the combined shape.

EXAMPLES

Follow-up shapes for N = 2, M = 3 (digits mark garden A = 1, garden B = 2):

```
11222
11222
  222
```

or

```
11
11
222
222
222
```

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
def snake_cells(n, m):
    # row 0 left to right, row 1 right to left, ...
    return [(i, j if i % 2 == 0 else m - 1 - j)
            for i in range(n) for j in range(m)]

def fill_along(grid, path, counts):
    # hand out the path to crop 1, crop 2, ... in turn
    k = 0
    for crop, cnt in enumerate(counts, 1):
        for _ in range(cnt):
            r, c = path[k]
            grid[r][c] = crop
            k += 1
    return grid

# BASE QUESTION (optimal): Fill crops along a snake path
# Idea: Generate the snake order of the cells and hand out
# count[0] cells to crop 1, the next count[1] to crop 2,
# and so on. Consecutive cells of the snake share an edge,
# so each crop is connected.
```

```
def arrange_crops(n, m, counts):
    # every cell is used exactly once
    assert sum(counts) == n * m
    grid = [[0] * m for _ in range(n)]
    # touching path cells keep each crop connected
    return fill_along(grid, snake_cells(n, m), counts)
```

281 Rotate Any m×n 2D Matrix

MEDIUM

*Template fit: clockwise: new[r][c] = old[rows - 1 - c][r] on a cols-by-rows result (or zip(*matrix[::-1]))*

QUESTION

EXAMPLES

```
[[1,2,3],[4,5,6]] (2×3) rotated 90° CW -> [[4,1],[5,2],[6,3]] (3×2).
```

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Rotate 90 degrees clockwise by
# reverse + transpose
# Idea: Rotating clockwise puts the last row first, so
# reverse the rows and transpose (zip with *). A non-
# square matrix changes shape from m x n to n x m, which
# is why it cannot be done in place the way LeetCode 48
# is.
def rotate_cw(matrix):
    # reverse the rows, then swap rows and columns
    return [list(row) for row in zip(*matrix[::-1])]
```

282 Solve matrix groups and recipe inventory

MEDIUM

Template fit: block (r // k, c // k) holds the minimum of every cell that maps to it; the recipe part is a topological sort

QUESTION

Overview: This pair of problems evaluates block-wise grid processing and aggregation for minimum-value selection alongside dependency-resolution and resource-allocation reasoning for recipe inventory planning, testing competencies in array manipulation, tie-breaking and constraint handling, graph-based dependency management and cycle detection, and algorithmic complexity analysis; they are commonly asked to gauge an applicant's ability to implement correct, efficient solutions under input-size and edge-case constraints. They belong to the Coding & Algorithms domain with subdomains in data structures (arrays/grids), algorithm design and analysis, and graph theory, and primarily require practical application of algorithmic techniques with attention to time/space complexity rather than purely theoretical proofs.

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Blocks: one pass updating a
# minimum per block
# Idea: Each cell updates the minimum of block (r // k, c
# // k). No block list is built; the last blocks may be
# smaller when k does not divide the size.
def block_mins(matrix, k):
    # the minimum of every k x k block; edge blocks may be
    # smaller, which r // k and c // k handle by
    # themselves
    m, n = len(matrix), len(matrix[0])
    rows, cols = (m + k - 1) // k, (n + k - 1) // k
    out = [[float("inf")] * cols for _ in range(rows)]
    for r in range(m):
        for c in range(n):
```

```
            cell = out[r // k][c // k]
            out[r // k][c // k] = min(cell, matrix[r][c])
    return out
```

283 Range Sum Query 2D (Immutable Matrix) MEDIUM

Template fit: prefix table p[r + 1][c + 1] = cell + p[r][c + 1] + p[r + 1][c] - p[r][c]; a query is four lookups

QUESTION

Given a 2D integer matrix, handle multiple queries of the following type: return the sum of the elements inside the rectangle defined by its upper-left corner (row1, col1) and lower-right corner (row2, col2) , inclusive. Implement a class NumMatrix that is initialized once with the matrix and can then answer many sum_region queries efficiently (each query in O(1) after preprocessing).

Function signature

class NumMatrix:
    def __init__(self, matrix: list[list[int]]) -> None:
        """Preprocess the matrix (e.g., build a 2D prefix-sum table)."""
        ...
    def sum_region(self, row1: int, col1: int, row2: int, col2: int) -> int:
        """Return the sum of the submatrix with corners (row1, col1) and (row2, col2),
inclusive."""
        ...

Follow-up (as reported): Follow-up constraint change from the report: what if the matrix is mutable (values can be updated between queries)? Hint: switch to a 2D Binary Indexed Tree (Fenwick) for O(log rows * log cols) updates and queries.

EXAMPLES

Example 1	
Input	`matrix = [[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]] sum_region(2, 1, 4, 3)`
Output	`8`
Why	`The 3x3 block rows 2..4, cols 1..3 is [[2,0,1],[1,0,1],[0,3,0]] which sums to 8.`

Example 2	
Input	`same matrix; sum_region(1, 1, 2, 2)`
Output	`11`
Why	`Block [[6,3],[2,0]] sums to 11.`

Example 3	
Input	`same matrix; sum_region(1, 2, 2, 4)`
Output	`12`
Why	`Block [[3,2,1],[0,1,5]] sums to 12.`

CONSTRAINTS

- 1 <= rows, cols <= 200
- -10^5 <= matrix[i][j] <= 10^5
- 0 <= row1 <= row2 < rows , 0 <= col1 <= col2 < cols
- There may be up to 10^4 calls to sum_region .

MAIN CODE | helper functions first, then the answer

```python
# BASE QUESTION (optimal): 2D prefix sums and inclusion-
# exclusion
# Idea: P holds the sum of the rectangle from the top-left
# corner; a query combines four entries.
class NumMatrix:
    def __init__(self, matrix):
        rows, cols = len(matrix), (
            len(matrix[0]) if matrix else 0
        )
        self.p = [
            [0] * (cols + 1) for _ in range(rows + 1)
        ]
        for r in range(rows):
            for c in range(cols):
                # the cell plus the block above and the
                # block to the left,
                # minus the corner block that was counted
                # twice
                self.p[r + 1][c + 1] = (
                    matrix[r][c]
                    + self.p[r][c + 1]
                    + self.p[r + 1][c]
                    - self.p[r][c]
                )

    def sum_region(self, r1, c1, r2, c2):
        p = self.p
        return (
            p[r2 + 1][c2 + 1]
            - p[r1][c2 + 1]
            - p[r2 + 1][c1]
            + p[r1][c1]
        )
```

Split into pieces, fix each piece, join (and pack words into lines)

284 Implement string formatting and request tracking tasks — EASY

Template fit: split, fix each piece, join; the request-tracking part is a small dict

QUESTION

Overview: This multi-part question evaluates string manipulation, numeric formatting, character-mapping validation under 180° rotation, and event-tracking/order management competencies, assessing parsing, edge-case handling, mapping validation, and design of data structures for ordered output.

MAIN CODE | helper functions first, then the answer

```
from collections import deque


# HELPERS for the base question
ROTATE = {"0": "0", "1": "1", "6": "9",
          "8": "8", "9": "6"}

# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
def snake_to_camel(s):
    # foo_bar -> fooBar (empty parts are ignored)
    words = [w for w in s.split("_") if w]
    if not words:
        return ""
    rest = [w.capitalize() for w in words[1:]]
    return words[0] + "".join(rest)

# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
def format_number(n, sep=",", sizes=(3,)):
    sign = "-" if n < 0 else ""
    digits = str(abs(n))
    parts, i = [], 0
    while len(digits) > sizes[min(i, len(sizes) - 1)]:
        size = sizes[min(i, len(sizes) - 1)]
        # peel one group off the right end
        parts.append(digits[-size:])
        digits = digits[:-size]
        i += 1
    parts.append(digits)
    return sign + sep.join(reversed(parts))

# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
def is_rotation_valid(s, mapping=ROTATE):
    # does the number read the same upside down (180
```

```
    # degrees)?
    # a character not in the map makes it invalid
```

```
    if any(ch not in mapping for ch in s):
        return False
    # rotate every digit and read the result backwards
    turned = "".join(mapping[ch] for ch in reversed(s))
    return turned == s

# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
class RequestTracker:
    """begin(id, t) and end(id, t). Finished requests are
    printed in the order they BEGAN: one is printed only
    when everything that began before it is done."""

    def __init__(self):
        self.order = deque()
        self.begin_at, self.end_at = {}, {}

    def begin(self, rid, t):
        if rid in self.begin_at:
            raise ValueError("duplicate begin")
        self.begin_at[rid] = t
        self.order.append(rid)

    def end(self, rid, t):
        if rid not in self.begin_at:
            raise ValueError("end without begin")
        if t < self.begin_at[rid]:
            raise ValueError("ends before it begins")
        self.end_at[rid] = t
        # print from the front while the oldest are
        # finished
        out = []
        while self.order and self.order[0] in self.end_at:
            first = self.order.popleft()
            out.append((first, self.begin_at[first],
                        self.end_at[first]))
        return out
```

285 Snake Case to Camel Case with Repository-Wide Refactor

MEDIUM

Template fit: the same function applied to every identifier found in the files

QUESTION

Convert a valid snake_case identifier to camelCase, then design a safe repository-wide migration for variables and function names without changing third-party symbols, public interfaces, or the tests' ability to detect regressions.

Implement a conversion from a valid snake_case identifier to camelCase.
The base input is guaranteed valid: no leading underscore, trailing underscore, or doubled separator.
Example: foo_bar -> fooBar.
Extend the discussion to a repository too large for manual review: replace snake-case variables and functions across the codebase.
Explain how to verify that the repository still works after the automated rewrite, including when test-package identifiers are rewritten too.
Preserve third-party functions and public interfaces; explain how the migration distinguishes symbols that may change from contracts that must remain stable.

EXAMPLES

```
foo_bar -> fooBar
```

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Convert names, rename only
# defined identifiers via tokens
# Idea: snake_to_camel capitalises each part after the
# first. owned_names collects the functions, arguments and
# variables the code defines (using the syntax tree),
# minus an allow-list of names that must stay.
def snake_to_camel(name):
    # foo_bar -> fooBar (input is a valid snake_case name)
    head, *rest = name.split("_")
    caps = [p[:1].upper() + p[1:] for p in rest]
    return head + "".join(caps)
```

286 Google Doc Line Wrap (Word Wrap with Whitespace and Newlines) MEDIUM

Template fit: count lines: go paragraph by paragraph, add a word if it fits (current + 1 + length <= width), else start a new line

QUESTION

EXAMPLES

```
text  = "Hello world\nFoo bar baz qux"
width = 10
Line 1: "Hello"             (5 chars; "world" + space would be 11 -> break)
Line 2: "world"
Line 3: "Foo bar"            (7 chars; "baz" -> "Foo bar baz" = 11 -> break)
Line 4: "baz qux"
Output: 4
```

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
def wrap_lines(text, width, break_long=False):
    lines = []
    # a newline always starts a new paragraph (and a new
    # line)
    for paragraph in text.split("\n"):
        cur = ""
        words = paragraph.split()
        if break_long:
            pieces = [split_long(w, width) for w in words]
            words = [p for chunk in pieces for p in chunk]
        for w in words:
            if not cur:
                cur = w
            elif len(cur) + 1 + len(w) <= width:
                # the word and one space still fit
                cur += " " + w
            else:
                lines.append(cur)
                cur = w
        # the last (or only, or empty) line of the
        # paragraph
        lines.append(cur)
    return lines

def split_long(word, width):
    # a word longer than the line is cut into pieces
    starts = range(0, len(word), width)
    return [word[i:i + width] for i in starts]

# BASE QUESTION (optimal): Greedy with a running line
# length
# Idea: Split on newlines first (each paragraph ends its
# line), then for each word compare cur_len + 1 +
```

```
# len(word) with the width. Only the length is tracked,
# never the joined line.
def wrap_count(text, width):
    return len(wrap_lines(text, width))
```

287 Implement a function to justify a list of words within a given width. HARD

Template fit: pack_lines for the words, then spread the extra spaces over the gaps (the left gaps get the remainder); the last line is left-aligned

QUESTION

Implement a function to justify a list of words within a given width.

MAIN CODE | helper functions first, then the answer

```
# HELPERS for the base question
def left_align(line, width, measure=len):
    text = " ".join(line)
    return text + " " * (width - measure(text))

def center(line, width, measure=len):
    # leftover spaces split on both sides, the odd one
    # right
    text = " ".join(line)
    spare = width - measure(text)
    left = spare // 2
    return " " * left + text + " " * (spare - left)

def pack_greedy(words, width, measure=len):
    # put as many words as fit on every line
    lines, line, used = [], [], 0
    for w in words:
        size = measure(w)
        # `used` counts the words and one space between
        # each
        if line and used + 1 + size > width:
            lines.append(line)
            line, used = [], 0
        used += size + (1 if line else 0)
        line.append(w)
    if line:
        lines.append(line)
    return lines

def spread(line, width, measure=len):
    # fully justified: spaces between the words, extra
    # ones
    # go to the LEFT gaps first. One word stays left-
    # aligned.
    if len(line) == 1:
        return line[0] + " " * (width - measure(line[0]))
    spaces = width - sum(measure(w) for w in line)
    gaps = len(line) - 1
    each, extra = divmod(spaces, gaps)
    out = ""
    for i, w in enumerate(line[:-1]):
        out += w + " " * (each + (1 if i < extra else 0))
    return out + line[-1]

# BASE QUESTION (optimal): Greedy line packing
```

```
# Idea: Pack as many words as fit on a line (counting one
# space between words). Spread the spare spaces over the
# gaps, giving the extra ones to the leftmost gaps.
def justify(words, width, last="left", measure=len,
```

```
        lines=None):
    # last: "left" (the classic rule), "full" (spread it
    # too)
    # or "center". `lines` lets the caller bring its own
    # line breaks (see pack_optimal).
    lines = lines or pack_greedy(words, width, measure)
    out = []
    for i, line in enumerate(lines):
        if i < len(lines) - 1:
            out.append(spread(line, width, measure))
        elif last == "full":
            out.append(spread(line, width, measure))
        elif last == "center":
            out.append(center(line, width, measure))
        else:
            out.append(left_align(line, width, measure))
    return out
```

288 Convert Valid Snake Case to Lower Camel Case

HARD

Template fit: reformat(): split on '_', keep the first piece, capitalise the others

QUESTION

Convert Valid Snake Case to

Lower Camel Case Given a valid snake-case identifier, convert it to lower camel case. The input consists of one or more lowercase ASCII words separated by single underscores. There is no leading underscore, trailing underscore, or pair of adjacent underscores. The output keeps the first word lowercase, removes each underscore, and capitalizes the first letter of every later word. Implement:

```
def solve(name: str) -> str: ...
```

Constraints

- `1 <= len(name) <= 100_000`
- `name` matches `[a-z]+(?:_[a-z]+)*`

Examples

```
"foo_bar" -> "fooBar" "already" -> "already" "http_api_name" -> "httpApiName"
```

MAIN CODE | helper functions first, then the answer

```
# BASE QUESTION (optimal): Split on underscores and
# capitalize
# Idea: Split on '_', keep the first word, and capitalize
# the first letter of each other word with a join. For
# valid input there are no empty parts.
def snake_to_camel(s):
    # foo_bar_baz -> fooBarBaz
    first, *rest = s.split("_")
    caps = [w[:1].upper() + w[1:] for w in rest]
    return first + "".join(caps)
```